

PrimeSim™ XA Command Reference

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	10
Related Products, Publications, and Trademarks	10
Conventions	11
Customer Support	12

1. Using PrimeSim XA Commands	13
PrimeSim XA Command Usage	13
Using XA Commands in a Command Script File	14
Using XA Commands Within the Netlist	14
Command Precedence Rules in Configuration Scripts	15
Using XA Commands During Interactive Debugging Mode	16
Enabling Tcl Mode	17
Using Wildcards	17
Common Syntax Definitions	18
Command Scoping	19
The -inst Option	19
The -node and -except_node Options	19
The -subckt and -except_subckt Options	20
The -inst and -subckt Options	20
The -node and -subckt Options	20
Specifying the <i>enable_value</i> Argument	21
Common 3DIC Syntax Definitions	21
Command Categories	23

2. PrimeSim XA Batch Commands	24
.chk_portpwr	29
.dd_check_block_power	33
.dspf_include	35
add_rc_element	35
enable_print_statement	37
force_node_voltage	37

Contents

keep_top_element	38
load_ba_file	39
load_gndcap_file	46
load_gpd_data	49
load_operating_point	50
load_parameter_file	52
load_spf_file	53
load_vector_file	54
load_verilog_file	55
map_ba_terminal	56
meas_post	57
primesim_xa_dcalg	59
print_pcm_state	60
probe_waveform_current	61
probe_waveform_ixba	66
probe_waveform_logic	71
probe_waveform_pcm	74
probe_waveform_va	75
probe_waveform_voltage	77
protect_element	84
pulse_oscillator	86
release_node_voltage	87
report_dangling_node	88
report_dc_unstable_node	88
report_floating_node	90
report_model	92
report_mram_state	93
report_node_alias	93
report_node_cap	94
report_operating_point	98
report_power	100
report_sim_activity	113

Contents

set_active_net_flow	116
set_analysis_core	117
set_analysis_post	118
set_array_option	121
set_ba_active_file	123
set_ba_option	124
set_bus_format	135
set_capacitor_option	136
set_ccap_level	140
set_ccap_option	141
set_certification_option	143
set_circuit_flash	144
set_circuit_transceiver	145
set_current_option	146
set_dc_icsweep_option	147
set_dc_option	149
set_disable_rawimage	154
set_inductor_option	155
set_dp_option	158
set_duplicate_rule	163
set_flash_option	165
set_floating_node	166
set_hotspot_option	169
set_interactive_option	171
set_interactive_stop	171
set_latch_control	173
set_latch_option	174
set_logic_threshold	175
set_macro_model_option	176
set_meas_option	179
set_message_option	181
set_model_level	183

Contents

set_model_option	185
set_monte_carlo_option	192
set_multi_core	196
set_option XA_SHOW_TIME	197
set_oscillator	197
set_parameter_value	199
set_partition_option	200
set_postlayout_meas	202
set_powernet_ccap_option	205
set_powernet_level	206
set_powernet_option	207
set_probe_option	209
set_probe_window	214
set_ra_functional_resistor	216
set_ra_net	218
set_ra_net_type	219
set_ra_option	220
set_ra_pwnet_driver	224
set_ra_pwnet_option	225
set_ra_reuse	226
set_rc_network_option	228
set_rc_option	229
set_resistor_option	230
set_restore_option	235
set_rf_option	235
set_robust_model	236
set_sample_point	237
set_save_state	238
set_sim_case	244
set_sim_hierid	245
set_sim_level	245
set_sim_mode	247

Contents

set_sim_stop	247
set_spectre_mc	249
set_speed_scale	249
set_sram_characterization	252
set_ssl_version	254
set_struct_verilog_option	254
set_synchronization_level	254
set_synchronization_option	255
set_tolerance_level	257
set_tolerance_option	259
set_va_view	260
set_vector_char	262
set_vector_option	263
set_waveform_option	266
set_waveform_sim_stat	273
set_wildcard_rule	275
set_zstate_option	279
skip_circuit_block	282
source	284
vcd2vec	284

3. PrimeSim XA Interactive Commands	286
PrimeSim XA Interactive Commands	286
alias	288
icheck_node_zstate	288
iclose_log	291
icontinue_sim	291
idelete_break_point	292
iforce_node_voltage	293
ilist_break_point	294
ilist_force_node	295
imatch_elem	296
imatch_node	296
iopen_log	298

Contents

iprint_connectivity	298
iprint_dcpath	302
iprint_flash_cell	303
iprint_elem_info	304
iprint_exi	306
iprint_help	308
iprint_node_info	308
iprint_pcm_state	309
iprint_subckt	310
iprint_time	310
iprint_tree	311
iprobe_waveform_current	312
iprobe_waveform_va	314
iprobe_waveform_voltage	315
iquit_sim	317
irelease_node_voltage	317
ireport_node_cap	318
ireport_operating_point	322
isearch_node	323
iset_break_point	326
iset_diagnostic_option	326
iset_env	327
iset_interactive_option	328
iset_interactive_stop	329
iset_save_state	330
iset_waveform_option	331
iset_zstate_option	331
iset_speed_scale	333
DC Interactive Mode Commands	334
exit	335
iclose_log	335
icontinue_dc	335
idelete_node_ic	336
imatch_elem	337
imatch_node	337
iopen_log	339
iprint_connectivity	339
iprint_elem_info	341

iprint_exi	343
iprint_help	344
iprint_node_info	345
isearch_node	346
iset_node_ic	348
4. PrimeSim XA Dynamic CCK Commands	349
cck_analog_pdown	350
cck_excess_ipath	359
cck_disable	365
cck_post	367
cck_resistive_path_report	368
cck_signal	369
cck_soa	381
cck_substrate	401
cck_toggle_count	405
check_measure	412
check_node_excess_rf	413
check_node_hotspot	415
check_node_quick_rf	417
check_node_zstate	419
check_timing_edge	424
check_timing_hold	428
check_timing_post	433
check_timing_pulse_width	433
check_timing_setup	435
set_report_option	440

About the PrimeSim XA PrimeSim XA Command Reference

The *PrimeSim XA Command Reference* describes the Synopsys PrimeSim XA batch and interactive commands and options.

This preface includes the following sections:

- [New in This Release](#)
 - [Related Products, Publications, and Trademarks](#)
 - [Conventions](#)
 - [Customer Support](#)
-

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in PrimeSim XA: What's New or the *PrimeSim XA Release Notes* on the [SolvNetPlus](#) site.

To see the *PrimeSim XA Release Notes*:

1. Go to the SolvNet Download Center located at the following address:
<https://solvnet.synopsys.com/DownloadCenter>
2. Select "PrimeSim XA," and then select a release in the list that appears.

Note:

You must be logged in to SolvNetPlus for the link to connect directly to the release notes site. If you are prompted to log in to SolvNetPlus upon clicking the link, log in, then click the link again to reach the article.

Related Products, Publications, and Trademarks

For additional information about the PrimeSim XA tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

- Cadence® Spectre® Circuit Simulator
- PrimeWave™ Design Environment WaveView
- Eldo®
- PrimeSim™ HSPICE®
- IC Validator
- StarRC™
- VCS® PrimeSim™ AMS

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
<code>Courier</code>	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.

Convention	Description
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

1

Using PrimeSim XA Commands

This chapter describes how to use PrimeSim XA commands and options.

This chapter includes the following sections:

- [PrimeSim XA Command Usage](#)
- [Enabling Tcl Mode](#)
- [Using Wildcards](#)
- [Common Syntax Definitions](#)
- [Command Categories](#)

PrimeSim XA Command Usage

PrimeSim XA commands are used to control simulator options and features. You can enter the commands in any of the following ways:

- [Using XA Commands in a Command Script File](#)
- [Using XA Commands Within the Netlist](#)
- [Command Precedence Rules in Configuration Scripts](#)
- [Using XA Commands During Interactive Debugging Mode](#)

All commands and their arguments are case-sensitive—even those commands placed within PrimeSim HSPICE and Eldo netlists. Command arguments that refer to netlist identifiers are treated as case-insensitive in PrimeSim HSPICE and Eldo format netlists, and as case-sensitive in Spectre format netlists. See the *PrimeSim XA User Guide* for more information about working with other netlist formats, such as PrimeSim HSPICE Netlist Compatibility, Eldo Netlist Compatibility, or Spectre Netlist Compatibility.

The PrimeSim XA tool uses Tcl, which is a non-proprietary scripting language, as its command language. To facilitate simple and easy use, many of the Tcl features are disabled by default. No previous knowledge of Tcl is required to use PrimeSim XA

commands. You can also set the PrimeSim XA tool to use a full Tcl interpreter, as described in [Enabling Tcl Mode](#).

Using XA Commands in a Command Script File

To use a command script file, enter a `-c` switch on the command line when starting the PrimeSim XA tool, as shown in the following example:

```
xa netlist.sp -c command_file
```

All commands and settings in the command file are processed as if the command were placed on the final line of the netlist. That is, the commands used in the command file override any PrimeSim XA commands used in the netlist file.

Note:

The `set_dp_option` and `set_probe_option` commands need to be placed in the command script file to work.

Using XA Commands Within the Netlist

PrimeSim HSPICE and Eldo syntax netlists use an `.option` command to insert the PrimeSim XA tool-specific commands into a netlist. To enter PrimeSim XA commands directly into the netlist, use the `.option` or `options` statement appropriate for that netlist, in the following syntax:

```
.opt[ion] XA_CMD="xa_command arg arg arg"
```

For example:

```
.option XA_CMD="set_sim_level -level 5"
```

Spectre format netlists use an `options` analysis statement. See the following syntax:

```
user_xa_commands options xa_cmd="xa_command arg arg arg"
```

For example:

```
myoptions options xa_cmd="set_sim_level -level 5"
```

Command Precedence Rules in Configuration Scripts

Commands within configuration scripts follow a set of precedence rules, which determine the order in which they are applied. Following is the rule for what the precedence is when the configuration commands are specified in different places or ways:

- Commands in configuration scripts are processed in the order in which they appear. Later commands take precedence over earlier ones.
- Local commands have higher precedence than global commands. The local command takes precedence if a local command and a global command conflict.
- Commands applied lower in the simulation hierarchy have higher precedence. This means that settings applied to a specific subcircuit take precedence over settings applied to a higher level such as netlist.
- Configuration script commands have higher precedence over netlist commands. If there is a conflict between a command in the configuration script and a command in the netlist, the configuration script command takes precedence

Example of Global Command Order Precedence

The following example has two configuration files, File 1 and File 2. In this example, the global `set_sim_level 5` setting in File 2 takes precedence, as it appears later. Instantiations of subcircuit `bandgap` have `set_sim_level 6` because it is lower in the hierarchy.

File 1:

```
set_sim_level 4
set_sim_level 5
set_sim_level 6 -subckt bandgap
```

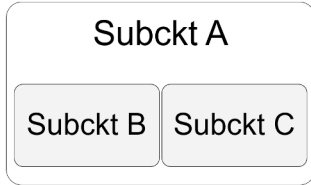
File 2:

```
set_sim_level 6 -subckt bandgap
set_sim_level 4
set_sim_level 5
```

Example of Local Command Order Precedence

The following example demonstrates the local command precedence:

```
set_sim_level 4 -subckt A
set_sim_level 5 -subckt B
set_sim_level 6 -subckt B
```



Commands are processed in order, so the lowest hierarchy setting takes precedence.

This example shows,

- Missing lower hierarchy rule, where subcircuit C has no specific setting, so it gets `set_sim_level 4` from subcircuit A.
- Lower hierarchy rule takes precedence and duplicate command order, where subcircuit B has `set_sim_level 6` because the commands are processed in order, and the most recent local command for B is `set_sim_level 6`.

Using XA Commands During Interactive Debugging Mode

The PrimeSim XA tool provides an interactive mode, in which you can enter commands at the `XA>` command prompt. The PrimeSim XA tool interactive mode commands are listed in [PrimeSim XA Interactive Commands](#).

To enter the PrimeSim XA tool interactive mode, use the `-intr` command-line option. This command-line option is used for all supported netlist formats.

```
-intr [time[unit]]
```

`time`

The time at which the PrimeSim XA tool enters interactive mode. If you do not specify a time, the PrimeSim XA tool does not enter interactive mode until you enter Ctrl-C.

`unit`

The unit of measure for `time` argument. The default is second. There is no space between the `time` and the `unit` arguments, or the PrimeSim XA tool terminates the simulation with an error message.

Ctrl-C

After entering the `-intr` option, you can optionally press Ctrl-C to enter the interactive mode.

Caution:

If you do not specify the `-intr` option on the command line, the PrimeSim XA tool cannot enter the interactive mode. If you enter

Ctrl-C, the PrimeSim XA tool prompts you to abort or continue the simulation.

Interactive Mode With Tcl Commands

The interactive mode defaults to the Tcl OFF mode. You can change to Tcl ON using the `-tcl` command-line switch. All Tcl syntax rules are applied in the Tcl ON mode.

For more information about the special characters in the Tcl ON mode, see the [Enabling Tcl Mode](#) section.

Enabling Tcl Mode

When invoking the simulator, you can enable a full Tcl interpreter by using the `-tcl` switch on the command line (Tcl ON behavior). The Tcl mode cannot be toggled on-and-off during a session, as shown in the following table.

Table 1 *Tcl On-and-Off Behavior*

Character	Tcl ON behavior	Tcl OFF behavior
\$	Variable substitution	Literal \$
[]	Command substitution	Literal []
\	Backslash substitution	Same as Tcl ON
" "	Argument grouping	Same as Tcl ON
{ }	Argument grouping	Same as Tcl ON
#	Comment if first character on line	Same as Tcl ON
;	End of command delimiter	Same as Tcl ON

Using Wildcards

A wildcard character is used to match instance and node names when setting up a simulation using the PrimeSim XA commands and netlist statements. Refer to the individual command for options that support a wildcard. The case sensitivity of the wildcard matching is set according to case sensitivity of the netlist where the names are defined.

The behavior of a wildcard in the PrimeSim XA tool might be different from the PrimeSim HSPICE, Eldo, and Spectre tools. The use of wildcards in the PrimeSim XA tool is an enhanced feature for FastSPICE applications.

[Table 2](#) lists the PrimeSim XA-supported wildcard patterns:

Table 2 *PrimeSim XA-Supported Wildcard Patterns*

Pattern	Description
?	Matches any single character, except the period character (.) and white space.
*	Matches any contiguous group of characters, except white space. The set_wildcard_rule command provides you with many options, including matching at one level, or all levels, for instance and node names, and reports hierarchical aliases for node names.

In the following example, the PrimeSim XA tool probes only the voltage of nodes `x1.a2`, `x1.ao`, and `x1.aout`. Nodes `x1.ini` and `x1.out` are not probed because they are the hierarchical aliases for `ain` and `aout`, respectively.

```
x1 ain aout resnetwork
.subckt resnetwork ini out
r1 ini a2 10
r2 a2 ao 100
r3 a2 out 15
r4 ao aout 10
r5 aout 0 20
.ends

.probe v(x1.*)
```

If `.probe v(x1.*)` is replaced with `.probe v(x1.????)`, then only node `x1.aout` is probed.

The following example evaluates the average voltage of all the nodes with names that are `out` at the first level of hierarchy, and assigns the result to `avgval*`, where `*` follows the name of the first-level instance.

```
.MEASURE TRAN avgval* AVG v(x*.out) FROM=0ns TO=100ns
```

Common Syntax Definitions

The standard syntax conventions are documented in the [Conventions](#) section of this manual. There are also some common syntax definitions shared between many PrimeSim XA commands, as described in the following topics:

- [Command Scoping](#)
- [Common 3DIC Syntax Definitions](#)

Command Scoping

The PrimeSim XA tool supports commands with many scoping options to apply to certain instances, nodes, or subcircuits only.

The -inst Option

The `-inst` option applies the command only to the specified instances. A wildcard character is supported for *instance_name*. See the [Using Wildcards](#) section for more information.

Syntax:

```
-inst {instance_name}
```

The following example applies the `set_sim_level -level 5` command only to the `x1.x2a` and `x1.x1a` instances. All other instances use the default value of the `set_sim_level` command.

```
set_sim_level -level 5 -inst x1.x1a x1.x2a
```

The -node and -except_node Options

The `-node` option applies the command only to the specified nodes. A wildcard character is supported for *node_name*. When *node_name* uses a wildcard, you can use the `-except_node` option to exclude the specified nodes from applying to the command. A wildcard is also supported for *x_node_name*. See the [Using Wildcards](#) section for more information.

Note:

Only a subset of commands supports the `-node` option or the `-except_node` option. Refer to each command for details.

Syntax:

```
[-node node_name {node_name}] \  
[-except_node x_node_name {x_node_name}]
```

The following example applies the `set_powernet_level -level 5` command only to the `x1.node1` and `x1.x1a.n2` nodes.

```
set_powernet_level -level 5 -node x1.node1 x1.x1a.n2
```

The following example applies the `check_node_quick_rf` command to all nodes except for node `out`.

```
check_node_quick_rf -node * ... -except_node out
```

The -subckt and -except_subckt Options

The `-subckt` option applies the command only to the specified subcircuits. The `-except_subckt` option excludes the commands from applying to the specified subcircuits. A wildcard character is not supported for `subckt_name` and `x_subckt_name`.

When the `-subckt` and `-except_subckt` options are specified in a command, the `-except_subckt` option has precedence over the `-subckt` option.

Syntax:

```
[-subckt subckt_name {subckt_name}] \  
[-except_subckt x_subckt_name {x_subckt_name}]
```

The following example applies the `check_node_zstate` command only to the `sensesim` subcircuit.

```
check_node_zstate -node * ... \  
-subckt sensesim cpump -except_subckt cpump
```

The -inst and -subckt Options

When the `-inst` and `-subckt` options are specified, they apply the command only to the specified instances inside the specified subcircuits. A wildcard character is supported for `instance_name`. See the [Using Wildcards](#) section for more information.

Syntax:

```
[-inst instance_name {instance_name}] \  
[-subckt subckt_name {subckt_name}]
```

The following example applies the `set_sim_level -level 5` command only to instances with names beginning with `x2` inside the `cpump` subcircuit. All other instances inside `cpump` use the default value of the `set_sim_level` command.

```
set_sim_level -level 5 -inst x2* -subckt cpump
```

The -node and -subckt Options

When the `-node` and `-subckt` options are specified, they apply the command to the specified nodes inside the specified subcircuits only. A wildcard character is supported for `node_name`. See the [Using Wildcards](#) section for more information.

Syntax:

```
[-node node_name {node_name}] \  
[-subckt subckt_name {subckt_name}]
```

The following example applies the `check_node_excess_rf` command only to the nodes with names beginning with `n` inside the `cpump` subcircuit.

```
check_node_excess_rf ... -node n* -subckt cpump
```

Specifying the *enable_value* Argument

Several commands use *enable_value* as an argument. Possible values are:

Table 3 *Possible Values for the enable_value Argument*

Positive	Negative
1	0
yes	no
true	false
on	off

Common 3DIC Syntax Definitions

Integrated circuits (ICs) of different functions and technologies can be packaged as part of a three-dimensional IC (3DIC) that needs to be simulated as a single circuit. Previously, the 3DIC netlist syntax and features supported parallel configuration in 3DIC integration using an interposer. Now, 3DIC hierarchical configuration is supported to allow for more flexibility in 3DIC integration, including stacked ICs. This enhancement allows the migration of an existing single IC technology flow to a multiple-technology flow, incorporating different process nodes, without major changes in the netlisting process.

Existing PrimeSim XA commands and the probing syntax of voltages and currents in a circuit can be specified using wildcard patterns for instances and nodes. As an extension to this capability, PrimeSim XA commands and probing syntax now support IC module scope. This could be useful in circuit analyses of module instances that appear at different levels of the circuit hierarchy.

Syntax

```
-inst [ic_module_label::]instance_spec
-node [ic_module_label::]node_spec
```

Arguments

Argument	Description
<code>ic_module_label::</code>	Specifies the name that references an IC module that is part of a 3DIC. The IC module label acts as additional condition to compliment any existing scope filter options. So it forces the instance and node matching scopes specified with <code>-inst</code> or <code>-node</code> to be limited within the IC module. All PrimeSim XA commands that support the <code>-inst</code> or <code>-node</code> argument can use the <code>ic_module_label</code> argument.
<code>instance_spec</code>	Specifies the instances upon which the command is applied. You can use wildcard (" <code>*</code> ") characters in instance names.
<code>node_spec</code>	Specifies the nodes upon which the command is applied. You can use wildcard (" <code>*</code> ") characters in node names.

For example:

```
set_model_level -level 5 -inst cismod::*
```

The above example sets the model level to 5 for all instances in the `cismod` module.

The same syntax enhancement that supports the IC module label reference also applies to probing in an HSPICE netlist:

Syntax

```
.print V([ic_module_label::]node_pattern)
.print I([ic_module_label::]instance_pattern)
```

Arguments

Argument	Description
<code>ic_module_label::</code>	Specifies the IC module scope reference.
<code>instance_spec</code>	Specifies the wildcard pattern for node matching.
<code>node_spec</code>	Specifies the wildcard pattern for instance matching.

For example:

```
.probe v(cismod::*)
```

The above command probes all node voltages under the scope of the `cismod` IC module.

Command Categories

[Table 4](#) categorizes the PrimeSim XA batch commands. For a list of the interactive commands, see [PrimeSim XA Interactive Commands](#).

Table 4 *PrimeSim XA Command Categories*

Command Category	Command
Diagnostics	check_node_excess_rf , check_node_hotspot , check_node_quick_rf , check_node_zstate , check_timing_edge , check_timing_hold , , check_timing_pulse_width , check_timing_setup , force_node_voltage , probe_waveform_current , probe_waveform_logic , probe_waveform_va , probe_waveform_voltage , release_node_voltage , report_power , report_sim_activity , set_hotspot_option , set_zstate_option
Netlist Control	keep_top_element , pulse_oscillator , set_va_view , set_capacitor_option , set_duplicate_rule , set_floating_node , set_inductor_option , set_logic_threshold , set_resistor_option , set_wildcard_rule , skip_circuit_block
Output Control	enable_print_statement , report_model , report_operating_point , set_meas_option , set_probe_option , set_probe_window , set_waveform_option
Post-layout Back-annotation	load_ba_file , map_ba_terminal , set_ba_option
Speed and Accuracy Control	set_ccap_level , set_dc_option , set_flash_option , set_model_level , set_model_option , set_oscillator , set_partition_option , set_pownet_option , set_sample_point , set_sim_level , set_tolerance_level , set_tolerance_option
Vector Stimulus	load_vector_file
Reliability Analysis	set_ra_functional_resistor , set_ra_net , set_ra_net_type , set_ra_option , set_ra_pwnet_driver , set_ra_pwnet_option , set_ra_reuse

2

PrimeSim XA Batch Commands

This chapter provides the syntax for all of the PrimeSim XA batch commands.

The PrimeSim XA commands are:

- `.chk_portpwr`
- `.dd_check_block_power`
- `.dspf_include`
- `add_rc_element`
- `enable_print_statement`
- `force_node_voltage`
- `keep_top_element`
- `load_ba_file`
- `load_gndcap_file`
- `load_gpd_data`
- `load_operating_point`
- `load_parameter_file`
- `load_spf_file`
- `load_vector_file`
- `load_verilog_file`
- `map_ba_terminal`
- `meas_post`
- `primesim_xa_dcalg`
- `print_pcm_state`

- [probe_waveform_current](#)
- [probe_waveform_ixba](#)
- [probe_waveform_logic](#)
- [probe_waveform_pcm](#)
- [probe_waveform_va](#)
- [probe_waveform_voltage](#)
- [protect_element](#)
- [pulse_oscillator](#)
- [release_node_voltage](#)
- [report_dangling_node](#)
- [report_dc_unstable_node](#)
- [report_floating_node](#)
- [report_model](#)
- [report_mram_state](#)
- [report_node_alias](#)
- [report_node_cap](#)
- [report_operating_point](#)
- [report_power](#)
- [report_sim_activity](#)
- [set_active_net_flow](#)
- [set_analysis_core](#)
- [set_analysis_post](#)
- [set_array_option](#)
- [set_ba_active_file](#)
- [set_ba_option](#)
- [set_bus_format](#)
- [set_capacitor_option](#)

- `set_ccap_level`
- `set_ccap_option`
- `set_certification_option`
- `set_circuit_flash`
- `set_circuit_transceiver`
- `set_current_option`
- `set_dc_icsweep_option`
- `set_dc_option`
- `set_disable_rawimage`
- `set_inductor_option`
- `set_dp_option`
- `set_duplicate_rule`
- `set_flash_option`
- `set_floating_node`
- `set_hotspot_option`
- `set_interactive_option`
- `set_interactive_stop`
- `set_latch_control`
- `set_latch_option`
- `set_logic_threshold`
- `set_macro_model_option`
- `set_meas_option`
- `set_message_option`
- `set_model_level`
- `set_model_option`
- `set_monte_carlo_option`
- `set_multi_core`

- `set_option XA_SHOW_TIME`
- `set_oscillator`
- `set_parameter_value`
- `set_partition_option`
- `set_postlayout_meas`
- `set_powernet_ccap_option`
- `set_powernet_level`
- `set_powernet_option`
- `set_probe_option`
- `set_probe_window`
- `set_ra_functional_resistor`
- `set_ra_net`
- `set_ra_net_type`
- `set_ra_option`
- `set_ra_pwnet_driver`
- `set_ra_pwnet_option`
- `set_ra_reuse`
- `set_rc_network_option`
- `set_rc_option`
- `set_resistor_option`
- `set_restore_option`
- `set_rf_option`
- `set_robust_model`
- `set_sample_point`
- `set_save_state`
- `set_sim_case`
- `set_sim_hierid`

- [set_sim_level](#)
- [set_sim_mode](#)
- [set_sim_stop](#)
- [set_spectre_mc](#)
- [set_speed_scale](#)
- [set_sram_characterization](#)
- [set_ssl_version](#)
- [set_struct_verilog_option](#)
- [set_synchronization_level](#)
- [set_synchronization_option](#)
- [set_tolerance_level](#)
- [set_tolerance_option](#)
- [set_va_view](#)
- [set_vector_char](#)
- [set_vector_option](#)
- [set_waveform_option](#)
- [set_waveform_sim_stat](#)
- [set_wildcard_rule](#)
- [set_zstate_option](#)
- [skip_circuit_block](#)
- [source](#)
- [vcd2vec](#)

Note:

The following PrimeSim XA batch commands are not available in a VCS PrimeSim AMS mixed-signal simulation.

.chk_portpwr

- `set_interactive_stop`
- `set_save_state` (For saving or restoring information, see the *VCS® PrimeSim™ AMS User Guide*)
- `set_restore_option`

.chk_portpwr**Description**

Checks that the current flowing into/out of specific subckt port is beyond the given current threshold *ith*, then reports the statistical Average (Avg), Root-Mean-Square (RMS) and the Maximum (Max) of the port current with the associated time window.

Syntax

```
.chk_portpwr label = cmd_label
+ port = port_name [port_name ...]
+ ith = real_number
+ [node= node_name [node_name]]
+ [inst = inst_pattern [inst_patten ...]]
+ [limit = integer]
+ [numv = integer]
+ [inst_hierid = post-layout_hier_char]
+ [twindow = start end [start end ...]]
+ [power = off | on]
+ [except_fanout = none | hier_gate]
+ [subckt = subckt_pattern [subckt_patten ...]]
```

Arguments

Argument	Description
<code>port = port_name</code> <code>[port_name ...]</code>	Specifies the full port name or port name as wildcard pattern from subckt definitions with hierarchical netlist input. With post-layout flat netlist, the top-level node name will assume the <code>port</code> name based on the devices connected at the level. Post-layout netlist example: - Command: <pre>.chk_portpwr inst=x* port=vdd limit=3 inst_hierid=/ v1 vdd ... pw1 (...) * top level source connection to vdd</pre> - Post-layout connectivity: *Post-layout device connectivity <pre>x1/m1 vdd ... x1/x2/m1 vdd ... x2/x4/x6/m1 vdd ...</pre> - Subckt port power reporting <pre>x1.vdd = i1(x1.m1) + i1(x1.x2.m1) x1/x2.vdd = i1(x1.x2.m1) x2.vdd = i1(x2.x4.x6.m1) x2/x4.vdd = i1(x2.x4.x6.m1) x2/x4/x6.vdd = i1(x2.x4.x6.m1) * skip reporting as exceeding hierarchy limit 3.</pre>
<code>ith = real_number</code>	Specifies the current threshold for violation reporting. The <code>ith=</code> enforces the current reports filtering only. The power report entries will not be affected.

Argument	Description
<pre>[node = node_name [node_name ...]]</pre>	Specifies the top node or instance port name for the hierarchical port connectivity traversal. The port list matched by this parameter are those hierarchical ports connected to this top-level node or the instance port, regardless of the port names if they matched the <code>port=</code> entries. With both <code>port=</code> and <code>node=</code>, the resulting port list contains both the entries matched from instance port names from the <code>port=</code> list and those found by traversing the top-level node or instanced port name through the netlist hierarchies. With <code>node=</code> specified with instance port name entry, - The instance port entry format is, <pre><instance_1><hierarchical_separator><instance_2> \ <...><hierarchical_separator><port_name></pre> - The instance port name searching is only applied to the hierarchical netlist topology, not the post-layout flat netlist input. - The <code>limit=</code> applies to the matched port full hierarchical path. - This instance hierarchical acts as minimal instance port reporting scope. The output only reports the lower-level instance ports connecting to the specified instance port, but not the higher level instance port even it connects to the same top-level node. Example: - Node connectivity hierarchies: <pre>vdd -> x1.vdd -> x1.x2.vdd -> x1.x2.x3.vdd</pre> - Command input: <pre>.chk_portpwr \ label=my_checkinst="X*"node="x1.x2.vdd" \ limit=10...</pre> - Output entries: <pre>x1.x2.vdd x1.x2.x3.vdd</pre>
<pre>[inst = inst_pattern [inst_pattern ...]]</pre>	Specifies the instance name wildcard patterns. Default as all instance searched as <code>*</code>.
<pre>[limit = integer]</pre>	Specifies port hierarchy depth (N) = instance hierarchy (N-1) + port (1). Default as 2^31 port name hierarchy level.
<pre>[numv = integer]</pre>	Specifies maximum number of violations reported for the given command. Default as 10,000.
<pre>[inst_hierid = post-layout_hier_char]</pre>	Specifies hierarchy delimiter character for instance hierarchy matching with <code>limit</code> control. No default.
<pre>[twindow = start end [start end ...]]</pre>	Specifies transient time windows activating the power check. Default from transient time zero (0) to transient end time.

Argument	Description
[power = off on]	• off- (default) Skips the instance port power reporting and instance block power reporting. • on - Enables instance port power reporting and instance block power reporting. • Instance port power and instance block power entries are scoped by the instance port current entries. • The instance block power will be generated with either pre-layout netlist, post-layout netlist or mixed netlist (pre-layout with extract post-layout blocks) input.
[except_fanout = none hier_gate]	• none - (default) No filtering applied based on device terminal connectivity. • hier_gate: -Filters out any entries (instance port) connecting to MOSFET gate terminals only. -The connectivity check is limited to lower hierarchy level connectivity with schematic hierarchical netlist input, not applicable to the flatten post-layout netlist virtual ports.
[subckt = subckt_pattern [subckt_pattern ...]]	Specifies the subcircuit name wildcard patterns. If no patterns are specified, the default is to search for all subcircuits as (*).

Subckt Port Power Check Report Description

Report File Extension:

- Output file naming convention as *output_prefix.portpwr*

Table 5 PrimeSim-XA subckt port power report format

Statement	Label	Count	Arguments
.chk_portpwr	<i>cmd_label</i>	<i>statement based report count</i>	<i>statement arguments</i>

Field Descriptions:

Statement - Subckt port power command keyword.

Label - Command label string.

Count - Total entry count reported from the given command statement.

Arguments - Full command input string.

.dd_check_block_power**<cmd_label>: Subckt Port Power Check - Port Current Report**

Index	Label	Port	Top-Node	Avg(A)	RMS(A)	Max(A)	Max Time(s)	From(s)	To(s)
<idx>	<cmd_label>	<inst><hier_id><port_name>	<node_name>						

<cmd_label>: Subckt Port Power Check - Port Power Report

Index	Label	Port	Top-Node	Avg(W)	RMS(W)	Max(W)	Max Time(s)	From(s)	To(s)
<idx>	<cmd_label>	<inst><hier_id><port_name>	<node_name>						

<cmd_label>: Subckt Port Power Check - Instance Total Power Report

Index	Label	Instance	Avg(W)	RMS(W)	Max(W)	Max Time(s)	From(s)	To(s)
<idx>	<cmd_label>	<inst>						

Field Descriptions:

Index - Unique output entry index (starting from '0') for each command statement.

Label - Command label string.

Port - Port name: *full_instance_path hier_id port_name*

Top-Node - Top-level node name connected by the instance port.

Avg(A) - Average of the port current. Current unit is "amp" (A).

RMS(A) - Root-Mean-Square of the port current. Current unit is "amp" (A).

Max(A) - Maximum port current flowing into the block (positive) or out from the block (negative). Current unit is "amp" (A).

Max Time(s) - Time that peak port current happened at specific. Time unit is "second" (s).

From(s) - Checking starting time. Time unit is "second" (s).

To(s) - Checking end time. Time unit is "second" (s).

Avg(W) - Average of the instance port power or instance block power. Power unit is "watt" (W).

RMS(W) - Root-Mean-Square of the instance port power or instance block power. Power unit is "watt" (W).

Max(W) - Maximum instance port power or instance block power. Power unit is "watt" (W).

Instance - Full instance hierarchy name.

.dd_check_block_power**Description**

Checks the block power.

Note:

This command cannot be used within the `.opt xa_cmd`.

Syntax

```
.dd_check_block_power label = label_name
+ [inst = inst_pattern [inst_patten ...]]
+ [limit = integer]
+ [twindow = start end [start end ...]]
+ [report = hier | block | both]
+ [iunit= m | u | n | p | f | 0]
+ [subckt = subckt_name {subckt_name}]
+ [file = file_infix]
+ [except_inst = inst_pattern [inst_pattern ...]]
+ [except_subckt = subckt_name [ subckt_name ...]]
```

Arguments

Argument	Description
label = label_name	Specifies the label name that appears in the report file, which makes it easier to search the report.
[inst = inst_pattern [inst_pattern ...]]	Specifies the instance name wildcard patterns. Default as all instance searched as <code>*</code> .
[limit = integer]	Specifies port hierarchy depth (N) = instance hierarchy (N-1) + port (1). Default as 2^31 port name hierarchy level.
[twindow = start end [start end ...]]	Specifies transient time windows activating the power check. Default from transient time zero (0) to transient end time.
[report = hier block both]	Report type: hierarchy, block, or both. Default is both.
[iunit= m u n p f 0]	Specifies the current value unit: • <i>m</i>: 1e-3 • <i>u</i>: 1e-6 • <i>n</i>: 1e-9 • <i>p</i>: 1e-12 • <i>f</i>: 1e-15 • <i>0</i>: no scale
[subckt = subckt_name {subckt_name}]	Specifies the subcircuit names.
[file = file_infix]	Specifies the output file infix. For example, if <code>file=test</code> , the output file will be <code>xa.test.blkpwr</code> .
[except_inst = inst_pattern [inst_pattern ...]]	Excludes the specified instances from checking the block power.

Argument	Description
<code>[except_subckt = subckt_name [subckt_name ...]]</code>	Specifies the names of subcircuits whose instances are to be excluded from checking the block power.

.dspf_include

Includes one Detailed Standard Parasitic Format (DSPF) file.

Syntax

```
.dspf_include "file.spf" [bus_delim="busdelim_schematic  
[busdelim_parasitic]] [port_order=sch|spf]
```

Description

Use this command to include one DSPF file.

- "bus_delim": maps bus-delimiter when schematic and DSPF files are different, if `busdelim_parasitic` is not specified, PrimeSim XA takes it from the DSPF file `"|BUSBIT []"` or `"|BUS_DELIMITER []"`, otherwise PrimeSim XA takes it from the command.
- "port_order": specifies the port-order definition of PrimeSim XA and uses if its value is `"sch"`. PrimeSim XA uses schematic port-order definition, if its value is `"spf"`, PrimeSim XA uses port-order definition in the `spf` file.

Examples

Example 1

```
.dspf_include "file.spf" bus_delim="<> []"
```

In the command, PrimeSim XA replaces all `A[0]` in `file.spf` to `A<0>` when including the `file.spf`.

```
.dspf_include "file.spf" port_order=sch
```

In the command, PrimeSim XA uses the port-order definition of the pre-layout netlist.

add_rc_element

Inserts a resistor or a capacitor between two nodes. If you specify a subcircuit, the extra resistance or capacitance element is inserted between the two nodes locally inside the subcircuit.

Syntax

```
add_rc_element -type (r|R|c|C) -n1 node1 -n2 node2 -val RCval \  
-subckt subcircuit_name
```

Arguments

Argument	Description
-type (r R c C)	Specifies if the element to be added is a resistor or capacitor.
-n1 node1	Specifies that the first node of the element is connected to <i>node1</i> , which a port, global, or local node to the subcircuit name. Supports wildcard characters in <i>node1</i> . However, if both <i>node1</i> and <i>node2</i> contain wildcard characters, the PrimeSim XA exits with error. If no matching node name is found, the entire command will be ignored.
-n2 node2	Specifies that the second node of the element is connected to <i>node2</i> , which a port, global, or local node to the subcircuit name. Supports wildcard characters in <i>node2</i> . However, if both <i>node1</i> and <i>node2</i> contain wildcard characters, the PrimeSim XA exits with error. If no matching node name is found, the entire command will be ignored.
-val RCval	Specifies the value of the element connected between <i>node1</i> and <i>node2</i> . Units are in Ohm for resistors and Farads for capacitors.
-subckt subcircuit_name	Adds the elements to the specified subcircuit.

Examples

```
add_rc_element -type R -n1 nv1 -n2 vdd -val 1e6 -subckt memcell
```

Adds a resistor with a value of 1e6 Ohm between the nv1 and vdd nodes inside the memcell subcircuit.

```
add_rc_element -type C -n1 *_o -n2 gnd -val 1f -subckt addr4
```

Adding 1f capacitor to all output ports of subcircuit *addr4* which have a (*_o*) suffix. In this example, (**_o*) uses a wildcard character to match all node names ending with (*_o*) within the subcircuit *addr4*.

enable_print_statement

Writes the `.PRINT` statements in HSPICE and Eldo format netlists to an ASCII file with a `*.print` extension. By default, the PrimeSim XA tool treats them as `.PROBE` statements. See *PrimeSim XA User Guide* for more information.

Syntax

```
enable_print_statement -switch enable_value
```

Arguments

Argument	Description
<code>-switch enable_value</code>	Enables the command. See the Command Scoping section for more information.

force_node_voltage

Forces the specified nodes to stay at the specified constant voltage during the simulation.

Syntax

```
force_node_voltage -node node_name {node_name} [-voltage voltage_value] \
  [-time time_value] [-subckt subckt_name] [-slope t_value]
```

Arguments

Argument	Description
<code>-node node_name {node_name}</code>	Specifies the node names to force to a constant voltage.
<code>-voltage voltage_value</code>	Specifies the constant voltage value to be applied to the node. If not specified, the node voltage at the specified time is frozen until <code>release_node_voltage</code> command is applied.
<code>-time time_value</code>	Specifies the simulation time when the node is forced. The default is 0 s.
<code>-subckt subckt_name</code>	Forces only the nodes inside the specified subcircuit.
<code>-slope t_value</code>	Specifies the slope of the voltage ramping in s/volt. The default is 10 ps/V.

Description

This command forces the specified nodes to stay at the specified constant voltage at the specified time during the simulation. The node voltage stays at the same value until the end of the simulation or when the `release_node_voltage` command is applied at the specified nodes. Nodes that have been optimized cannot be forced by this command. If an optimized node is forced, a warning message is issued.

Examples

Example 1

The following example forces the `vpump` node to 2.5 volts at 10 ns. This node remains at this value until the end of the simulation or when the `release_node_voltage` command is applied.

```
force_node_voltage -node vpump -voltage 2.5 -time 10ns
```

Example 2

The following example freezes the voltage of node `a1` at time 10ns to its value of the `a1` at that time.

```
force_node_voltage -node a1 -time 10ns
```

keep_top_element

Simulates the specified top-level instance with respect to the other elements in the netlist. You can use this command to keep the specified elements in the top-level hierarchy.

This command works with the `-top` command line option. By default, when you use the `-top` command line option, all elements in the top-level hierarchy will be retained.

Syntax

```
keep_top_element -inst instance_name -all 0|1
```

Arguments

Argument	Description
<code>-inst instance_name</code>	Specifies the instance name to be simulated as the top-level instance.
<code>-all 0 1</code>	By default, the <code>-all</code> option is set to 1 to simulate all top-level elements when no instance is specified. The option is automatically set to 0 when <code>-inst</code> is specified to simulate specific top-level elements only.

Examples

The following example specifies all elements inside the subA subcircuit as top-level elements to be simulated. All elements (including stimulus) at the top-level hierarchy will be retained.

```
$> xa netlist.sp -top subA
```

The following example specifies all elements inside the subA subcircuit as top-level elements, and also keeps the XtopB instance as the top-level instance for simulation.

```
$> xa netlist.sp -top subA -c cfg
```

And the configuration file contains the following information:

```
keep_top_element -instance XtopB
```

load_ba_file

Specifies a postlayout back-annotation file.

Syntax

```
load_ba_file [-file] filename \  
  [-icmodule ic_module_name] \  
  [-xba 0|1] \  
  [-min_res value] \  
  [-max_res value] \  
  [-min_cap value] \  
  [-min_ind value] \  
  [-skipnet net_name {net_name}] \  
  [-rcnet net_name {net_name}] \  
  [-ccnet net_name {net_name}] \  
  [-cnet net_name {net_name}] \  
  [-ccap_to_gcap cap_value] \  
  [-dpf switch value] \  
  [-add_netpin_by_xy_file file_name] \  
  [-add_netpin DSPF_netname DSPF_node {DSPF_node}] \  
  [-add_netpin_file netpin_file_name] \  
  [-add_instpin_file file_name] \  
  [-delete_netpin DSPF_netname DSPF_node {DSPF_node}] \  
  [-delete_netpin_file netpin_file_name] \  
  [-what_if cmd_file] \  
  [-layout_only_device_models model_name {model_name}] \  
  [-gpd gpd_path] \  
  [-report_no_ba value] \  
  [-corner corner_name]
```

Arguments

Argument	Description
<code>-file filename</code>	Specifies the name of the parasitic netlist file (SPEF or SPF format). It can be a gzipped file to save the disk space. The StarRC tool can directly generate gzipped files. For more information, see the <code>NETLIST_COMPRESS_COMMAND</code> command in the <i>StarRC Command Reference</i> .
<code>-icmodule ic_module_name</code>	Specifies the module name to be used in 3DIC designs. For more information, see the Three-Dimensional Integrated Circuit (3DIC) Modularization section in the <i>PrimeSim XA User Guide</i> .
<code>-xba 0 1</code>	Enables the extended back-annotation (XBA) flow. The default is 0. For more information about XBA, see the Enabling the XBA Flow section.
<code>-min_res value</code>	Specifies the lower threshold of resistors from the specified back-annotation file to be kept. It overrides the value specified by the set_ba_option command for the back-annotation file. All resistors with an absolute value below the specified value are shorted by the <code>set_resistor_option -rule=1</code> option. The default is 0.01.
<code>-max_res value</code>	Specifies the upper threshold of resistors from the specified back-annotation file to be kept. It overrides the value specified by the set_ba_option command for the back-annotation file. All resistors with an absolute value above the specified value are treated as open-circuit by <code>set_resistor_option rule=1</code> . The default is 1e12.
<code>-min_cap value</code>	Specifies the lower threshold of capacitors from the specified back-annotation file to be kept. It overrides the value specified by the set_ba_option command for the back-annotation file. All capacitors with an absolute value below the specified value are treated as open-circuit by <code>set_capacitor_option rule=1</code> . The default is 1e-2*CSHUNT. The default of CSHUNT is 1e-20.
<code>-min_ind value</code>	Specifies the upper threshold of inductors from the specified back-annotation file to be kept. It overrides the value specified by the set_ba_option command for the back-annotation file. All inductors above the specified value are shorted by <code>set_inductor_option rule=1</code> . The default is 0.

Argument	Description
<code>-skipnet net_name {net_name}</code>	Specifies the name of nets that should not be back-annotated. You can specify multiple net names. You can use wildcard characters in the net names.
<code>-rcnet net_name {net_name}</code>	Specifies the nets to use for full RC back-annotation. By default (when the <code>-rcnet</code> option is not specified), all nets are back-annotated as full RC. If you specify the <code>-rcnet</code> option, all other nets not specified by this option use only lumped capacitance back-annotation. The lumped capacitance value uses the net capacitance value in the <code>* NET</code> line. This option works in both SPF and SPEF formats and accepts wildcard characters.
<code>-ccnet net_name {net_name}</code>	Specifies for the nets to use Cg+Cc back-annotation (and to remove all resistors from a given net). All the nets not specified by this option use full RC back-annotation (by assuming the net itself contains RCs). This option works in both SPF and SPEF formats and accepts wildcard characters. Also, you cannot use this argument with the <code>-lump_c_only</code> option of the set_ba_option command.
<code>-cnet net_name {net_name}</code>	Specifies the nets to use lumped capacitance back-annotation. The lumped capacitance value uses the net capacitance value in the <code>* NET</code> line. All the nets not specified by this option use full RC back-annotation (by assuming the net itself contains RCs). This option works in both SPF and SPEF formats and accepts wildcard characters. Also, you cannot use this argument with the <code>-lump_c_only</code> option of the set_ba_option command.
<code>-ccap_to_gcap cap_value</code>	Converts all the coupling capacitors to ground capacitors if the value of the coupling capacitor is below the <code>cap_value</code> . If the back-annotation file contains no coupling capacitor, there is no impact. This option works for both SPF and SPEF formats. The default is <code>1e-19</code> .
<code>-dpf switch_value</code>	Specifies to back-annotate the DPF section (that is, the <code>*Instance</code> section in the DSPF file). By default <code>switch_value</code> is 1 and <code>*Instance</code> section is back-annotated.

Argument	Description
<code>-add_netpin_by_xy_file file_name</code>	Instructs the PrimeSim XA tool to insert a "** P" net pin by specifying XY coordinates. It finds the closest location to the specified XY coordinates on the defined metal layer. It uses the following formula to calculate the distance: $d=\sqrt{dX^2+dY^2}$ where • $dX = X_n - X_u$; $dY = Y_n - Y_u$ • X_n and Y_n are node coordinates. • X_u and Y_u are user-specified coordinates. The specified file has the following file format. <pre><net_name> <x> <y> <layer itflayer>=<layer_name> <distance range>=<value></pre> The file format has the following rules: • <code>net_name</code> is the net name where new net pins are added. <code>x</code> and <code>y</code> are the point coordinates. • <code>layer_name</code> is the extracted layer name or ITF layer name as reported in LAYER_MAP in SPF file. • If <code>distance</code> is provided, the node closest to XY point is selected if the distance to the point from the node is smaller than <code>value</code>. • If <code>range</code> is provided, all the nodes with distance to the point smaller than <code>value</code> are selected. • <code>X</code>, <code>Y</code>, <code>distance</code>, and <code>range</code> are defined in DSPF file units. • PrimeSim XA reports a table with names of the selected nodes, or "not found" if no node satisfies argument parameters. • Specify this option together with the <code>-xba 1</code> option. Otherwise, an error message is issued. • Specifying multiple <code>net_name</code> entries inside this file is supported. • Specify only one add net pin by XY coordinates per command line. • Wildcard characters are not supported. • Spaces are allowed between arguments and on either side of the equal symbol ("=").
<code>-add_netpin DSPF_netname DSPF_node {DSPF_node}</code>	Adds a new net pin. The first value is a DSPF net name, followed by one or more DSPF node names. The DSPF node name can be a sub-node (specified with <code>* S</code>), instance pin (specified with <code>* I</code>), or node observation point (<code>** OP</code>).

Argument	Description
<code>-add_netpin_file netpin_file_name</code>	Specifies a user-defined file to add with two columns. The first column is a DSPF net name, and the second column is a DSPF node name. Specify one DSPF net name and one DSPF node name pair per line. The DSPF node name can be a sub-node (specified with <code>* S</code>), instance pin (specified with <code>* I</code>), or node observation point (<code>** OP</code>). Empty lines are supported. Lines starting with an asterisk symbol (<code>"*</code>) are ignored.
<code>-add_instpin_file file_name</code>	Specifies a user-defined file with four columns per line. Each line defines the connection between the DSPF net to the prelayout device pin name in the following format: <pre>DSPF_netname DSPF_nodename Prelayout_instance_name Prelayout_port_name</pre> <code>DSPF_netname</code> is the net name from the DSPF file. For example: <code>* NET vdd ...</code> where vdd is a DSPF net name. <code>DSPF_nodename</code> can be a net pin, an instance pin, or a sub-node. <code>Prelayout_instance_name</code> is a prelayout instance or device name. <code>Prelayout_port_name</code> is a prelayout port name for the instance and device. You can specify only one file name. Only the top-level DSPF file is supported.
<code>-delete_netpin DSPF_netname DSPF_node {DSPF_node}</code>	Deletes a net pin. The first value is a DSPF net name, followed by one or more DSPF node names. The DSPF node name can be a sub-node, an instance pin, or a probe text node. You can specify multiple <code>-delete_netpin</code> options.
<code>-delete_netpin_file netpin_file_name</code>	Specifies a user-defined file to delete with two columns. The first column is a DSPF net name, and the second column is a DSPF node name. Specify one pair of "DSPF net name" and "DSPF node name" per line. The DSPF node name can be a sub-node, an instance pin, or a probe text node. You can specify only one <code>-delete_netpin_file</code> option. Empty lines are supported. Lines starting with an asterisk symbol (<code>"*</code>) are ignored.

Argument	Description
<code>-what_if cmd_file</code>	Specifies a command file to control what modifications can be done to the data read from the SPF file. You can apply only one command file to an SPF file. The command file supports the following commands: <pre>modifyresistor net_name resistor_name new_value</pre> This command specifies the new value for the resistor in the specified net. The width and area of the existing resistor is updated according to the ratio calculated as old resistance divided by the new resistance. <pre>addressistor net_name resistor_name node1 node2 value [[-layer] layer] [-w -a value]</pre> This command specifies where the new resistor is to be added. If you do not specify a layer or width/area, the EM analysis is not performed for the resistor. The resistor is assigned a name as follows: <pre>res_what_if_unique_number node1 node2</pre> <pre>addgcap net_name node1 value</pre> This command specifies where the new ground capacitance is to be added. <pre>modifyresistor vdd R3346 1000</pre> <pre>addressistor vdd vdd:7 vdd:76 100 -layer metall -a 10</pre> <pre>addgcap vdd vdd:193 1e-12</pre> In this example, when applying the command-based what-if analysis feature, the PrimeSim XA tool modifies the R3346 resistor that belongs to the vdd net to 1000 ohm. Also, a new resistor is added to the vdd net that connected between the vdd:7 internal node and vdd:76 internal node with the metall layer and area of 10 um*um. A new ground cap is added to the vdd net on the vdd:193 node with value of 1pF.
<code>-report_no_ba value</code>	Specify one of the following values: • 0 does not report missing SPF nets or instances (default). • 1 reports instance names that contain missing SPF nets. • 2 reports any missing SPF nets.
<code>-layout_only_device_models model_name {model_name}</code>	Provides the layout device model name that only exists in layout to allow proper back-annotation.

Argument	Description
<code>-gpd gpd_path</code>	Specifies the path to the design Galaxy Parasitic Database (GPD).
<code>-corner corner_name</code>	Specifies the name of the corner to simulate. The default is the first corner inside the GPD.

Description

Use the `load_ba_file` command to specify a parasitic back-annotation file. You can specify multiple files using multiple `load_ba_file` commands. You can use a mixture of SPF and SPEF formats.

Enabling the XBA Flow

The traditional back-annotation flow is based on device-to-device name matching between the prelayout and postlayout netlists. In some cases this approach does not provide 100% matching due to stacking device configurations, finger nets, and other discrepancies.

The XBA flow provides another solution by replacing the entire contents of prelayout subcircuit with the postlayout contents of the same subcircuit. It guarantees a clean (100% device and RC) back-annotation and incorporates existing RC back-annotation functionality. The assumption is that SPF file contains the `.SUBCKT` definitions. A top-level (flat) circuit definition is not supported.

After parsing of the prelayout netlist, preprocessing of the SPF file is done. During SPF parsing all the RC networks are collapsed and an ideal netlist is created from the instance section of the SPF file. Then the prelayout content of subcircuit to be back-annotated is replaced by ideal netlist.

After that the RC back-annotation flow runs. This approach provides correct back-annotation results and keeps all back-annotation features available, such as selective back-annotation. You can specify multiple `.SUBCKT` definitions in the SPF file.

During XBA, the following information is saved in the database:

- Node names specified by `*|NET` statement in the SPF file (/ is replaced by .).
- Device names in the instance section of the SPF file.

You can only use those names in the analysis statement.

Note:

The XBA flow has the following limitations:

- Subcircuit and instanced-based simulation command control is not supported if the subcircuit or instance is nested inside the CBA subcircuit/instance.
- `probe_waveform_current [-isub|-x subckt_instance_name.port]` is not supported if the subcircuit or instance is nested inside the XBA subcircuit/instance.

Examples

In the command script file:

```
load_ba_file -file parasitic_net.spf
```

In an HSPICE or Eldo netlist:

```
.opt xa_cmd="load_ba_file -file parasitic_net.spf -skipnet X1.in*"
```

In a Spectre netlist:

```
baoptions options Xa_cmd="load_ba_file -file parasitic_net.spf  
-skipnet X1.in*"
```

The following command enables the XBA flow and specifies the `vdd` net for full RC back-annotation.

```
load_ba_file -file test.SPF - xba 1 -rcnet vdd
```

The following example uses the FF corner in the Galaxy Parasitic Database (GPD) for back-annotation.

```
load_ba_file -file dir/path_to_gpd -corner FF
```

See Also

- [check_node_excess_rf](#)
- [map_ba_terminal](#)

load_gndcap_file

Adds ground capacitors to nodes without modifying the netlist.

Syntax

```
load_gndcap_file [-file] gcap_file {gcap_file}  
[-protect_gndcap 0|1|2]
```

Arguments

Argument	Description
<i>gcap_file</i>	Specifies the name of the file that contains the node names where the ground capacitors and values are added.
-protect_gndcap 0 1 2	0 (default) specifies not to protect capacitors. 1 specifies to protect capacitors. 2 uses the add_rc_element command.

Description

The format of *gcapfilename* is:

```
nodename1 capvalue1 [occ|otc]
nodename2 capvalue2 [occ|otc]
...
nodenamen capvaluen [occ|otc]
```

By default, if you do not specify a keyword, the PrimeSim XA tool adds a ground capacitor of the specified value to the node. The `occ` keyword overwrites the constant capacitance of the node. The `otc` keyword overwrites the total capacitance of the node.

You can specify a node name multiple times (with multiple entries) in the *gcapfilename*. The keyword of the first entry is used for all entries on the same node name. The ground capacitance value applied to the node is the sum of all the capacitance value of the same node name.

Examples

The following example reads in the *cap_file* file and processes the capacitance information in the file.

```
load_gndcap_file -file cap_file
```

The next example uses the schematic shown in [Figure 1](#). If you do not specify the `load_gndcap_file` command, the total capacitance for *out1* and *out2* is:

```
totalc(out1) = c(c1) + c(c12) + c(mp1+mn1) = 5f + 8f + 0.5605f = 13.5605f
totalc(out2) = c(c2) + c(c12) + c(mp2+mn2) = 5f + 8f + 0.5605f = 13.5605f
```

If you use `load_gndcap_file -file capfile` and *capfile* contains:

```
out1 35f
out2 10f
```

The PrimeSim XA tool adds an additional 35f ground capacitor to `out1`, and 10f ground capacitor to `out2`:

$$\begin{aligned}\Sigma c(out1) &= c(c1) + c(c12) + c(mp1+mn1) + c(capfile:out1) \\ &= 5f + 8f + 05605f + 35f = 48.5605f\end{aligned}$$

$$\begin{aligned}\Sigma c(out2) &= c(c2) + c(c12) + c(mp2+mn2) + c(capfile:out2) \\ &= 5f + 8f + 05605f + 10f = 23.5605f\end{aligned}$$

If `capfile` contains:

```
out1 35f occ  
out2 10f occ
```

The PrimeSim XA tool overwrites the constant capacitor of `out1` with 35f, and `out2` with 10f:

$$\begin{aligned}\Sigma c(out1) &= c(capfile:out1) + c(c12) + c(mp1+mn1) \\ &= 35f + 8f + 05605f = 43.5605f\end{aligned}$$

$$\begin{aligned}\Sigma c(out2) &= c(capfile:out2) + c(c12) + c(mp2+mn2) \\ &= 10f + 8f + 05605f = 18.5606f\end{aligned}$$

If `capfile` contains:

```
out1 35f otc  
out2 10f otc
```

The PrimeSim XA tool overwrites the total capacitor of `out1` with 35f, and `out2` with 10f:

$$\begin{aligned}\Sigma c(out1) &= c(capfile:out1) \\ &= 35f\end{aligned}$$

$$\begin{aligned}\Sigma c(out2) &= c(capfile:out2) \\ &= 10f\end{aligned}$$

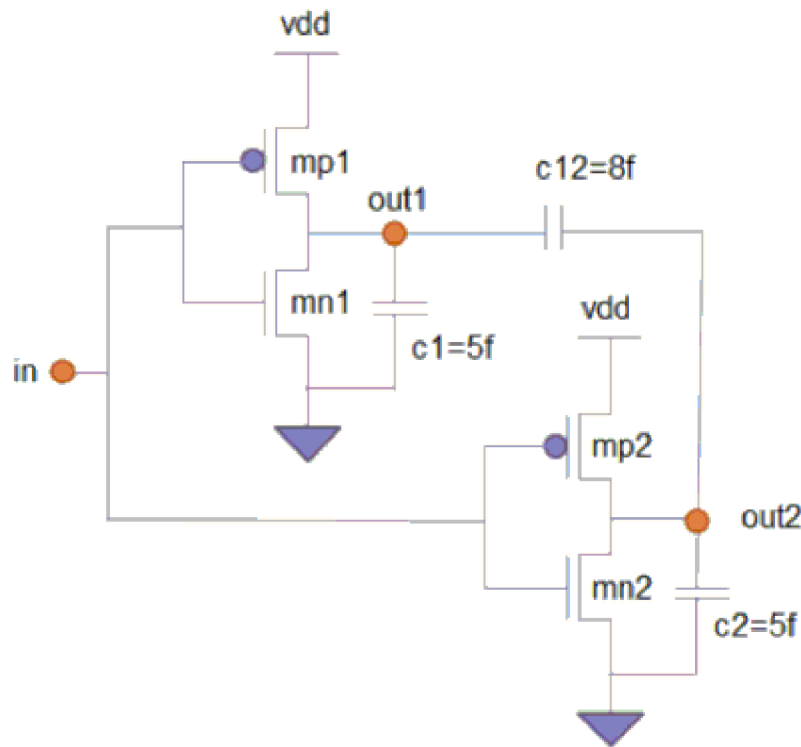
If `capfile` contains:

```
out1 35f otc  
out2 10f occ
```

Because `out1` is specified twice, the PrimeSim XA tool only uses the keyword of the first entry (`otc`) to apply to all other nodes with the same node name. In this case, it overwrites the total capacitor of `out1` with 45f:

$$\begin{aligned}\Sigma c(out1) &= c(capfile:out1) \\ &= 45f\end{aligned}$$

Figure 1 Schematic Example for Adding Capacitance



load_gpd_data

Description

Performs a post-layout simulation on a corner used in the Galaxy Parasitic Database (GPD).

Syntax

```
load_gpd_data [-src] gpd_path [-corner corner_name]
               [-dump_spf 0|1]
```

Arguments

Argument	Description
-src gpd_path	Specifies the path to the design GPD.
-corner corner_name	Specifies the name of the corner to simulate. The default is the first corner inside the GPD.

Argument	Description
<code>-dump_spf 0 1</code>	Specifies whether to dump an SPF format file: • 0(default) does not dump an SPF file. • 1 specifies to dump an SPF file and exit the simulation.

Examples

The following example performs post-layout simulation with the FF corner provided in the GPD.

```
load_gpd_data dir/path_to_gpd -corner FF
```

load_operating_point

Description

Determines how the PrimeSim XA tool handles a file containing initial conditions.

Syntax

```
load_operating_point [-file filename {filename}]
[-node_type node_type]
[-type ic_type]
[-subckt subckt {subckt}]
[-inst inst {inst}]
[-remove_hier hier_name {hier_name}]
[-optimize 0|1]
```

Arguments

Argument	Description
<code>-file filename {filename}</code>	Specifies the name of the file to load. The tool supports files in either native netlist (default) or non-netlist format. You must specify the <code>-type</code> option if the file to load is in non-netlist format. For example, when loading a <code>spectre.ic</code> file, you must specify the <code>-type ic</code> option.
<code>-node_type node_type</code>	Limits the application of initial conditions to nodes detected as <code>node_type</code> . A list of arguments is supported. Currently only the node type <code>latch</code> is supported to limit the application of initial conditions to the detected latches.

Argument	Description
<code>-type ic_type</code>	Specifies the type of initial condition to apply. The <code>ic_type</code> can be <code>ic</code> or <code>nodeset</code>. This option overrides what is specified in the file. You must specify the <code>-type</code> option if the file specified by the <code>-file</code> option is in non-netlist format. When the <code>-type</code> option is not specified, the PrimeSim XA tool treats the file as a netlist file.
<code>-subckt subckt {subckt}</code>	Limits the application of initial conditions to nodes of the instances of <code>subckt</code>. A list of arguments is supported. Wildcards are not supported.
<code>-inst inst {inst}</code>	Limits the application of initial conditions to the specified instances. Wildcards are supported.
<code>-remove_hier hier_name {hier_name}</code>	Specifies a hierarchy to remove from the instance path before applying the DC solution. Note: - The <code>load_operating_point</code> command with the <code>-remove_hier</code> argument is resolved independently. - Arguments such as the <code>-type</code> and <code>-node_type</code> do not take effect if they are defined in the <code>load_operating_point</code> command with the <code>-remove_hier</code> argument.
<code>-optimize 0 1</code>	Enables special processing of the <code>.ic</code> file to optimize performance: <code>0</code> (default): Flattens the hierarchical netlist and loads the IC file. <code>1</code>: Preserves netlist hierarchy while loading initial conditions (ICs) for performance and capacity on designs with many levels of hierarchy. When specified, only the <code>-file</code> and <code>-type</code> options are effective; all other options are ignored.

Examples

The following example reads initial conditions from the `op-file` file and applies them as `nodeset` to latch nodes only.

```
load_operating_point -file op-file -node_type latch -type nodeset
```

The following example reads initial conditions from the `op-file` file and applies them to the nodes in the `xtop.analog` instance only, as defined in the `.ic` or `.nodeset` file.

```
load_operating_point -file op-file -inst xtop.analog
```

The following example reads initial conditions from the `op-file` file and applies them only to latch nodes and the nodes in instances of the `latch_clr` subcircuit, as defined in the `.ic` or `.nodeset` file.

```
load_operating_point -file op-file -subckt latch_clr -node_type latch
```

The following example shows how to use the `-remove_hier` argument to remove a hierarchy from the instance path. In this example, the hierarchy `x1.x2` is removed, so that the instance `x4.x5` is considered. This means that any `.ic` statements that apply to `x1.x2` is also applied to `x4.x5`.

```
load_operating_point -file op1.ic -subckt sensAmpl -inst x4.x5  
-remove_hier x1.x2
```

If there is an `.ic` statement `v(x1.x2.a)=3.3` in the `op1.ic` file, then:

- `load_operating_point -file op1.ic -subckt sensAmpl -inst x4.x5 -remove_hier x1.x2` reads the statement as `v(x4.x5.a)=3.3`.

If there are 2 X instances named `x51` and `x52` under `x4.x5`, then the final resolved `.ic` statements will be two:

```
v(x4.x5.x51.a)=3.3  
v(x4.x5.x52.a)=3.3
```

If there are 2 X instances named `x41` and `x42` under `x4`, then the final resolved `.ic` statements will be two:

```
v(x4.x41.x5.a)=3.3  
v(x4.x42.x5.a)=3.3
```

Note:

The `-type` and `-node_type` arguments are global arguments, and they do not take effect if they are defined in the `load_operating_point` command with the `-remove_hier` argument. To make these arguments take effect, you need to specify them before the `load_operating_point` command with the `-remove_hier` argument. For example,

```
load_operating_point -type ic -node_type latch  
load_operating_point -file op1.ic -subckt sensAmpl -inst x4.x5  
-remove_hier x1.x2
```

load_parameter_file

Lets you override instance parameter values in the netlist.

Syntax

```
load_parameter_file -file filename
```

Arguments

Argument	Description
<code>-file filename</code>	Specifies the name of the file that contains the instance parameter information.

Description

This command lets you specify a file that contains instance parameter values that override the corresponding definitions in the netlist. All other instance parameters keep their original values.

You can specify multiple lines in the file and use # to denote a comment. The file format for each line is:

```
instance_name.parameter_name value
```

You can use `load_parameter_file` in a `.alter` statement.

Examples

```
load_parameter_file -file param.txt
```

The `param.txt` file contains:

```
x1.m1.dtemp -25  
x1.m2.dtemp 0  
x2.m1.dtemp 125  
x2.m2.dtemp 25
```

In the previous example the `dtemp` parameter value overwrites the values for the `x1.m1`, `x1.m2`, `x2.m1`, and `x2.m2` instances. The new `dtemp` value is -25 for the `x1.m1` instance, 0 for the `x1.m2` instance, 125 for the `x2.m1` instance and 25 for the `x2.m2` instance. All other occurrences of `dtemp` keep their original values defined in the netlist.

load_spf_file

Specifies a post-layout file to be included for simulation.

Syntax

```
load_spf_file -file filename [-check_temperature 0|1]
```

Arguments

Argument	Description
<code>-file filename</code>	Specifies the file name of the extracted netlist (SPF) to be included for simulation.
<code>-check_temperature 0 1</code>	Enables checking of the temperature difference between the top-level and extracted netlist. The default is 0.

Description

When you want to include an SPF file and check the temperature:

- Do not use a `.inc` statement in the netlist to include the SPF file.
- Use `load_spf_file` with `-check_temperature 1`.

load_vector_file

Loads a vector stimulus file. It can be a VEC, VCD, or EVCD file. See Vector Stimulus Files in the *PrimeSim XA User Guide* for more information.

Syntax

```
load_vector_file -file filename [-format format_specification]
```

Arguments

Argument	Description
<code>-file file_name</code>	Specifies the file name of a vector stimulus file.
<code>-format format_specification</code>	Specifies the format of a vector stimulus file. If the format of the vector stimulus file is VCD or EVCD, a control signal file is also needed.

Argument of *format_specification*

VEC	Reads in the file as an HSPICE netlist file.
VCD -ctl <i>ctrlfile</i> vcd -ctl <i>ctrlfile</i>	Reads in the file as a VCD file. The <i>ctrlfile</i> specifies the name of the VCD control file. See Using a Value Change Dump File in the <i>PrimeSim XA User Guide</i> for more information.

Argument	Description
<code>ctrlfile</code> EVCD [-ctl] evcd [-ctl <i>ctrlfile</i>]	Reads in the file as EVCD file. The <i>ctrlfile</i> specifies the name of the EVCD control file. See Using the Extended Value Change Dump File in the <i>PrimeSim XA User Guide</i> for more information.

Examples

Instructs the PrimeSim XA tool to load an HSPICE VEC file named `input.vec`.

```
load_vector_file -file input.vec -format VEC
```

Instructs the PrimeSim XA tool to load a VCD file named `stimulus.vcd` and a signal control file named `mapfile`.

```
load_vector_file -file stimulus.vcd -format VCD -ctl mapfile
```

load_verilog_file

Loads a structural Verilog netlist. The PrimeSim XA tool uses the connectivity from Verilog netlist but does not support Verilog functions.

Syntax

```
load_verilog_file -file filename [-icmodule ic_module_name]
```

Arguments

Argument	Description
<code>-file <i>filename</i></code>	Specifies the name of the structural Verilog netlist file.
<code>-icmodule <i>ic_module_name</i></code>	Specifies the module name to be used in 3DIC designs. For more information see Three-Dimensional Integrated Circuit (3DIC) Modularization in the <i>PrimeSim XA User Guide</i> .

Examples

```
#load a structural Verilog netlist
load_verilog_file -file top.v
```

```
#load multiple structural Verilog netlists
load_verilog_file -file inv.v
load_verilog_file -file ring_oscillator.v
```

map_ba_terminal

Specifies the terminal name mapping between the back-annotation file and the terminal names recognized by the simulator.

Syntax

```
map_ba_terminal -name ba_file_term_name  
               [-alias] valid_terminal_name [-subckt subcircuit_name]
```

Arguments

Argument	Description
-name <i>ba_file_term_name</i>	Name of the terminal as it appears in the SPF file in the * I line.
-alias <i>valid_terminal_name</i>	Name of the device terminal (valid names are shown in Table 6) or name of the subcircuit ports.
-subckt <i>subcircuit_name</i>	The terminal mapping that applies only to the specified subcircuit.

Description

The PrimeSim XA tool uses the first character, and optional subsequent characters, to determine which terminal is represented. [Table 6](#) shows the terminal identification characters.

If the instance terminals in the back-annotation file contains names that are not recognized based upon the characters shown in [Table 6](#), the `map_ba_terminal` command must be used.

Table 6 *map_ba_terminal* Identification Characters

Terminal Index	M (MOS)	Q (BJT)	R, C, D (Resistor, Capacitor, Diode)
1	D[R][A][I][N]	C[O][L][L][E][C][T][O] [R]	A[N][O][D][E], P[L][U][S], P[O][S][I][T][I][V][E]
2	G[A][T][E]	B[A][S][E]	B, C[A][T][H][O][D][E], M[I][N][U][S], N[E][G][A][T][I][V][E]
3	S[O][U][R][C][E]	E[M][I][T][T][E][R]	S[U][B][S][T][R][A][T][E]

Table 6 *map_ba_terminal Identification Characters (Continued)*

Terminal Index	M (MOS)	Q (BJT)	R, C, D (Resistor, Capacitor, Diode)
4	B [U] [L] [K]	S [U] [B] [S] [T] [R] [A] [T] [E]	N/A

Examples

In the following example UDRN is used for the drain connection in the back-annotation file. The PrimeSim XA tool does not recognize UDRN, so you must use the following command to back-annotate correctly:

```
map_ba_terminal -name UDRN -alias D
```

```
*|NET Xarr1/Xarr<11>/XI1/mc_n5 0.000826547PF
*|I (Xarr1/Xarr<11>/XI1/MM8:UDRN Xarr1/Xarr<11>/XI1/MM8 UDRN B 0 33.06 40.34)
*|I (Xarr1/Xarr<11>/XI1/MM5:SRC Xarr1/Xarr<11>/XI1/MM5 SRC B 0 32.85 40.815)
*|I (Xarr1/Xarr<11>/XI1/MM4:SRC Xarr1/Xarr<11>/XI1/MM4 SRC B 0 32.67 39.51)
*|I (Xarr1/Xarr<11>/XI1/MM3:UDRN Xarr1/Xarr<11>/XI1/MM3 UDRN B 0 32.95
39.035)
Cg39 Xarr1/Xarr<11>/XI1/MM8:UDRN 0 8.26142e-17
Cg40 Xarr1/Xarr<11>/XI1/MM5:SRC 0 1.2562e-17
Cg41 Xarr1/Xarr<11>/XI1/MM4:SRC 0 1.40334e-16
Cg42 Xarr1/Xarr<11>/XI1/MM3:UDRN 0 6.31581e-18
R51 Xarr1/Xarr<11>/XI1/MM8:UDRN Xarr1/Xarr<11>/XI1/MM5:SRC 10.3526
R52 Xarr1/Xarr<11>/XI1/MM8:UDRN Xarr1/Xarr<11>/XI1/MM4:SRC 31.6749
R53 Xarr1/Xarr<11>/XI1/MM4:SRC Xarr1/Xarr<11>/XI1/MM3:UDRN 3.52783
```

See Also

- [load_ba_file](#)
- [check_node_excess_rf](#)

meas_post

Performs measurements using existing simulation results.

Syntax

```
meas_post -waveform file_name
```

Arguments

Argument	Description
<code>-waveform file_name</code>	You must specify an existing waveform file in <code>fsdb</code> or <code>wdf</code> , or <code>grp</code> format. For information about the <code>grp</code> format, see the <code>-parallel_dump</code> argument in set_waveform_option . Split waveforms are not supported.

Description

When you use `meas_post` in a command script file or with `.option xa_cmd`, the PrimeSim XA tool:

- Only reads in the `.measure` command with the related parameters in the netlist (when the `.measure` command uses parameters).
- Performs the measurement specified in the netlist using the data in the specified waveform file.

You can use only one `meas_post` in a simulation. If you specify more than one command, the PrimeSim XA tool uses the last one and ignores the previous one. the PrimeSim XA tool issues a warning in the log file to point out which `meas_post` command was used and which ones were ignored.

When you run the PrimeSim XA tool with `meas_post`, use the `-o` command line option to redirect the new output data. This option avoids overwriting the simulation log file.

You can use `meas_post` with a SPICE netlist that only includes `.measure` commands. This convention does not support wildcard characters.

Examples

Suppose a previous PrimeSim XA simulation generated the following files: `top.fsdb` and `top.log`. To perform a measurement for the simulation results, do the following steps:

1. Use a measurement command from a previous simulation or add a new one. For example:

```
.meas tran delay trig v(clk) val=1.5 rise=1 targ v(d[1]) val=1.5  
rise=1
```

2. Add the following command in the existing command script file, for example:

```
meas_post -waveform top.fsdb
```

3. Run the PrimeSim XA tool:

```
xa original_netlist -c config -o measNewResults
```

Two new files are created: `measNewResults.log` and `measNewResults.meas`.

Note that you can also use the `-wavefile waveform_file` command line option to rerun measurements with existing simulation results. The following example runs measurements written in the `.meas.spi` file using the results from the `RES/xa.fsdb` file.

```
xa meas.spi -wavefile RES/xa.fsdb
```

primesim_xa_dcalg

Sets the approach for DC convergence in the PrimeSim XA tool.

Syntax

```
.option primesim_xa_dcalg=[0|1|2|3|4|5|6|7]
```

Arguments

Argument	Description
0	Use the XA default DC convergence algorithm (default)
1	Gate partition algorithms for digital and memory circuits.
2	Gate partition merging algorithms for memory and mixed-signal circuits
3	Hybrid mode that uses gate partitioning followed by SPICE mode for final convergence accuracy for memory and mixed-signal circuits.
4	Global pseudo transient algorithm with fast convergence for memory and mixed-signal circuits.
5	Global pseudo transient algorithm for SRAM and mixed-signal circuits.
6	Channel partition algorithm for analog circuits.
7	SPICE algorithm for transient RF, transceiver, and oscillator circuits.

print_pcm_state

Description

Prints the states of PCM cells at a given time to an ASCII file for inspection, or saved for initialization in a later simulation.

Syntax

```
print_pcm_state -time value -file file_name [-mode value]
[-inst inst_name {inst_name}] [-subckt subckt_name {subckt_name}]
[-limit level] [-except_inst inst_name {inst_name}]
```

Arguments

Argument	Description
-time value	Specifies the time for printing the states of selected PCM cells.
-file file_name	Specifies the output file name.
-mode value	Specify one of the following values for the printing mode: • 0 for inspection mode (default), for which the output has the format: <i>inst_name state_value</i>. • 1 for save mode, for which the output has the format: <i>set_parameter_value -name state -value param_value -model model_name -inst inst_name</i>
-inst inst_name {inst_name}	Selects PCM cells to be printed. You can specify a wildcard character in the <i>inst_name</i> .
-subckt subckt_name {subckt_name}	Selects the PCM cells to be printed by subcircuit name.
-limit level	Specifies the hierarchical level down to which the PCM cells are printed. When you specify <i>-subckt</i> the <i>-limit level</i> is relative to where the particular instance is located in the hierarchy. A value of 0 is the top level of the subcircuit. The default for <i>level</i> is 3.
-except_inst inst_name {inst_name}	Specifies the instances to exclude from printing.

Examples

The following example prints the states of all PCM cells inside `x1.x2` to the file `state100ns.txt`.

```
print_pcm_state -time 100n -file state100ns.txt -inst x1.x2.*
```

This example prints the states of all PCM cells inside `x1.x2` to the `state100ns.txt` file:

```
x1.x2.r1 0
x1.x2.r2 1
x1.x2.r3 0
```

probe_waveform_current

Creates current waveform output.

Syntax

```
probe_waveform_current -i | il instance_name {instance_name}
[-in instance_name {instance_name}]
[-iall instance_name {instance_name}]
[-isub | -x subckt_instance_name.port {subckt_instance_name.port}]
[-subckt subckt_name] [-limit level] [-level level_val]
[-except_inst instance_name {instance_name}]
[-except_subckt subcircuit_name {subcircuit_name}]
[-except_port except_pattern] [-filetag file_tag]
```

Arguments

Argument	Description
<code>-il</code> or <code>-i instance_name {instance_name}</code>	Specifies the name of the instance at which the current through instance terminal 1 is probed.
<code>-in instance_name</code>	Specifies the name of the instance at which the current of terminal <i>n</i> is probed, where <i>n</i> is a positive integer. Note this argument applies only to instances not specified by <code>-subckt</code> .
<code>-iall instance_name</code>	Specifies the name of the instance at which the current of all terminals are probed. Note this argument applies only to instances not specified by the <code>-subckt</code> option.

Argument	Description
<code>-isub -x</code> <code>subckt_instance_name.port</code> <code>{subckt_instance_name.port}</code>	<code>-isub</code> or <code>-x</code> returns the total current flowing into the subcircuit port. The <code>subckt_instance</code> is a subcircuit instance name, and <code>port</code> is a name of the subcircuit port. Positive values indicate the current is flowing into the subcircuit port. Negative values indicate the current is flowing out of the subcircuit port. Note that <code>-x</code> is not supported in Eldo netlist syntax.
<code>-subckt subckt_name</code>	Specifies the subcircuit of the instances at which the current is probed.
<code>-limit limit_val</code>	Specifies the hierarchy level down to which currents are probed when you use wildcard characters. A value of 0 specifies the top level. The default for <code>limit_val</code> is 3. When you specify <code>-subckt</code> , <code>limit_val</code> is relative to the scope of <code>subckt_name</code> . See Table 7 for probing behavior with different combinations of <code>-limit</code> and <code>-level</code> .
<code>-level level_val</code>	Specifies the relative hierarchical level down to which currents are probed when you use wildcard characters. Numbering starts with 1 at the hierarchical level where the first wildcard appears. When you specify <code>-subckt</code> , <code>level_val</code> is relative to the scope of <code>subckt_name</code> in addition to the position of the first wildcard. See Table 7 for probing behavior with different combinations of the <code>-limit</code> and <code>-level</code> option.
<code>-except_inst instance_name</code> <code>{instance_name}</code>	Specifies the instances to exclude from probing.
<code>-except_subckt subcircuit_name</code> <code>{subcircuit_name}</code>	Specifies the subcircuits to exclude from probing.
<code>-except_port except_pattern</code>	Excludes ports that match the <code>except_pattern</code> from probing. You can specify wildcards in the pattern name. This option only works together with the <code>-isub</code> or <code>-x</code> option.
<code>-filetag file_tag</code>	Specifies a file tag to direct the signals probed with this command to a separate waveform file. The waveform file has the standard name with <code>.filetag</code> inserted before the normal file suffix, for example, <code>xa.filetag.wdf</code>. The <code>-filetag</code> option is not supported for the <code>tr0</code> and <code>psf</code> formats.

Description

Probes the current through an instance pin. The current waveform is written to the output file in the format specified by the `post` option in the netlist.

The PrimeSim XA tool-supported wildcard characters (*) can be used in the `instance_name` identifiers. The wildcard character can be used to match one level—or all levels—of the hierarchy. See the [set_wildcard_rule](#) command.

Note:

The `probe_waveform_current` command cannot be used to probe subcircuit terminal currents.

Probes specified by this command are in addition to the `.probe` statement in the PrimeSim HSPICE or Eldo netlist files or `save` statements in the Spectre netlist files.

Long simulations, or simulations in which some nodes have a high level of activity, can produce very large waveform files. To minimize waveform file loading time in these files, you can direct signals to separate waveform files and keep file sizes smaller.

To direct signals to a separate waveform file, use the `-filetag` argument. All of the signals probed by that instance of the command are directed to a separate waveform file. The file name has the same format as the standard waveform file name, except for the suffix; for example, `xa.mytag.wdf`. All of the settings made by [set_waveform_option](#) also apply to the tagged output files. A signal can be directed to multiple files if it is probed with another command that specifies a different `file_tag`. Additionally, multiple `probe_waveform_current` commands can use the same file tag. The other probing commands (voltages and logic) can also use the same file tag.

When you use the `probe_waveform_current -isub|x subckt.port -except_port port_list`, the command returns the current flowing in the specified ports of the subcircuit specified after `-isub` or `-x` and excludes the current in the ports specified with `-except_port`.

The `-except_port` option can only work together with `or -x`. If `-except_port` is used, but neither `-isub` nor `-x` is specified in the `probe_waveform_current` command, then `-except_port` is ignored.

The `-except_port` option can work together with the following `probe_waveform_current` options:

- `-isub|x` (mandatory)
- `-subckt`
- `-except_inst`

- -filetag
- -limit

Examples

```
probe_waveform_current -i vdd vss vda
```

```
probe_waveform_current -iall X1.X2.M37  
probe_waveform_current * -except_inst *clk
```

The last example probes all instances down to the default level of hierarchy (3), except those finishing with the `clk` pattern.

The following example probes the `i1` current in `mrxdrv` in all instances of `rxblock` and diverts them to the `rx_block` waveform file. If the PrimeSim XA tool is run as `xa net.sp -o xa -wavefmt wdf`, the waveform file is named `xa.rx_block.wdf`.

```
probe_waveform_current mrxdrv -subckt rxblock -filetag rx_block
```

If the netlist contains:

```
.subckt inv  in out  
mn1 out in gnd gnd n w=5.00u l=1.00u  
mp1 out in vdd vdd p w=10.00u l=1.00u  
.ends  
.subckt xfer  gn gp d s  
mp1 d gp s vdd p w=10.00u l=1.00u  
mn1 d gn s gnd n w=5.00u l=1.00u  
.ends  
.subckt xor2  in1 in2 out  
mn1 out n3 n2 gnd n w=5.00u l=1.00u  
mp1 out n3 n4 vdd p w=10.00u l=1.00u  
x2  in2 n3 inv  
x3  n4 n2 inv  
x4  n2 n4 n3 out xfer  
x1  in1 n4 inv  
.ends  
x1  b  a  p  xor2  
x2  cin p  sum xor2
```

And the configuration file contains:

```
probe_waveform_current -isub x1.* -except_port out
```

The `probe_waveform_current` command probes the currents flowing in all ports of instance `x1` except for the `out` port. For example:

```
isub(x1.in1)  
isub(x1.in2)  
  
isub(x1.x1.in)  
isub(x1.x2.in)
```



```
isub(x1.x3.in)
isub(x1.x4.gn)
isub(x1.x4.gp)
isub(x1.x4.d)
isub(x1.x4.s)
```

Use the same netlist as the previous example, but with the following `probe_waveform_current` command:

```
probe_waveform_current -isub * -subckt xor2 -except_port in*
```

This `probe_waveform_current` example probes all currents flowing in the XOR2 subcircuit ports and in the ports of the subcircuits below XOR2 in the hierarchy, except for the ports starting with `in`. For example:

```
isub(x1.out)
  isub(x1.x1.out)
  isub(x1.x2.out)
  isub(x1.x3.out)
  isub(x1.x4.gn)
  isub(x1.x4.gp)
  isub(x1.x4.d)
  isub(x1.x4.s)
isub(x2.out)
  isub(x2.x1.out)
  isub(x2.x2.out)
  isub(x2.x3.out)
  isub(x2.x4.gn)
  isub(x2.x4.gp)
  isub(x2.x4.d)
  isub(x2.x4.s)
```

Use the same netlist as the first example, but with the following `probe_waveform_current` command:

```
probe_waveform_current -isub * -subckt xor2 -except_port in* -except_inst
x2
```

This `probe_waveform_current` command probes all currents flowing in the XOR2 subcircuit ports, except for the ports starting with `in`. It also excludes the `x2` instance of the XOR2 subcircuit. You can see in the waveform file:

```
isub(x1.out)
  isub(x1.x1.out)
  isub(x1.x3.out)
  isub(x1.x4.gn)
  isub(x1.x4.gp)
  isub(x1.x4.d)
  isub(x1.x4.s)
isub(x2.out)
  isub(x2.x1.out)
  isub(x2.x3.out)
```

```
isub(x2.x4.gn)
isub(x2.x4.gp)
isub(x2.x4.d)
isub(x2.x4.s)
```

The following example probes all currents from hierarchy level 1 to level 3 inside subcircuit volgen.

```
probe_waveform_current x1.* -level 3 -subckt volgen
```

The following two examples use a netlist with:

```
x1 from subckt l1;
x1.x2b from l2 inside l1;
x1.x2b.x3 from base inside l2 then inside l1;
x1.x2b.x3.xb2 from base2 inside base then inside l2 then inside l1.
```

The following example probes all instances in the subcircuit named l2, except those instances inside l2 and named r3.

```
probe_waveform_current * -subckt l2 -except_subckt l2.r3
```

The following example probes all instances the subcircuit named l2, except those instances inside the subcircuits whose name matches ba*. All instances in x1.x2b.x3 are excepted.

```
probe_waveform_current * -subckt l2 -except_subckt ba*
```

See Also

- [probe_waveform_logic](#)
- [probe_waveform_va](#)
- [probe_waveform_voltage](#)
- [set_wildcard_rule](#)

probe_waveform_ixba

Probes the sum of the all the device terminal currents of the specified sub-circuit instance touching the specified node. In the waveform file, the signal of the command has the syntax of *inst_name_to_spf.i(*:node)*.

Syntax

```
probe_waveform_ixba -ipattern ipattern -node node_name
```

Arguments

Argument	Description
<code>-ipattern ipattern</code>	Specifies the device pattern, or full hierarchical instance name down to the instance in the SPF file. You can use wildcard characters in the instance name. If the instance is not inside SPF file, the PrimeSim XA tool issues a warning in the log file.
<code>-node node_name</code>	Specifies a top-level schematic node name to be traced. You cannot use wildcard characters in the node name.

Description

The PrimeSim XA tool supports extended back-annotation (XBA) flow (see the [Enabling the XBA Flow](#) section) that removes the contents of the specified prelayout subcircuit and replaces it with the contents of the Instance Section in the SPF file. The XBA flow ensures a clean back-annotation. However, because it creates a flat SPF netlist, the circuit hierarchy of the replaced subcircuit no longer exists, so the XBA flow has the following limitations.

- Subcircuit-based simulation command control is not supported.
- ISUB/X probing option is not supported.
- The [report_power](#) command is not supported.
- Prelayout instance names are replaced by postlayout instance names, so some commands need to change to reflect the instance name changes, such as probing.

This command addresses the ISUB/X probing limitation.

This command also supports using IXBA in the `.MEASURE` and `.PROBE` statements. The syntax is:

```
ixba(ipattern:node_name)
```

For example:

```
.probe ixba(x0.xx5.*:x0.a5)
.measure tran avg_xp avg ixba(x0.xx5.*:x0.a5)
```

The `ixba()` probe statement is equivalent to `probe_waveform_ixba -ipattern x0.xx5.* -node x0.a5`. You can use the `ixba(ipattern:node)` function to access the signal if you need to use it for analysis statements.

Examples

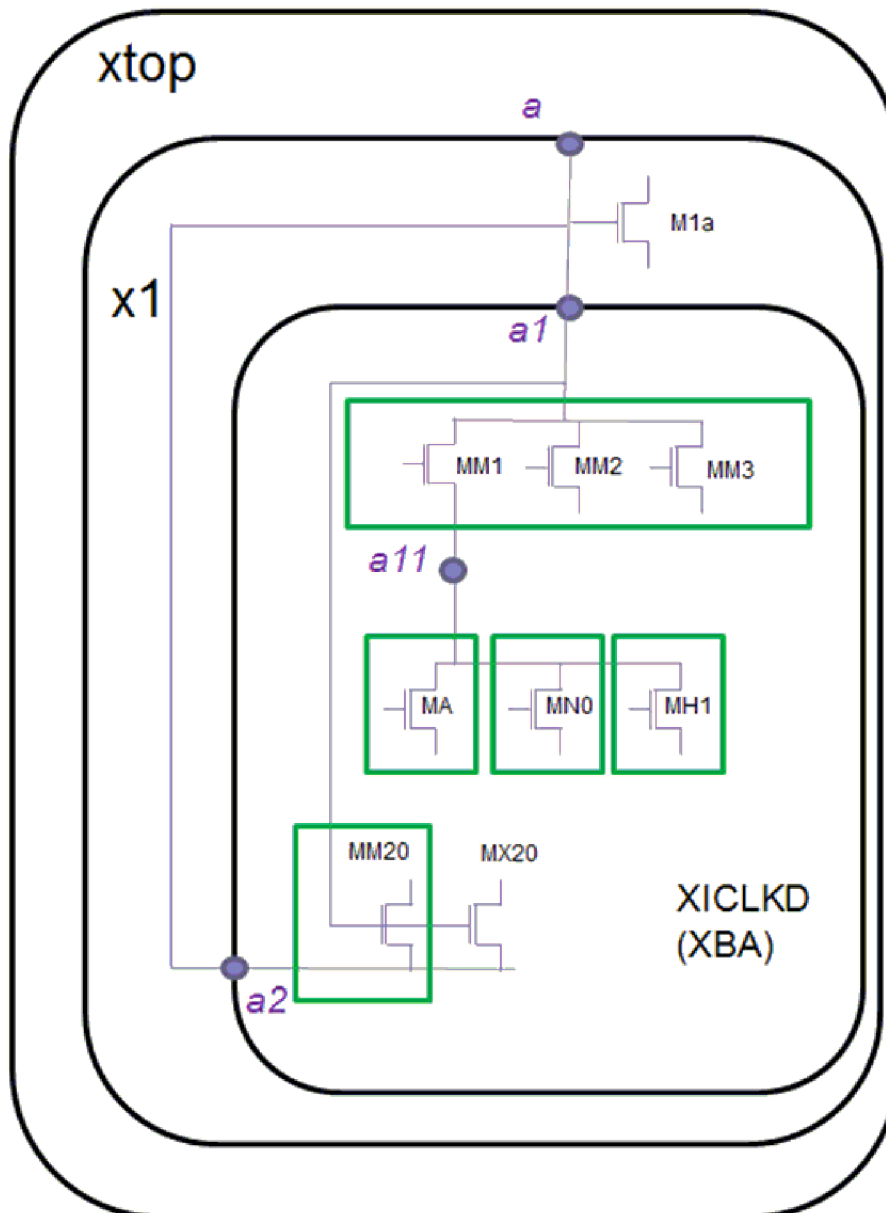
The following example shows how to use the `probe_waveform_ixba` command to replace ISUB functionality. ISUB is often used to probe the port current into a subcircuit. For more information about the ISUB functionality, see the *PrimeSim XA User Guide*.

Because the XBA flow flattens the hierarchy of the SPF file, ISUB cannot be applied because there are no more subcircuits, and their instances, in the netlist hierarchy. To probe the sum of the device currents from a subcircuit instance, use `probe_waveform_ixba`. For example:

```
probe_waveform_ixba -ipattern X1.X2.X3 -node VDD
```

In [Figure 2](#), the green box represents the original subcircuit structure. The XBA flow is enabled on instance `xtop.x1.xa` with `load_ba_file -file xafile.spf -xba 1`.

Figure 2 ISUB Example



The xafile.spf file has the following content:

```
...
Instance Section
X1/CLKD/X1/XCKGD1/MM1...
XI/CLKD/X1/XCKGD1/MM2...
XI/CLKD/X1/XCKGD1/MM3...
XI/CLKD/X1N/XPLL/XCLK/MM20...
```

```
X1/CLKD/MX20...
...
X1/CLKD/X2/X1NAND/MA...
X1/CLKD/X1N202[5]/MNO...
X1/CLKD/XXXTIEL/MH1...
...
```

The following command traces all the devices below the hierarchy of `xtop.x.xiclk` touching the node `a` and probes the sum of all the device terminal currents below the hierarchy `xtop.x.xiclk`.

```
probe_waveform_ixba -ipattern xtop.x1.xiclk.* -node a
```

The PrimeSim XA tool probes the sum of device current terminal of the following devices.

```
Ixba(xtop.x1.xiclk :a) = Iterminal (XI/CLKD/X1/XCKGD1/MM1)
                        + Iterminal (XI/CLKD/X1/XCKGD1/MM2)
                        + Iterminal (XI/CLKD/X1/XCKGD1/MM3)
                        + Iterminal (XI/CLKD/XIN/XPLL/XCLK1/MM20)
                        + Iterminal (XI/CLKD/MX20)
```

Where:

```
Iterminal (XI/CLKD/XIN/XPLL/XCLK1/MM20) = Ig (XI/CLKD/XIN/XPLL/XCLK1/MM20)
                                           + Is (XI/CLKD/XIN/XPLL/XCLK1/MM20)
```

```
Iterminal (XI/CLKD/MX20) = Ig (XI/CLKD/MX20) + Is (XI/CLKD/MX20)
```

In reference to ISUB, the current output from `probe_waveform_xba` is slightly different from ISUB. Originally if XBA is not enabled and ISUB is used:

```
Isub(xtop.a) = Iterminal (XI/CLKD/X1/XCKGD1/MM1)
              + Iterminal (XI/CLKD/X1/XCKGD1/MM2)
              + Iterminal (XI/CLKD/X1/XCKGD1/MM3)
              + Iterminal (XI/CLKD/XIN/XPLL/XCLK1/MM20)
              + Iterminal (XI/CLKD/MX20)
              + Iterminal (M1a)
```

```
Isub(xtop.x1.a1) = Iterminal (XI/CLKD/X1/XCKGD1/MM1)
                  + Iterminal (XI/CLKD/X1/XCKGD1/MM2)
                  + Iterminal (XI/CLKD/X1/XCKGD1/MM3)
                  + Ig (XI/CLKD/XIN/XPLL/XCLK1/MM20)
                  + Ig (XI/CLKD/MX20)
```

The following command traces all the devices below the hierarchy of `xtop.x.xiclk.x2` touching the node `xtop.x1.xiclk.a11`. Based on the SPF file, there are six devices touching node `xtop.x1.xa.a11`, but there is only one device below the hierarchy `xtop.x1.xiclk.x2`.

```
probe_waveform_ixba -ipattern xtop.x1.xiclk.x2.* -node
  xtop.x1.xiclk.a11
```

The outcome of this command is the current terminal of `XI/CLKD/X2/XINAND/MA`:

```
Ixba(xtop.x1.xiclkd.all) = Iterminal(XI/CLKD/X2/XINAND/MA)
```

You can specify:

```
probe_waveform_ixba -ipattern xtop.x1.xiclkd.mn* -node a
```

To probe current from devices with name starting from `xtop.x1.xiclkd.mn`, or:

```
probe_waveform_ixba -ipattern xtop.x1.*.* -node a
```

To probe currents from all devices under hierarchy `xtop.x1.*` (third level of hierarchy) that are connected to node `a`.

Note:

The name pattern for `-ipattern` is applied only to devices. The extracted (parasitic) resistors and capacitors from SPF are not included in the current summation.

See Also

- [load_ba_file](#)
- [probe_waveform_logic](#)
- [probe_waveform_va](#)
- [probe_waveform_voltage](#)
- [set_wildcard_rule](#)

probe_waveform_logic

Generates logic values of specified nodes in the output waveform file.

Syntax

```
probe_waveform_logic -node node {node} [-nN inst_name {inst_name}]  
[-nall inst_name {inst_name}] [-loth low_threshold]  
[-hith high_threshold] [-subckt subckt_name {subckt_name}] [-limit level]  
[-level level_val] [-except_node node_name {node_name}]  
[-except_inst instance_name {instance_name}]  
[-except_subckt subckt_name {subckt_name}] [-port enable_value]  
[-filetag file_tag]
```

Arguments

Argument	Description
<code>-node node {node}</code>	Specifies the node to be printed as a logical signal.
<code>-nN inst_name {inst_name}</code>	Specifies the name of the instance. The Nth terminal is probed as a logic signal, where N is a positive integer. Note that this option cannot probe subcircuit instance terminals.
<code>-nall inst_name {inst_name}</code>	Specifies the name of the instance in which all terminals are probed as a logic signal. Note that this option cannot probe subcircuit instance terminals.
<code>-loth low_threshold</code>	Specifies the threshold voltage for the LOW logic state, 0.
<code>-hith high_threshold</code>	Specifies the threshold voltage for the HIGH logic state, 1.
<code>-subckt subckt_name {subckt_name}</code>	Probes the nodes of the specified subcircuits as logic signals.
<code>-limit limit_val</code>	Specifies the hierarchy level down to which node logic values are probed when you use wildcard characters. A value of 0 specifies the top level. The default for <i>limit_val</i> is 3. When you specify <code>-subckt</code> , <i>limit_val</i> is relative to the scope of <i>subckt_name</i> . See Table 7 for probing behavior with different combinations of <code>-limit</code> and <code>-level</code> .
<code>-level level_val</code>	Specifies the relative hierarchical level down to which node logic values are probed when you use wildcard characters. Numbering starts with 1 at the hierarchical level where the first wildcard appears. When you specify <code>-subckt</code> , <i>level_val</i> is relative to the scope of <i>subckt_name</i> in addition to the position of the first wildcard. See Table 7 for probing behavior with different combinations of the <code>-limit</code> and <code>-level</code> options.
<code>-except_node node_name {node_name}</code>	Specifies the nodes to exclude from probing.
<code>-except_inst instance_name {instance_name}</code>	Specifies the instances to exclude from probing.
<code>-except_subckt subckt_name {subckt_name}</code>	Specifies the subcircuits to exclude from probing.
<code>-port enable_value</code>	If you specify an <i>enable_value</i> value of 1, a wildcard character matches the subcircuit port names.

Argument	Description
<code>-filetag file_tag</code>	Specify a file tag to direct the signals probed with this command to a separate waveform file. The waveform file has the standard name with <code>.filetag</code> inserted before the normal file suffix, for example, <code>xa.filetag.wdf</code> . The <code>-filetag</code> option is not supported for the <code>tr0</code> and <code>psf</code> formats.

Description

The default settings for `-loth` and `-hith` are derived from [set_logic_threshold](#). By default, `-loth` is set to 30% of the high voltage, and `-hith` is set to 70% of the high voltage. The PrimeSim XA tool uses a search algorithm to determine the high/low voltage for a given node if there are multiple supply domains.

If you only specify `-loth`:

- Voltage $\leq$ the `low_threshold` signal is 0.
- Voltage $\geq$ 70% of the high-voltage signal is 1.

The in-between signal is U (unknown).

If you only specify `-hith`:

- Voltage $\geq$ the `high_threshold` signal is 1.
- Voltage $\leq$ 30% of the high-voltage the signal is 0.

The in-between signal is U (unknown).

If you specify both `-loth` and `-hith`:

- If `-loth` $\geq$ `-hith`, error out.
- If voltage $<$ `low_threshold`, the signal is 0.
- If voltage $\geq$ `high_threshold`, the signal is 1.
- If `low_threshold` $<$ voltage $<$ `high_threshold`, the signal is U (unknown).

Different signals can use different logic thresholds; but for a single signal, there can be only one logic threshold, even if the signal is being probed to several file tags as specified by the `-filetag` option.

Note:

You can only apply the `-nN` and `-nall` arguments to primitive instances.

Examples

This example prints the logic state of the `data` node in the waveform file. If `data` has a voltage of $\leq 0.6V$, the logic state is 0; otherwise, the logic state is 1.

```
probe_waveform_logic -node data -loth 0.6
```

See Also

- [probe_waveform_current](#)
- [probe_waveform_va](#)
- [probe_waveform_voltage](#)
- [set_wildcard_rule](#)

probe_waveform_pcm

Description

Specifies how to probe the states and resistances of selected PCM cells.

Syntax

```
probe_waveform_pcm pcm -lvs value -lvr value  
[-inst inst_name {inst_name}] [-subckt subckt_name {subckt_name}]  
[-limit level] [-except_inst inst_name {inst_name}]
```

Arguments

Argument	Description
-lvs value	Specifies one of the following values for state printing: • 0 disables state printing. • 1 enables state printing (default).
-lvr value	Specifies one of the following values for resistance printing: • 0 disables resistance printing. • 1 enables resistance printing (default).
-inst inst_name {inst_name}	Selects the PCM cells to be probed. You can use a wildcard character in the <i>inst_name</i> .
-subckt subckt_name {subckt_name}	Selects the PCM cells to be probed by subcircuit name.

Argument	Description
<code>-limit level</code>	Specifies the hierarchical level down to which the PCM cells are probed. When you specify <code>-subckt</code> the <code>-limit</code> level is relative to where the particular instance is located in the hierarchy. A value of 0 is the top level of the subcircuit. The default for <code>level</code> is 3.
<code>-except_inst</code> <code>inst_name</code> { <code>inst_name</code> }	Specifies the instances to exclude from probing.

Examples

The following example probes the resistances of all PCM cells inside `x1.x2`.

```
probe_waveform_pcm -lvs 0 -lvr 1 -inst x1.x2.*
```

probe_waveform_va

Syntax

```
probe_waveform_va \  
-var variable_name {variable_name} | parameter_name {parameter_name} \  
[-subckt subckt_name] [-limit level] [-level level_val] \  
[-branch_voltage named_branch_list] \  
[-branch_current named_branch_list]
```

Arguments

Argument	Description
<code>-var variable_name</code> { <code>variable_name</code> } <code>parameter_name</code> { <code>parameter_name</code> }	Specifies the hierarchical path to the Verilog-A variable or parameter.
<code>-subckt subckt_name</code>	Specifies to probe the variables in all instances of the specified subcircuit. If you specify <code>-subckt</code> the <code>var_name</code> is local to the subcircuit.
<code>-limit limit_val</code>	Specifies the hierarchy level down to which values of Verilog-A variables and parameters are probed when you use wildcard characters. A value of 0 specifies the top level. The default for <code>limit_val</code> is 3. When you specify <code>-subckt</code> , <code>limit_val</code> is relative to the scope of <code>subckt_name</code> . See Table 7 for probing behavior with different combinations of <code>-limit</code> and <code>-level</code> .

Argument	Description
<code>-level <i>level_val</i></code>	Specifies the relative hierarchical level down to which values of Verilog-A variables and parameters are probed when you use wildcard characters. Numbering starts with 1 at the hierarchical level where the first wildcard appears. When you specify <code>-subckt</code> , <i>level_val</i> is relative to the scope of <i>subckt_name</i> in addition to the position of the first wildcard. See Table 7 for probing behavior with different combinations of the <code>-limit</code> and <code>-level</code> options.
<code>-branch_voltage</code> <code><i>named_branch_list</i></code>	Specifies the hierarchical path to the named Verilog-A branch for which branch voltage is probed.
<code>-branch_current</code> <code><i>named_branch_list</i></code>	Specifies the hierarchical path to the named Verilog-A branch for which branch current is probed.

Description

Note that `probe_waveform_va`:

- Probes the value of Verilog-A variables or parameters, branch voltages, branch currents and writes them to the plot file. It does not probe Verilog-A ports or electrical nodes. Use the [probe_waveform_voltage](#) command to probe electrical signals.
- Scopes only to Verilog-A instances and modules, and does not print SPICE MOS model parameters.

The output to the waveform file is the hierarchical path and the variable name. There are no signal access functions such as `v()` or `i()` for the voltage and current signals. The separator between the hierarchical path and the variable name is the hierarchical separator for the simulation as defined by the [set_sim_hierid](#) command or the `.hier` command (Eldo format only).

The PrimeSim XA tool-supported wildcard characters (*) can be used in the *variable_name* identifier. The wildcard character can be used to match one level, or all levels, of the hierarchy. See the [set_wildcard_rule](#) command.

Examples

Probes the variable count in the x1 module. The variable appears in the waveform file as x1.count.

```
probe_waveform_va -var x1.count
```

Probes all variables and parameters in the x1 module.

```
probe_waveform_va -var x1.*
```

Probes all variables and parameters in the x1 module matching the pattern, cout*.

```
probe_waveform_va -var x1.cout*
```

Probes all variables and parameters in all instances of the subcircuit mymodule.

```
probe_waveform_va * -subckt mymodule
```

Probes the branch voltage of b1 inside x1 module.

```
probe_waveform_va -branch_voltage x1.b1
```

Probes the branch current of b1 inside x1 module.

```
probe_waveform_va -branch_current x1.b1
```

See Also

- [probe_waveform_current](#)
- [probe_waveform_logic](#)
- [probe_waveform_voltage](#)
- [set_wildcard_rule](#)

probe_waveform_voltage

Creates a voltage waveform output.

Syntax

```
probe_waveform_voltage -v node_name {node_name}
[-vn instance_name {instance_name}] [-vall instance_name {instance_name}]
\
[-vsub subckt_instance_name.port {subckt_instance_name.port}]
[-subckt subckt_name] [-limit limit_val] [-level level_val]
[-except_inst instance_name {instance_name}]
[-except_subckt subcircuit_name {subcircuit_name}]
[-except_node node_name {node_name}] [-port enable_value]
[-ba_net net_name {net_name}] [-filetag file_tag]
[-ba_net_pin_type pin_name {pin_name}]
[-ba_net_except_node node_name {node_name}] [-tstep period_value]
[-twindow {start_time stop_time} start_time [stop_time]]
```

Arguments

Argument	Description
-v node_name {node_name}	Specifies the node name at which the voltage is probed.

Argument	Description
<code>-vn instance_name</code>	Specifies the name of the instance at which the voltage of terminal <i>n</i> is probed, where <i>n</i> is a positive integer.
<code>-vall instance_name</code>	Specifies the name of the instance at which the voltage of all terminals are probed.
<code>-vsub subckt_instance_name.port</code> {subckt_instance_name.port}	Probes the ports of the subcircuit using local port names. You can use wildcard characters. This probe matches ports identically to an isub probe, except that it reports voltage instead of current.
<code>-subckt subckt_name</code>	Probes the nodes in all instances of the named subcircuit. If you specify this argument, the nodes or instances should be in the subcircuit instance hierarchy level.
<code>-limit limit_val</code>	Specifies the hierarchy level down to which voltages are probed when you use wildcard characters. A value of 0 specifies the top level. The default for <i>limit_val</i> is 3. When you specify <code>-subckt</code> , <i>limit_val</i> is relative to the scope of <i>subckt_name</i> . See Table 7 for probing behavior with different combinations of the <code>-limit</code> and <code>-level</code> options.
<code>-level level_val</code>	Specifies the relative hierarchical level down to which voltages are probed when you use wildcard characters. Numbering starts with 1 at the hierarchical level where the first wildcard appears. When you specify <code>-subckt</code> , <i>level_val</i> is relative to the scope of <i>subckt_name</i> in addition to the position of the first wildcard. See Table 7 for probing behavior with different combinations of the <code>-limit</code> and <code>-level</code> .
<code>-except_inst instance_name</code> {instance_name}	Specifies the instances to exclude from probing.
<code>-except_subckt subcircuit_name</code> {subcircuit_name}	Specifies the subcircuits to exclude from probing.
<code>-except_node node_name {node_name}</code>	Specifies the nodes to exclude from probing.
<code>-port enable_value</code>	If you specify an <i>enable_value</i> value of 1, a wildcard character matches the subcircuit port names.

Argument	Description
<code>-ba_net net_name {net_name}</code>	Probes the voltage waveforms for the SPF/SPEF pin names. You can specify a wildcard character in a net name. Note that this argument only works in back-annotation and is not applicable to the <code>-subckt</code> and <code>-limit</code> arguments.
<code>-filetag file_tag</code>	Specifies a file tag to direct the signals probed with this command to a separate waveform file. The waveform file has the standard name with <code>.filetag</code> inserted before the normal file suffix, for example, <code>xa.filetag.wdf</code> . The <code>-filetag</code> option is not supported for the <code>tr0</code> and <code>psf</code> formats.
<code>-ba_net_pin_type pin_name {pin_name}</code>	Probes the voltage waveforms only for the specified Instance pins in <code>pin_name</code> from the SPF/SPEF out of the back-annotated nets specified by the <code>-ba_net node_name</code> argument. Note that the probe scope of this option depends on the nodes specified by <code>-ba_net</code> . Currently <code>pin_name</code> is supported only for gate instance pins.
<code>-ba_net_except_node node_name {node_name}</code>	Specifies node names not to probe. You can specify a wildcard character in the node names. Note that node names specified are in the <code>* NET</code> section in the SPF/SPEF.
<code>-tstep period_value</code>	Specifies the time step period value to use with <code>-twindow</code> .
<code>-twindow {start_time stop_time} start_time [stop_time]</code>	Defines a printing window for the specified signal. In that window the signal is printed using the time step defined with <code>-tstep</code> . The <code>-twindow</code> and <code>-tstep</code> arguments are useful when the <code>set_sample_point -period period_value</code> <code>-twindow time_windows_values</code> command is used and you want to later run FFT analysis. It is recommended to use <code>probe_waveform_voltage -v net_name -tstep period_value -twindow</code> with the same window as for <code>set_sample_point -twindow</code> . The signal is dumped at a regular time step within the <code>-twindow</code> . Note: The <code>-twindow</code> option must be used together with the <code>-tstep</code> option. Otherwise, the <code>-twindow</code> option will be ignored.

Description

Probes the voltage on a node or on the pin of a primitive instance. The voltage waveform is written to the output file in the format specified by the `post` option in the netlist.

The PrimeSim XA tool-supported wildcard character (`*`) can be used in the `node_name` and `instance_name` identifiers. The wildcard character can be used to match one level—or all levels—of the hierarchy. See the `set_wildcard_rule` command.

Probes specified by this command are in addition to the `.probe` statement in the PrimeSim HSPICE or Eldo netlist files or `save` statements in the Spectre netlist files.

Long simulations, or simulations in which some nodes have a high level of activity, can produce very large waveform files. To minimize waveform file loading time in these files, you can direct signals to separate waveform files and keep file sizes smaller.

To direct signals to a separate waveform file, use the `-filetag` argument. All of the signals probed by that instance of the command are directed to a separate waveform file. The file name has the same format as the standard waveform file name, except for the suffix; for example, `xa.mytag.wdf`. All of the settings made by `set_waveform_option` also apply to the tagged output files. A signal can be directed to multiple files if it is probed with another command that specifies a different `file_tag`. Additionally, multiple `probe_waveform_voltage` commands can use the same file tag. The other probing commands (voltage and logic) can also use the same file tag.

Understanding the Difference Between the -level and -limit Options

The `-level` and `-limit` option are relative to the current hierarchy level set by `-subckt`. When you use the `*` symbol as wildcard character, `-level` is relative to the hierarchical level at which the first `*` appears.

Table 7 *-limit and -level Combinations*

	-limit	-level	Comment
1	Not specified	Not specified	Default with -limit 3
2	Specified	Not specified	-limit takes effect
3	Not specified	Specified	-level takes effect
4	Specified	Specified	The more restrictive one between -level and -limit takes effect

For example:

```
probe_waveform_voltage * -limit 0
```


Probes all nets at the top level only. It is the same as:

```
probe_waveform_voltage * -level 1
```

The following example probes all nets at the top level and 1 level of hierarchy below.

```
probe_waveform_voltage * -limit 1
```

It is the same as:

```
probe_waveform_voltage * -level 2
```

The following example shows no probes because limit 0 is the top level.

```
probe_waveform_voltage x1.* -limit 0
```

The following example shows no probes because limit 1 is the first level of hierarchy from the top, and x1.x2.* is the second level of hierarchy, so a minimum limit of 2 is required.

```
probe_waveform_voltage x1.x2.* -limit 1
```

The following example probes all nets in x1 (but does not probe in x1.x2, x1.x3, and so on).

```
probe_waveform_voltage x1.* -level 1
```

It is the same as:

```
probe_waveform_voltage x1.* -limit 1
```

The following example probes all nets in x1.x2 (but does not probe in x1.x2.x1, x1.x2.x3, and so on).

```
probe_waveform_voltage x1.x2.* -level 1
```

It is the same as:

```
probe_waveform_voltage x1.x2.* -limit 2
```

Probing Back-Annotated Nets

The `-ba_net_pin_type` and `-ba_net_except_node` arguments can enhance the probing of back-annotated nets together with the `-ba_net` argument. If you want to extract and probe gate-only instance pins from SPF/SPEF, you can use the `-ba_net_pin_type` argument. Also, If you want to filter out any specific pattern you do not want to probe from SPF/SPEF, you can use the `-ba_net_except_node` argument with `-ba_net`.

Note that the `-ba_net` argument is the parent option, and `-ba_net_pin_type` and `-ba_net_except_node` arguments are child options, so you should define the parent option to get the effects from the child options. If not, warning messages are printed on the screen and in the log file.

For example:

```
probe_waveform_voltage -ba_net * -ba_net_pin_type gate
```

Filters out gate ipins out of all back-annotated nets (*).

```
probe_waveform_voltage -ba_net * -ba_net_except_node *net*
```

Filters out the specified patterns such as *net*, *net, and net* from the SPF/SPEF.

```
probe_waveform_voltage -ba_net * -ba_net_except_node *net*
```

Probes only gate pins with patterns such as *net*, *net and net* from the SPF/SPEF.

Examples

Probes all voltage nodes down to the default level of hierarchy, which is 3.

```
probe_waveform_voltage *
```

Probes all voltage nodes from hierarchy level 1 to level 4.

```
probe_waveform_voltage *.* -limit 4
```

Does not probe the voltage of any node. The asterisk characters *.* refer to all of the nodes at the first level and below, but the `-limit 0` argument limits the nodes (to be probed) to only the top level. The arguments are, therefore, contradictory.

```
probe_waveform_voltage *.* -limit 0
```

Probes all nodes down to the second level of hierarchy, except those finishing with `clk` pattern.

```
probe_waveform_voltage * -except_node *clk
```

Probes the voltage waveform from the SPF/SPEF instance pins for any net name that matches `clk`.

```
probe_waveform_voltage -ba_net clk*
```

Specifies to probe the `ctrl` signal and all signals that begin with `sig` in all occurrences of the `mysub` subcircuit. The `-limit 2` option is relative to the top level of the subcircuit and limits the depth of the wildcard probe.

```
probe_waveform_voltage ctrl sig* -subckt mysub -limit 2
```

It is equivalent to the following SPICE definition:

```
.subckt mysub
.probe v(ctrl)
.probe v(sig*) level=2
...
ends
```

Probes the signal `rx_data` in all instances of `rxblock` and diverts them to the `rx_block` waveform file. If the PrimeSim XA tool is invoked as `xa net.sp -o xa -wavefmt wdf`, the waveform file is named `xa.rx_block.wdf`.

```
probe_waveform_voltage rx_data -subckt rxblock -filetag rx_block
```

The following examples use this sample circuit:

```
.subckt toplevel t1 t2
X1 t1 t2 downsub
...
.ends
.subckt downsub d1 d2
...
.ends
X0 a b toplevel
X01 c d toplevel
```

Probes all of the ports in `xtoplevel` and any subcircuits contained in `toplevel` down to the limit of 1. The following probes are in the waveform file: `v(x0.t1)`, `v(x0.t2)`.

```
probe_waveform_voltage -vsub x0.* -limit 1
```

Probes all of the ports in `x0` and any subcircuits contained in `x0` down to the limit of 1. The following probes are in the waveform file: `v(x0.t1)`, `v(x0.t2)`, `v(x0.x1.d1)`, and `v(x0.x1.d2)`.

```
probe_waveform_voltage -vsub x0.*
```

You can combine the `vsub` probe with the `-subckt` option. The `vsub` instance.port becomes local to the subcircuit definition:

```
probe_waveform_voltage -vsub * -subckt toplevel
```

This command probes all subcircuit ports in all instances of the named subcircuit as well as the ports of any subcircuit instance in `toplevel` down the default limit of 3. This example probes: `v(x0.t1)`, `v(x0.t2)`, `v(x0.x1.d1)`, `v(x0.x1.d2)`, `v(x01.t1)`, `v(x01.t2)`, `v(x01.x1.d1)`, and `v(x01.x1.d2)`.

To probe only the ports of the named subcircuit use `-limit 0` to prevent matching any deeper into the hierarchy:

```
probe_waveform_voltage -vsub * -subckt toplevel -limit 0
```

This command probes all subcircuit ports in all instances of the named subcircuit, `toplevel`: `v(x0.t1)`, `v(x0.t2)`, and `v(x01.t1)`, `v(x01.t2)`.

Probes only the port voltages of all instances of the `downsub` subcircuit. The following probes are in the waveform file: `v(x0.x1.d1)`, `v(x0.x1.d2)`, `v(x01.x1.d1)`, and `v(x01.x1.d2)`.

```
probe_waveform_voltage -vsub * -subckt downsub -limit 0
```

When you use wildcard matching, except patterns can be used to exclude some nodes:

```
probe_waveform_voltage -vsub * -subckt downsub -limit 0 -except_node x0.*
```

This command probe only the ports of all instances of the `downsub` subcircuit, but excludes any instance of `downsub` in `x0`. The following probes are added to the waveform file: `v(x01.x1.d1)`, `v(x01.x1.d2)`.

Probes all voltage nodes from hierarchy level 2 to level 5.

```
probe_waveform_voltage x1.x2.* -level 4
```

Probes all voltage nodes from hierarchy level 0 to level 2 inside subcircuit `volgen`.

```
probe_waveform_voltage * -limit 2 -level 6 -subckt volgen
```

The following two examples use a netlist with:

```
x1 from subckt l1;  
x1.x2b from l2 inside l1;  
x1.x2b.x3 from base inside l2 then inside l1:  
x1.x2b.x3.xb2 from base2 inside base then inside l2 then inside l1.
```

The following example probes all nodes the subcircuit named `l2`, except those nodes inside `l2` and starting with `i1`.

```
probe_waveform_voltage * -subckt l2 -except_subckt l2.i1*
```

The following example probes all nodes the subcircuit named `l2`, except those nodes inside the subcircuits whose name matches `b*`. All nodes in `x1.x2b.x3` are excepted.

```
probe_waveform_voltage * -subckt l2 -except_subckt b*
```

See Also

- [probe_waveform_current](#)
- [probe_waveform_logic](#)
- [probe_waveform_va](#)
- [set_wildcard_rule](#)

protect_element

Protects resistor, inductor, capacitor, and diode (R/L/C/D) elements from optimization.

Syntax

```
protect_element [-type] element_type [-min min_threshold]  
[-max max_threshold] [instance_spec]
```

Arguments

Argument	Description
<code>-type element_type</code>	Specifies the type of element to protect from optimization: • R for resistor. • L for inductor. • C for capacitor. • D for diode. You can specify only one type of element in one command, but you can include multiple <code>protect_element</code> commands in a simulation. If you specify the <code>-type</code> argument without any other arguments, it applies to all the elements.
<code>-min min_threshold</code> <code>-max max_threshold</code>	Specifies the range of values of the elements not to be optimized. <code>-min min_threshold</code> specifies the lower threshold and <code>-max max_threshold</code> specifies the upper threshold. Any element in the range is protected from optimization. If you specify <code>minthres</code> without <code>maxthres</code>, all R/L/C elements larger than or equal to <code>minthres</code> are protected from optimization. If you specify <code>maxthres</code> without <code>minthres</code>, all R/L/C elements smaller than or equal to <code>maxthres</code> are protected from optimization. Note that these values are not applied (ignored) to diodes.
<code>instance_spec</code>	See the Common Syntax Definitions section for details about the <code>instance_spec</code> argument.

Description

By default, the PrimeSim XA tool might optimize R/L/C/D elements in the circuit based on their values and topology. There are cases when you do not want any optimization done on certain R/L/C/D elements because it would affect the functionality or accuracy of the simulation. So you can use the `protect_element` command to disable the optimization of those elements. Full diode models are used for protected diodes.

Examples

The following example protects all the top-level resistors from being optimized:

```
set_wildcard_rule one
protect_element -type R -inst *

or

protect_element R -inst R*

or

protect_element -type R -inst R*
```

The following example protects all resistors from being optimized:

```
protect_element -type R
```

or

```
protect_element -type R -inst *
```

The following command protects all resistors with value from 10 ohms to 20 ohms from being optimized:

```
protect_element -type R -min 10 -max 20 -inst *
```

The following command protects the top-level resistor whose name begins with R12 and other hierarchical resistors whose name begins with R24 that has value from 10 ohms to 18 ohms from being optimized:

```
protect_element -type R -min 10 -max 18 -inst R12* *.R24*
```

The following command protects all the resistors whose value from 10 ohms to 100 ohms only inside the p11 subcircuit from being optimized:

```
protect_element -type R -min 10 -max 100 -subckt p11
```

The following example protects all the top level resistors whose value is exactly 10 ohms from being optimized:

```
protect_element -type R -min 10 -max 10 -inst R*
```

The following example protect all the top level resistors with values larger than or equal to 10 ohms and all capacitors from all hierarchy levels with value smaller than or equal to 1e-20 farads from being optimized:

```
protect_element -type R -min 10 -inst R*  
protect_element -type C -max 1e-20 -inst *C*
```

pulse_oscillator

Applies a current kick to a specified node.

Syntax

```
pulse_oscillator -node node_name {node_name} -pw pulse_width  
-time value [value ...] [-amp amp_value] [-rt rt_value]
```

Arguments

Argument	Description
<code>-node node_name</code> <code>{node_name}</code>	Defines the node at which the current source is connected.
<code>-pw pulse_width</code>	Specifies the duration of the current pulse.
<code>-time value</code>	Specifies the time at which the current pulse starts. You can specify multiple pulses by listing multiple times.
<code>-amp amp_value</code>	Specifies the amplitude of the current pulse.
<code>-rt rt_value</code>	Specifies the rise and fall time of the current pulse.

Description

A current pulse with a pulse width of half the expected oscillation period is usually sufficient to start oscillations, but sometimes you need to experiment with the pulse amplitude.

If a circuit has a fully differential structure, then use two `pulse_oscillator` commands: one connected to the positive and negative branch of the differential structure and one applied with opposite polarity.

Examples

```
pulse_oscillator -node xosc.pl -pw 1n -time 10u -amp 1u  
pulse_oscillator -node xosc.nl -pw 1n -time 10u -amp -1u
```

release_node_voltage

Description

Releases the node voltages from the values fixed by [force_node_voltage](#). When you specify this command, the simulation results determine the node voltages.

Syntax

```
release_node_voltage -node node_name {node_name} [-time time_value]  
[-subckt subckt_name] [-soft time_value]
```

Arguments

Argument	Description
<code>-node node_name</code> <code>{node_name}</code>	Specifies the node names to release.

Argument	Description
<code>-time time_value</code>	Specifies the time to release the nodes. The default is 0.
<code>-subckt subckt_name</code>	Releases only the nodes inside the specified subcircuit.
<code>-soft time_value</code>	When a node has been forced and then is released, unexpected spikes can occur on that node because the node is suddenly driven/loaded again. This argument adds a resistive path to the node to take over the current when the node is released. The user-defined <i>time_value</i> specifies the time needed by the node to stabilize again after it has been released.

Examples

In the following example the `vpump` signal previously forced to a given value is released at 150ns. The simulation results determine the `vpump` voltage value until the end of the run unless you use a new [force_node_voltage](#) command.

```
release_node_voltage vpump -time 150ns
```

report_dangling_node

Description

Reports dangling nodes in a separate file.

Syntax

```
report_dangling_node enable_value
```

Arguments

Argument	Description
<i>enable_value</i>	By default, when this option is off, the PrimeSim XA tool reports dangling nodes in the log file. Turn this option on to report dangling nodes in a separate file with the following format: <i>sim_output_file_name.dng</i> .

report_dc_unstable_node

Reports unstable nodes during DC initialization, including the percentage of total iterations and the voltage change value for each node. The unstable nodes are reported in the output `.unstable` files.

Syntax

```
report_dc_unstable_node
[-time wall_time_value]
[-iter num_of_iter]
[-volt_thresh voltage_threshold_value]
[-flush flush_time_value]
[-sort 0|1]
```

Arguments

Argument	Description
-time wall_time_value	Reports a node as unstable if matrix solving takes more than wall_time_value to converge. The value is in seconds. Default is 3600s.
-iter num_of_iter	Reports a node as unstable if matrix convergence takes more than the num_of_iter of iterations to converge. Default is 10,000.
-volt_thresh voltage_threshold_value	Reports a node with a voltage change during two consecutive iterations larger than the voltage_threshold_value. Default is 0.5V.
-flush flush_time_value	Forces to write the buffered data to a new .unstable report every flush_time_value. Default is 300s. The output .unstable report is generated with a suffix n, where n is an incremental number. For example: xa.unstable.iter.1, xa.unstable.iter.2, xa.unstable.iter.3 and so on.
-sort 0 1	Controls data sorting in the output .unstable files. • 1: Reports unstable nodes with the highest value on top in the output .unstable files. • 0 (default): Reports unstable nodes in the output .unstable files without data sorting.

Description

Use the report_dc_unstable_node command to report nodes as unstable when the specified option settings are violated during DC initialization. Nodes that go beyond vdd and ground are always reported. The command reports unstable nodes in the following output .unstable files:

- output.unstable.time

node name (solving time) (voltage change value) (iteration number)
(relative iteration percentage)
vdd (2.000000e-02) (1.503) (677) (2.00000e+01%)
...

- output.unstable.iter

```
node name (iteration number) (voltage change value) (solving time)
  (relative iteration percentage)
vdd ( 677 ) ( 1.503 ) ( 0.000000e+00 ) ( 2.00000e+01% )
...
```

- output.unstable.volt

```
node name (voltage change value) (solving time) (iteration number)
  (relative iteration percentage)
vdd ( 1.503 ) ( 0.000000e+00 ) ( 677 ) ( 2.00000e+01% )
...
```

Limitation

When the `report_dc_unstable_node` command is run with the `set_dc_option -method spice` command, the number of iterations is meaningless because there is only one single matrix. Thus, all nodes have the same number of iterations.

Examples

In the following example, DC initialization is run with `set_dc_option method=2`. Nodes are reported as unstable and are sorted in the output `.unstable` files, if the time to converge is greater than 1000s.

```
set_dc_option -method 2
report_dc_unstable_node -time 1000 -sort 1
```

In the following example, three `.unstable` files (`.unstable.time`, `.unstable.iter`, and `.unstable.volt`) are generated. The `.time` file uses the default 3600s threshold; the `.iter` file uses the user-specified 5000 iterations; and the `.volt` file uses the default 0.5V threshold.

```
report_dc_unstable_node -iter 5000
```

report_floating_node

Reports all floating nodes in a file with a `.fnode` extension, or `.fgate` if you only request floating gates.

Syntax

```
report_floating_node [-type type] [-format format] [-limit limit]
[-file file_name]
```

Arguments

Argument	Description
<code>-type type</code>	Specify one of the following values: <code>gate</code> generates a list of floating gates in a <code>.fgate</code> file. <code>all</code> generates a list of floating nodes in a <code>.fnode</code> file (default).
<code>-format format</code>	Can specify: <code>ic</code> specifies the format in the form of a <code>.ic</code> statement (default). <code>vsrc</code> specifies the format in the form of voltage source. <code>report</code> specifies the format in the form of node and port connection table.
<code>-limit limit</code>	Specifies the hierarchy level down to which a floating node is checked and reported. You can only use this option with <code>-format report</code> . By default, all floating nodes are reported.
<code>-file file_name</code>	Specifies the name of the output file that contains the floating node information. It has a <code>.fnode</code> extension if you use <code>-type all</code> and a <code>.fgate</code> extension if you use <code>-type gate</code> .

Description

A floating node is a node that has no DC path to ground and touches at least one of the following:

- The gate of a MOSFET.
- A current source.
- The controlling input node of controlled source

Floating gates are the most common floating nodes.

Note:

You cannot use this command with the `set_floating_node` command. If you specify both the `set_floating_node` and `report_floating_node` commands, then the last one is used.

Examples

Specifies a report that includes all floating nodes, including floating bulks. The report is in the form of a table.

```
report_floating_node -type all -format report
```

Specifies a report that includes all floating nodes, except floating bulks. The report is in the form of a `.ic` statement.

```
report_floating_node -type all -format ic
```

report_model

Description

Reports detailed model information. This command lets you generate reports that contain detailed model information similar to what the Eldo format provides in the `.chi` file.

Syntax

```
report_model -report report_value [-generate enable_value]  
[-stop enable_value] [-count model|subckt|all|none]
```

Arguments

Argument	Description
<code>-report <i>report_value</i></code>	Specifies one of the following values: 1 to generate a condensed report. DEVICE, MODEL, TYPE, LEVEL, and INSTANCE information is provided, but the detailed list of model parameters is suppressed. 2 to generate a full report, including detailed model parameters. 3 to generate a full report with all devices (not only devices that have a <code>.model</code>) and all instances, even when they use identical models.
<code>-generate <i>enable_value</i></code>	Generates a netlist that contains <code>.model</code> cards and a single instantiation of each unique model instance. None of the original circuit netlist connectivity is in the file. This information is written to a file with a <code>.model</code> extension.
<code>-stop <i>enable_value</i></code>	Stops the simulation after front-end processing before starting DC or transient analysis. This argument can provide a quick analysis to help debug models.
<code>-count <i>model subckt all none</i></code>	Specifies one of the following options to control how to report the list of models. <code>model</code> to report the list of all models and their count in the <code>.modelinfo</code> file. <code>subckt</code> to report the list of all subcircuits and their count in the <code>.modelinfo</code> file. <code>all</code> to report the list of all models and subcircuits and their count in the <code>.modelinfo</code> file. <code>none</code> (default) to not report model and subcircuit information in the <code>.modelinfo</code> file.

report_mram_state

Monitors and reports initial state conditions and state transition information of any MRAM bit cells.

Syntax

```
report_mram_state -type 0|1|2
```

Arguments

Argument	Description
<code>-type 0 1 2</code>	Specifies one of the following values: • 0 to turn off the reporting (default). • 1 to report MRES state transitions to the output .statemres file. • 2 to report MRES state transitions to the output .statemres file, and print the state change message to the screen and the log file.

report_node_alias

Reports all the aliased nodes in the report files.

Syntax

```
report_node_alias -hierarchy enable_value | -short enable_value
```

Arguments

Argument	Description
<code>-hierarchy enable_value</code>	If set to 1, reports all hierarchy node aliases to a file. The default is 0, which does not report hierarchy node aliases.
<code>-short enable_value</code>	If set to 1, reports all hierarchy node aliases to a file. The default is 0, which does not report shorted node aliases.

Description

This command prints the nodes that have aliased node names. A node might have an alias because of:

- Hierarchical alias names from instances of subcircuits.
- Shorted alias names due to very small resistors or a DC voltage source of 0 volt connected to a node.

The primary node name is the node name that remains in the database. The alias node name is the name that is removed from database. The alias output file has two or more columns such as:

```
<primary_node_name> <alias_node_name_0> ... <alias_node_name_n>
```

Examples

Reports all hierarchy alias nodes in *output_file.nodealias*.

```
report_node_alias -hierarchy 1
```

Reports all shorted nodes in *output_file.nodealias*.

```
report_node_alias -short 1
```

Reports all hierarchy alias nodes and all shorted nodes in *output_file.nodealias*.

```
report_node_alias -hierarchy 1 -short 1
```

report_node_cap

Reports capacitance information for the specified nodes.

Syntax

```
report_node_cap -node node_name {node_name} [-short_resistor value]
[-group group_name] [-limit limit_value] [-report basic|detail]
[-time time_vale {time_value}]
```

Arguments

Argument	Description
-node node_name {node_name}	Reports capacitance for the node names you specify. You can use wildcard characters in the node names.
-short_resistor value	Specifies the value of the resistor to be shorted. The default value is 0, which means none of the resistors are shorted.
-group group_name	Creates a group name for the nodes you specify with -node. If you specify this option, all nodes for a report_node_cap command are grouped together. the PrimeSim XA tool reports the capacitance information based on this group. Use this option only for a flat, postlayout design.
-limit limit_value	Specifies the hierarchy level down to which the PrimeSim XA tool reports the capacitance information. The default is 3.

Argument	Description
<code>-report basic detail</code>	Specifies the type of report to print. Use the <code>basic</code> keyword (the default) to print only the basic capacitance information. Use the <code>detail</code> keyword to print a detailed report.
<code>-time time_vale {time_value}</code>	Reports the capacitance value at each specified <code>time_value</code> time. For example: <code>report_node_cap ... -time 1ns 50ns 650ns</code> When you do not specify <code>-time</code> the capacitance is reported at time 0.

Description

The reported node capacitance information includes:

- Total node capacitance
- Design and net (parasitic) capacitance
- Capacitance of MOSFET devices
- Capacitance of other connected devices, including diodes

`report_node_cap` outputs the capacitance information in a `*.cap#` file.

In a prelayout design, capacitance is reported as:

$C_{total} = C_{dn} +$

$C_{mos} + C_{other}$

$C_{dn} = C_{design}$

C_{mos}

C_{other}

In postlayout design, the PrimeSim XA tool expands a single node from the prelayout design into multiple nodes because of the RC parasitics. The postlayout flow is divided into 2 scenarios:

- A back-annotated postlayout.
- A flat postlayout.

In the back-annotated postlayout flow, you can trace the connectivity back to the prelayout design with the information from the prelayout netlist and the back-annotation file:

`*|NET na 0.00458507PF <-- Net Capacitance (CBAnet)`

```
*|I (x02/mp:GATE x02/mp GATE I 4.8e-16 22.75 3.25) // $llx=22.55
$lly=3.25 $urx=22.95 $ury=3.25 $lvl=5

*|I (x02/mn:GATE x02/mn GATE I 2.4e-16 22.75 1.05) // $llx=22.55 $lly=0.6
$urx=22.95 $ury=1.05 $lvl=4

*|I (x01/mp:DRN x01/mp DRN B 0 8.45 3.25) // $llx=8.45 $lly=2.65 $urx=9.2
$ury=3.85 $lvl=7

*|I (x01/mn:DRN x01/mn DRN B 0 8.45 1.05) // $llx=8.45 $lly=0.75 $urx=9.2
$ury=1.35 $lvl=6

*|S (na:1 22.75 2.65) // $llx=22.55 $lly=2.65 $urx=22.95 $ury=2.65 $lvl=3
*|S (na:2 22.75 3.925) // $llx=22.55 $lly=3.85 $urx=22.95 $ury=4 $lvl=5

Cg1 na:1 0 4.33011e-17
Cg2 na:2 0 3.99892e-17

...

R158 na:1 x02/mp:GATE 4.8 $l=0.6 $w=0.4 $lvl=5
R159 na:1 na:3 30 $l=0.8 $w=0.4 $lvl=3
R160 x02/mp:GATE na:2 5.016 $l=0.675 $w=0.4 $lvl=5
R161 na:3 na:4 16.875 $l=0.5 $w=0.4 $lvl=3
R162 na:3 na:5 18.75 $l=0.5 $w=0.4 $lvl=3
R163 na:3 na:8 3.96 $a=0.04 $lvl=10
R164 na:4 na:6 3.96 $a=0.04 $lvl=1

...

R186 na:18 x01/mn:DRN 5.38888 $a=0.09 $lvl=12
R187 na:19 na:20 0.992002 $l=0.8 $w=0.5 $lvl=2
R188 na:19 na:21 13.1234 $l=6.45 $w=0.3 $lvl=2
R189 na:20 na:24 0.744002 $l=0.6 $w=0.5 $lvl=2
R190 na:20 x01/mp:DRN 5.38888 $a=0.09 $lvl=12

...
```

In back-annotated post-layout flow, the capacitance information is reported as:

$C_{total} = C_{dn} + C_{mos} + C_{other}$

$$C_{dn} = C_{design} + C_{net}$$

$$C_{mos}$$

$$C_{other}$$

In the previous back-annotated file, the `report_node_cap -node na` command reports:

$$C_{total} = C_{dn} + C_{mos} + C_{other}$$

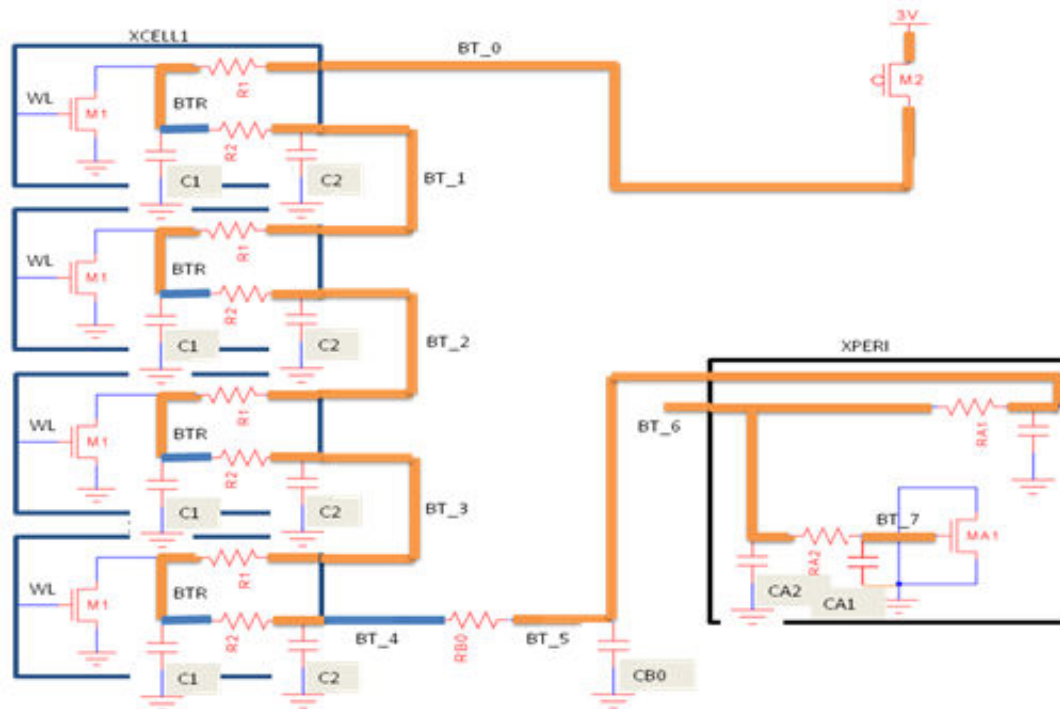
$$C_{dn} = C_{design} + C_{net} (0.00458507\text{pF})$$

$$C_{mos} = C_{gate}(x02/mp) + C_{gate}(x02/mn) + C_{junc}(x01/mp) + C_{junc}(x01/mn)$$

$$C_{other} = 0$$

In a flat postlayout flow, the PrimeSim XA tool treats all nodes as unique and independent. To accurately calculate the capacitance information, you need to group the nodes and then run `report_node_cap`. For example, see [Figure 3](#).

Figure 3 Flat Postlayout Flow Example



In [Figure 3](#), a prelayout node, BT, has been expanded into different nodes in the postlayout. Because the postlayout netlist is flat, and there is no trace of connectivity from the prelayout netlist and back-annotation flow, all nodes are treated as unique and independent.

To accurately report the capacitance information, you need to tell the PrimeSim XA tool which nodes can be grouped together for `report_node_cap` command to calculate the capacitance information. To group a list of nodes into one group, use `-group` argument and list the names of nodes to be grouped: `BT_0`, `BT_1`, `BT_2`, `BT_3`, `BT_4`, `BT_5`, `BT_6`, `XPERI.BT_7`, `XCELL1.BTR`, `XCELL2.BTR`, `XCELL3.BTR`, and `XCELL4.BTR` and do the following steps:

1. Specify the following `report_node_cap` command.

```
report_node_cap -node BT_* XPERI.BT_7 XCELL?.BTR -group BT
```

This command groups all the specified node names into one group named `BT`.

2. Assuming all parasitic capacitor has a value of 1 fF, the PrimeSim XA tool calculates the capacitance information as:

```
Ctotal=Cdn+Cmos
Cdn=12*1fF
Cmos=Cgate(xperi.ma1)+Cjunc(m2)+Cjunc(xcell1.m1)+Cjunc(xcell2.m1)+Cjunc(xcell3.m1)+Cjunc(xcell4.m1)+Cjunc(xperi.ma1)
```

Another way to get the correct capacitance report of a net in a flat postlayout netlist is to use the `-short_resistor` option to short all the parasitic resistors. Assuming all resistors of the `BT` net in the previous example are less than 10K ohm, the following command shorts the resistors and reports the same node capacitance.

```
report_node_cap -node BT_0 -short_resistor 1e4
```

report_operating_point

Description

Determines how the PrimeSim XA tool exports initial conditions it computes in DC analysis. It also defines how the PrimeSim XA tool reports the results of latch (and other circuit types) detection. The initial conditions are dumped in a file named `prefix.time.ic`. Note that you cannot specify multiple `report_operating_point` commands.

Syntax

```
report_operating_point -time dc|end|time_value {dc|end|time_value}
[-report all|core]
[-file file_name]
[-node_type node_type]
[-type ic_type]
[-subckt subckt {subckt}]
[-inst inst {inst}]
[-node_details switch_value]
[-trigger warn_converge]
```

Arguments

Argument	Description
<code>-time</code> <code>dc end time_value</code>	Specifies the time to dump the operating point: • <code>dc</code> for a time value of 0 (default). • <code>end</code> for the transient end time. • <code>time_value</code> for a user-defined time.
<code>-report all core</code>	Specify <code>core</code> (the default) to dump the subset of the core nodes that the PrimeSim XA tool needs to determine the operating point. Specify <code>all</code> to force all nodes to be written to a file.
<code>-file file_name</code>	Adds operating point information to the specified file. The default file name is <code>prefix.time.ic</code> .
<code>-node_type node_type</code>	Limits the application of initial conditions to nodes detected as <code>node_type</code> . You can specify a list of arguments with either or both of the following keywords to limit the reporting: • <code>latch</code> reports only the detected latches and the back-annotated nets. • <code>latch_uninit</code> reports the detected uninitialized latches as warnings in the PrimeSim XA log file. For example, Warning: FOUND 0 out of 100 total latch groups not initialized. See test/xa.fdi.latch_group logfile for more details. • <code>memcell</code> reports only detected memory cells. If you want to also report the optimized nodes, specify <code>-node_type memcell</code> with the <code>-report all</code> option.
<code>-type ic_type</code>	Specifies the type of initial condition to be applied. The <code>ic_type</code> can be <code>ic</code> or <code>nodeset</code> . This option overrides what is specified in the file.
<code>-subckt subckt</code> <code>{subckt}</code>	Limits the application of initial conditions to nodes of the instances of <code>subckt</code> . A list of arguments is supported. Wildcards are not supported.
<code>-inst inst {inst}</code>	Limits the application of initial conditions to the specified instances. Wildcards are supported.
<code>-node_details</code> <code>switch_value</code>	You can specify one of the following for <code>switch_value</code> : • 0 (default) does not list node details. • 1 lists node details in a separate file. The file name is created by appending <code>.op_table</code> to the name of the initial conditions file (for example, <code>xa.0.ic.op_table</code>).

Argument	Description
-trigger warn_converge	Dumps the operating point when the PrimeSim XA tool encounters a convergence problem during transient simulation and displays the message: Convergence failed for small step. You can use this option with the set_message_option command to limit the number of messages and define the action when the limit is reached. For example: <pre>report_operating_point -trigger warn_message set_message_option -limit 5 -pattern "Convergence Failed for small step" -action exit</pre> Dumps up to five operating points when the PrimeSim XA tool has convergence problems and exits the simulation after this limit has been reached.

Examples

Writes out a *prefix.0.ic* file containing initial conditions from time 0 as *nodeset* for latch nodes only.

```
report_operating_point 0 -node_type latch -type nodeset
```

Writes out a *10n-op* file containing initial conditions from time 10n as *ic* only for latch nodes and nodes in instances of the *latch_fast* subcircuit.

```
report_operating_point 10n -file 10n-op -subckt latch_fast -node_type
latch
```

Writes a file *<prefix>.0.ic* that contains initial conditions from time 0 as *nodeset* for latch nodes only.

```
report_operating_point 0 -node_type latch -type nodeset
```

Writes a file *<prefix>.0.ic* that contains initial conditions from time 0 as *nodeset* for memory cell nodes only.

```
report_operating_point 0 -node_type memcell -type nodeset
```

Produces a *file .0.ic* file that contains an operating point at time 0 with *.ic* for the core nodes and also generates a *.0.ic.op_table* file with node initialization details.

```
report_operating_point -time 0 -node_details 1
```

report_power

Generates power consumption reports.

Reporting by port name:

```
report_power -port port_name {port_name}
[-label label_name]
[-twindow tstart [{tstop}] {tstart [tstop]}]
[-limit level]
[-subckt subckt]
[-avg enable_value]
[-rms enable_value]
[-max enable_value]
[-min enable_value] [-probe enable_value]
[-except_port except_pattern]
```

Reporting by node name:

```
report_power -by_node node_name {node_name}
[-label label_name]
[-twindow tstart [{tstop}] {tstart [tstop]}]
[-limit level] [-subckt subckt] [-avg enable_value]
[-rms enable_value]
[-max enable_value]
[-min enable_value]
[-probe enable_value]
[-except_port except_pattern]
[-report basic|detail]
[-format csv]
```

Arguments

Argument	Description
-port port_name {port_name}	Specifies the currents to report based on port names. A preceding “*” or “.” character in the port name is optional.
-by_node node_name {node_name}	Specifies the currents to report based on connectivity. You can use wildcards in the node names.
-label label_name	Specifies the text label for the report file.
-twindow tstart tstop {tstart tstop}	Performs the check within the time window defined by tstart tstop {tstart tstop}. The tstart and tstop must come in pairs, except for the final window where if tstop is not specified it is be assumed to be the end of the simulation.
-limit level	Specifies the maximum hierarchical depth for the report. The default is 3.

Argument	Description
<code>-subckt subckt</code>	Specifies a subcircuit name for current reporting. Note that you cannot use this argument with the <code>-by_node</code> argument. Be aware that the search for the subcircuit instances is limited to three levels of hierarchy. If the subcircuit instances are in a lower hierarchical level, use the <code>-limit</code> option.
<code>-avg enable_value</code>	Specifies whether to report the average value. The default is 1 for printing the average value. See the Common Syntax Definitions section for details about the <code>enable_value</code> argument.
<code>-rms enable_value</code>	Specifies whether to report the rms value. The default is 1 for printing the rms value. See the Common Syntax Definitions section for details about the <code>enable_value</code> argument.
<code>-max enable_value</code>	Specifies whether to report the maximum value. The default is 1 for printing the maximum value. See the Common Syntax Definitions section for details about the <code>enable_value</code> argument.
<code>-min enable_value</code>	Specifies whether to report the minimum value. The default is 1 for printing the minimum value. See the Common Syntax Definitions section for details about the <code>enable_value</code> argument.
<code>-probe enable_value</code>	Enables or disables the probing of ports: 1, true, or on to enable; 0, false, or off to disable. The default is 0.
<code>-except_port except_pattern</code>	Excludes ports that match the <code>except_pattern</code> from the report. You can specify wildcards in the pattern name.
<code>-report basic detail</code>	If detail is set, the following information is printed: <pre>Sub-circuit Level: xx Percentage of top sub-circuit: Avg(%)= xx Rms(%)= xx Percentage of parent sub-circuit: Avg(%)= xx Rms(%)= xx</pre> The default is basic. This argument is only supported with <code>-by_node</code>.
<code>-format csv</code>	When you specify <code>-format csv</code>, the <code>.power</code> report is in CSV (Comma Separated Value) format only. The output file extension is <code>.power.csv</code>. The <code>-report detail</code> option is ignored when you use <code>-format csv</code>. Only basic format is reported.

Description

The `report_power` command produces a text report file that contains subcircuit port currents. There are two methods for specifying the reporting ports: by the port name or by connectivity. The report is named `file.power`. The `-limit` argument specifies the absolute hierarchical depth. The top level of the netlist hierarchy is 0.

To specify reporting by port name, use the `-port` argument. A subcircuit port name is defined by a subcircuit definition (`.subckt subname portname1 portname2 ...`). A leading `"*"` wildcard and `"."` hierarchical delimiter are optional. You can use the `-subckt` argument to scope specific subcircuits. You can also use the `-except_port` argument to exclude some ports. Any port names matching the exclude pattern are not reported.

To specify reporting by connectivity, use the `-by_node` argument. The ports/terminals of the instances connected to the named nodes have their power reported. The PrimeSim XA tool automatically creates a port/terminal list based on the connecting subcircuit and ideal voltage and current sources to the specified nodes. The list of ports and terminals traverses the hierarchy based on the `-limit` option. A power/current report is not generated for any primitive element other than ideal voltage and current sources. MOS, BJT, resistors, capacitors, and so on are excluded from the power/current report. Subcircuit instances that the PrimeSim XA tool detects as a MOS macro model are also excluded. You can use the `-except_port` argument to exclude some ports. Any port names matching the exclude pattern are not reported.

A warning is issued when any of the excluded elements do connect to the specified node. The warning message is consistent with other PrimeSim XA tool warnings. By default, only the first 10 warning are printed and can be controlled with the [set_message_option](#) command.

If `-by_node` is applied, the report is generated for those subcircuits/elements at that level of hierarchy, even if the hierarchical level is exceeded in the `-limit` value. Additional levels of hierarchy are only included if they meet the `-limit` value. If a wildcard is used in the `-by_node` pattern, the `-limit` setting limits the hierarchical depth of the wildcard match. Power is reported as if all matched nodes had been explicitly named. Thus the maximum port depth in this scenario is the limit value + 1.

If the `-by_node` is also the name of a port, its power is reported, as well as that of any connected subcircuit ports further down in the circuit hierarchy. It does not report any ports connected up in the hierarchy.

The `-except_port` argument excludes some ports from being reported. The proper way to conceptualize this behavior is to first determine which ports match the patterns specified by `-port` or `-by_node` and exclude any ports that match the pattern. For more details about excluding ports, see the following examples.

Examples

The following example shows sample report file content.

```
##### LABEL=x1_ports_0_1u FROM=0 TO=1e-06 #####

Port Name: x1.vdd (vdd)
Sub-circuit Definition: buf8
Max(A)= 4.3845644e-03 at = 2.0847267e-07 Min(A)= 4.8172598e-12 at
= 5.1890557e-09 Avg(A)= 1.2179682e-04 Rms(A)= 6.3463403e-04
Max(W)= 1.3153693e-02 at = 2.0847267e-07 Min(W)= 1.4451779e-11 at
= 5.1890557e-09 Avg(W)= 3.6539046e-04 Rms(W)= 1.9039021e-03

Port Name: x1.x_buf1.vdd (vdd)
Sub-circuit Definition: buf4
Max(A)= 4.3099853e-03 at = 3.0768178e-07 Min(A)= 2.3878731e-12 at
= 5.1890557e-09 Avg(A)= 5.9134919e-05 Rms(A)= 4.2502434e-04
Max(W)= 1.2929956e-02 at = 3.0768178e-07 Min(W)= 7.1636193e-12 at
= 5.1890557e-09 Avg(W)= 1.7740476e-04 Rms(W)= 1.2750730e-03
```

The `report_power` supports on-the-fly reporting. At the conclusion of a report window, as specified by the `-from` and arguments, the report is written to the output file.

```
.global vdd vss

.subckt level1 a
R1 vdd a r=1k
R2 a vss r=1k
X2 a level2
.ends

.subckt level2 a
R1 vdd a r=1k
R2 a vss r=1k
X3 a level3
.ends

.subckt level3 a
R1 vdd a r=1k
R2 a vss r=1k
X4 a level4
.ends

.subckt level4 a
R1 vdd a r=1k
R2 a vss r=1k
.ends

*Top level netlist start here, level=0
vvdd vdd 0 dc=3
vvss vss 0 dc=0
v0 a 0 dc=2
X1 a level1

.tran 1n 5n
.opt xa_cmd="set_sim_level 7"
```



```
*-limit is used to specify a hierarchical level:
.opt xa_cmd="report_power -port a -label atlevel0 -limit 0"
*The above does not report anything because 'a' is not a port,
*but is a net at top level (level 0).

.opt xa_cmd="report_power -port a -label atlevel1 -limit 1"
*The above reports power in X1.a, because a is a port for X1
*which is the 1st level of hierarchy (-limit 1)

.opt xa_cmd="report_power -port a -label sublevel2 -subckt level2"
*The above reports power in a port named "a" within subckt
*level2: X1.X2.a

.opt xa_cmd="report_power -port a -label porta"
*The above reports power thru all ports "a" in all subcircuits
*down to the hierarchical level 3 because if -limit is unspecified
*it defaults to 3:
*X1.A, X1.X2.A, X1.X2.X3.A; but not X1.X2.X3.X4.a

.opt xa_cmd="report_power -port vdd"
*The above report power thru all ports VDD in all subcircuits
*down to hierarchical level 3.

.end

.global vdd
vvdd vdd 0 3

X1 in out vdd 0 ckt1

.subckt ckt1 in out vdd vss
X1 in n1 vdd vss block1
X2 n1 n2 vdd vss block2
X3 n2 out block3
M1 vdd n1 n5 vss nmos
R1 n5 vss
.ends

.subckt block1 clkkin clkout vdd vss
X1 clkkin intl1 vdd vss inv
X2 intl1 clkout vdd vss inv
.ends

.subckt block2 input output vcc vss
X1 clkkin intl1 vcc vss inv
X2 intl1 clkout vcc vss inv
.ends

.subckt block3 clkkin clkout
X1 clkkin intl1 vdd vss inv
X2 intl1 clkout vdd vss inv
.ends
```

If you specify the following command:

```
report_power -by_node vdd -limit 1
```

For the previous netlist, `report_power` generates a power/current report of the following ports:

vvdd

x1.vdd

If you specify the following command:

```
report_power -by_node vdd -limit 2
```

This command generates a power/current report of the following ports:

vdd

x1.vdd

x1.x1.vdd

x1.x2.vcc

x1.x2.vdd

x1.x3.vdd

Note:

Because of the global `vdd` statement exists in all subcircuit instances, but since the `vdd` port of the `x1.x2` subcircuit is not connected, the report for this subcircuit is 0. Also, power/current report is generated for subcircuit and ideal voltage and current sources only. A report is not generated for other primitive elements or macro-models connected to the specified node, for example `x1.M1`.

If you specify the following command:

```
report_power -by_node vdd -limit 3
```

For the previous netlist, `report_power` generates a power/current report of the following ports:

vvdd

x1.vdd

x1.x1.vdd

x1.x1.x1.vdd

x1.x1.x2.vdd

```
x1.x2.vcc
x1.x2.vdd
x1.x2.x1.vcc
x1.x2.x1.vdd
x1.x2.x2.vcc
x1.x2.x2.vdd
x1.x3.vdd
x1.x3.x1.vdd
x1.x3.xd.vdd
```

Note:

A power/current report is not generated for non-subcircuit elements or macro-models, for example, X1.M1.

To further illustrate port reporting, here is another sample netlist:

```
X1 nodeA out ckt1
.subckt ckt1 in out
X2 nodeB out ckt2
...
.ends
.subckt ckt2 in out
X3 nodeC out VBIAS
...
.ends
.subckt VBIAS node1 VBIASOUT
X4 node1 VBIASOUT VREG
X5 VBIASOUT node2 LOAD
...
.ends
.subckt VREG IN OUT
...
.ends
.subckt LOAD IN OUT
...
.ends
```

If you specify the following command:

```
report_power -by_node X1.out -limit 3
```

For the previous netlist, `report_power` generates a power/current report of the following ports:

```
X1.out
```

X1.X2.out

X1.X2.X3.VBIASOUT

If you specify the following command:

```
report_power -by_node X1.X2.X3.VBIASOUT -limit 2
```

For the previous netlist, `report_power` generates a power/current report of the following ports:

X1.X2.X3.X4.OUT

X1.X2.X3.X5.IN

X1.X2.X3.VBIASOUT

Note:

Although VBIASOUT is at level 3 of the hierarchy and `-limit 2` is specified the reports are still generated because the node is explicitly named.

Here is another example netlist with port reporting examples:

```
.subckt bottom a b
R1 a 0 r=1k
R2 b 0 r=1k
.ends
```

```
.subckt top 1 2
Rtop1 1 z r=100
Rtop2 2 y r=100
Xbottom 1 y bottom
.ends
```

```
Xtop a b top
Xtop2 a b top
```

If you specify the following command:

```
report_power -by_node xstop1
```

For the previous netlist, `report_power` generates a power/current report as follows:

- Node `xstop.1` is a port so it is reported. This port connects the node up the hierarchy.
- For `Xtop.xbottom.a`, this port is connected to the node `xstop.1`, but is down the hierarchy. Note that if you did not use the `-by_node` argument this port is not reported.
- This example does not report `xstop2.1`.

```
.subckt superhuge3 vdd en
...
.ends
```

```
.subckt huge1 vdd en
...
.ends
.subckt huge2 vdd en
...
.ends
.subckt regulator vdd regvdd
...
.ends
.subckt tlc din dout vdd
Xreg vdd regvdd regulator
X1huge regvdd en1 huge1
X2huge regvdd en2 huge2
X3huge regvdd en3 superhuge3
.ends
Xchip din dout vdd tlc
Vvdd vdd 0 pw1 0 0 100n 3.3
```

If you specify the following command:

```
report_power -port vdd -except_port xchip.x3huge*
```

For the previous netlist, `report_power` first finds all the ports that match `vdd`:

Xchip.vdd

Xchip.xreg.vdd

Xchip.x1huge.vdd

Xchip.x2huge.vdd

Xchip.x3huge.vdd

Then `report_power` removes anything from the above that matches `xchip.x3huge.*`, leaving:

Xchip.vdd

Xchip.xreg.vdd

Xchip.x1huge.vdd

Xchip.x2huge.vdd

If you specify the following command:

```
report_power -by_node xchip.regvdd -except_port xchip.x3huge*
```

For the previous netlist, `report_power` first finds all the ports connected to `xchip.regvdd`:

Xchip.xreg.regvdd

Xchip.x1huge.vdd

Xchip.x2huge.vdd

Xchip.x3huge.vdd

Then `report_power` removes anything from the above that matches `xchip.x3huge.*`, leaving:

Xchip.xreg.regvdd

Xchip.x1huge.vdd

Xchip.x2huge.vdd

Here is another sample netlist example to illustrate the `-except_port` argument.

```
.subckt regulator vdd vdd3v3 vdd5v vaa vcc vee vss vss3v3 vss5v
...
.ends
.subckt huge vdd vdd3v3 vdd5v vaa vcc vee vss vss3v3 vss5v
...
.ends
.subckt tlc
Xreg vdd vdd3v3 vdd5v vaa vcc vee vss vss3v3 vss5v regulator
X1 vdd vdd3v3 vdd5v vaa vcc vee vss vss3v3 vss5v hugeX3 vdd vdd3v3 vdd5v
  vaa vcc vee vss vss3v3 vss5v huge
.ends
Xchip tlc
```

If you specify the following command:

```
report_power -port v* -except_port *vee *vss* xchip.x3*
```

For the previous netlist, `report_power` first finds all the ports that match `v*`:

Xchip.xreg.vdd

Xchip.Xreg.vdd3v3

Xchip.Xreg.vdd5v

Xchip.Xreg.vaa

Xchip.Xreg.vcc

Xchip.Xreg.vee

Xchip.Xreg.vss

Xchip.Xreg.vss3v3

Xchip.Xreg.vss5v

Xchip.x1.vdd

chip.X1.vdd3v3
Xchip.X1.vdd5v
Xchip.X1.vaa
Xchip.X1.vcc
Xchip.X1.vee
Xchip.X1.vss
Xchip.X1.vss3v3
Xchip.X1.vss5v
Xchip.x3.vdd
Xchip.X3.vdd3v3
Xchip.X3.vdd5v
Xchip.X3.vaa
Xchip.X3.vcc
chip.X3.vee
chip.X3.vss
Xchip.X3.vss3v3
Xchip.X3.vss5v

Then `report_power` excludes ports that match `*vee *vss* xchipx3*`, leaving:

Xchip.xreg.vdd
Xchip.Xreg.vdd3v3Xchip.Xreg.vdd5v
Xchip.Xreg.vaa
chip.Xreg.vcc
chip.x1.vdd
Xchip.X1.vdd3v3
Xchip.X1.vdd5v
Xchip.X1.vaa
Xchip.X1.vcc

```
.subckt fee r
rr r 0 r=1k
.ends

.subckt bar p q
r1 p 0 r=10
iq q 0 dc=0.1m
fee p fee
.ends

.subckt foo a b c
ra a a1 r=10
a a1 0 dc=1u
b b b1 r=10
ib b1 0 dc=2u
rc c c1 r=10
ic c1 0 dc=0.5u
xbar a b bar
xbar2 a1 b bar
.ends

.subckt top 1 2 3
X1 1 1 1 foo
x2 2 3 3 foo
x3 3 3 3 foo
.ends

xcut 1 2 3 top

v1 1 0 dc=1
v2 2 0 dc=2
v3 3 0 dc=3

.opt xa_cmd="report_power -by_node xcut.x1.* -label wilddcardlimit -limit
1"
* This reports nothing because all matched patterns exceed limit value

.opt xa_cmd="report_power -by_node xcut.x1.a* -label wilddcard_a*_limit
-limit 1"
* This reports nothing because all matched patterns exceed limit value

.opt xa_cmd="report_power -by_node xcut.x1.a* -label wilddcard_a*_limit2
-limit 2"
* xcut.x1.a
* xcut.x1.ia (power_element)
* xcut.x1.xbar.p
* xcut.x1.xbar2.p

.opt xa_cmd="report_power -by_node xcut.x1.a -label defined_a_limit2
-limit 2"
.opt xa_cmd="report_power -by_node xcut.x1.a1 -label defined_a1_limit2
-limit 2"
```



```
* These two command together should produce the same output as the
previous

.opt xa_cmd="report_power -by_node xcut.1 -label defined_1_limit2 -limit
2"
.opt xa_cmd="report_power -by_node xcut.1 -label defined_1_limit1 -limit
1"
* because a named node always report the power of a port connected to it,
the above
* produces the same result.

.opt xa_cmd="report_power a -label port_a_limit2 -limit 2"
* note that when specifying by port, limit is a hard limit to the port
hierarchical level
* but with -by_node you can get 1 level deeper.
```

See Also

- [set_wildcard_rule](#)

report_sim_activity

Identifies performance problems in a simulation.

Syntax

```
report_sim_activity -type report_type [-node node_name {node_name}]
[-except_node node_name] [-num max_nodes] [-twindow tstart [{tstop}]
{tstart [tstop]}] [-file file_name] [-flush interval[%]]
[-subckt subcircuit_name] [-except_subckt node_name]
[-inst subcircuit_name] [-except_inst node_name] [-probe_format format]
[-profile 0|1] [-limit limit_value] [-max_conn value]
```

Arguments

Argument	Description
<code>-type report_type</code>	Specifies one of the following report types: • <code>node</code> reports the most highly active nodes. This option is the default if you do not specify <code>-type</code>. • <code>partition</code> reports only the most active partition information, which also includes the most active nodes in the partition. • <code>all</code> reports both the most highly active nodes and the most active partition information.
<code>-node node_name</code> <code>{node_name}</code>	Defines the signal node name, which can be the node name of a single node or a node name with the asterisk (*) wildcard character that represents a group of node names. The behavior of asterisk (*) character is controlled by set_wildcard_rule .

Argument	Description
<code>-except_node node_name</code>	Specifies the nodes to be excluded from reporting. You can use a wildcard character in the node name.
<code>-num max_nodes</code>	Specifies the maximum number of nodes to report. The default is 10.
<code>-twindow tstart tstop</code> <code>{tstart tstop}</code>	Performs the check within the time window defined by <code>tstart tstop {tstart tstop}</code> . The <code>tstart</code> and <code>tstop</code> must come in pairs, except for the final window where if <code>tstop</code> is not specified it is assumed to be the end of the simulation. If you specify more than one window in <code>-twindow</code> , the windows must be in ascending order.
<code>-file file_name</code>	Specifies the name of the file that contains the activity information. The default file name is <code>prefix.cpa_time</code> .
<code>-flush interval [%]</code>	Specifies when the information is flushed to the activity report. If the value is followed by the percent sign (%), the information is flushed every percentage of the transient run time. If the value is not followed by the percent sign, then the information is flushed every "wall time" period. This period is specified in hours. Decimal numbers are allowed.
<code>-subckt subcircuit_name</code>	The activity report applies only to the specified subcircuit. You can specify multiple subcircuit names and use wildcard characters in a subcircuit name.
<code>-except_subckt</code> <code>subcircuit_name</code>	The activity report does not apply to the specified subcircuit. Note that you cannot use wildcard characters in an subcircuit name in this case.
<code>-inst inst_name</code>	The activity report applies only to the specified instance. You can specify multiple instance names and use wildcard characters in an instance name.
<code>-except_inst inst_name</code>	The activity report does not apply to the specified instance. You can use wildcard characters in an instance name.
<code>-probe_format format</code>	Specify one of the following options to dump probe statements to the activity report. An <code>output_filename.probe</code> file is generated with a list of probes. - hspice format is <code>.probe tran v()</code>. - xa format is <code>probe_waveworm_voltage ...</code>.
<code>-profile 0 1</code>	When set to 1, an instance-based cost estimation is dumped to a <code>.cpa</code> file. The default is 0. This option helps you identify CPU-consuming blocks. You can speed up a simulation by replacing these blocks with models.

Argument	Description
<code>-limit limit_value</code>	Limits the number of instances reported by <code>-profile</code> . The default value is 10.
<code>-max_conn value</code>	Specifies the number of nodes to report with maximum connections in the most expensive partition. The default is 10.

Description

If you specify both the `-twindow` and `-flush` arguments, the activity information is written only within the time window. If you use Ctrl+C to stop a simulation, the activity information is written in a separate file (with a `.cpa` extension).

If you specify multiple time windows and use Ctrl+C, the reported activity is from the beginning of the last time window until the time use Ctrl+C. For example, if you specify `-twindow 10u 20u 50u 60u`:

- Ctrl+C at 15u (in the time window) writes activity information from 10u to 15u to the `.cpa` file.
- Ctrl+C at 30u (out of the time window) writes activity information from 10u to 30u to the `.cpa` file.
- Ctrl+C at 70u (out of the time window) writes activity information from 50u to 70u is written to the `.cpa` file.

Note:

Because this feature is based on statistical sampling, some randomness in the results is to be expected.

Examples

Analyzes performance in the 1ms to 1.5ms and 3ms to 5ms time windows and stores results in the `MyAnalysis.cpa_1.5m` and `MyAnalysis.cpa_5m` files.

```
report_sim_activity -type all -twindow 1e-3 1.5e-3 3e-3 5e-3 \
  -file MyAnalysis
```

Enables the performance analyzer and flushes results every 6 minutes (1/10th of an hour).

```
report_sim_activity -type all -flush 0.1
```

Stores performance analysis results in two files, `MyRpt.cpa_40m` and `MyRpt.cpa_110m`. They contain the activity report for the `ldo_iso` subcircuit only for the 20m to 40m and 100m to 110m time windows.

```
report_sim_activity -type node -subckt ldo_iso -file MyRpt \  
-twindow 20m 40m 100m 110m
```

set_active_net_flow

Triggers the automated active net flow.

Syntax

```
set_active_net_flow [-switch] switch_value [-vtol numeric_value]  
[-twindow tstart tstop {tstart tstop}] [-reuse_active_net enable_value]  
[-reuse_ic enable_value] [-setup_cmd cmd_file]
```

Arguments

Argument	Description
-switch switch_value	Runs the active net flow. The default is 1.
-vtol numeric_value	Specifies the tolerance voltage difference. A node is considered active if the node voltage varies larger than the specified value. The default value is 100mV.
-twindow tstart tstop {tstart tstop}	Specifies the time windows to be checked for active nodes. If you specify this argument, the check is limited to the specified time windows. The default for tstart is 0 and tstop is the end of transient simulation time. The time window has to be specified in a pair, except for the last entry.
-reuse_active_net enable_value	Instructs the PrimeSim XA tool to reuse the active net file information and avoid rerunning the first pass for a data sweep. The default is 1, which specifies to reuse the active net file. Set this option to 0 to rerun the entire flow and regenerate the active net file. This reuse of the active net file is also the default behavior for .alter/bisection optimization flow.
-reuse_ic enable_value	By default the PrimeSim XA tool reuses the prelayout .nodeset. If you enable this option, the PrimeSim XA tool uses .ic instead of .nodeset.
-setup_cmd cmd_file	By default, the PrimeSim XA tool uses the same command for both the first and second run. If you specify this option, you can change three commands in the setup run to further speed-up the setup simulation. The three commands are: set_sim_level , set_model_level , and set_synchronization_level .

Description

The active net flow automatically runs a prelayout simulation (ignores the `load_ba_file` command) and generates the active net information to use in the postlayout simulation. Its usage is limited to the back-annotation flow.

Examples

Invokes the active net flow. A net is active when its voltage variation is greater than 100mV.

```
set_active_net_flow 1 -vtol 100m
```

Invokes the active net flow. A net is active when its voltage variation is greater than 100mV within the [1us, 2us] window.

```
set_active_net_flow 1 -vtol 100mv -twindow 1u 2u
```

Invokes the active net flow and reruns the entire flow to regenerate the active nets. A net is active when its voltage variation is greater than 100mV within the [1us, 2us] window or [4us,6us] window.

```
set_active_net_flow 1 -vtol 0.1 -twindow 1u 2u 4e-6s 6e-6s \
-reuse_active_net 0
```

set_analysis_core

Assesses the PrimeSim XA Analysis Module (AM) processing requirements and can allocate up to two additional cores.

Syntax

```
set_analysis_core -core [0|1|2]
```

Arguments

Argument	Description
-core 0 1 2	Sets one of the following AM multicore options: 0 disables multicore processing so the AM uses the same number of cores as the simulation engine. 1 detects if an additional thread is needed for analysis processing based on the processing load. When there are many active analyses, the PrimeSim XA tool might enable up to an additional core to speed up processing. 2 detects if additional threads are needed for analysis processing based on the processing load. When there are many active analyses, the PrimeSim XA tool might enable up to two additional cores to speed up processing. This is the default when the <code>set_multi_core</code> command or <code>-mt</code> command line option is not specified.

Description

The AM performs many different types of analysis, such as signal probing, signal measuring, CircuitCheck, and so on. The AM detects the amount of processing needed and might enable additional cores to speed up processing according to the `-core` value you specify.

Note:

The `set_analysis_core` command is not compatible when multicore simulation is enabled with the `set_multi_core` command or `-mt` command line option. If you specify both of these commands, the PrimeSim XA tool issues a warning message.

set_analysis_post

Lets you control postprocessing for the specified timing check, CCK, vector check and `.measure` analysis commands.

Syntax

```
set_analysis_post [-cmd cmd_name {cmd_name}] [-check 0|1]
```

or

```
set_analysis_post -waveform file_name [-cmd cmd_name {cmd_name}]
```

Arguments

Argument	Description
<code>-cmd cmd_name {cmd_name}</code>	Specifies the analysis commands to post-process. You can set <code>cmd_name</code> to: • <code>meas</code> to enable post-processing with the <code>.measure</code> commands that are specified in the <code>config.tcl</code> file • <code>cck</code> to enable postprocessing with the CCK commands that are specified in the <code>config.tcl</code> file. The supported CCK commands are: <code>cck_soa</code> and <code>cck_signal</code>. • <code>tcheck</code> to enable post-processing with the time checking commands that are specified in the <code>config.tcl</code> file. The supported commands are: <code>check_timing_setup</code>, <code>check_timing_hold</code>, <code>check_node_excess_rf</code>, <code>check_timing_pulse_width</code>, and <code>check_node_quick_rf</code>. • <code>vchk</code> to check the expected output signal values that are specified in a vector file. See the Vector Checks section below for more information. • <code>all</code> to enable postprocessing and run all supported analysis commands.

Argument	Description
<code>-check 0 1</code>	0 (default) runs simulation and generates waveforms without executing the specified analysis commands. 1 runs simulation and generates waveforms. All the specified analysis commands will be executed. Note that these commands are executed only when the <code>-check</code> argument is set to 1.
<code>-waveform file_name</code>	Specifies the input waveform file for postprocessing. Do not use this argument with <code>-check</code> .

Description

The `set_analysis_post` command can generate the signal waveforms required for postprocessing the first simulation run. It can also enable postprocessing with the input waveform file in the second simulation run. Note that postprocessing only supports the timing check, CCK, vector check, and `.measure` analysis commands.

In the first syntax form, the tool collects the signal data related to the simulation results for specified commands. These commands do not run if you specify the `-check 0` argument.

In the second syntactic form the tool runs the specified commands in postprocessing based on the input waveform file.

Note:

- You cannot specify multiple `set_analysis_post` commands in a simulation run.
- If you specify the `-wavefile` command line option, the `set_analysis_post` command will be disabled.

Vector Checks

You can modify the output signal values in the vector file and run postprocessing to check for the impact of the vector value changes. The checking results are stored in an `.err` file.

To postprocess vector check results, include the `set_analysis_post` command in the command file and run the following:

```
% xa input.sp -o out -c config.tcl
```

where `input.sp` is the input netlist file in which a vector file is specified for use in vector check postprocessing. The `config.tcl` command file contains the `set_analysis_post` command for postprocessing.

The PrimeSim XA tool performs vector check postprocessing by using the waveforms stored in the `.fsdb` waveform file and the expected output signal values that are specified in the vector file. You can either specify the vector file in the `input.sp` file or include the

`load_vector_file` command in the `config.tcl` command file. For example, use the following statement in the `input.sp` file to specify the vector file (`check.vec`) for vector check postprocessing:

```
.vec check.vec
```

To specify the vector file with the `load_vector_file` command, include the following command in the `config.tcl` command file:

```
load_vector_file -file check.vec -format vec
```

When the vector check postprocessing run is complete, you can check the vector check results in the `out.err` file.

Examples

This section includes examples for running the `set_analysis_post` command.

Example 1

The following three commands are equivalent. The tool collects the signals waveforms of all the supported analysis commands in the simulation but does not run the analysis commands.

```
set_analysis_post  
set_analysis_post -cmd all  
set_analysis_post -cmd all -check 0
```

Example 2

The following two commands are equivalent. The tool collects the signal waveforms of all the supported analysis commands and runs them in a simulation.

```
set_analysis_post -check 1  
set_analysis_post -cmd all -check 1
```

Example 3

In the following example, the tool collects signal waveforms related to the `.measure` statements in the netlist but does not execute them.

```
set_analysis_post -cmd measure
```

Example 4

In the following example, the tool collects signal waveforms related to the `.measure` statements in the netlist, as well as the `cck_soa` and `cck_signal` commands, and executes these commands.

```
set_analysis_post -cmd measure cck -check 1
```

Example 5

The following two commands are equivalent. The tool does postprocessing for all the supported analysis commands based on the `xa.fsdb` file.

```
set_analysis_post -waveform xa.fsdb  
set_analysis_post -waveform xa.fsdb -cmd all
```

Example 6

The following example uses the `set_analysis_post` command in two simulations.

1. Add the `set_analysis_post -cmd all` command to the command file and run the first simulation. The tool generates the waveform file (`xa.fsdb`) which contains signals related to all supported analysis commands in the case.

The output `xa.fsdb` waveform file contains signals related to all the supported analysis commands.

2. Change the command to `set_analysis_post -cmd all -waveform xa.fsdb` and run the second simulation.

The PrimeSim XA tool does postprocessing with the `xa.fsdb` file that is generated in the first run.

Example 7

In the following example, the first simulation run only collects signal waveforms related to the `.measure` statement for postprocessing, but does not generate an output file. The second simulation executes only the `.measure` statements and outputs the results.

```
set_analysis_post -cmd measure  
set_analysis_post -cmd measure -waveform xa.fsdb
```

Example 8

The following example enables vector check during postprocessing. When the `-cmd vchk` option is specified, the tool writes waveforms to the `.fsdb` file that is generated from the simulation run by using the same circuit netlist as in postprocessing. This input netlist is required to build necessary data for postprocessing vector check results, though no simulation is performed in the postprocessing run.

```
set_analysis_post -cmd vchk -waveform file_name.fsdb
```

set_array_option

Provides user control over the effects of the PrimeSim XA SPICE optimization for array (SOFA) technology.

Syntax

```
set_array_option -array_detection enable_value  
[-cell_gndcap_tol tol_value]  
[-cell_subckt cell_subckt_name[.controlling_port]]  
[-flash_array enable_value] [-rc_optimize enable_value]  
[-keep_cell_ccap enable_value] [-merge_supply_nodes 0|1]
```

Arguments

Argument	Description
-array_detection enable_value	Determines whether the PrimeSim XA tool applies array optimization. Specify 0 to turn array optimization off. The default is 1.
-cell_gndcap_tol tol_value	Sets the relative tolerance for equalizing capacitor values for isolated ground capacitors inside memory cells. Values of ground caps are modified by not more than tol_value (abs((c-c_modified)/c)<=tol_value) to make more in-cell capacitors equal to each other across the different cells forming an array. This helps array optimization for some post-layout memory arrays. The valid range is 0% to 100%. The default is 15%.
-cell_subckt cell_subckt_name .controlling_port	Lets you manually provide the memory cell subcircuit name with optional controlling ports. the PrimeSim XA tool automatically detects the controlling port of the memory cell, but if the PrimeSim XA tool failed to do that, you can add one or more controlling port names right after the cell_subckt_name with the "." concatenation. Controlling ports generally represent word line ports of the memory subcircuit.
-flash_array enable_value	If set to 1, this option enables detection of arrays of flash cells of single MOSFET or floating gate. The default is 0.
-rc_optimize enable_value	Set this option to 1 (default) to apply post-layout array optimization (SOLE4SOFA). Set it to 0 to disable post-layout array optimization.
-keep_cell_ccap enable_value	if you enable this argument, for coupling capacitors between different cells, or between cell internal nodes and bitlines or wordlines, or between cell internal nodes and power nets, the global coupling capacitor optimization algorithm is applied, which means no extra aggressive lumping is applied. Cell coupling capacitors are treated the same way as other general coupling capacitors are treated. If you do not enable this argument (default), cell coupling capacitors are always converted to ground capacitors by default as long as their value is less than 1e-10F. In this case, extra aggressive lumping is applied.

Argument	Description
<code>-merge_supply_nodes 0 1</code>	If set to 1 (default) merges power supply nodes directly connected to memory cells that form an array. This optimization of circuit topology improves simulation performance for memory arrays with large resistive networks in their power supplies. Do not use this optimization if the simulation needs to analyze the detailed behavior of the power supply nodes connected to memory arrays.

Description

The SOFA algorithm transforms a memory array cell to an optimized array model to minimize their solve times while maintaining accuracy from the current [set_sim_level](#) settings. `set_array_option` can be useful for debugging simulations, or in other special circumstances when it is desirable to have the ability to disable the array optimization.

set_ba_active_file

Description

Creates an active nets file to run the selective extraction and back-annotation flow.

Syntax

```
set_ba_active_file [-file] file_name [-node node_name {node_name}]  
[-vtol numeric_value] [-twindow {tstart tstop} tstart [tstop]]
```

Arguments

Argument	Description
<code>-file file_name</code>	Specifies the active nets file name.
<code>-node node_name {node_name}</code>	Specifies the nodes that are identified as active.
<code>-vtol numeric_value</code>	Specifies the voltage tolerance voltage difference, which identifies a node as active. The default is 100mV.
<code>-twindow {tstart tstop} tstart tstop</code>	Specifies time windows to be checked for active nodes and performs the check within the time window defined by: <code>tstart tstop {tstart tstop}</code> . The <code>tstart</code> and <code>tstop</code> must come in pairs, except for the final window where if <code>tstop</code> is not specified it is assumed to be the end of the simulation.

Examples

The following example checks for all nodes that have a voltage difference more than 100mV and lists those nodes in an `activenet.rcxt` file.

```
set_ba_active_file -file activenet
```

The following example checks for all nodes under the `x1` instance that have a voltage difference more than 10mV during the time window of 10ns and 100ns. All those nodes are listed in a `test1.rcxt` file.

```
set_ba_active_file -file test1 -node x1.* -vtol 10m -twindow 10n 100n
```

The following example checks for all top level nodes with names starting with `a` or `b` during two different time windows of 10n and 15ns and 40ns and the end of simulation. If the voltage difference of the node is larger than 100mV, it is listed in a `file1.rcxt` file.

```
set_ba_active_file -file file1 -node a* b* -vtol 100m -twindow 10n 15n 40n
```

After the active nets file is created, you can use this file in the StarRC™ tool to extract a selective SPF/SPEF file. With this active nets file, StarRC extraction performance is also improved, and the SPF/SPEF file size can be reduced.

Finally, use the new SPF/SPEF file to perform the normal back-annotation simulation.

set_ba_option

Adjusts how the PrimeSim XA tool handles back-annotation.

Syntax

```
set_ba_option [-short_pins switch_value] [-lump_c_only switch_value]
[-dpf_scale scale_value] [-dpf_elem_type type {type}]
[-finger_prefix prefix_string]
[-active_net_file file_name {file_name}]
[-report_large_net value]
[-enable_error_net setting_value]
[-keep_prelayout_cap switch_value]
[-bus_delimiter left_char right_char]
[-swap_port port1 port2 sub_name]
[-min_res value]
[-max_res value]
[-min_cap value]
[-report_trim_rlc enable_value]
[-rcnet net_name {net_name}]
[-rcgnet net_name {net_name}]
[-rnet_powernet mode_value]
[-cnet_powernet mode_value]
[-ccnet_powernet mode_value]
[-rnet_powernet mode_value]
```

```
[-cnet net_name {net_name}]
[-ccnet net_name {net_name}]
[-skipnet net_name {net_name}]
[-dpf switch_value]
[-skip_powernet mode_value]
[-skip_signalnet switch_value]
[-spftlv enable_value]
[-position file file]
[-keep_prelayout_model switch_value]
[-auto_map_ba_terminal [0|1]]
[-cc_scale scale_value]
[-ccap_to_gcap value]
[-select_ipin_method method_value]
[-select_ipin_gate [0|1]]
[-layout_only_device_models model_name {model_name}]
[-ba_net_success_ratio percentage_vale]
[-check_temperature 0|1]
[-multi_subckt_suffix "suffix_name"]
[-subckt_mode 0|1|2]
[-skip_vsrc 1|0] [-keep_layout_only_net 0|1]
```

Arguments

Argument	Description
-short_pins switch_value	Sets one of the following values: 0 off no false to disable shorting multiple pins together from the DSPF/SPEF file. 1 on yes true to short multiple pins together (the default).
-dpf_scale scale_value	Applies a <i>scale_value</i> to the DPF section of the DSPF file. This is a global setting and applies to all DSPF files that are used for a given simulation. The default value for <i>scale_value</i> is 1, meaning no scale applies to the DPF section and the PrimeSim XA tool assumes the actual proper dimension is used in the DPF section. This argument takes both a unit or numerical value such as 1u or 1e-6.
-lump_c_only switch_value	When set to 1, reads in the net capacitance value from the NET line and only back-annotate the lumped capacitance of that net and ignore the distributed RC back-annotation. The default is 0.
-dpf_elem_type type {type}	Lets you specify the types of elements you want to back-annotate from a DPF file. Table 8 shows the supported element types. Note that -dpf_elem_type mos is equivalent to the -dpf_mos_only 1 option in the previous PrimeSim XA release.

Argument	Description
<code>-finger_prefix prefix_string</code>	Instructs the PrimeSim XA tool to use the <i>prefix_string</i> value to identify finger devices. The default string is the @ character.
<code>-active_net_file file_name</code> <code>{file_name}</code>	Instructs the PrimeSim XA tool to back-annotate the full RC for the active nets from the SPF/SPEF file. For inactive nets, the lump C value is added. You must use this argument with the load_ba_file command.
<code>-report_large_net value</code>	Reports large SPF nets as warnings in the PrimeSim XA log file. By default, any net with an RC element count greater than 1000 is reported in the PrimeSim XA log file.
<code>-enable_error_net setting_value</code>	Specifies one of the following three options: <code>0 off no false</code> ignores back-annotation of the net (the default) if the error is critical. <code>1 on yes true</code> ignores elements connected to the incorrect terminal only. This argument tells the PrimeSim XA tool to ignore the back-annotation of an instance (*I) with an incorrect terminal, and any element connected to it. All other elements are back-annotated to the net. <code>lump</code> back-annotates lumped capacitance to the net. This argument tells the PrimeSim XA tool to add the total lumped capacitance specified in the back-annotation file to the net. If source (SRC) is misconnected to drain (DRN), or vice-versa, the PrimeSim XA tool automatically swaps the connection of those two terminals. If any other misconnection exists, the simulator does not back-annotate the net unless the <code>-enable_error_net</code> option is used. You can use this option only once. Otherwise the last entry overrides all previous entries.

Argument	Description
<code>-keep_prelayout_cap switch_value</code>	Determines how the PrimeSim XA tool deals with a prelayout netlist that contains ground capacitors with the same node name as the SPF net in the DSPF file. Specify one of the following values: 0 overwrites the prelayout net capacitance with what is defined in the DSPF netlist. 1 (default) keeps all the ground capacitors in the prelayout netlist and add the remaining capacitors in the back-annotation file. This option applies only to ground capacitors. Coupling capacitors are not considered. Also, the PrimeSim XA tool issues a warning message when the prelayout net capacitance is overwritten by the DSPF netlist.
<code>-bus_delimiter left_char right_char</code>	Modifies the bus character only inside the SPEF file. You can specify the <i>left_char</i> and <i>right_char</i> to match the prelayout node names. The supported bus delimiters are [] and <>.
<code>-swap_port port1 port2 sub_name</code>	Allows port swapping for subcircuit (macro model) terminals.
<code>-min_res value</code>	Specifies the global lower threshold of the resistors from the back-annotated DSPF/SPEF to be kept. All resistors with an absolute value below the specified value are shorted.
<code>-max_res value</code>	Specifies the global upper threshold of the resistors from the back-annotated DSPF/SPEF to be kept. All resistors with an absolute value above the specified value are treated as open-circuit.
<code>-min_cap value</code>	Specifies the global lower threshold of the capacitors from back-annotated DSPF/SPEF to be kept. All capacitors with an absolute value below the specified value are treated as open-circuit.
<code>-report_trim_rlc enable_value</code>	Generates an <code>.rlcignore</code> file with the list of resistor, capacitor, and inductor elements that have been optimized. The default is 0.
<code>-rcnet net_name {net_name}</code>	Specifies the nets to use for full RC back-annotation. All other nets not specified by this argument only use lumped capacitance back-annotation. The lumped capacitance value uses the net capacitance value in the * NET line. This argument works in both SPF and SPEF.

Argument	Description
<code>-rcgnet net_name {net_name}</code>	Specifies for nets to use R(resistor)+C _g (lumped capacitor) back-annotation. C _c (coupling capacitor) is always based on both NETs status. It might be skipped, lumped on one side, or lumped on both sides for C _g . All nets not specified by this argument use full RC back-annotation (assuming the net itself contains RCs). This argument works in both SPF and SPEF formats and accepts wildcard characters.
<code>-rnet_powernet mode_value</code>	Specify one of the following options. <code>external</code> to enable the power net back-annotation for resistors into external power nets (or connected to VSRC). <code>internal</code> to enable the power net back-annotation for resistors into internal power nets. <code>all</code> (default) to enable the power net back-annotation for resistors into all power nets.
<code>-cnet_powernet mode_value</code>	Specify one of the following options. <code>external</code> to enable the power net back-annotation for lumped capacitors into external power nets (or connected to VSRC). <code>internal</code> to enable the power net back-annotation for lumped capacitors into internal power nets. <code>all</code> (default) to enable the power net back-annotation for lumped capacitors into all power nets.
<code>-ccnet_powernet mode_value</code>	Specify one of the following options. <code>external</code> to enable the power net back-annotation for C_c (coupling capacitors) into external power nets (or connected to VSRC). <code>internal</code> to enable the power net back-annotation for C_c (coupling capacitors) into internal power nets. <code>all</code> (default) to enable the power net back-annotation for C_c (coupling capacitors) into internal power nets.
<code>-cnet net_name {net_name}</code>	Specifies for the nets to use lumped capacitance back-annotation. The lumped capacitance value uses the net capacitance value in the * NET line. All the nets not specified by this argument use full RC back-annotation (assuming the net itself contains RCs). This argument works in both SPF and SPEF formats and accepts wildcard characters.

Argument	Description
<code>-ccnet net_name {net_name}</code>	Specifies for the nets to use Cg+Cc back-annotation (removes all resistors from a given net). All the nets not specified by this argument use full RC back-annotation (assuming the net itself contains RCs). This argument works in both SPF and SPEF formats and accepts wildcard characters.
<code>-skipnet net_name {net_name}</code>	Specifies the name of nets that should not be back-annotated. You can specify multiple net names. You can use wildcard characters in the net names.
<code>-dpf switch_value</code>	Specifies to back-annotate the DPF section. The default <i>switch_value</i> is 1.
<code>-skip_powernet mode_value</code>	Specify one of the following options for <i>mode_value</i> : <code>external</code> to disable back-annotation into external power nets (ground or connected to VSRC). <code>internal</code> to disable back-annotation into internal power nets. <code>all</code> to disable back-annotation into all power nets. The default is not to skip any power nets.
<code>-skip_signalnet switch_value</code>	Specifies signal nets to be skipped for back-annotation.
<code>-spftlv enable_value</code>	Enables the top-level view of the SPF file for the EMIR analysis flow. The default is 0.
<code>-position_file file</code>	Reads in a position file when multiple SPF files are read in for RA.
<code>-keep_prelayout_model switch_value</code>	Sets which model the PrimeSim XA tool uses, the model in the prelayout netlist or in SPF file (default). Usually, different models are used for prelayout netlist and post-layout netlist. When running simulation with a prelayout netlist and back- annotation with SPF extracted from layout, the value for this option can decide which model should be used.
<code>-auto_map_ba_terminal [0 1]</code>	Enables or disables automatic mapping for macro models terminal names used in the prelayout netlist to the pin names in the post-layout netlist. The default is 1. For more information about this options, see the Mapping Terminals section.

Argument	Description
<code>-cc_scale <i>scale_value</i></code>	Applies a <i>scale_value</i> to the parasitic coupling capacitors in the DSPF files. This is a global setting and applies to all DSPF files that are used for a given simulation. The default value is 1, meaning no scale applies to the parasitic coupling capacitors.
<code>-ccap_to_gcap <i>value</i></code>	Specifies the global capacitance value of the coupling capacitors to be kept as a result of back-annotated DSPF/SPEF. This argument converts all the coupling capacitors to ground capacitors if the value of the coupling capacitor is below the specified <i>value</i> .
<code>-select_ipin_method <i>method_value</i></code>	Instructs the PrimeSim XA tool to select which ipin to use for any type of analysis such as <code>.probe/.measure/.ic/.nodeset</code>, or any statement that uses the <code>NET</code> name. This option has no impact for <code>NETs</code> with a <code>* P</code> statement. You can specify one of the following values: • 0 to choose the first instance pin as the representative node. This is the default value. • 1 to choose the last instance pin as the representative node. • 2 to choose the first instance pin as the representative node after sorting of <code>(* I)</code>. • 3 to choose the last instance pin as the representative node after sorting of <code>(* I)</code>. • 4 specifies to choose the representative node with the same name as <code>* NET</code>. This node may come from any of the following: <code>* S</code> statement, <code>* I</code> statement, or port of R or C. If the searching process is not successful, then <code>set_ba_option</code> uses <code>-select_ipin_method 0</code>.
<code>-select_ipin_gate 0 1</code>	The argument is only for the first GATE instance pin selection. You can use it together with <code>-select_ipin_method</code>. This argument has the precedence over <code>-select_ipin_method</code>. Also, it has no impact for <code>NETs</code> with a <code>* P</code> statement. • 0 has no impact. the rule is the same as <code>-select_ipin_method</code>. • 1 chooses the first GATE pin as the representative node of <code>(* I)</code>. If the instance GATE pin does not exist on <code>(* I)</code>, this argument is discarded and follows the <code>-select_ipin_method</code> rule, either the default or user-specified.
<code>-layout_only_device_models <i>model_name</i> {<i>model_name</i>}</code>	Provides the layout device model name that only exists in layout to allow proper back-annotation.

Argument	Description
<code>-ba_net_success_ratio percentage_vale</code>	Stops the simulation if the number of nets successfully back-annotated is lower that the <i>percentage_value</i>.
<code>-check_temperature 0 1</code>	When set to 1, checks if the temperature specified in the extracted (SPF) file matches the temperature specified with <code>.TEMP</code> in the main netlist. If it does not match, the PrimeSim XA tool errors out. If the SPF file does not have a specified temperature, it also errors out. Note that this argument only works for the back- annotation flow. The default is 0.
<code>-multi_subckt_suffix "suffix_name"</code>	Lets you specify a subcircuit suffix pattern name, which can include a wildcard character, for multi-back-annotation so that the subcircuit from the DSPF is annotated to the prelayout subcircuit and includes the same pattern of the subcircuit name from the DSPF file. For example, if you specify <code>"foo*"</code> for <code>subckt foo</code> from the DSPF, it is back-annotated to <code>.subckt foo_x</code>, <code>.subckt foo_y</code>, and so on in the prelayout netlist. Note: You must enclose the subcircuit suffix name in double quotes ("").
<code>-subckt_mode 0 1 2</code>	Determines how to process a subcircuit name in the prelayout netlist that is not matched in the DSPF file: • <code>-subckt_mode 0</code> (default) applies top-level back-annotation to the subcircuit. • <code>-subckt_mode 1</code> ignores back-annotation for the subcircuit. • <code>-subckt_mode 2</code> specifies to do back-annotation for the first ordered DSPF file to the subcircuit, and ignores the rest of the DSPF files invoked by multiple <code>load_ba_file</code> commands. This option is compatible with similar functionality in the PrimeSim tool.
<code>-skip_vsrc 1 0</code>	When set to 1, specifies that all the voltage sources from the user-defined stimulus are not back-annotated from the DSPF file. The default is 0.
<code>-keep_layout_only_net 0 1</code>	When set to 1, keeps the coupling capacitors overlapping between schematic and layout only parasitic nets. The default is 0.

Description

By default, the PrimeSim XA tool uses [load_ba_file](#) for back-annotation. You can use [set_ba_option](#) when you want to make adjustments to how the PrimeSim XA tool handles files to perform lumped capacitance back-annotation only, or when you want to short out the SPF pins or apply the DPF scaling to the SPF files.

Note:

This command now has the functionality of the `enable_ba_error_net` command, which is now obsolete.

Supported Element Types

[Table 8](#) lists the elements you can specify with the `-dpf_elem_type` option.

Table 8 Supported Element Types

Supported Element Keywords	Expected To Match
mos	m* or M*
diode	d* or D*
resistor	r* or R*
bjt (BJT)	q* or Q*
capacitor	c* or C*
instance (Xxxx macro-model)	x* or X*

Mapping Terminals

When parsing the prelayout netlist, the PrimeSim XA tool finds all macro models from the model libraries (`.lib` files) and picks up a device from the macro model definition that represents a macro model. For each macro model, the PrimeSim XA tool provides subcircuit port-to-device pin mapping. This information is used to generate corresponding [map_ba_terminal](#) commands to map macro model ports to device pin names.

When you run the [load_ba_file](#) command, the `-auto_map_ba_terminal` option automatically generates a [map_ba_terminal](#) command for macro models that maps the terminal name used in the prelayout netlist to the pin name in the postlayout netlist. This option provides correct back-annotation and accurate simulation results. Also, for devices with interchangeable terminals such as MOSFETs, resistors, capacitors and inductors, a corresponding `set_ba_option -swap_port` command is automatically generated to define swappable device terminals. Without this functionality you need to provide mapping manually.

Table 9 show the devices that are supported in macro model definitions with their pin names used for mapping.

Table 9 Swappable Device Pins

Device	Pin Name	Swappable Pins
MOSFET	D, G, S, B	D and S
JFET	D, G, S	D and S
BJT	C, B, E	No pin swapping
DIODE	A, C	No pin swapping
RES, CAP, IND	P, N	P and N

Examples

Given the following `vss` net call:

```
*|NET vss 0.00924425PF
*|I (x04/mn:BULK x04/mn BULK B 0 20.15 8.55)
*|I (x02/mn:BULK x02/mn BULK B 0 22.75 1.05)
*|P (vss B 0 0.325 0.151)
*|P (vss_1 B 0 20.15 8.55)
*|P (vss_2 B 0 22.75 1.05)
```

The `vss_1` and `vss_2` pins are shorted to `vss` if `set_ba_option -short_pins 1` is applied.

```
set_ba_option -bus_delimiter < >
```

This command changes the bus delimiters in the SPEF file. For example:

```
xtop/xsram/xcol1/wl[0] becomes xtop/xsram/xcol1/wl<0>
xtop/xsram/xcol<1>/wl[0] becomes xtop/xsram/xcol<1>/wl<0>
xtop/xsram/xcol[1]/wl[0] becomes xtop/xsram/xcol<1>/wl<0>
set_ba_option -dpf_elem_type mos
```

Back-annotates only MOSFET DPF devices and ignores other devices in the DPF section.

```
set_ba_option -dpf_elem_type mos resistor diode
```

Back-annotates only MOSFET/resistor/capacitors DPF devices and ignores other devices in the DPF section.

```
set_ba_option -dpf_elem_type mos instance
```

Back-annotates only MOSFET and Xx instance DPF devices and ignores other devices in the DPF section.

```
schematic netlist:
M1 a b c d NCH l=45e-9 w=180e-9

SPF:M1 a b c d NMOS l=45e-9 w=90e-9 ...
M1@2 a b c d NMOS l=45e-9 w=90e-9 ....
```

The default value for `-keep_prelayout_model` is 0, which means it uses the model from SPF file. When setting `-keep_prelayout_model` to 1, it uses the model from the schematic netlist.

The following command enables the resistor only back-annotation for the `vdd` and `nclk` nets.

```
set_ba_option -rnet vdd nclk
```

The following command enables the resistors and lumped capacitors back-annotation for the `vdd` and `nclk` nets.

```
set_ba_option -rcgnet vdd nclk
```

The following example sets the precedence between two `-rnet` and `-rcgnet` arguments. In this case, `-rnet vdd` is ignored since the `vdd` net is specified in `-rcgnet vdd`.

```
set_ba_option -rnet vdd nclk -rcgnet vdd
```

The following commands identify the power nets external or internal power nets first and do the back-annotation to the corresponding power nets according to the arguments.

The following example enables back-annotation for R-only nets into the external power nets, internal power nets, or all power nets.

```
set_ba_option -rnet_powernet external
set_ba_option -rnet_powernet internal
set_ba_option -rnet_powernet all
```

The following example enables back-annotation for C_g (lumped capacitor) nets into the external power nets, internal power nets, or all power nets.

```
set_ba_option -cnet_powernet external
set_ba_option -cnet_powernet internal
set_ba_option -cnet_powernet all
```

The following example enables back-annotation for C_c (coupling capacitor) nets into the external power nets, internal power nets, or all power nets.

```
set_ba_option -ccnet_powernet external
set_ba_option -ccnet_powernet internal
set_ba_option -ccnet_powernet all
```

set_bus_format

Sets the bus delimiters.

Syntax

```
set_bus_format -open open_delimiter [-close close_delimiter]
```

Arguments

Argument	Description
-open open_delimiter	Specifies the opening delimiter character, for example, [, <, _, or nothing (no open delimiter).
-close close_delimiter	Specifies the close delimiter character, for example,], >, _, or nothing (no close delimiter).

Description

You can specify two types of bus delimiters: both open delimiter and close delimiter, or just the open delimiter. If you do not use a `set_bus_format` command, by default the PrimeSim XA tool uses `[ ]` as bus delimiters, for example, `a[0]`, `a[1]`, `a[2]`, and so on. Wildcard expansion is supported for bus nodes with a bus delimiter (`:`).

Examples

Specifies `a<0>`, `a<1>`, `a<2>`, and so on.

```
set_bus_format -open < -close >
```

Specifies `a/0`, `a/1`, `a/2`, and so on.

```
set_bus_format -open /
```

Specifies `a0]`, `a1]`, `a2]`, and so on.

```
set_bus_format -open -close ]
```

Specifies `a0`, `a1`, `a2`, and so on.

```
set_bus_format -open -close
```

Specifies `a#0`, `a#1`, `a#2`, and so on.

```
set_bus_format -open #
```

set_capacitor_option

Lets you control capacitor optimization. You can specify multiple `set_capacitor_option` commands.

Syntax

```
set_capacitor_option -rule rule_value
[-min min_value]
[-report report_value]
[-process_negative negcap_switch]
```

Arguments

Argument	Description
<code>-rule rule_value</code>	You must specify one of the following optimization rules for resistor elements: • 1 specifies all capacitors below the <code>min_value</code> are treated as open circuit. The PrimeSim XA tool processes the capacitors at the netlist parsing stage after solving the parameters. All voltage and current probes of the optimized capacitors cannot be recovered for analysis during simulation. This rule has the advantage of using less memory. • 2 specifies that all capacitors below the <code>min_value</code> are lumped. It has the advantage of recovering the optimized capacitors for voltage and current probes. However, this rule can impact the performance of the simulation. Rule 1 is executed before rule 2, so you can apply both rules in one simulation. See Figure 4. When you specify multiple commands, only rule 1 can override rule 1, and rule 2 can override rule 2. It is recommended to specify the absolute minimum capacitance value of rule 1 to be less than absolute minimum capacitance value of rule 2. See Figure 5.
<code>-min min_value</code>	Specifies the lower threshold value for the capacitors to be kept. All capacitors with absolute capacitance value below <code>min_value</code> ($\text{abs}(\text{Capacitance}) < \text{min_value}$) are treated as follow by the <code>rule_value</code>. • <code>-rule 1</code> treats capacitors as open circuit. • <code>-rule 2</code> treats capacitors as lumped.
<code>-report report_value</code>	Reports the list of optimized capacitors. Parasitic capacitors from the back-annotation file are not reported. It can be set to: • 0 (default) does not report any capacitors. • 1 reports the list of optimized capacitors to an output file with a <code>*.rlcignore</code> extension.

Argument	Description
<code>-process_negative</code> <code>negcap_switch</code>	Controls the optimization of negative capacitors for simulation. This is only valid as part of the <code>-rule 1</code> option. You must specify <code>-rule 1</code> if you use this option. It can be set to: <code>normal</code> (default) keeps negative capacitors for optimization by the <code>-rule</code> and <code>-min</code> options. <code>keep</code> specifies to keep all negative capacitors for simulation. <code>open</code> treats all negative capacitors as open circuit for simulation.

Description

This command optimizes capacitors to improve performance for post-layout simulation. The PrimeSim XA tool supports two rules previously described.

If you do not specify this command, the PrimeSim XA tool supports the following behavior.

- For PrimeSim XA, Eldo and Spectre netlist formats, all capacitors are kept.
- For the SPF or SPEF netlist formats (back-annotation), the PrimeSim XA tool supports [load_ba_file](#) command and [set_ba_option](#) command to read in the back-annotation file and control the capacitor optimization. The `-min_cap` option of those commands sets the lower threshold of the capacitors in the back-annotation file to be kept. All capacitors with absolute value less than the value specified by `-min_cap` option are treated as open circuit with the `-rule 1` option. The default is `1e-2*CSHUNT`. The default of `CSHUNT` option is `1e-20`.

The capacitor optimization from the back-annotation file is affected by these commands in the following order of priority:

1. [load_ba_file](#)
2. [set_ba_option](#)
3. [set_resistor_option](#)

Figure 4 Simulation Flow of -rule 1 and -rule 2

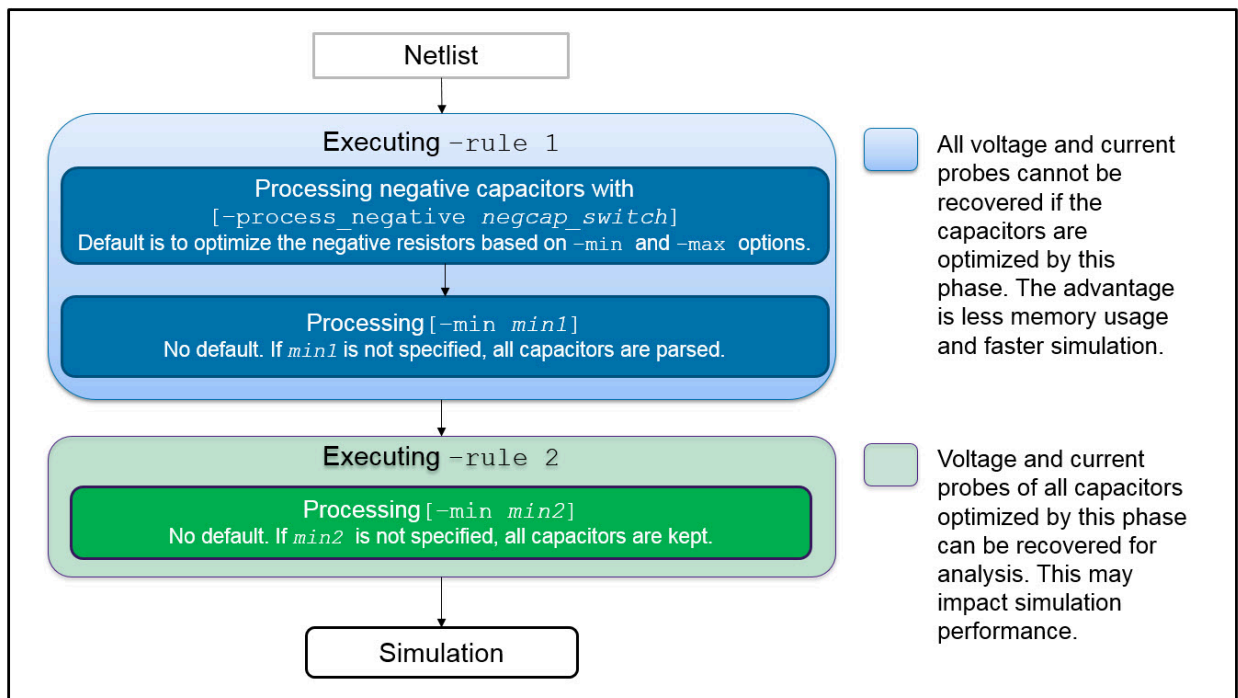
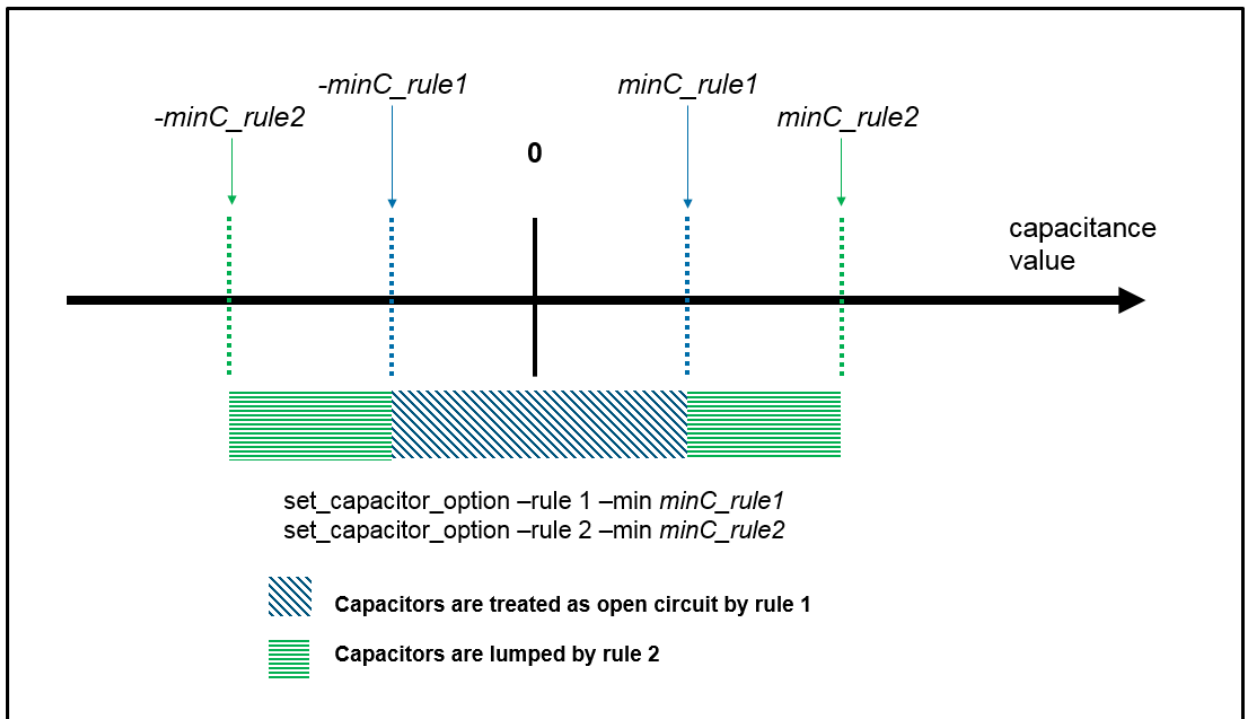


Figure 5 Using -rule 1 and -rule 2 Options in the Same Simulation



Examples

The following example opens all the capacitors with value between $-5\text{e-}15$ Farad and $5\text{e-}15$ Farad.

```
set_capacitor_option -rule 1 -min 5e-15
```

The following example treats all the capacitors with value less than $1\text{e-}15$ Farad as open circuit and lumps all capacitors with value between $1\text{e-}15$ Farad and $5\text{e-}15$ Farad. All negative capacitors are treated as open circuit.

```
set_capacitor_option -rule 1 -min 1e-15 -process_negative open  
set_capacitor_option -rule 2 -min 5e-15
```

The following example treats all capacitors with value less than $1\text{e-}12$ Farad as open circuit with rule 1 and reports them in *.rlcignore file, and also lumps all the capacitors with value less than $1\text{e-}10$ Farad with rule 2 and reports them in *.rlcignore file.

```
set_capacitor_option -rule 1 -min 1e-12 -report 1  
set_capacitor_option -rule 2 -min 1e-10 -report 1
```

set_ccap_level

Lets you override the default coupling capacitor tolerances.

Syntax

```
set_ccap_level -level ccap_level  
[-inst inst_name {inst_name}]  
[-subckt subckt_name {subckt_name}]
```

Arguments

Argument	Description
-level ccap_level	Sets the level of the coupling effect. The <i>ccap_level</i> is an integer from 1 to 7, where 1 produces the fastest simulation and 7 the most accurate. The default is the level you specify using set_sim_level , or, if none was specified, level 3. See Table 10 for details about each level.
-inst inst_name {inst_name}	Defines the instance name at which the <i>ccap_level</i> applies.
-subckt subckt_name {subckt_name}	Defines the subcircuit name at which the <i>ccap_level</i> applies.

Description

During simulation, the PrimeSim XA tool uses the default coupling capacitor tolerances pre-set in each [set_sim_level](#). You can use `set_ccap_level` to override the default coupling capacitor tolerances. You can use this command locally on specific instances or subcircuits.

[Table 10](#) provides guidelines for when to use `set_ccap_level` and what level to choose.

Table 10 Guidelines for using set_ccap_level

Level	Description
1	Specifies the most aggressive setting. All coupling capacitors are split. No coupling effects are modeled at this level.
2	Specifies an aggressive setting. All coupling capacitors, except those with both terminals that belong to the same resistive network, are split to ground. Almost no coupling effects are considered during simulation, in particular no inter-block coupling effects are modeled. Note that you should not use this level when charge-coupling effects are important for circuit functionality, such as in charge-pump circuits.

Table 10 Guidelines for using set_ccap_level (Continued)

Level	Description
3	Performs maximum optimizations of coupling capacitors. A small amount of coupling effects in blocks propagating digital signals and power distribution networks, and some coupling effects in blocks propagating analog signals are modeled. This level is suitable for functional verification or for simulation of only the strongest coupling effects in digital, memory, low-sensitivity analog, mixed-signal and full-chip circuits.
4	Performs moderate optimizations of coupling capacitors. It models strong coupling effects, particularly in blocks propagating analog signals. This level is suitable for functional verification or for simulation of only the strongest coupling effects in all circuits.
5	Performs balanced optimizations of coupling capacitors. Coupling effects in both digital and analog circuits are accurately modeled for all types of circuits. This level is suitable for accurate timing and power simulation for all circuits with coupling effects.
6	Performs conservative optimizations of coupling capacitors. Detailed coupling effects are accurately modeled to provide SPICE-like accuracy for all types of circuits. All circuits with coupling effects would generate SPICE-like accuracy.
7	No coupling capacitor optimizations are performed. This level yields full-SPICE accuracy.

Note:

In all cases, the `set_ccap_level` corresponds to the matching level used by the `set_sim_level` command. That is, if you set `set_ccap_level` to 3, the results are the same as if you had set `set_sim_level` to 3.

Examples

In the following example, the PrimeSim XA tool overrides the coupling capacitor tolerances from 3 (set by `set_sim_level`) to level 6.

```
set_sim_level 3
set_ccap_level 6
```

set_ccap_option

Use this command with `set_ccap_level` to get maximum performance simulating designs with many small coupling capacitors.

Syntax

```
set_ccap_option -ccap_to_gcap ccap_threshold_value  
[-ccap_to_scap switch_value]  
[-ccap_threshold ccap_low_threshold_value]
```

Arguments

Argument	Description
<code>-ccap_to_gcap</code> <code>ccap_threshold_value</code>	Converts all coupling capacitors to ground capacitors if the coupling capacitor value is smaller than the <code>ccap_threshold_value</code> . There is no default value for the <code>ccap_threshold_value</code> . You must specifies a value to perform the conversion.
<code>-ccap_to_scap</code> <code>switch_value</code>	Converts all coupling capacitors to simplified capacitors. The simplified capacitor is an approximated model that allows partitioning across two regions previously connected by the coupling capacitor. The switch value can specify: • 1 for a more efficient way of handling the capacitor model. • 2 for a more conservative way of handling the capacitor model.
<code>-ccap_threshold</code> <code>ccap_low_threshold_value</code>	Specifies to keep any coupling capacitors with values greater than the <code>ccap_low_threshold_value</code> value. The default is 1.e-10F.

Description

The [set_ccap_level](#) command controls how the PrimeSim XA tool handles the coupling capacitors. In general, this command provides a good performance/accuracy tradeoff. However, in designs with many small coupling capacitors, especially from a large back-annotation file, [set_ccap_level](#) might not provide enough tuning capability to get maximum performance. `set_ccap_option` provides more advanced controls.

The `set_ccap_option` command splits small coupling capacitors to ground capacitors before the [set_ccap_level](#) takes effect, so specify `set_ccap_option` before you use [set_ccap_level](#). The `set_ccap_option` command converts small coupling capacitors prior to the RC optimization.

Examples

Splits all coupling capacitors to ground capacitors if the couple capacitor value is less than 1e-18.

```
set_ccap_option -ccap_to_gcap 1e-18
```

Splits all coupling capacitors to simplified capacitors.

```
set_ccap_option -ccap_to_scap 1
```

set_certification_option

Controls time stepping and sets a fixed global time step for transient simulation. You can also use the option to enable MOS model checking for accurate model validation.

Syntax

```
set_certification_option -enable 0|1|2  
[-tstep_control 0|1] [-tstep step_value]  
[-mos_check 0|1]
```

Arguments

Argument	Description
-enable 0 1 2	Enables this option. • 0 (default): Disables the option setting for the command. • 1: Enables the option setting. • 2: Enables the option setting and prints negative diagonal capacitance and conductance in the log file during transient simulation when there is no encrypted element in the netlist.
-tstep_control 0 1	Sets to 0 to disable time stepping. When set to 1, PrimeSim XA chooses a time step for simulation based on the value specified in the .tran line in the netlist. When used in combination with the -tstep option, PrimeSim XA uses the specified time step for transient simulation. The default is 0.
-tstep <step_value>	Sets a user-specified time step for transient simulation. This option is valid only when the -tstep_control option is set to 0.
-mos_check 0 1	Controls MOS model checking. • 0 (default): Disables MOS model checking. • 1: Enables MOS model checking.

Examples

The following example specifies a time step and uses it for transient simulation.

```
set_certification_option -enable 1 -tstep_control 0 -tstep 1e-13
```

set_circuit_flash

Specifies a single, easy-to-use command to simulate flash designs.

Syntax

```
set_circuit_flash -app func|power -acc value
```

Arguments

Argument	Description
-app func power	Use -app func to check the functionality of a flash circuit. Use -app power for accurate measurement of power consumption.
-acc value	Specifies the performance/accuracy setting. See Table 11 .

Description

A flash design typically is a complex circuit with memory cells, charge pumps, voltage regulators and digital logic, each of which might operate at its own voltage range and have its own accuracy requirement. In addition, you may want to simulate a flash circuit in different flows such as ideal, post-layout extracted, and power-up/power-down. These complexities often make it difficult to come up with a set of commands that optimize accuracy and performance for the simulation.

To meet this challenge, the `set_circuit_flash` command has been created to enable internal customization according to the characteristics of the flash design, accuracy requirement, and type of the simulation. See [Table 11](#)..

Table 11 Performance/Accuracy Settings

set_circuit_flash	-app func	-app power
-acc 1	Functional simulation with no charge pump or oscillator (best performance).	Static power simulation (best performance).
-acc 2	Functional simulation with no charge pump or oscillator.	Static power simulation.
-acc 3	Functional simulation with no charge pump or oscillator.	Dynamic power simulation.
-acc 4	Functional simulation with charge pumps and oscillators.	Dynamic power simulation.

Table 11 Performance/Accuracy Settings (Continued)

set_circuit_flah	-app func	-app power
-acc 5	Functional simulation with charge pumps and oscillators (highest accuracy).	Leakage power simulation (highest accuracy).

This command is a global command. When using it, remove all other global commands that affect accuracy, such as `set_sim_level 5`. However, you can keep all other global commands such as `set_duplicate_rule -select_subckt last`, and all local commands such as `set_model_level 5 -subckt osc_23`.

Examples

Specifies a dynamic power simulation.

```
set_circuit_flash -app power -acc 4
```

Specifies a functional simulation for a flash circuit with charge pumps.

```
set_circuit_flash -app func -acc 5
```

Specifies a functional simulation for a flash circuit with a known block that has a charge pump. Note that for this example you can also use the setting in the previous example. However, the setting in this last example gives you better performance because it uses `-acc 1`, so you have a local command for the charge pump to satisfy the high accuracy requirement for this block.

```
set_circuit_flash -app func -acc 1
set_model_level 5 -subckt chg_pmp
```

set_circuit_transceiver

Specifies the setup for Fin Field-Effect Transistor (FinFET) transceiver simulations.

Syntax

```
set_circuit_transceiver -enable enable_value
[-process 0|1] [-powerup 0|1] [-app leakage]
```

Arguments

Argument	Description
-enable [enable_value]	Sets to 1 to enable FinFET transceiver simulation.

Argument	Description
<code>-process 0 1</code>	Specifies to simulate designs by process technology. • 0 (default): Runs FinFET simulation for 16nm designs and above. • 1: Runs FinFET simulation for 7nm designs and below.
<code>-powerup 0 1</code>	Sets to 1 to inform the simulator that this simulation starts mostly from 0 at time 0, and a power-up sequence will be run during transient analysis to ramp up all of the supplies and internal powers. Default is 0.
<code>-app leakage</code>	Sets to <code>leakage</code> to inform the simulator that this is a leakage simulation.

For planar bulk process transceivers, this command is not needed.

See Also

- [FinFET Transceiver Simulations](#)

set_current_option

Description

Adds an extra control for the current accuracy in addition to the [set_sim_level](#) setting.

Syntax

```
set_current_option [-level] option_value
```

Arguments

Argument	Description
<i>option_value</i>	You can specify an integer value between 0 and 7. The higher the value, the more accurate the current measurement. The default is 0. A value of 1 or 2 is recommended for peak, average and RMS current measurements. A value equal to or greater than 3 is recommended for leakage current or sensitive measurement. In general, the more the number of decades needed to settle from the preceding peak, the higher the option value should be. When tiny changes in voltages can influence the supply currents cumulatively, a value greater than 3 might be needed. For extremely detailed current measurements, a value of 6 is recommended. The specified <i>option_value</i> does not correlate to the value set by the <code>set_sim_level -level</code> command.

set_dc_icsweep_option

Description

Specifies how the IC sweep of each run in a sweeping or multi-run simulation (sweep, Monte Carlo, `.alter`) is initialized with a previously saved operating point.

Syntax

```
set_dc_icsweep_option [-enable] 0|1  
[-type nodeset|ic]  
[-mode previous|first]  
[-skip_dc 0|1]
```

Arguments

Argument	Description
<code>-enable</code>	The default setting of this argument is 0 to disable the automatic operating point reuse functionality. If this command is not specified, and the netlist contains a sweep statement, the parametric analysis is run normally without the automatic operating point reuse. Set this argument to 1 to enable automatic operating point reuse.

Argument	Description
<code>-type nodeset ic</code>	Specifies the type of initial condition to apply for the reused operating point. You can specify <code>ic</code> or <code>nodeset</code> (default). This argument has no effect on the first run of a parameter sweep analysis, or the nominal run of a Monte Carlo analysis in which IC sweep works normally by honoring the original initial conditions from a <code>load_operating_point</code> file, or from the netlist <code>.ic/nodeset</code> statements. The original initial conditions are ignored in the subsequent runs. When used with <code>-skip_dc</code>, this argument is ignored and a warning message is issued.
<code>-mode previous first</code>	Specifies how the runs are initialized by a previously saved operating point result. Without <code>-mode</code> set, the value of <code>previous</code> specifies to initialize the current run with the operating point result from the previous run of the analysis. The value of <code>first</code> specifies to always initialize the current run with a common operating point result from the first run. This argument does not apply to Monte Carlo analysis, which always initializes its DC with the operating point from its nominal run to provide consistent behavior between a distributed and sequential Monte Carlo run. The PrimeSim XA tool issues a warning message when <code>-mode</code> is specified with a Monte Carlo command.
<code>-skip_dc 0 1</code>	Specifies whether to perform the usual IC sweep or to skip it. Without <code>-skip_dc</code>, the default value of 0 performs an IC sweep prior to transient analysis. A value of 1 specifies to skip the DC evaluation and go straight to the transient analysis for all subsequent runs after the first one. This command always performs an IC sweep for the first run. Use this setting only for debugging the initial conditions file from another simulator imported in the PrimeSim XA tool.

Examples

Performs a data sweep analysis. The IC sweep is performed for each run after the first run and is automatically initialized with the initial conditions of the `nodeset` type from the previous run.

```
.tran 1n 100n sweep data=dataname
.opt xa_cmd="set_dc_icsweep_option -enable 1"
```

Performs a data sweep analysis. An IC sweep is performed for the first run. Subsequent runs skip the DC and start the transient analysis with a common initial conditions of type `ic` at time 0 from the first run. The `-type` argument is ignored and a warning message is issued.

```
.tran 1n 100n sweep data=dataname
.opt xa_cmd="set_dc_icsweep_option 1 -type ic -mode first -skip_dc 1"
```

The `set_dc_icsweep_option` command in the following example is invalid. The `previous` mode is not compatible with the enabled `-skip_dc` option. The PrimeSim XA tool errors out during the front-end processing with the appropriate error message about the invalid usage.

```
.tran 1n 100n sweep data=dataname  
.opt xa_cmd="set_dc_icsweep_option 1 -mode previous -skip_dc 1"
```

Performs a Monte Carlo analysis with a nominal run and 9 run samples. An IC sweep is performed for the nominal run. Subsequent runs perform an IC sweep that is consistently initialized from a common initial conditions of `nodeset` type from the nominal run. This is the normal OP reuse usage scenario for a Monte Carlo analysis.

```
.tran 1n 100n sweep monte=10  
.opt xa_cmd="set_dc_icsweep_option 1 -mode first"  
.opt xa_cmd="set_monte_carlo_option -enable 1"
```

Performs a Monte Carlo analysis with a DC nominal run and 9 run samples. A DC-only nominal run with the initial conditions from `low_vt_typ.ic` is performed to generate the OP for reuse as `nodesets` for all the MC samples. The `-mode` option is ignored with the appropriate warning messages.

```
.tran 1n 100n sweep monte=10  
.opt xa_cmd="load_operating_point -file low_vt_typ.ic"  
.opt xa_cmd="set_dc_icsweep_option 1 -mode first"  
.opt xa_cmd="set_monte_carlo_option -enable 1"
```

Performs a Monte Carlo analysis with a nominal run and 9 run samples in a DP running mode with five processes. An IC sweep is performed for the nominal run. Subsequent runs perform an IC sweep that is consistently initialized from a common initial conditions of `nodeset` type from the nominal run. The option `-mode first` is ignored with the appropriate warning message. This is the normal OP re-use usage scenario of a Monte Carlo analysis.

```
.tran 1n 100n sweep monte=10 -dp 5  
.opt xa_cmd="set_dc_icsweep_option 1 -mode first"  
.opt xa_cmd="set_monte_carlo_option -enable 1"
```

set_dc_option

Controls DC convergence.

Syntax

```
set_dc_option  
[-method method_type [method_type ...]]  
[-skip_dc value]  
[-skip_uic 0|1]  
[-cont_dc enable_value]
```

```
[-min_res min_res_value]
[-report report_type]
[-iteration iter_value]
[-loop_solve loop_solve_val]
[-max_v max_val]
[-min_v min_val]
[-check_vsrc 0|1]
[-latch latch_value]
{-cck_filepath directory_path}
[-algorithm algorithm_type]
[-gmin_level gmin_level_val]
[-analog_level val]
[-init init_val]
```

Arguments

Argument	Description
-method <i>method_type</i>	Specifies the method for DC convergence. Set <i>method_type</i> to one of the following values: • <i>auto</i> (or 0) is the default sequence that the PrimeSim XA tool uses to achieve DC convergence. The PrimeSim XA tool starts with the most aggressive method and, if DC convergence is not achieved, goes to a more conservative method in the following order: <i>static</i> > <i>adaptive</i> > <i>spice</i>. • <i>static</i> (or 1) partitions the circuit before solving the operating point. It uses the switch level for its initial estimate and the Newton Raphson iteration and pseudo-tran approaches for single-matrix solving. • <i>adaptive</i> (or 2) automatically adjusts the partition during the solving stage for better DC convergence. When you specify this method, the block size is often larger than the one with the <i>static</i> method. It also uses a more robust Newton Raphson iteration and pseudo-tran approach for single-matrix solving. • <i>spice</i> (or 3) is the most accurate DC method. It does not partition and uses the Newton Raphson iteration and pseudo-tran approaches for single-matrix solving. For more information about choosing a method for DC convergence, see the following Description section.
-skip_dc <i>value</i>	Specifies one of the following values: • 0 (default) uses the usual DC methods. • 1 skips the DC evaluation and goes straight to transient analysis. Use this setting only for debugging the IC file from another simulator imported into the PrimeSim XA tool for initial condition testing.

set_dc_option

Argument	Description
<code>-skip_uic 0 1</code>	Specifies one of the following values: • 0 (default) honors the <code>UIC</code> option in the <code>.tran</code> statement in the netlist. • 1 ignores the <code>UIC</code> option in the <code>.tran</code> statement in the netlist when running DC initialization as if there was no <code>UIC</code> option in the netlist. To use the <code>-skip_uic</code> option, you must place the <code>set_dc_option</code> command either in the netlist after the <code>.tran</code> statement, or in a Tcl configuration file. If the <code>set_dc_option</code> command is placed in the netlist before the <code>.tran</code> statement, it will be overwritten by <code>UIC</code> and be ignored.
<code>-cont_dc enable_value</code>	When DC does not converge, the PrimeSim XA tool stops and sets all the unsolved nodes to 0V and continues with the transient simulation. This is the default behavior of PrimeSim XA DC when <code>enable_value</code> is off. If <code>enable_value</code> is on, the PrimeSim XA tool continues solving the rest of the nodes at the last algorithm chosen in the case of non-convergence, and applies the initial condition appropriately to the nodes. For example: • When <code>-method static -cont_dc 1</code> is specified and the PrimeSim XA tool does not converge using the <code>static</code> method, it continues solving the rest of the nodes using the <code>static</code> method. • When <code>-method static adaptive -cont_dc 1</code> is used and the PrimeSim XA tool does not converge in both methods, the PrimeSim XA tool continues solving the rest of the nodes using the <code>adaptive</code> method. See Common Syntax Definitions for more information about <code>enable_value</code>.
<code>-min_res min_res_value</code>	Without this argument, the DC solution keeps all the resistors when it solves the operating point. This argument tells DC processing to short small resistors below the specified <code>min_res_value</code>. The default is 0. This option only supports the <code>static</code> method of the <code>-method</code> option.
<code>-report report_type</code>	Specifies one of the following <code>report_type</code> values: • <code>always</code> specifies to always dump the DC convergence report. • <code>fail</code> (default) specifies to only dump the DC convergence report when the DC operating point fails to converge. • <code>off</code> does not dump the DC convergence report. • <code>unstable</code> reports the percentage of total iterations for each node during DC initialization. The report is dumped into the <code>.unstable</code> file.
<code>-iteration iter_value</code>	Specifies the maximum number of DC iterations. The default is 100,000,000.

Argument	Description
<code>-loop_solve</code> <code>loop_solve_val</code>	If you set this value greater than or equal to 1, the tool controls the DC maximum number of region solving only in static analysis. The default is 400.
<code>-max_v</code> <i>max_val</i>	If you set this value greater than or equal to 0, the tool clamps voltage in DC analysis. The default value is ,1000,000.
<code>-min_v</code> <i>min_val</i>	If you set this value less than or equal to 0, the tool clamps voltage in DC analysis. The default value is -1,000,000.
<code>-check_vsrc</code> 0 1	Default is 0. If you specify 1 tool enables verification of user-defined voltage sources (VSRC) during DC analysis. If a VSRC is outside the range of <code>-max_v</code> limit, the tool errors out with a message indicating "VSRC maximum limit is too small". Similarly, if a VSRC is lower than the <code>-min_v</code> limit the tool errors out with a message indicating "VSRC minimum limit is too large".
<code>-latch</code> <i>latch_value</i>	Specify one of the following values: 0 ignores front-end latch detection. 1 (default) for front-end latch detection based on group latch nodes, with one node of each group assigned a node set based on vsupply detection on this group. 2 for front-end latch detection based on group latch nodes, with one node of each group assigned initial conditions (ic) based on vsupply detection on this group.
<code>-cck_filepath</code> <i>directory_path</i>	Specifies the directory path to check when searching for the CircuitCheck (CCK) latch detection files, If you do not specify a path, the PrimeSim XA tool does not load any CCK latch detection files.
<code>-algorithm</code> <i>algorithm_type</i>	Specifies the algorithm type to solve nonlinear equations: <code>default</code> enables the PrimeSim XA tool to choose the more efficient algorithm to solve the nonlinear equations. It can be the Newton Raphson method or Homotopy method. <code>newton</code> overrides the default nonlinear equation-solving algorithm to use Newton Raphson method. <code>homotopy</code> overrides the default nonlinear equation-solving algorithm to use Homotopy method.
<code>-gmin_level</code> <i>gmin_level_val</i>	Specifies one of the following values to set the GMIN value during DC convergence. 0 (default) takes the global GMIN value. 1 sets the GMIN value to 1e-28. 2 sets the GMIN value to zero.

Argument	Description
<code>-analog_level val</code>	Specifies the level for analog simulation with an integer value of 3 through 7, which corresponds to a set_sim_level value.
<code>-init init_val</code>	Specifies one of the <code>init_val</code> values: • 0 disables any initial guessing for the <code>spice</code> method. • 1 (default) enables initial guessing from the <code>static</code> method with limited iterations for the <code>spice</code> method. • 2 disables additional initial guessing for special instances, such as ring-oscillators for all the methods. • 3 disables adaptive IC initialization algorithm for all the methods. • 4 enables a hybrid mode of the <code>spice</code> and <code>static</code> methods.

Description

This command controls options related to the DC solver. You can use the command to output a DC convergence report to aid in debugging a DC convergence problem. The report is written to a `.dc0` file in the same directory as the other simulator output files. The converged and non-converging nodes are listed in separate sections together with the node voltage, delta-v, delta-I, and the element contributing the largest current to the node.

You can also choose the DC method or skip DC analysis to go straight to transient analysis.

If you specify multiple options for the `-method` argument, the PrimeSim XA tool starts with the most aggressive DC method and proceeds to more conservative DC methods if necessary to achieve DC convergence. [Table 12](#) lists the guidelines that apply to the `-method` option.

Table 12 How To Choose a DC Convergence Method

-method option	Guideline
<code>auto</code>	This method lets the PrimeSim XA tool choose the DC convergence method automatically.
<code>static</code>	This method provides the best performance for most circuits of any size.
<code>adaptive</code>	The method is for medium-sized analog or mixed-signal circuits.

Table 12 *How To Choose a DC Convergence Method (Continued)*

-method option	Guideline
spice	This method provides the most accurate DC result in the PrimeSim XA tool. It is not recommended for large circuits because of its impact on performance. If the <code>spice</code> method still does not provide DC convergence, the PrimeSim XA tool issues a non-DC convergence warning and starts the transient analysis from the non-converged operating point derived from this option.

Example 1

The following example always dumps the DC convergence report.

```
set_dc_option -report always
```

Example 2

The following example uses the `static` method for DC analysis.

```
set_dc_option -method static
```

Example 3

In the following example, the netlist contains the following lines:

```
.ic v(bg)=0.8
.tran 1p 2u UIC
```

If you run the PrimeSim XA simulation without `set_dc_option -skip_uic` or with `set_dc_option -skip_uic 0`, PrimeSim XA skips DC initialization and starts the transient simulation using `.ic (bg)=0.8`.

If you run the PrimeSim XA simulation with `set_dc_option -skip_uic 1`, PrimeSim XA runs DC initialization using `.ic (bg)=0.8`, and then runs the transient simulation.

set_disable_rawimage

Disables generation of a `.rawimage` file.

Syntax

```
set_disable_rawimage [0|1]
```

Arguments

Argument	Description
0 1	Specify 1 to disable generation of a <code>.rawimage</code> file. The default is 0.

Description

The PrimeSim XA tool generates a `.rawimage` file to reduce peak memory usage. By default, in a single transient run, this file is automatically deleted once the data is no longer needed (before the simulation enters the DC stage).

For any inner-loop simulations (that include `.alter/bisection/data sweep/Monte Carlo`) the `.rawimage` file remains until the entire simulation is completed. The `.rawimage` file size, and the duration of its existence, can become a critical gating issue for large postlayout simulations or heavy Monte Carlo analysis. This command makes it possible to force the PrimeSim XA tool not to generate the `.rawimage` file.

set_inductor_option

Lets you control inductor optimization. You can specify multiple `set_inductor_option` commands.

Syntax

```
set_inductor_option -rule rule_value
[-min min_value]
[-report report_value]
```

Arguments

Argument	Description
<code>-rule rule_value</code>	You must specify one of the following optimization rules for inductor elements: • 1 processes inductors at the netlist parsing stage after solving the parameters. As a result, All inductors below the <i>min_value</i> are shorted. All voltage and current probes of the optimized inductors cannot be recovered for analysis during simulation. This rule has the advantage of using less memory. • 2 has the advantage of recovering the optimized inductors for voltage and current probes. However, this rule can impact the performance of the simulation. All inductors below the <i>min_value</i> are shorted. Rule 1 is executed before rule 2, so you can apply both rules in one simulation. See Figure 6. When you specify multiple commands, only rule 1 can override rule 1, and rule 2 can override rule 2. It is recommended to specify the absolute minimum inductance value of rule 1 to be less than absolute minimum inductance value of rule 2. See Figure 7.
<code>-min min_value</code>	Specifies the lower threshold value for the inductors to be kept. • With rule 1, all inductors with absolute value below <i>min_value</i> ($\text{abs}(\text{Inductance}) < \text{min_value}$) are shorted. • With rule 2, only positive inductors with value below <i>min_value</i> ($\text{Inductance} < \text{min_value}$) are shorted.
<code>-report report_value</code>	Reports the list of inductor that were optimized. It can be set to: • 0 (default) does not report any optimized inductors. • 1 reports the list of optimized inductors to an output file with a *.rlcignore extension.

Description

This command optimizes inductors to improve performance for post-layout simulation. The PrimeSim XA tool supports two rules as previously described.

If you do not specify this command, the PrimeSim XA tool keeps all the inductors for simulation. The PrimeSim XA tool also supports [load_ba_file](#) command and [set_ba_option](#) command to read in the back-annotation file (SPF and SPEF) and control the inductor optimization.

Figure 6 Simulation Flow of -rule 1 and -rule 2

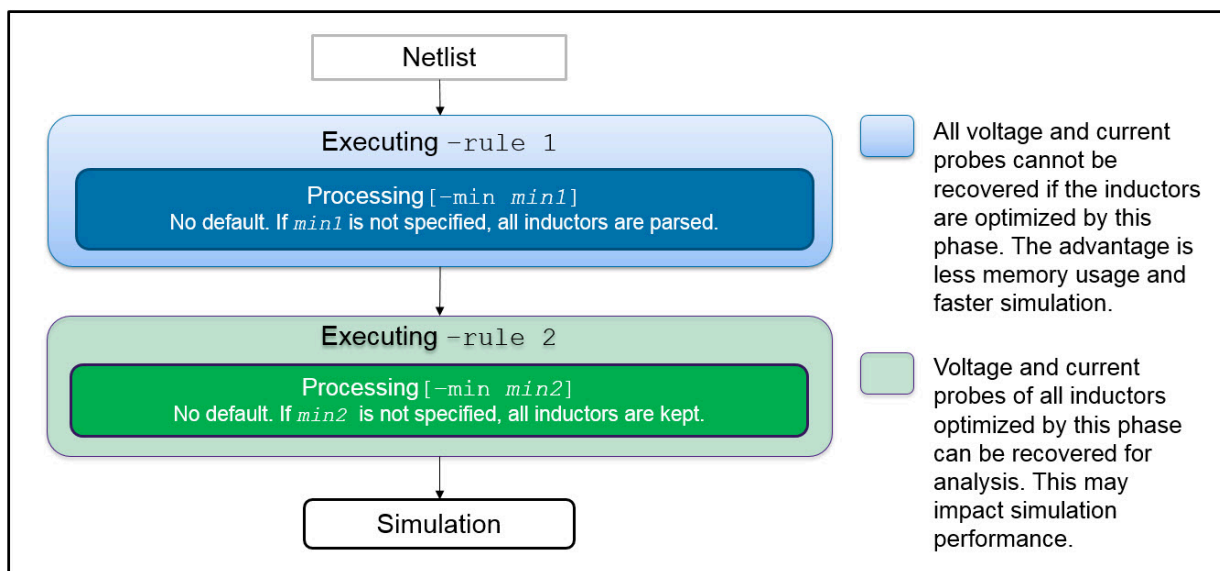
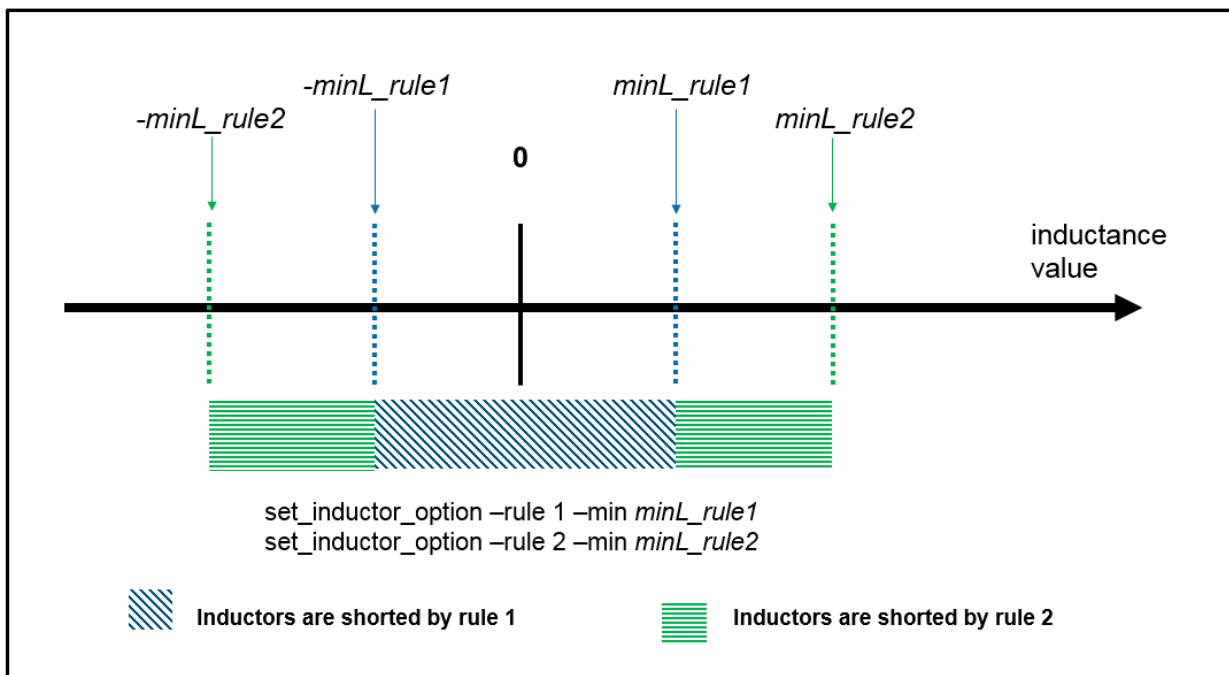


Figure 7 Using -rule 1 and -rule 2 Options in the Same Simulation



Examples

The following example shorts all the inductors with a value 5e-12 Henry with rule 1.

```
set_inductor_option -rule 1 -min 5e-12
```

The following example shorts all inductors with value less than 1e-12 Henry with rule 1 and reports them in a *.rlcignore file, and also shorts all the inductors with value less than 1e-10 Henry with rule 2 and reports them in a *.rlcignore file.

```
set_inductor_option -rule 1 -min 1e-12 -report 1
set_inductor_option -rule 2 -min 1e-10 -report 1
```

set_dp_option

Controls Distributed Processing (DP) simulation behavior.

Syntax

```
set_dp_option [-task_max_attempt value]
[-task_init_timeout value]
[-task_timeout_mult value]
[-worker_timeout value]
[-monitor_disk_space value]
[-monitor_memory_free value]
[-monitor_memory_usage value]
[-monitor_cpu_load value]
[-worker_max_count value]
[-worker_idle_timeout value]
[-check_config_interval value]
```

Arguments

Argument	Description
-task_max_attempt value	Sets the maximum number of times a task can be resubmitted if the simulation fails due to either the timeout mechanism or the monitor check function. The <i>value</i> is an integer greater than or equal to 1. The default is 3.
-task_init_timeout value	Sets an initial timeout value for the completion of the first task on any given worker. The <i>value</i> is a positive integer. There is no default value. Supported suffixes for unit are <i>m</i> or <i>M</i> for minutes and <i>h</i> or <i>H</i> for hours. If you do not specify a suffix, the default unit is hour and a warning message is issued. This argument works with the <code>-task_timeout_mult</code> argument to set the worker-specific effective task timeout value for all tasks after the first task. You use a task timeout to allow a worker to identify a runaway task and stop that task. A nominal run, either requested by the Monte Carlo analysis or automatically executed for DC initialization in a DP simulation mode (see the set_dc_icsweep_option command), is also affected by this timeout value.

Argument	Description
<code>-task_timeout_mult value</code>	Sets a task timeout multiplier factor. The timeout for each subsequent task after the first task of a worker is defined by the run time of the first completed task, scaled up by this value. The <i>value</i> must be 1 or larger. The default is 2. Table 14 lists the possible <code>-task_init_timeout</code> use combinations and the corresponding effects: Column A is the value of <code>-task_init_timeout</code>. Column B is the value of <code>-task_timeout_mult</code>.
<code>-worker_timeout value</code>	Limits the maximum lifespan for a worker to complete the total assigned tasks. The <i>value</i> is a positive integer. This argument should be set in accordance with the policy of your computer grid environment. The default is infinite. Supported suffixes for unit are <code>m</code> or <code>M</code> for minutes and <code>h</code> or <code>H</code> for hours. If you do not specify a suffix, the default unit is hour and a warning message is issued. The runtime of each task is estimated by multiplying the runtime of the first, non-nominal simulation task by a factor of 1.2. If there is not enough remaining time for the upcoming task, the worker is shut down and the task redistributed to a different worker. If the worker timeout is reached in mid-simulation, the task stops and is reassigned to a different worker provided that its <code>-task_max_attempt</code> value has not been exceeded.
<code>-monitor_disk_space value</code>	Monitors the lower threshold of the available working (scratch) disk space of a DP worker, including <code>/tmp</code> or NFS. The monitored disk is specified by the <code>-dplocation</code> command line option. When the free disk space of a DP worker falls under the threshold, the simulation task of the DP worker stops. The <i>value</i> is a positive integer. The default is 10MB. Supported suffixes for unit are <code>m</code> or <code>M</code> for MB and <code>g</code> or <code>G</code> for GB. If you do not specify a suffix, the default unit is MB and a warning message is issued.
<code>-monitor_memory_free value</code>	Monitors the lower threshold of available amount of physical memory. When the amount of free physical memory of a DP worker falls lower than the threshold, the simulation task of the DP worker stops. The <i>value</i> is a positive integer. The default is 10MB. Supported suffixes for unit are <code>m</code> or <code>M</code> for MB and <code>g</code> or <code>G</code> for GB. If you do not specify a suffix, the default unit is MB and a warning message is issued.
<code>-monitor_memory_usage value</code>	Monitors the upper threshold percentage of memory usage. The memory usage percentage is calculated as "PrimeSim XA worker process virtual memory usage / total system physical memory * 100". For example, if a worker process uses 10GB of memory in a server with 16GB physical memory, the usage is 62.5. The <i>value</i> is between 0 and 99 can be a fractional number greater than 0 and less than 100. The default is 95.

Argument	Description
<code>-monitor_cpu_load value</code>	Monitors the upper threshold of the load average of a worker CPU core. When the average CPU loads of 1, 5, and 10 minutes of the DP worker are higher than the threshold, the simulation task of the DP worker stops. This <i>value</i> is a fractional number greater than 1. The default is 2. The PrimeSim XA tool issues a warning when its internal limit is exceeded before the full number of samples is obtained.
<code>-worker_max_count value</code>	Lets you change the <code>-dp</code> setting after launching the job. The argument must be set in <code>xa_dp.ini</code> file under the current simulation directory. The PrimeSim XA tool periodically reads in the file every minute. The default is to use the initial <code>-dp</code> value from command line option setting.
<code>-worker_idle_timeout value</code>	Sets the timeout value a worker waits a task. The value is a positive integer and the default is 5 minutes. Set this argument in accordance with the policy of your computer grid environment. The supported suffixes for <i>value</i> are <code>m</code> or <code>M</code> for minutes and <code>h</code> or <code>H</code> for hours.
<code>-check_config_interval value</code>	Checks the DP configuration change during the runtime. The value is a positive integer and the default is 60 seconds. Set this argument in accordance with the policy of your computer grid environment. The supported suffixes for <i>value</i> are <code>m</code> or <code>M</code> for minutes and <code>h</code> or <code>H</code> for hours.

Description

DP uses the term definitions shown in [Table 13](#).

Table 13 DPM Term Definitions

Term	Definition
Master	An independent PrimeSim XA process that only initializes and manages the distributed processing flow and collects final result to provide summary. The Master does not run a simulation. Figure 8 shows the master and worker relationship.
Worker	The host in the grid that is assigned to execute a task. A worker can be assigned to execute multiple tasks.
Job	A PrimeSim XA DP invocation submitted to the grid. For example, a job of a DMC run consists of multiple individual execution tasks.

Table 13 DPM Term Definitions (Continued)

Term	Definition
Task	The individual independent execution that makes up the DP job. It corresponds to the Monte Carlo sample that makes up the DMC job, or a temperature value of a DP temperature sweep, or a corner of a DP .ALTER block sweep. A task can be single or multiple threaded, but is usually single-threaded.

Figure 8 Master and Worker Relationship

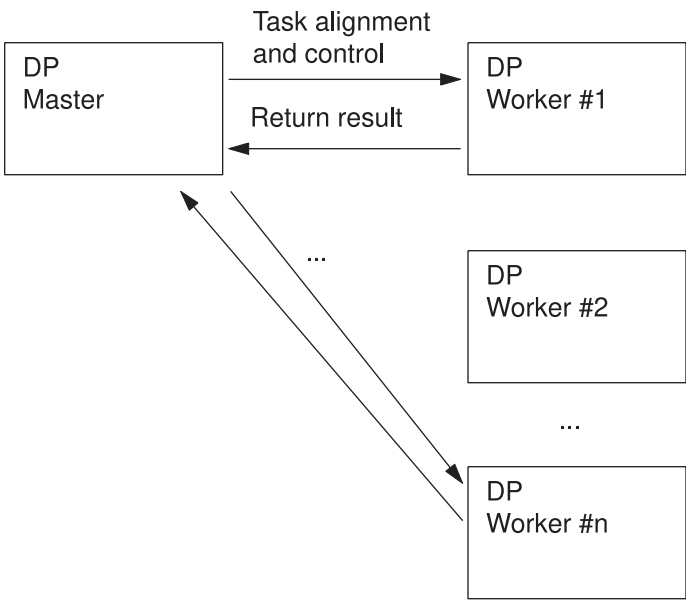


Table 14 Multiple Use Combinations

A	B	Effect on Task Timeout
Not set	Not set	Timeout of the first task (including the nominal run) of each worker is infinite. Timeout of subsequent tasks is the runtime of the first non-nominal task * 2.
Not set	Set	Timeout of the first task (including the nominal run) of each worker is infinite. Timeout of subsequent tasks is the runtime of the first non-nominal task * B.
Set	Not set	Timeout of the first task (including the nominal run) of each worker is A. Timeout of subsequent tasks is (A * 2).

Table 14 Multiple Use Combinations (Continued)

A	B	Effect on Task Timeout
Set	Set	Timeout of the first task (including the nominal run) of each worker is A. Timeout of subsequent tasks is (A * B).

Licensing

A master does not consume a PrimeSim XA license. Each worker checks out one license token for one distributed process with up to two cores. A PrimeSim HSPIICE license is needed for the Monte Carlo data mining feature.

Supported DP Connection Methods

DP supports the following connection methods.

- RSH [Remote Shell]
- SSH [Secure Shell]
- SGE [Oracle Grid Engine, originally Sun Grid Engine]
- LSF [Load Sharing Facility, from Platform Computing]
- SH [shell process on the local host]
- PBS [Portable Batch System]
- RTDA [Runtime Design Automation Network Computer]
- Netbatch [Intel Netbatch Compute Farm]

Preparing the Environment

The following conventions apply to the life span policy for single process in farm system.

Master

- Master waits for all workers to complete tasks before exiting. Therefore, the master should not be launched in machine that has a set time limitation for a single process. If master reaches the time limitation and stops, the remaining workers stop.

Worker

- Make sure the time slot for a single process is long enough for a worker to complete at least one sample run.

Disk space

- Make sure there is enough disk space in `temp` directory or the output directory given through `-o` command line option (depending on the setting of `-dplocation`). Running out of disk space causes the DP flow to exit abnormally. You can set `-monitor_disk_space` to help the DP flow exit and report error messages properly if there isn't enough disk space.

Machine load

- If a worker runs on a heavily loaded machine in terms of high CPU/memory usage, it may cause the machine to hang. The task running on a hanging machine cannot be completed, which eventually leads to all DP flow hanging. The monitor thread automatically terminates those workers on a heavily loaded machine when its load hits the monitored threshold specified by the `-monitor_cpu_load` option.. You can set `-monitor_memory_free`, `-monitor_memory_usage` or `-monitor_cpu_load` to adjust the monitor threshold so the master launches a new worker and assigns uncompleted tasks.

Network protocol PV4/IPV6 setup

- The PrimeSim XA tool adopts IPV4 for the DP flow unless the IPV6 network is explicitly set with an environment control variable through `setenv CDPL_IPV6`. You can set it to IPV4 through `unsetenv CDPL_IPV6`. If IPV6 is enabled, it displays the message:

The environment variable `CDPL_IPV6` is setting, XA DP will adopt the `ipv6` protocol

set_duplicate_rule

Lets you process multiple subcircuit definitions with the same name, multiple ports in a subcircuit with the same port name, or multiple model definitions with the same name.

Syntax

```
set_duplicate_rule -select_subckt select_value
[-subckt sub_name {sub_name}]
[-select_model select_value
[-model model_name {model_name}]
[-subckt_port enable_value]
```

Arguments

Argument	Description
<code>-select_subckt <i>select_value</i></code>	Specifies if the <i>select_value</i> of the first or last subcircuit definition is used when duplicate definitions exist. The default setting is 0, which raises an error if duplicate subcircuit definitions exist.
<code>-subckt <i>sub_name</i> {<i>sub_name</i>}</code>	Scopes the <code>-select_subckt</code> setting to only the named subcircuit definitions.

Argument	Description
<code>-select_model</code> <code>select_value</code>	Specifies if the <code>select_value</code> of the <code>first</code> or <code>last</code> subcircuit definition is used when duplicate definitions exist. The default setting is 0, which raises an error if duplicate model definitions exist.
<code>-model model_name</code> { <code>model_name</code> }	Scopes the <code>-select_model</code> setting to only the named model definitions.
<code>-subckt_port</code> <code>enable_value</code>	If a subcircuit definition contains a duplicate port name, the PrimeSim XA tool raises an error unless this option is enabled. If enabled, duplicate ports are shorted together.

Description

This command controls the simulator behavior when duplicate model or subcircuit definitions are found in the netlist. It also controls the behavior when duplicate subcircuit port names in exist in a subcircuit definition. A duplicate model or subcircuit definition raises an error unless this command is used. It can be used to specify if the first or last definition of a subcircuit or model is used and can be issued multiple times. The last command issued takes precedence. As with other the PrimeSim XA tool specifications, a global command does not override a previously set local command.

Duplicate subcircuit port names are allowed by default in Eldo simulation mode. In other netlist formats, duplicate subcircuit port definitions result in an error unless you specify the `-subckt_port` argument to enable duplicate ports. All duplicate ports are electrically shorted.

Examples

Specifies a global setting to select the last subcircuit.

```
set_duplicate_rule -select_subckt last
```

Overrides all previous `set_duplicate_rule` commands and uses the first subcircuit definition.

```
set_duplicate_rule -select_subckt last
set_duplicate_rule -select_subckt first
```

A global setting does not override a previously set local setting. In this example, subcircuit B uses the first definition and the remaining duplicated subcircuits use the last definition.

```
set_duplicate_rule -select_subckt first -subckt B
set_duplicate_rule -select_subckt last
```

If a global setting is made, the PrimeSim XA tool does not raise any parsing errors for duplicated definitions. If only local settings are made the PrimeSim XA tool raise a parsing error for any other duplicate definition. If the netlist contained:

```
set_duplicate_rule -select_subckt last -subckt A B C

.subckt A
*first a definition
...
.ends

.subckt A
*second A definition (expected)
...
.ends

.subckt D
...
.ends

.subckt D
* not intentional or expected
...
.ends
```

The PrimeSim XA tool selects the last definition of subcircuit A, but raises an error because subcircuit D has duplicate definitions.

set_flash_option

Sets options for the flash core cell.

Syntax

```
set_flash_option -delvto value
[-inst inst_name {inst_name}]
[-subckt subcircuit_name {subcircuit_name}]
```

Arguments

Argument	Description
-delvto value	Specifies the change in threshold voltage.
-inst inst_name {inst_name}	Selects flash cells to be initialized. Wildcards can be used in the <i>inst_name</i> .
-subckt subcircuit_name {subcircuit_name}	Selects flash cells for initialization by subcircuit name.

Description

This command specifies the initial change in threshold voltage to be applied to flash cells. If `-inst` or `-subckt` is not used to specify specific instances the command applies to all flash cells in the netlist. A `delvto` setting applied by this command overrides the instance parameter `delvto` for a flash cell.

Examples

Sets an initial vth change of -0.5V on all flash cells in the netlist.

```
set_flash_option -delvto -0.5
```

Sets an initial vth change of +2V on the `x1.x2.x3.mcell` flash cell.

```
set_flash_option -delvto 2 -inst x1.x2.x3.mcell
```

Sets an initial vth change of -1.5V on all flash cells in all `memword_16` instances of the subcircuit.

```
set_flash_option -delvto -1.5 -subckt memword_16
```

set_floating_node

Sets the voltage value of floating nodes at the DC operating point, or throughout the transient simulation. This command only works for floating nodes that have no driving path.

Syntax

```
set_floating_node
-val value
[-type gate | all]
[-format ic | vsrc]
[-outfile enable_value fileName]
[-dev_type n | p]
[-subckt name]
[-inst name]
```

where

```
enable_value ::= 1 | on | yes | true | 0 | off | no | false
```

Arguments

Argument	Description
-val value	Specifies a voltage value applied to the floating nodes.

Argument	Description
<code>-type gate all</code>	Specifies one of the following options: <code>gate</code>: Sets floating gates. <code>all</code> (Default): Sets all floating nodes, except floating bulks.
<code>-format ic vsrc</code>	Specifies one of the following options: <code>ic</code> (Default): Initializes floating nodes with the <i>value</i>. <code>vsrc</code>: Sets floating nodes to the <i>value</i> throughout the simulation. Using this option is equivalent to applying voltage sources of <i>value</i> to the floating nodes.
<code>-outfile enable_value fileName</code>	Enables (1 or on) or disables (0 or off) creation of an output file. See Common Syntax Definitions for more about the <i>enable_value</i>. The <i>fileName</i> is the output file name specified with the <code>-o</code> option when invoking the PrimeSim XA tool (or the input file name, if <code>-o</code> was not specified). The file extension is <i>.fnode</i>. If you create an output file and set <code>-format ic</code>, the PrimeSim XA tool saves output data in the form of <i>.ic</i> statements; if <code>-format vsrc</code>, the PrimeSim XA tool saves output data in the form of voltage sources. You can modify the output file to include it into the simulation.
<code>-dev_type n p</code>	Specifies the device type. <code>n</code>: Sets only <i>n</i> device gates to the given <code>-val value</code>. Other device gates (<i>p</i> devices) are left floating, or can be set to a different value using the <code>set_floating_node</code> command. <code>p</code>: Sets only <i>p</i> device gates to the given <code>-val value</code>. Other device gates (<i>n</i> devices) are left floating, or can be set to a different value using the <code>set_floating_node</code> command. The <code>-dev_type</code> argument only applies if the <code>-type</code> argument is set to <code>gate</code>. If the <code>-type</code> argument is set to <code>all</code>, <code>-dev_type</code> is ignored and a warning is issued in the log file.
<code>-subckt name</code>	Specifies the name of the sub-circuit to be assigned with the given voltage value. Use this option when you want to assign a different gate voltage to different sub-circuits of the same device type. Wildcard usage is supported.
<code>-inst name</code>	Specifies the name of the instance to be assigned with the given voltage value. Use this option when you want to assign a different gate voltage to different instances of the same device type. Wildcard usage is supported.

Description

Behavior of floating nodes can vary considerably with simulation algorithms and models. In a few cases, simulation results for a critical part of a circuit can depend on the voltage value at one or more floating nodes in the circuit. This can lead to hard-to-detect design issues and cause different results from different simulators. Forcing such nodes to a known value by means of `set_floating_node` can help isolate design problems.

You can use multiple `set_floating_node` commands with different scope control arguments to meet your design requirements.

Note:

You cannot use this command with the `report_floating_node` command. If you specify both the `set_floating_node` and `report_floating_node` commands, then the last one is used.

Using the -dev_type Argument

To allow different settings for pmos and nmos devices, multiple `set_floating_node` commands are allowed in the same configuration file. If multiple commands are set for the same device type, the last one overwrites the previous ones.

If two devices with a different type are connected to the same floating node, the last command takes effect. For example, in the following netlist:

```
.subckt inv A Z
mn1 Z A 0 0 n ...
mp1 Z A vdd vdd p ...
.ends
Xinv1 in out inv
```

Suppose that the `in` net is floating. If you specify:

```
set_floating_node 0 -dev_type n -type gate
set_floating_node 3.3 -dev_type p -type gate
```

Then the `in` net takes an IC value of 3.3. If you specify:

```
set_floating_node 3.3 -dev_type p -type gate
set_floating_node 0 -dev_type n -type gate
```

Then the `in` net takes an IC value of 0.

In a case when a net has multiple values, it appears followed by an asterisk (*) character in the `.fnode` report. See an example of an `.fnode` report below. If the value set by the `set_floating_node` command for the `in` net has been overwritten, it is followed by the '*' character. The value of `xtop.xbg.net3` has not been overwritten.

```
*** .IC/VSRC generated by report_floating_node command
*** When multiple set_floating_node commands are used,
*** overwritten values are marked with *.
```



```
.ic in = 3.3 *  
.ic xtop.xbg.net3 = 0.0
```

You need to use the `-outfile 1` option to generate the `.fnode` report.

Examples

Initializes all floating nodes at 0v in OP. No output file is generated.

```
set_floating_node 0 -type all -format ic
```

Sets all floating gates to 3.3v throughout the simulation. the PrimeSim XA tool prints an output file with an extension `.fnode0`, containing data in the form of voltage sources.

```
set_floating_node 3.3 -type gate -format vsrc -outfile 1
```

Applies an initial value of 0V to the gate of all n devices and connects pmos gates to a value of 2volt.

```
set_floating_node -val 0 -dev_type n -type gate  
set_floating_node -val 2 -dev_type p -type gate -format vsrc
```

Applies an initial value of 0V to the gate of all n devices and connects pmos gates to a value of 2volt. It also sets an initial value of 0 for all other (non-gate) floating nodes.

```
set_floating_node -val 0 -dev_type n -type gate  
set_floating_node -val 2 -dev_type p -type gate -format vsrc  
set_floating_node -val 0
```

In the following example, there are three devices that are in the same sub-circuit, called inverter:

```
xfloat 99 nflt_gate 55 100 inverter  
xmy_float1 99 nflt_gate1 5 100 inverter  
xmy_float2 99 nflt_gate2 6 100 inverter
```

The following command sets the voltage value of 1 to the floating nodes under `xmy_float1` and `xmy_float2`.

```
set_floating_node -val 1 -format vsrc -dev_type n -pode 1 -type gate \  
-outfile 1 -inst xmy_.
```

set_hotspot_option

Description

Provides more control along with the [check_node_hotspot](#) command.

Syntax

```
set_hotspot_option -factor value
```

Arguments

Argument	Description
<code>-factor value</code>	Specifies the value used in the calculation to determine which nodes are reported by the check_node_hotspot command. The default is 0.5.

Examples

The following example sets up the hot spot check for all nodes in the design. The PrimeSim XA tool reports nodes with the sum of capacitive charging and discharging currents greater than the hot spot factor of 0.1 multiplied by the sum of the node with the largest capacitive current.

```
check_node_hotspot -node *
set_hotspot_option -factor 0.1
```

```
*-----
* Hotspot Check
* node    = *
* factor  = 0.1
*-----
```

Node Name	Cap (fF)	Toggle	Icin (uA)	Icout (uA)
x4.bn	745.6	19	20.08	22.31
b[0]	604.1	19	15.68	14.11
x4.b	519.4	19	14.38	12.94
x4.p	405.9	18	11.49	11.51
x3.bn	745.6	9	8.926	11.16
x4.x1.n4	248.1	19	8.343	9.25
b[1]	604.1	9	7.84	6.272
x4.x2.n3	217.2	18	6.548	6.561
x3.b	519.4	9	7.192	5.75
x3.p	405.9	9	6.248	5.014
x4.k	261.2	15	5.254	5.9
x2.bn	745.6	4	4.463	4.464
x3.x1.n4	248.1	9	3.703	4.625
x3.x2.n3	217.2	9	3.059	3.789
x4.x1.n2	103.5	19	3.537	3.193
x3.k	261.2	9	3.003	3.688
b[2]	604.1	4	3.136	3.136
s[0]	83.6	18	3.057	3.06
x2.b	519.4	4	2.877	2.875
x4.n1	216.1	11	2.957	2.385
x2.p	405.9	4	2.503	2.508
x1.bn	745.6	2	2.231	2.232

```
Total non-input nodes : 81
Total node toggles    : 406
```

```
Total charging current      : 185.9uA
Total discharging current   : 186.1uA
```

set_interactive_option

Enables the dumping of all interactive commands into a separate log file named `inter.log` in the output directory. By default, interactive commands are dumped into the regular log file.

Syntax

```
set_interactive_option -log 1
```

Arguments

Argument	Description
<code>-log 1</code>	Dumps interactive commands to a specific log file.

set_interactive_stop

Enables the node voltage monitoring/checking capability.

Syntax

```
set_interactive_stop -check v(node_name) {v(node_name)}
[-max vmax]
[-min vmin]
[-twindow tstart tstop {tstart tstop}]
[-cmf script_file_name]
[-at interactive_mode_time]
```

Arguments

Argument	Description
<code>-check v(node_name) {v(node_name)}</code>	Specifies the voltage signals to be monitored during transient simulation.
<code>-max vmax</code>	If any monitored signal rises above the upper limit you specify within the specified time window, the PrimeSim XA tool interrupts the transient simulation and enters interactive mode to facilitate debugging.

Argument	Description
<code>-min vmin</code>	If any monitored signal falls below the lower limit you specify within the specified time window, the PrimeSim XA tool interrupts the transient simulation and enters interactive mode to facilitate debugging. Note that for monitored signals you should specify at least one <code>-max</code> or <code>-min</code> argument.
<code>-twindow tstart tstop</code> <code>{tstart tstop}</code>	Specifies the time periods for the signals to be monitored in the simulation.
<code>-at</code> <code>interactive_mode_time</code>	Specifies the time for the PrimeSim XA tool to enter interactive mode.
<code>-cmf script_file_name</code>	Specifies the script file to be run when the PrimeSim XA tool enters interactive mode due to a violation at the upper or lower limits of <code>-max</code> or <code>-min</code> . Note that the commands in the script file must be interactive commands.

Description

If any monitored signal rises above the upper limit or falls below lower limit, the PrimeSim XA tool interrupts the transient simulation and enters interactive mode. You can also use this command to specify a script file to run when entering interactive command due to either an upper/lower limit violation, or specified time with the `-at` argument. The secondary use of this command is to specify a time to re-enter interactive mode using the `-at` argument.

Note:

The `set_interactive_stop` command is not available in a VCS PrimeSim AMS mixed-signal simulation.

Examples

Stops the transient simulation and enters interactive mode at 150us.

```
set_interactive_stop -at 150us
```

Stops the transient simulation and enters interactive mode at 150us and runs the commands in `cmd_file`.

```
set_interactive_stop -at 150us -cmf cmd_file
```

Stops the transient simulation and enters interactive mode when `v(net1)` is greater than 1.2V and runs the commands in `cmd_file`.

```
set_interactive_stop -check v(net1) -max 1.2 -cmf cmd_file
```

set_latch_control

Detects latch nodes to prevent meta-stable states during power-up simulation.

or

Syntax

```
set_latch_control -twindow t0 t1
set_latch_control [-check 1]
```

Arguments

Argument	Description
-twindow t0 t1	Specifies the power ramp-up period when force and release commands are applied to detected latch nodes.
-check enable_value	Creates a .metainfo file with detected latch nodes, related VSRC sources, power node names and the suggested set_latch_control command to be used with the case.

Description

In a power-up simulation, the functional results or accuracy can fail due to meta-stable states of memory cells. Latch nodes can enter meta-stable states and cause larger supply current or unexpected current fluctuation during power ramping up. When power ramp-up is done, all the latches in the memory cells should be initialized with proper states, such as 0v and VDD. If the latch cells enter meta-stable states, a larger current is drawn from the power and can lead to other performance issues.

You can use the set_latch_control command to prevent latch nodes from entering meta-stable states to avoid such problems. This command can report the latch nodes using -check 1, or automatically apply force/release commands to the latch nodes during transient simulation by specifying -twindow t0 t1. Currently the type of latch nodes detected by this command are mostly SRAM memory cell nodes. Some latches with back-to-back inverters used in data buffer or cache could also be detected.

Examples

Latch node voltages are controlled during the power-up period from 1.5ms to 1.7ms.

```
set_latch_control -twindow 1.5m 1.7m
```

Generates a .metainfo file, which lists detected memory cell latch nodes that can potentially lead to meta-stable states during power-up simulation. Only one out of the two back-to-back latch nodes is reported in the .metainfo file. In addition to the latch nodes,

related power nodes are also reported. When the detected power nodes are external PWL VSRC, or DC VSRC, their VSRC definitions are also reported.

```
set_latch_control -check 1
```

If the power nodes are internal, only their names are reported.

The following example shows a `.metainfo` file generated by `set_latch_control -check 1`.

```
FDI metastable nodes stats:
xsram.xcore7.xb7_d75_wl240_241.xi0.q
xsram.xcore6.xb0_d19_wl22_23.xi0.q
xsram.xcore1.xb0_d70_wl0_1.xi1.q
xsram.xcore5.xb0_d65_wl12_13.xi1.q
...
xsram.xcore7.xb7_d28_wl248_249.xi0.q
xsram.xcore6.xb0_d70_wl20_21.xi0.q
Total forced latch nodes=32768
Total released latch nodes=32768
supply_node=vddx_rf(external)
* vvddx_rf Vsrc, pw1=0,0 1.6e-07,0.945 [Time-dependent]
supply_node=xsram.vdd_core(internal)
#cell_latch=32768 #noncell_latch=0
2 supply nodes found driving 65536 metastable latch nodes
Suggested command:
set_latch_control -twindow 0 1.6e-07
```

In this example, `vvddx_rf` is a PWL VSRC defined as `Vvddx_rf vddx_rf 0 pw1 (0 0 1.6e-07 0.945 )`. Therefore you can specify `set_latch_control -twindow 0 1.6e-7` to prevent metastable states during 0n and 1.6e-07s.

set_latch_option

Description

Reports uninitialized latches in the netlist.

Syntax

```
set_latch_option -report_uninit_latch 0|1
```

Arguments

Argument	Description
<code>-report_uninit_latch 0 1</code>	0 (default) does not report uninitialized latches. 1 specifies to report uninitialized latches.

Examples

The uninitialized latch information is stored in an `output.uninitialized_latch` file in the following format.

```
* The following latch nodes are uninitialized
* - Node name in "<MOS_name>:::gate" format is not a real name.
* - It is for representing node (from back-annotation) with no *
*   name only.
"x8.x7.x6.x5.x4_r00.x2_00.n_t0"
"x8.x7.x6.x5.x4_r00.x2_00.n_c0"
...
```

set_logic_threshold

Specifies a default logic high/low threshold value.

Syntax

```
set_logic_threshold -loth low_threshold_value
                   -hith high_threshold_value
                   [-node node_names]
                   [-event_type value]
```

Arguments

Argument	Description
-loth low_threshold_value	Sets the logic low threshold value. The default unit is Volt.
-hith high_threshold_value	Sets the logic high threshold value. The default unit is Volt.
-node node_names	Specifies the node names to apply the threshold value. If this argument is not used, the default is all nodes. A wildcard is supported. You can specify multiple nodes separated by spaces.
-event_type value	Determines when a node generates a digital event. When value is set to 0, a node generates a digital event when its voltage rise above, or falls below, the middle voltage, such as 0.5*(high_voltage + low_voltage). This setting is suitable to simulate most circuits. When value is set to 1 (the default), a node generates a digital event for a node when its voltage rises above high_threshold_value or falls below its low_threshold_value. With this setting, the digital event is generated later than the one with the 0 setting.

Description

The node state is determined by the relationship between its voltage and the high/low logic threshold value:

- If a node voltage is equal to or larger than the *high_threshold_value*, its state is ONE.
- If the node voltage is between the *high_threshold_value* and *low_threshold_value*, its state is undefined (U-state).
- If a percentage sign (%) is applied to the *high_threshold_value* and *low_threshold_value* arguments, a percentage value of the high/low voltage is taken, rather than taking the absolute value.

When the `set_logic_threshold` command is not used, the default *high_threshold_value* and *low_threshold_value* are 70% and 30%, respectively.

set_macro_model_option

Description

Sets sub-circuits as macro models. The sub-circuits that are treated as black boxes, and any of the interior nodes or instances in these sub-circuits will be skipped from waveform probing.

When you run the `set_macro_model_option` command without specifying any option, the command searches for all sub-circuit definitions in `.lib` files that are loaded by `.lib` statements, and then labels them as macro models.

Syntax

```
set_macro_model_option [-file file_name] [-report 0|1] [instance_spec]
```

where

```
[instance_spec] ::= [-inst inst_name {inst_name}] [-subckt subckt_name  
{subckt_name} ] [-except_inst inst_name {inst_name}]  
[-except_subckt subckt_name {subckt_name}]
```

Arguments

Argument	Description
<code>-file file_name</code>	Specifies the netlist library (.lib) files in which the PrimeSim XA tool searches for sub-circuits to be labeled as macro models by tracing through the <code>.include</code> and <code>.lib</code> files reference structures. Wildcard pattern matching is supported. When not specified, the PrimeSim XA tool searches all netlist library (.lib) files for sub-circuits to be labeled as macro models.

Argument	Description
-inst <i>inst_name</i>	Specifies the names of the instances to be labeled as macro models. Wildcard pattern matching is supported.
-subckt <i>ckt_name</i>	Specifies the names of the sub-circuits to be labeled as macro models. Wildcard pattern matching is supported.
-except_inst <i>inst_name</i>	Specifies the names to the instances to be excluded from being labeled as macro models. Wildcard pattern matching is supported.
-except_subckt <i>subckt_name</i>	Specifies the names to the sub-circuits to be excluded from being labeled as macro models. Wildcard pattern matching is supported.
-report 0 1	Chooses to write the labeled macro models to an output file (*.macmodel). The supported values are: 0 (default): Not to write the labeled macro models to the macro model report file. 1: Writes the labeled macro models to the macro model report file.

Examples

The circuit has the following netlist files:

top.sp:

myMOS.lib:

```
.lib mos_typ
.option scale=0.9e-6 shrink_factor=1.0
.lib 'lib1.lib' TT
.lib 'lib1.lib' TT_hvt
.lib 'lib1.lib' TT_lvt
.endl mos_typ
```

lib1.lib:

```
.lib TT
.include pmos.sp
...
.subckt sub1
...
.ends
...
.endl TT

.lib TT_hvt
...
.subckt sub2
...
.ends
...
```

```
.endl TT_hvt  
  
.lib TT_lvt  
...  
.subckt sub3  
...  
.ends  
...  
.endl TT_lvt
```

pmos.sp:

```
...  
.subckt pmos_mac  
...  
.ends
```

nmos.sp:

```
...  
.include "mod.sp"  
.subckt nmos_mac  
...  
.ends
```

mod.sp:

```
...  
.subckt mod  
...  
.ends  
  
...  
.lib 'myMOS.lib' mos_typ  
.include nmos.sp  
...
```

The following command labels all sub-circuits (that is, sub1, sub2, sub3 and pmos_mac) as macro models. No macro model report (.macmodel) file is generated.

```
set_macro_model_option
```

The following command labels sub-circuits prefixed with "sub" as macro models and generates a macro model report file (.macmodel).

```
set_macro_model_option -subckt sub* -report 1
```

The following command excludes the sub-circuits prefixed with "pmos" from macro model identification and generates a macro model report file (.macmodel).

```
set_macro_model_option -except_subckt pmos* -report 1
```

If the nmos.sp netlist file includes sub-circuits nmos_mac and mod, the following command labels only the sub-circuits nmos_mac and mod as macro models, and generates a macro model report file (.macmodel). Sub-circuits (sub1, sub2, sub3 and pmos_mac) are not labeled as macro models.

```
set_macro_model_option -file nmos.sp -report 1
```

Affected Waveform Probing Commands

The PrimeSim XA tool treats macro models as black boxes and skips the device interior nodes or instances (including ports) of the macro models from waveform probing. If one port is kept, the command keeps the sub-currents of this port current for waveform probing.

The following waveform probing commands are affected:

- `probe_waveform_voltage -v -i`
- `probe_waveform_ba -v`
- `probe_waveform_logic -node -n`
- `probe_waveform_current -i - inst -isub`
- `probe_waveform_pcm`
- `probe_waveform_ixba -ipattern -node`
- `probe_waveform_ispf -node -level`
- `probe_waveform_va`

The `set_macro_model_option` command does not work on the Verilog-A modules directly. However, if any VA instances inside the macro model devices are skipped from wildcard matching, the tool skips waveform probing of all the internal nodes and variables of those Verilog-A instances, too.

set_meas_option

Description

Controls the output format of the measurement values computed by the simulation for the .measure statements in the netlist. The tool supports the PrimeSim HSPICE (column based), PrimeSim XA (row based) and CSV formats for measurements for ease of post-process.

The command can also be used to adjust global tolerances and trigger/target behaviors globally, for improving the consistency of the measurement values computed by the simulator.

Syntax

```
set_meas_option [-format hspice|xa|csv]
[-bisect_meas final|all]
[-flush 0|1][
[-rf_vtol vtol_value]
[-targ_td trig]
[-measfile 0|1]
[-name_conflict 0|1]
{-dump_to_log 0|1}
```

Arguments

Argument	Description
-format hspice xa csv	Specifies <code>hspice</code> for *.mt output format (for HSPICE formatting), <code>xa</code> for *.meas output format (for PrimeSim XA formatting), or <code>csv</code> for comma-separated values format (for example, .a#.t#.mt.csv). The default is <code>xa</code> .
-bisect_meas final all	<code>final</code> is the default and means the PrimeSim XA tool only writes the .measure results of the bisection iteration (transient simulation) that meets the goal of the bisection process. <code>all</code> outputs the .measure results for each iteration of the bisection process. This is useful for debugging the bisection process/results.
-flush 0 1	If you set this argument to 1 it flushes the results of measurements in the .meas file as soon as the measurement is completed. The default is 0.
-rf_vtol vtol_value	Sets the rise/fall voltage value tolerance for all the measures that calculate rise/fall/cross events by setting <code>vlg = val - vtol_value</code> , <code>vhg = val + vtol_value</code> . <code>vtol_value</code> can only be a positive value.
-targ_td trig	Specifies to: • Globally set all the <code>trig-targ</code> type measurements to use <code>td=trig</code>. • Override any <code>td</code> setting if there is an existing <code>td=org_td</code>.
-measfile 0 1	When set to 1, dumps all alter measure results into a single file. It is only available with <code>-format hspice</code> or <code>-format csv</code> . The default is 0.
-name_conflict 0 1	Controls the behavior when there is a measurement and parameter name conflict. The supported values are: • 0 (default): Issues a warning without existing the tool when there is a measurement and parameter name conflict. • 1: Issues an error message and existing the tool when there is a measurement and parameter name conflict.
-dump_to_log 0 1	When set to 1, writes the measurement results to the log file. The default is 0.

Examples

The following example generates the measurement into `hspice` format, with the `.mt` extension:

```
set_meas_option -format hspice
```

The following example measures a delay between `clock` and `out` in the 100ns and 500ns time window.

```
.meas tran delay trig v(clock)=0.3 rise=1 targ v(out)=0.3 fall=1  
from=100e-9 to=500e-9
```

If you do not use the `-flush 1` option, all measurement results are dumped into the `.meas` file once the simulation has completed. If you use the `-flush 1` option, the measurement is dumped into the `.meas` file as soon as it is available (at 500ns in this example). The `.meas` files is then updated each time a new measurement result is available.

This feature applies only to the `trig-targ` and `find-when` features of the `.meas` command. The other limitation is that the following syntax is not supported.

```
set_meas_option -format hspice -flush 1
```

set_message_option

Controls the warning messages the PrimeSim XA tool prints.

Syntax

```
set_message_option  
[-limit lim_val]  
[-action warn|stop|exit]  
[-pattern pattern {pattern}]  
[-node pattern {<pattern>}]  
[-except_node pattern {<pattern>}]  
[-instance pattern {<pattern>}]  
[-except_instance pattern {<pattern>}]  
[-subckt pattern {<pattern>}]  
[-except_subckt pattern {<pattern>}]
```

Arguments

Argument	Description
<code>-limit <i>lim_val</i></code>	Limits the maximum number of error messages to print. The default is 10. When you specify 0 is for the message, no warning is printed in the log file.

Argument	Description
<code>-action warn exit stop</code>	Specifies one of the following options: <code>warn</code> (the default) to output warnings and continue the simulation. <code>exit</code> to exit the simulation if a warning message is encountered. <code>stop</code> to pause the simulation and prompt to either continue or exit. If you specify <code>exit</code> or <code>stop</code>, the action takes place on any occurrence of the warnings specified with <code>-pattern</code> or any warning if no <code>-pattern</code> is specified.
<code>-pattern pattern</code> <code>{pattern}</code>	If you specify a pattern, the settings of the current command instance apply only to the specified warning message. If the pattern contains white spaces, it needs to be enclosed with a grouping delimiter, double quotes (") or braces ({}). You can specify multiple patterns.
<code>-node pattern</code> <code>{<pattern>}</code>	Specifies the nodes to report in the log file.
<code>-except_node pattern</code> <code>{<pattern>}</code>	Excludes the specified nodes in the log file.
<code>-instance pattern</code> <code>{<pattern>}</code>	Specifies the instances to report in the log file.
<code>-except_instance pattern</code> <code>{<pattern>}</code>	Excludes the specified the instances in the log file.
<code>-subckt pattern</code> <code>{<pattern>}</code>	Specifies the sub-circuits to report in the log file.
<code>-except_subckt pattern</code> <code>{<pattern>}</code>	Excludes the specified the nodes in the log file.

Description

If you specify only the `-limit` argument, the PrimeSim XA tool prints the specified number of warning messages for every warning type. If you specify both the `-limit` and `-pattern` arguments, the PrimeSim XA tool prints the specified number of messages that match the specified pattern. By default, the messages that do not match the pattern are printed 10 times.

If you specify both the `-pattern` and `-action` arguments, the PrimeSim XA tool applies the action when the specified pattern is matched in a warning message: either continue, exit, or pause the simulation.

Caution:

It is recommended that the `set_message_option` command be used inside the command file included with the `-c` command line option.

Examples

In the following example, when the PrimeSim XA tool finds an ignored option or command, it stops and exits the simulation.

```
set_message_option -pattern "Option/Command ignored" -action exit
```

In the following example, for any warning that contains the "floating" or "not matched" patterns, the PrimeSim XA tool stops and exits the simulation.

```
set_message_option -pattern "floating" -action exit  
set_message_option -pattern "not matched" -action exit
```

In the following example, the PrimeSim XA tool reports up to 100 warnings for the "Option/Command ignored" pattern and stops and exits the simulation if any message contains the "floating" pattern.

```
set_message_option -limit 100 -pattern "Option/Command ignored"  
set_message_option -pattern "Floating" -action exit
```

In the following example, the PrimeSim XA tool stops the simulation if it finds the "forward biased" or "exceeding" patterns. It displays the following message:

```
Message with stop action has been met. Do you want to exit the  
simulation? [y|n]
```

```
set_message_option -pattern "forward biased" "exceeding" -action stop
```

In the following example, the PrimeSim XA tool tabulates and prints up to 50 occurrences of warnings containing the pattern "unsupported" and then exits with an error.

```
set_message_option -limit 50 -pattern "unsupported" -action exit
```

set_model_level

Overrides the automatic choice of table model or equation used for each [set_sim_level](#) command. This command provides the arguments to either choose the type of model used for other [set_sim_level](#) settings or override it with a specific type of model.

Syntax

```
set_model_level  
  -level level | -type model_type  
  [-force 0|1]  
  [-inst inst_name {inst_name}]  
  [-subckt subckt_name {subckt_name}]
```

Arguments

Argument	Description
<code>-level model_level</code>	Specifies an integer value of 1 through 7, which corresponds to a set_sim_level value.
<code>-type model_type</code>	Overrides the automatic model choice with a specific model specified by type. The type can be <code>digital</code> , <code>analog</code> , or <code>equation</code> . If you specify a less accurate type of model than the automatic model choice, the automatic model choice has precedence over your specification (unless you specify <code>-force 1</code>). <code>digital</code> sets both the MOSFET current and charge model to fast table. <code>analog</code> sets both the MOSFET current and charge model to table. <code>equation</code> sets both the MOSFET current and charge model to equation. You cannot use both the <code>-level</code> and <code>-type</code> arguments in the same command.
<code>-force 0 1</code>	This option is only valid when you use <code>-type</code> . If you specify <code>-force 1</code> , PrimeSim XA forces all MOSFETs to use the model type specified with <code>-type</code> .
<code>-inst instance_name {instance_name}</code>	Specifies the instance names.
<code>-subckt subcircuit_name {subcircuit_name}</code>	Specifies subcircuit names.

Description

By default, PrimeSim XA chooses the type of table model or equation used for each [set_sim_level](#) command. The `set_model_level` command overrides the automatic choice of table model or equation used for each [set_sim_level](#) command. It either chooses the type of model used for other [set_sim_level](#) commands or overrides it with a specific type of model.

Examples

The following example overrides the model strategy of level 3 with level 5. All other strategies remain as level 3.

```
set_sim_level -level 3
set_model_level -level 5
```

The following example overrides the model strategy of level 4 to promote digital table to analog (dynamic) table. If some MOSFETs use equation model, they are simulated using equation model.


```
set_sim_level -level 4
set_model_level -type analog
```

The following example overrides the model strategy of level 4 to force all the MOSFETs to use analog (dynamic) table.

```
set_sim_level -level 4
set_model_level -type analog -force 1
```

set_model_option

Controls the type of model, the enhanced table feature, the BSIM4 NQS feature, the grid, MOS binning, and model parameter checking.

Syntax

```
set_model_option [model_spec]
[-grid value]
[-grid_scale scale]
[-grid_lscale lscale_value]
[-charge_conserving value]
[instance_spec]
[-moscap 0|1]
[-bsim3_negcap 1]
```

where

```
model_spec ::= current_spec|charge_spec|stress_param_spec| \
    b4nqs_model|binning_spec|model_param_check_spec
instance_spec ::= [-inst inst_name {inst_name}] \
    [-subckt subckt_name {subckt_name}]

current_spec ::= -current current_model | -i current_model
charge_spec ::= -charge charge_model | -q charge_model \
stress_param_spec ::= -lod lod_value|-wpe wpe_value
b4nqs_model ::= -b4nqs b4nqs_value
binning_spec ::= -mos_bin_ratio tolerance -mos_bin_closest \
    model_name
model_param_check_spec ::= -model_param_check check_value

current_model ::= full|table| ids|fast|none
charge_model ::= full|table| fast|lump|none
```

Arguments

Argument	Description
<code>-current</code> <code>current_model-i</code> <code>current_model</code>	Specify one of the following keywords to override the automatic choice for the current model: <code>full</code> for a MOS equation current model with no optimization. <code>table</code> for a dynamic table built for each current in the MOS model. <code>ids</code> for a dynamic table built only for drain-source current (<code>ids</code>) for a MOS model. <code>fast</code> for a static table used for the current in a MOS model.
<code>-charge charge_model-q</code> <code>charge_model</code>	Specify one of the following keywords to override the automatic choice for the charge model: <code>full</code> for a MOS equation charge model with no optimization. <code>table</code> for a dynamic table built for all the charge components in the MOS model. <code>fast</code> for a static table used for all the charge components in a MOS model. <code>lump</code> for approximate lump capacitors used for a MOS: <code>Cgate</code>, <code>Csource</code>, and <code>Cdrain</code>.
<code>-b4nqs b4nqs_value</code>	Disables or ignores (0) <code>b4nqs</code> effects. Specify 1 (default) to invoke the <code>b4nqs</code> model.
<code>-wpe -lod -0 off</code>	Disables or ignores STI/LOD (for <code>-lod</code>) or WPE (for <code>-wpe</code>) effects.
<code>-wpe -lod 1 on</code>	Uses look-up tables, obtained from device equations, to handle stress effects. This is the default. See the Description section below for more details.
<code>-wpe -lod 2</code>	Uses an enhanced table model method, which uses a single base look-up table, to handle stress effects. See the Description section below for more details.

Argument	Description
<code>-mos_bin_ratio tolerance</code>	Specifies the ratio tolerance for binning the MOS model. It is an integer value between 0 and 500 and must always be followed by a percent sign (%). The default is 0. Post-layout simulations use extracted netlists with final sizes for devices. The final sizes extracted from layout are slightly different from the schematic sizes, so devices with the same size in prelayout (or schematic) netlist have slightly different sizes in the post-layout netlist. The difference is same but results in many devices each having its own size parameters. As a consequence, the PrimeSim XA tool generates a large number of unique models, which can increase the memory usage and run time. The purpose of this option is to create fewer models and improve the memory usage and run time for post-layout simulations. All devices with parameters that differ by less than a given ratio share the same model. All parameters are assumed to have the same importance and the same ratio is used for all parameters.
<code>mos_bin_closest model_name</code>	Enables the use of the closest bin for a device that does not fit any of the given model bins. This feature applies to all device models specified by the <i>model_name</i>, where only the designated models have the feature applied, and all other models do not. You can specify a wildcard character ("*") to apply this feature to all models. For more information about this option, see the Using mos_bin_closest section.
<code>-grid grid_value</code>	Sets the absolute LUT grid size in volts. You can only specify this option globally.
<code>-grid_scale scale</code>	Scales the dynamic table model grid size by the square of the specified <i>scale</i> value. Setting the scale too small can lead to a very small grid size, which can use up a lot of memory and slowdown overall simulation. You can only specify this option globally. The default value is 1.
<code>-grid_lscale lscale_value</code>	Sets the linear scaling of the grid size. If you set the scale too small it might lead to a very small grid size, which can use up a lot of memory and slowdown overall simulation. You can only specify this option globally. The default value is 1. The effective grid size is the <i>lscale_value</i> multiplied by the grid size determined by the PrimeSim XA tool.
<code>-charge_conserving value</code>	Specifies one of the following for charge conserving models. • -1 forces all devices to use non-charge-conserving models. • 0 (default) specifies that the PrimeSim XA tool decides whether to use charge-conserving models for certain devices/ • 1 forces all devices to use charge-conserving models.

Argument	Description
<code>-model_param_check</code> <code>check_value</code>	This option is equivalent to the PrimeSim HSPICE option <code>.option MODPARCHK</code> . Use this option to determine whether a simulation aborts when it encounters fatal errors in model side parameter checking. When set to 1 (default), the simulation aborts if it encounters a fatal error during parameter checking and reports the error. When set to 0, it checks limited model parameters and re-sets them to avoid fatal errors and the simulation runs to conclusion. Note that this option is only available for BSIM4 models.
<code>-inst</code> <code>instance_name{instance_name}</code>	Specifies the instance names.
<code>-subckt</code> <code>subcircuit_name{subcircuit_name}</code>	Specifies the subcircuit names.
<code>-moscap 0 1</code>	When set to 1 optimizes MOSCAP devices.
<code>-bsim3_negcap 1</code>	BSIM3 models can introduce negative capacitances, which can cause a simulation to slow down or fail. Setting the <code>-bsim3_negcap</code> option to 1 adds additional capacitances to the nodes that have negative capacitances.

Description

The PrimeSim XA tool automatically chooses the appropriate current and charge model. You can overwrite this automatic selection and take the following stress effects into consideration:

- Shallow trench isolation (STI), a compact way of separating transistors for submicron geometries.
- Length of oxide definition (LOD), a measure of transistor layout sizing, used to modify device characteristics due to submicron effects.
- Well proximity effect (WPE), a measure of device degradation due to closeness of the transistor well or body to the transistor itself.

By default, the PrimeSim XA tool builds a unique model (either an equation level model or a look-up table model) for each unique transistor geometry. A different model is required for a transistor even if the dimensions are the same as another transistor, differing only in the STI/LOD/WPE value. In this way, the PrimeSim XA tool preserves all STI/LOD/WPE effects.

The PrimeSim XA tool enables turning off the BSIM4 NQS parameter, if TRNQSMOD=1, without modifying the model file, to see if performance is impacted by using this parameter.

The PrimeSim XA tool can also use an enhanced table model method, applied for a given transistor dimension. This model uses derating factors to account for STI/LOD/WPE effects during simulation. The algorithm provides performance and memory capacity advantages with minimal loss of accuracy.

You set the `-lod` and `-wpe` values to `0` or `off` to ignore STI/LOD/WPE effects, to `1` or `on` (default) to use preserve all stress effects, and to `2` to use the enhanced table model method.

The PrimeSim XA tool can also:

- Control binning of MOS devices whose parameters fall within a given percent ratio of one another.
- Automatically choose the appropriate grid and grid scale. You can overwrite this automatic selection.
- Abort or continue when fatal model parameter checking errors occur.

Note:

This command replaces the `use_mos_instpar` and `set_mos_binning` commands.

Using mos_bin_closest

For a MOS device with width (W) and length (L) specified on the instance line, where W and/or L do not fit any of the given model bins, the closest bin is defined by the following, in which Lmin, Lmax, Wmin, and Wmax are the minimum L, maximum L, minimum W, and maximum W, respectively, of a bin as shown in [Table 15](#).

Table 15 Definition of Closest Bin

Condition	Closest Bin
No W bin, L bin exists	Lmin <= L < Lmax, if W > maximum Wmax of Lmin, Lmax bin else W <= minimum Wmin of Lmin, Lmax bin
W bin, no L bin	Wmin <= W < Wmax, if L > maximum Lmax of Wmin, Wmax bin else L <= minimum Lmin of Wmin, Wmax bin
No W bin, no L bin	If W > maximum available Wmax and (L > maximum Lmax of maximum available Wmax bin or L <= minimum Lmin of maximum available Wmax bin) else W <= minimum available Wmin and (L <= minimum Lmin of minimum available Wmin bin or L > maximum Lmax of minimum available Wmin bin)

In the following example [Table 16](#) gives the available bins and [Table 17](#) illustrates the closest bin selection for a given device. size:

Table 16 *Example Bins*

Bin	Lmin	Lmax	Wmin	Wmax
1	2u	3u	50u	60u
1	3u	4u	50u	60u
3	6u	7u	40u	80u
4	6u	7u	80u	90u

Table 17 *Closest Bin Selection Examples*

Condition	Device L	Device W	Closest Bin	Reasoning
No W bin, L bin exists	6u	100u	4	Lmin <= L < Lmax, W > maximum Wmax of Lmin, Lmax bin
No W bin, L bin exists	6u	60u	3	Lmin <= L < Lmax, W <= minimum Wmin of Lmin, Lmax bin
W bin, no L bin	5u	50u	2	Wmin <= W < Wmax, L > maximum Lmax of Wmin, Wmax bin
W bin, no L bin	1u	50u	1	Wmin <= W < Wmax, L <= minimum Lmin of Wmin, Wmax bin
No W bin, no L bin	5u	65u	2	If W > maximum available Wmax and L > maximum Lmax of maximum available Wmax bin
No W bin, no L bin	1u	65u	1	If W > maximum available Wmax and L <= minimum Lmin of maximum available Wmax bin
No W bin, no L bin	1u	40u	1	W <= minimum available Wmin and L <= minimum Lmin of minimum available Wmin bin

Table 17 Closest Bin Selection Examples (Continued)

Condition	Device L	Device W	Closest Bin	Reasoning
No W bin, no L bin	5u	40u	2	W <= minimum available Wmin and L > maximum Lmax of minimum available Wmin bin

If you use `-mos_bin_closest` a warning appears in the log file that the MOS device with its W, L dimensions does not fit any given model bin. Also, the closest bin information (Lmin, Lmax, Wmin, Wmax) used for the respective MOS device is noted in the log file.

Examples

The following example uses the full current model and full charge model.

```
set_model_option -i full -q full
```

The following example turns off the BSIM4 NQS parameter.

```
set_model_option -b4nqs 0
```

The following example uses the enhanced look-up table for all effects.

```
set_model_option -lod 2 -wpe 2
```

The following example uses the mos binning ratio feature set to 10%.

```
set_model_option -mos_bin_ratio 10%
```

The following example uses the grid and grid_scale feature, set to 0.01V and 0.5, respectively.

```
set_model_option -grid 0.01
set_model_option -grid_scale 0.5
```

The following example uses the model parameter checking feature, set to 0 for the simulation to continue with .reset model parameters that are fatal.

```
set_model_option -model_param_check 0
```

The following enables the closest bin selection feature for all models.

```
set_model_option -mos_bin_closest *
```

The following example enables the closest bin selection feature for all moscap* models only. All other models do not use the closest bin selection feature.

```
set_model_option -mos_bin_closest moscap*
```

set_monte_carlo_option

Specifies how to run traditional Monte Carlo analysis for transient simulation.

Syntax

```
set_monte_carlo_option [-enable enable_value]
[-sample_output mc_index]
[-parameter_file enable_value]
[-mc0_file enable_value]
[-mc0_header enable_value]
[-dump_waveform enable_value]
[-datamining on|off]
[-auto_replace enable_value]
[-auto_replace_max_attempt n]
[-compress_file none | "file_ext1{,file_ext2,...}"]
[-dp_parallel_post_process enable_value]
```

where

enable_value ::= 1 | on | yes | true | 0 | off | no | false

Arguments

Argument	Description
-enable enable_value	The default setting for this option is enabled and must be set to process Monte Carlo samples. If this option is disabled and the netlist contains a sweep monte statement, the sweep monte statement is ignored with a warning.
-sample_output mc_index	Specifies the Monte Carlo sample index in which the output files of measurement result files and the waveform files are kept in the output directory. • first keeps the output file from the first Monte Carlo run. • all keeps all the Monte Carlo output files from all runs. • last keeps the output file from last Monte Carlo run. • none specifies not to keep any output files. This is the default. • index [index2 index3 ...] keeps the Monte Carlo output files from the specified sample index. Specify an integer larger than 0. If you want to specify multiple index numbers, separate them with a space. For a list of output files generated when the <code>-sample_output</code> option is specified, see the Output Files From Monte Carlo Analysis section in <i>PrimeSim XA User Guide</i>.
-mc0_file enable_value	When enabled, the tool creates a .mc0 file which lists all of the random variable values used in the simulation, and is equivalent to the PrimeSim HSPICE *.mc0 file. When enabled, the tool creates sample files (.mc0) for independent random variable values.

Argument	Description
<code>-parameter_file enable_value</code>	When enabled, the tool creates a <code>.mc_params</code> parameter file.
<code>-mc0_header enable_value</code>	Sets the option to 1 to print the header information in the output <code>.mc0</code> file.
<code>-dump_waveform enable_value</code>	You must enable this option if you want to write out waveforms when simulation is complete. You can specify one of the following values: • 0 to disable waveform dumping. This is the default. • 1 to write out all signals in the waveform file. • 2 to write out waveforms for the samples that are specified with the <code>-sample_output</code> option.
<code>-datamining on off</code>	Enables Monte Carlo data mining post-processing feature.
<code>-auto_replace enable_value</code>	Specifies whether to automatically replace a failed simulation sample in DP by a new one when running Monte Carlo analysis. • 1: The PrimeSim XA tool continues to replace the failed simulation sample until the target number of samples (specified in <code>.tran</code> statement) has been successfully completed. If the target number of samples cannot be completed after replacement, the PrimeSim XA tool will try a maximum of two times the target number of samples. • 0 (Default): When specified, no auto-replacement is executed. To configure options for failed measurement detection, use the check_measure command. Note: When the <code>-auto_replace</code> option is specified, running Monte Carlo data mining generates reports based on the final successful Monte Carlo sample runs only. For more usage examples, see the Auto-Replacing Monte Carlo Samples When Measurements Fail section in <i>PrimeSim XA User Guide</i>.
<code>-auto_replace_max_attempt n</code>	Sets the maximum number of auto-replacing indexes. By default, the tool uses the number that is two times the sample space specified in <code>.tran</code> statement. After having the expected successful Monte Carlo samples, PrimeSim XA stops the simulation and skips the remaining attempts. To configure options for failed measurement detection, use the check_measure command. For more usage examples, see the Auto-Replacing Monte Carlo Samples When Measurements Fail section in <i>PrimeSim XA User Guide</i>.

Argument	Description
<code>-compress_file none "file_ext1{, file_ext2,...}"</code>	Enables file compression of the specified <i>file_ext</i> files. The default is <code>none</code> .
<code>-dp_parallel_post_process enable_value</code>	Sets the <code>-dp_parallel_post_process</code> option to enable DP parallel post-processing feature. The default is <code>0</code> .

Description

You can run Monte Carlo analysis as an additional check to ensure that your design functions as expected. The PrimeSim XA tool supports only traditional Monte Carlo analysis for transient simulations with the HSPICE netlist format.

For a detailed description on how to run Monte Carlo Analysis in PrimeSim XA, see the Monte Carlo Analysis chapter in *PrimeSim XA User Guide*.

For more information on traditional Monte Carlo analysis, see the “Monte Carlo Analysis—Flow and Output” section in [PrimeSim HSPICE User Guide: Basic Simulation and Analysis](#).

Examples

```
.tran 1n 100n sweep Monte=2
.opt xa_cmd="set_monte_carlo_option -sample_output all"
```

This example creates the following output files:

- `xa.log`
- `xa.mc`
- `xa.meas`
- Files for sample 1 (nominal simulation):
 - `xa.m1.wdf`
 - `xa.m1.meas`
 - `xa.m1.ic`
- Files for sample 2 (first random set):
 - `xa.m2.wdf`
 - `xa.m2.meas`
 - `xa.m2.ic`

```
* XA 64-bit LINUX D-2010.03-20100316 (built 17:04:04 Mar 16 2010)* Build
id: 1762407* Copyright (C) 2010 Synopsys Inc. All rights reserved.
```

```
meas_variable = out
nominal = 0.000000e+00
mean      = 8.228455e-05
varian    = 5.832803e-08
stddev    = 2.415120e-04          avgdev = 1.893783e-04
min       = -1.568056e-04
max       = 6.508503e-04
median    = 3.212345e-04
run_min   = 4
run_max   = 8
run_median = 7

-7.603999e-05, nb =      2, freq = 2.000000e-01 | *****
 4.725601e-06, nb =      4, freq = 4.000000e-01 | *****
 8.549119e-05, nb =      0, freq = 0.000000e+00 |
 1.662568e-04, nb =      2, freq = 2.000000e-01 | *****
 2.470224e-04, nb =      0, freq = 0.000000e+00 |
 3.277879e-04, nb =      1, freq = 1.000000e-01 | *****
 4.085535e-04, nb =      0, freq = 0.000000e+00 |
 4.893191e-04, nb =      0, freq = 0.000000e+00 |
 5.700847e-04, nb =      0, freq = 0.000000e+00 |
 6.508503e-04, nb =      1, freq = 1.000000e-01 | *****
```

The following .mc_csv file contains the same data as the .mc file, except for the ASCII histogram. It has CSV formatting. The first line is a header row:

```
meas_variable,nominal,mean,varian,stddev,avgdev,min,max,...
out,0.000000e+00,8.228455e-05,5.832803e-08,...
```

The following .meas file is the standard PrimeSim XA measurement file. It contains all the per-sample measurement values.

```
** XA 32-bit LINUX D-2010.03-SP1-ENG2 (built 14:07:40 Jul 19 2010)
* Build id: 1833524
* Copyright (C) 2010 Synopsys Inc. All rights reserved.
*

mc_index = 1

rt      = 8.7888150e-12          targ = 1.5616071e-09  \
  trig = 1.5528183e-09
dly     = 3.4741727e-10          targ = 5.7741727e-10
  trig = 2.3000000e-10
temper  = 1.2500000e+02
alter#  = 1.0000000e+00
```

set_multi_core

Runs a multicore simulation.

Syntax

```
set_multi_core [-core num_cores]
               [-check_model value]
               [-check_netlist value]
```

Arguments

Argument	Description
<code>-core num_cores</code>	Specifies the number of cores to use with either a numerical value of 2 or more or the <code>max</code> keyword. The number of cores requested is automatically adjusted to the maximum available in the machine. The <code>max</code> keyword specifies to use maximum number of cores available in the machine.
<code>-check_model value</code>	Lets you check for non-thread-safe and thread-safe models with one of the following values: • 0 does not perform the check (default). • 1 checks to see if thread-safe models are internally tagged as such and causes the PrimeSim XA tool to error out if the models are not tagged properly. • 2 reports internally tagged non-thread-safe models with a warning. • 3 combines 1 and 2.
<code>-check_netlist value</code>	Helps you determine whether a netlist is a good candidate for a multicore simulation. If you enabled a multicore simulation already, this option is ignored. Otherwise one of following three messages is dumped in <code>*.log</code> file: This netlist is likely to benefit from a multicore simulation. This netlist can possibly benefit from a multicore simulation. This netlist is unlikely to benefit from a multicore simulation. • 0 does not perform the check (default). • 1 checks the netlist, dumps the result in the log file, and continues the simulation. • 2 checks the netlist, dumps the result in the log file, and stops the simulation.

Examples

Runs a multicore simulation with four cores.

```
set_multi_core -core 4
```

Reports the multicore check results and stops the simulation.

```
set_multi_core -check_netlist 2
```

Checks if models are internally tagged as thread-safe and errors out if they are not tagged as thread-safe.

```
set_multi_core -check_model 1
```

set_option XA_SHOW_TIME

Description

Defines a user-specified percentage of the transient simulation time at which progress reports are generated.

Syntax

```
set_option XA_SHOW_TIME <value>
```

Arguments

Argument	Description
XA_SHOW_TIME <value>	The percentage of the transient simulation time (between 0 and 1) at which you want to receive progress reports.

Examples

This example sets the simulation to report progress every 1% of the transient time.

```
set_option XA_SHOW_TIME 0.01
```

set_oscillator

Applies the trapezoidal integration method to oscillator circuits.

Syntax

```
set_oscillator -inst inst_name|-subckt subckt_name  
[-disable value]  
[-report report_value]
```

Arguments

Argument	Description
<code>-inst inst_name</code>	Specifies an instance as an LC oscillator.
<code>-subckt subckt_name</code>	Specifies a subcircuit definition as an LC oscillator.
<code>-disable value</code>	Specifies a value of 1 to disable automatic detection of oscillator circuits. This option applies globally. You cannot use it to disable detection of instances or subcircuits. The default is 0.
<code>-report report_value</code>	Specifies a value of 1 to output additional log information. When automatic detection is on, all detected elements are listed in the log. The default is 0.

Description

Trapezoidal integration is often the best integration method for LC oscillator circuits but can apply to other types of oscillators as well. If the characteristic frequency on an inductive network is between 0.5Hz and 800 MHz, an oscillator circuit is often identified automatically. Because the capacitors in an LC oscillator have a critical impact on the oscillation frequency, this command causes conservative treatment of the capacitors in the oscillator.

If the PrimeSim XA tool fails to identify an oscillator automatically, you can use the `set_oscillator` command to manually specify the trapezoidal integration method. Trapezoidal integration is also applied to the regions (partitions) that contain the designated oscillators. You can also use this command for other circuit elements that require trapezoidal integration.

In a hierarchical netlist, apply trapezoidal integration to the inductor of the oscillator or the lowest-level subcircuit that contains the oscillator. In a flat netlist, apply it to the inductor.

Note:

If you do not specify `-subckt` or `-inst`, the PrimeSim XA tool issues a warning and does not apply trapezoidal integration to any instance. If automatic detection is enabled, the PrimeSim XA tool still applies trapezoidal integration to any detected oscillators.

Examples

The following example applies the trapezoidal integration method to an inductor instance.

```
set_oscillator -inst x1.xosc.L1
```

The following example applies the trapezoidal integration method to an oscillator instance.

```
set_oscillator -inst x1.xxtal
```

The following example applies the trapezoidal integration method to all instances of a subcircuit.

```
set_oscillator -subckt crystal
```

The following example disables automatic detection of LC oscillator circuits.

```
set_oscillator -disable 1
```

The following example prints additional log information.

```
set_oscillator -report 1
```

The following example disables automatic detection and applies the trapezoidal integration method to the specified instance.

```
set_oscillator -inst xxtal -disable 1
```

set_parameter_value

Description

Lets the PrimeSim XA tool change the value of a specified parameter for selected instances without modifying the netlist. This command supports device parameters of compact and macro models.

Syntax

```
set_parameter_value -name param_name  
-value param_value  
-model model_name  
[-inst inst_name {inst_name}] |  
[-subckt subckt_name {subckt_name}]
```

Arguments

Argument	Description
-name param_name	Specifies the macro model parameter name to apply the value to be set.
-value param_value	Specifies the parameter value of the macro model.
-model model_name	Specifies the name of the macro model.
-inst inst_name {inst_name}	Selects instance names of the macro model to set the parameter value. You can use wildcard characters in the inst_name.
-subckt subckt_name {subckt_name}	Selects instances of the macro model inside the specified subcircuit to set the parameter value.

Examples

The following example changes the `delvto` value for all devices with `nch` model inside `x1` to 0.5.

```
set_parameter_value -name delvto -value 0.5 -model nch x1.*
```

The following example changes the `state` value for all devices with `pcm` model to 1.

```
set_parameter_value -name state -value 1 -model pcm
```

The following example changes the `w` value of the `xr0.m0` instance of the `p0` model to 1000.

```
set_parameter_value -name w -value 1000 -subckt mysub
.model p0 pmos
.subckt mysub 1 2 3 val=300
rcp 1 2 r=val
m0 0 1 2 3 p0
.ends mysub
xr0 0 1 2 mysub
```

set_partition_option

Overrides the automatic setting for circuit partitioning.

Syntax

```
set_partition_option -sp enable_value
[-ap enable_value]
[-print_power enable_value]
[[-rule] gate|channel|never]]
[-inst inst_name {inst_name}]
[-subckt subckt_name {subckt_name }]
[-footer_switch 0|1]
```

Arguments

Argument	Description
-sp enable_value	Turns static partitioning on (1, the default) or off (0). Static partitioning is not instance specific and does not affect the idealized power-net block region.
-ap enable_value	Turns advanced partitioning on (1, the default) or off (0). Advanced partitioning is not instance specific.
-print_power enable_value	Writes a list of power cut nodes to the <code>.cut</code> file.

Argument	Description
<code>-rule gate never</code>	Allows partitioning of MOSFETs at the <code>gate</code> (high-impedance) terminal. The node touching the gate of the MOSFET is in a different partition than the source/drain terminals. If it is set to a Verilog-A instance, it allows partitioning of the ports of Verilog-A instance. The nodes touching the high-impedance ports of the Verilog-A instance are in different partitions, but the Verilog-A instance is still in one partition. Note that this option only allows the device to be partitioned, but does not guarantee it. The partitioning rules of surrounding devices can cause the device to remain un-partitioned in spite of the <code>gate</code> rule being applied to it. Specify <code>channel</code> to allow partitioning of MOSFETs at the channel (low-impedance) terminal. <code>never</code> turns off the partitioning. The nodes touching the MOSFET are in one partition.
<code>-inst inst_name {inst_name}</code>	Applies the partition rule defined by <code>-rule</code> to the specified instances.
<code>-subckt subckt_name {subckt_name }</code>	Applies the partitioning rule defined by <code>-rule</code> to the specified subcircuits.
<code>-footer_switch 0 1</code>	If your circuit has footer switches use the <code>-footer_switch 1</code> option to ensure that the footer switch is properly recognized. The default is 0.

Description

You can use the `set_partition_option` command to enable/disable the PrimeSim XA tool partitioning algorithms. The PrimeSim XA tool has two stages of partitioning: static partitioning and advanced partitioning. The static partitioning algorithm uses mature circuit heuristics to perform partitioning. The advanced partitioning algorithm uses more aggressive partitioning methods to try to further reduce partition sizes.

If `-sp 0` is set all partitioning is disabled and the `-ap` setting is ignored. You can disable the advanced partitioning with `-ap 0` to isolate partitioning issues or to assist in tightening simulation accuracy at a cost of simulation run time. Disabling static partitioning disables all circuit partitioning in the PrimeSim XA tool. This is not recommended for large circuits because of the prohibitive simulation runtime.

Examples

```
set_partition_option -ap 0
```

set_postlayout_meas

Checks all the back-annotated signals used in the `.meas` statements and reports the earliest or latest signals.

Syntax

```
set_postlayout_meas mname
-trig [early|late] | [min|max]
-targ [early|late] | [min|max]
```

Arguments

Argument	Description
<i>mname</i>	Specifies the name of the measurement as specified in the <code>.meas</code> statement. You can use wildcard characters in the name.
<code>-trig early late</code>	When you specify <code>-trig early</code>, the PrimeSim XA tool picks the first signal reaching <code>val=<value></code> among all the SPF signals of the <code>trig</code> net within the time window defined by the <code>.meas</code> statement. When you specify <code>-trig late</code>, the PrimeSim XA tool picks the last signal reaching <code>val=<value></code> among all the SPF signals of the <code>trig</code> net within the time window defined by the <code>.meas</code> statement. You must specify either the <code>-trig</code> or <code>-targ</code> argument.
<code>-trig min max</code>	When you specify <code>-trig min</code>, the PrimeSim XA tool picks the smallest of all rise/fall times among all the SPF signals of the <code>trig</code> net within the time window defined by the <code>.meas</code> statement. When you specify <code>-trig max</code>, the PrimeSim XA tool picks the largest of all rise/fall times among all the SPF signals of the <code>trig</code> net within the time window defined by the <code>.meas</code> statement.
<code>-targ early late</code>	When you specify <code>-targ early</code>, the PrimeSim XA tool picks the first signal reaching <code>val=<value></code> among all the SPF signals of the <code>targ</code> net within the time window defined by the <code>.meas</code> statement. When you specify <code>-targ late</code>, the PrimeSim XA tool picks the last signal reaching <code>val=<value></code> among all the SPF signals of the <code>targ</code> net within the time window defined by the <code>.meas</code> command.

Argument	Description
-targ min max	When you specify -targ min, the PrimeSim XA tool picks the smallest of all rise/fall times among all the SPF signals of the targ net within the time window defined by the .meas statement. When you specify -targ max, the PrimeSim XA tool picks the largest of all rise/fall times among all the SPF signals of the targ net within the time window defined by the .meas statement.

Description

The `set_postlayout_meas` command is a postlayout processing command. It applies only to PrimeSim XA simulations with DSPF back-annotation using the [load_ba_file](#) command. It checks all the back-annotated signals used in the `.meas` statements and reports the earliest or latest signals, depending on how you specify the `-trig` and `-targ` arguments. When you only specify `-trig`, the same signal (same device port) is used for the `targ` signal. When you only specify `-targ`, the same signal (same device port) is used for `trig` signal.

The `set_postlayout_meas` command picks either the first arriving signal among all net signals or the last arriving signal among all net signals. In the case of rise/fall time measurement, `trig` and `targ` are the same signal.

In addition, the following rules apply to `set_postlayout_meas`.

- The measurements affected by `set_postlayout_meas` must be at the top level. They are not honored if the measurements are embedded in subcircuits.
- Only `.measure (delay, rise and fall)` is supported. The `set_postlayout_meas` does not support any other `.measure` commands. Standard HSPICE `.MEAS` syntax is supported.
- The back-annotation net instance pins (lines starting with `*|I` in the DSPF file) are automatically dumped in the waveform file when `set_postlayout_meas` is used. Only `*|I` pins are considered. The `*|S` and `*|P` pins are ignored.
- You cannot apply multiple `set_postlayout_meas` commands to the same `.meas` statement. If you want to apply different time windows or different `trig/targ` values, then you must define multiple `.meas` statements. If multiple `set_postlayout_meas` commands apply to the same measurement name, then a warning is issued and the last one is used.

Other Affected Commands

If you use the `set_postlayout_meas` command together with the [meas_post](#) command, and if you save the postlayout signals with `probe_waveform_voltage -ba_net`, the

`set_postlayout_meas` command applies to the `.meas` commands. For example, suppose you run a simulation with `probe_waveform_voltage -ba_net A*` and after the simulation completes you want to perform measurements. You can write a SPICE file with the measurements:

```
.meas tran Delay1 trig v(A1) val=1 rise=1 targ v(A2) fall=1 val=1
.meas tran Delay2 trig v(B1) val=1 rise=2 targ v(B2) fall=2 val=1
```

And write a configuration file with:

```
meas_post -waveform xa.fsdb
set_postlayout_meas Delay1 -trig early -targ late
set_postlayout_meas Delay2 -trig early -targ late
```

Both the `delay1` and `delay2` measurements are honored, but only the first one (`Delay1`) uses the `set_postlayout_meas` options, because only the `A*` back-annotated nets are saved. The `B1` and `B2` back-annotated signal nets are not saved with `probe_waveform_voltage -ba_net`, so the `Delay2` measurement is done as usual. The `set_postlayout_meas Delay2` command is ignored.

The PrimeSim XA tool supports the Eldo `.EXTRACT` commands and generates a `.meas` file identical to the `.meas` file generated by the HSPICE `.meas` command. So the `set_postlayout_meas` command applies to the results of the `.EXTRACT` measurement. For example, if the following Eldo command is in the netlist:

```
.EXTRACT label=del1 tpddd(v(in), v(out), vth=1
```

After the simulation completes, a `.meas` file is created with the delay between `v(in)` falling and `v(out)` falling with `val=1V`. If you specify the following configuration file command:

```
set_postlayout_meas del1 -trig early -targ late
```

The measurement uses the early slope for the `trig` signal (`v(in)`) and the late slope for the `targ` signal (`v(out)`).

Examples

In the case of rise/fall (also called slope or transition) time measurement, `trig` and `targ` are the same signal, so the `-trig` and `-targ` arguments are optional. The `-trig` and `-targ` values can be `min` or `max`. For example, the following `.meas` statement measures the fall time (it's fall because `trig` value is greater than `targ` value) of signal `SUM0`.

```
.meas tran fall1 trig v(SUM0) val='0.9*vdd' fall=1 from=1n to=2n targ
v(SUM0) val='0.1*vdd' fall=1 from=1n to=2n
```

And uses this DSPF file:

```
*|NET SUM0 0.0134922PF
*|P (SUM0 0 1e-14 -459.55 80.95)
*|I (x0/x33/M2:SRC x0/x33/M2 SRC B 0 -462.5 36.25)
*|I (x0/x33/M1:SRC x0/x33/M1 SRC B 0 -462.5 11)
```

The PrimeSim XA tool measures all fall times for SUM0, that is:

Fall time for `x0/x33/M2:SRC` from 0.9vdd to 0.1vdd.

Fall time for `x0/x33/M1:SRC` from 0.9vdd to 0.1vdd.

The PrimeSim XA tool then compares all the results. If `set_postlayout_meas -trig max` is used, then the measurement reports the `max` value. If `set_postlayout_meas -trig min` is used, then the measurement reports the `min` value.

The following `.meas` statement in a netlist applies to a delay measurement:

```
.meas tran D1 trig v(X1.z) val=vdd/2 rise=1 from= 1n to= 2n targ v(X2.c)
fall=2 val=vdd/2 late from= 3n to= 4n
```

And the configuration file contains:

```
set_postlayout_meas D1 -trig early -targ late
```

To determine the triggering signal, the PrimeSim XA tool checks all SPF signals related to `X1.z`. It picks the first rising signal reaching `vdd/2` between 1ns and 2ns. To determine the target signal, the PrimeSim XA tool checks all SPF signals related to `X2.c`. It picks the last falling signal reaching `vdd/2` between 3ns and 4ns. Note that the PrimeSim XA tool ignores the first occurrences of `X2.c` signals falling, it picks the second occurrence of all `X2.c` signals falling (because of `fall=2`).

The `.meas0` file shows:

```
...D1 = 1.8657638e-09 targ = 1.8657638e-09 trig = 0.0000000e+00
targ_signal=x2.c::x1.x3.mn2:gate trig_signal=x1.z::x1.x3.mn1:drain
Temper = 2.5000000e+01
alter# = 1.0000000e+00
```

set_powernet_ccap_option

Description

Converts all the coupling capacitors touching the power net to two ground capacitors if the value of the coupling capacitance is below the specified value. The value of each ground capacitor is equal to the value of the coupling capacitor.

Syntax

```
set_powernet_ccap_option -ccap_to_gcap_threshold cap_value
```

Arguments

Argument	Description
- ccap_to_gcap_thresh old cap_value	Converts coupling capacitors touching power nets to two ground capacitors if coupling capacitance is below the <i>cap_value</i> .

Examples

This following example lumps all coupling capacitors less than 1e-15 to ground if any of the terminals of the coupling capacitor touch the power net. Each ground capacitor has a value equivalent to the coupling capacitor.

```
set_powernet_ccap_option -ccap_to_gcap_threshold 1e-15
```

set_powernet_level

Controls the performance/accuracy tradeoff for IR drop simulation.

Syntax

```
set_powernet_level -level level_value [-node node_name {node_name}]
```

Arguments

Argument	Description
-level level_value	Specifies the performance/accuracy tradeoff level for IR drop simulation. The default is the level you specify with the <i>set_sim_level</i> command. Levels 1 through 7 are valid. If you do not specify a <i>set_sim_level</i> command, the default value is 3.
-node node_name {node_name}	Defines the node names to which the power network accuracy level is applied. It can be the node name of a single node or a node name with the asterisk (*) wildcard character that represents a group of node names. The behavior of asterisk (*) character is controlled by <i>set_wildcard_rule</i> .

Description

The higher the *set_powernet_level*, the more parasitic elements are taken into consideration for IR drop simulation. If the resistor connected to the power supply is below the predetermined value for each *set_sim_level*, the resistor is not taken into consideration for IR drop simulation.

The PrimeSim XA tool automatically adjusts the resistance value based on the simulation strategy. You can override the automatically chosen idealized power-net strategy to meet your accuracy requirements, especially for IR-drop simulations. The smaller the value, the better the accuracy for IR-drop simulations.

The main usage for the `set_powernet_level` command is to use it along with `set_sim_level` to speed up the IR drop simulation. Set the `set_sim_level` command based on the circuit types (see [Table 18](#)) and use a higher `set_powernet_level` to override the pre-determined resistance value of `set_sim_level`. Otherwise, the resistance value of `set_powernet_level` by default matches the levels by `set_sim_level`. For a more detailed IR drop simulation, set a higher `set_powernet_level` to take more parasitic resistors connected to an idealized power-net block for simulation.

For a detailed description about the `set_sim_level` command options, see the [set_sim_level](#) command page.

Table 18 Guidelines for Appropriate Circuit Types for Each `set_sim_level`

Level	Digital	Memory	Low-Sensitivity Analog	High-Sensitivity Analog	Mixed-Signal	Full-Chip
7	X		X	X	X	
6	X	X	X	X	X	X
5	X	X	X	X	X	X
4	X	X	X	X	X	X
3	X	X	X		X	X

Examples

In the following example, the PrimeSim XA tool overrides only the idealized power-net resistor tolerances used in `set_sim_level 3` with the tolerances used in `set_sim_level 6`. All resistors connected to power supply with values larger or equal to 1 ohm are taken into the simulation. `set_sim_level 3` is suitable for IR drop simulation for digital, memory or low-sensitivity analog designs.

```
set_sim_level 3
set_powernet_level 6
```

set_powernet_option

Controls how resistors are partitioned into the idealized power-net block.

Syntax

```
set_powernet_option -ideal_rmax res_value
[-collapse_node node {node}]
[-except_node node_name {node_name}]
[-report enable_value]
[-supply_node node_name {node_name}]
```

Arguments

Argument	Description
res_value	Specifies the maximum resistance value of the resistor to be partitioned into the idealized power net block.
-collapse_node node {node}	Collapses the resistor network to the specified node name.
-except_node node_name {node_name}	Specifies a list of nodes whose resistive networks should not be identified as power nets. You can use wildcard characters in the node names.
-report enable_value	Generates a table of identified power networks in the log file.
-supply_node node_name {node_name}	Specifies a list of nodes to be added as power nets in addition to the power nets automatically identified by the PrimeSim XA tool. This argument is processed after the -except_node argument.

Description

This command controls how resistors are partitioned into the idealized power-net block. Power-net elements and nodes connected to ground through low-impedance elements (such as voltage sources and resistors smaller than the specified value) are put into the idealized power-net block. [Table 19](#) shows the defaults used at the different accuracy levels.

Table 19 Default -ideal_rmax res_value Values

Accuracy Level	Default -ideal_rmax res_value values
set_sim_level 3	10 ohms
set_sim_level 4	7.5 ohms
set_sim_level 5	5 ohms
set_sim_level 6	1 ohms
set_sim_level 7	1e-5 ohms

In some cases the PrimeSim XA tool might automatically adjust the resistance value based on the simulation strategy. To meet your accuracy requirements, especially for IR-drop simulations, you can override the automatically chosen *res_value* by using the `-ideal_rmax res_value` option. The smaller the *res_value* the better the accuracy for ID-drop simulations.

Examples

The following example only identifies VSS and VDD* as power nets.

```
set_powernet_option -except_node *  
set_powernet_option -supply_node VSS VDD*
```

set_probe_option

Controls probing options.

Syntax

```
set_probe_option  
[-probe_undefined_node enable_value]  
[-probe_rc 1|2]  
[-variable_separator separator_char]  
[-netlist_probe_control control_value]  
[-skip_flat_pl_node control_value]  
{-ba_net_probe_sync 0|1}  
[-ba_net_probe_gate 0|1]  
[-ba_net_probe_skip_finger 0|1]  
[-dump_waveform_file 0|1]  
[-ba_update_xprobe 0|1]  
[-compat 0|1|2|3|4]  
[-signame_mode 0|1|2|3]
```

Arguments

Argument	Description
<code>-probe_undefined_node enable_value</code>	When enabled, a probe of an undefined signal writes a constant signal with a value of 0 to the plot file. This option is recognized only in the Eldo netlist format.
<code>-probe_rc 1 2</code>	Controls the wildcard expansion used in probing. 1: Default. 2: Excludes simple RC elements from <code>.probe i(*)</code> for simulation performance.

Argument	Description
<code>-variable_separator separator_char</code>	You use this option to change the separator between the hierarchical path and the Verilog-A variable or parameter name in the plot file to a colon (:). The default separator is the regular hierarchical delimiter (.) or the delimiter set by the <code>set_sim_hierid</code> command.
<code>-netlist_probe_control control_value</code>	Controls whether the PrimeSim XA tool honors the netlist probe statements (<code>v()</code> , <code>i()</code> , and <code>x()</code>). Specify the option with one of the following values: • 0 (default): Honors netlist probe statements (<code>.probe/.print/.lprint/probe_waveform_abc</code>). • 1: Ignores all the netlist probe statements and any other commands that trigger probing. • 2: Ignores all the netlist probe statements and honors commands that potentially trigger probing (<code>.measure/vec/vcd</code>). • 3: Ignores current probes in the netlist. • 4: Ignores voltage probes in the netlist. • 5: Honors all the netlist probe statements and ignores commands that potentially trigger probing (<code>.measure/vec/vcd</code>).
<code>-skip_flat_pl_node control_value</code>	Controls whether the PrimeSim XA tool ignores the postlayout nodes for wildcard probing. You can specify one of the following values: • 0 (default): Does not skip any flattened postlayout nodes for wildcard probing. This is consistent with the current PrimeSim XA tool usage. • 1 to skip all flattened postlayout nodes for wildcard probing (name with “.”). • 2: Skips only internal nodes (ending with <code>{number}</code>) but keeps others, such as instance pins, for wildcard probing. • 3: Does not skip any flattened postlayout nodes (same as 0). It also treats all wildcard nodes as explicit nodes to prevent optimizing these nodes. This option is recommended only for debugging purposes.
<code>-ba_net_probe_sync 0 1</code>	When set to 1, back-annotated net instance pin voltages from SPF or SPEF file are probed for all nets that have voltage probes defined from <code>.probe</code> and <code>probe_waveform_voltage</code> commands. The exception options for <code>.probe</code> and <code>probe_waveform_voltage</code> are also honored. All <code>-ba_net {net_names}</code> statements from <code>probe_waveform_voltage -ba_net {net_names}</code> are ignored. The default is 0.

Argument	Description
<code>-ba_net_probe_gate 0 1</code>	When set to 1, only gate pins from (* I) are probed from the SPF or SPEF file. Note that this option only takes effect if <code>-ba_net_probe_sync 1</code> is set. Otherwise this option is ignored and a warning is printed in the log file. The default is 0.
<code>-ba_net_probe_skip_finger 0 1</code>	When set to 1, the finger device pins from (* I) that contain @ are not probed from the SPF or SPEF file. Note that this option only takes effect if <code>-ba_net_probe_sync 1</code> is set. It is ignored otherwise and a warning is printed in the log file. The default is 0.
<code>-dump_waveform_file 0 1</code>	When set to 0, this argument disables the generation of an output waveform file. All other aspects of the simulation, including probe commands and measure statements, run as specified and are not affected. The default is 1, meaning an output waveform file will be generated.
<code>-ba_update_xprobe 0 1</code>	When set to 1, this argument enables ISUB/X probing commands to include the current through parasitic RC elements in the back-annotation flow so the port current into a subcircuit becomes accurate in back-annotation. The default is 0, which accounts only for MOS device currents.

Argument	Description
<code>-compat 0 1 2 3 4</code>	Specifies the criteria for probe filtering. Explicit probing names are not filtered. The available values are: • 0 (default): No filtering is performed. • 1: Filters voltage, current and ISUB probes that match wildcard patterns for models and subcircuit devices in the foundry .lib file. ◦ Device terminal current probes coming from the .model or .subckt in the foundry's Process Design Kits (PDKs) are not automatically expanded for wildcard matching. ◦ Wildcard probe levels are not expanded to devices that are added or manipulated by the foundry *.so files. ◦ Inline subcircuit port currents and internal devices are not expanded wildcard matching. ◦ Internal RC currents and nodes are filtered, as long as they do not touch the subcircuit or the instance port. • 2: In addition to the filtering criteria from <code>-compat 1</code>, counts subcircuit port names or device terminal names at the hierarchical level as specified in the <code>save</code> statement. • 3: In addition to the filtering criteria from <code>-compat 2</code>, filters voltage and current probes on the following ports: netlist capacitance device ports, netlist MOS device ports, and lower-level ports that eventually connect to VDD and VSS. When used with the <code>set_circuit_transceiver</code> command, the <code>-compat</code> option is set to 3 by default. • 4: In addition to the filtering criteria from <code>-compat 2</code>, filters voltage and current probes on the lower-level ports that eventually connect to VSS.
<code>-signame_mode 0 1 2 3</code>	Sets the waveform signal label name mode. • 0 (default in simulator legacy run): Honors the original netlist format with simulator legacy syntax. For either HSPICE netlist or 3rd party netlist, each simulator dumps its legacy syntax. • 1 (default in PrimeSim Continuum run) : Honors the original netlist format with simulator common syntax. For either HSPICE netlist or 3rd party netlist, all simulators dump common syntax. • 2: Forces SPICE format common syntax. For either HSPICE netlist or 3rd party netlist, all dump HSPICE syntax. • 3: Forces third-party simulator non-SPICE format common syntax. For either HSPICE netlist or 3rd party netlist, all dump 3rd party syntax.

Description

When you specify the `-probe_undefined_node` option, the PrimeSim XA tool creates a signal in the plot file with a constant value of 0 V for any explicitly probed node. This feature is available only for Eldo syntax netlists.

The ISUB/X probing command is often used to probe the port current in a subcircuit. When you probe the port current of a subcircuit block after back-annotation, the ISUB/X probing command gathers the current of MOS devices in the SPF netlist. So the port current is not the same as the power source current that is directly connected to the port. This is due to the current of parasitic RC elements that are not considered in ISUB/X probing commands when you back-annotate the SPF netlist into the subcircuit block.

When you specify the `-ba_update_xprobe` option, the PrimeSim XA tool includes the current through parasitic RC elements for the ISUB/X probing command, so the port current into the subcircuit becomes accurate. The `-ba_update_xprobe` option is applicable only to the back-annotation flow.

Example 1

The following example causes probes of undefined signals to be written to the waveform file with a constant value of 0 (zero).

```
set_probe_option -probe_undefined_node 1
```

Example 2

The following example causes the probes of Verilog-A module variables and parameters to use a colon separator between the hierarchical path and the variable/parameter name.

```
probe_waveform_va -var x1.x2.va_var_name  
set_probe_option -variable_separator :
```

The signal name in the waveform file is `x1.x2:va_var_name`.

Example 3

The following example shows how `set_sim_hierid`, `probe_waveform_va` and `-variable_separator` interact.

```
set_sim_hierid -hierid / #Sets delimiter to foreslash  
probe_waveform_va x1/x2/xva_module/va_var #Must use foreslash delimiter  
set_probe_option -variable_separator : # Use colon as variable separator  
# in plot file  
  
#The wdf output is:  
#x1/x2/xva_module:va_Va
```

Example 4

The following example ignores all probe statements and other statements that may trigger probing.

```
set_probe_option -netlist_probe_control 1
```

Example 5

The following example explains the `-compat` option setting with the `set_probe_option` command for pattern matching.

Assume the `save` statement is defined as:

```
save X*:currents depth=2 sigtype=all
```

If `-compat 1` or `-compat 0` is specified, the following names are matched because the instance `XMissing.xfilter1` has two hierarchical levels and meets the criteria of `depth=2`:

- `XMissing.xfilter1:n1`
- `XMissing.xfilter1:n2`

If `-compat 2` is specified, the above names are not matched because the tool treats ports “n1” and “n2” to be at one hierarchical level. The setting of `depth=2` directs the tool to go two hierarchical levels down for pattern matching, which means only the name `XMissing:n1` is matched, not `XMissing.xfilter1:n1`. In other words, both “.” and “:” symbols are recognized as hierarchical delimiters when `-compat 2` is specified.

set_probe_window

Defines a printing window that reduces the waveform output file size.

where:

Syntax

```
set_probe_window [-split] window_spec [-period period_time]
window_spec ::= [-window] {window_start_value window_end_value} \
               window_start_value [time_end]

time_end ::= time_end_value | end
```

Arguments

Argument	Description
<code>-split</code>	Specifies to split and save the output waveform file in separate waveform files. The default is not to split waveform files as separated outputs.
<code>window_start_value</code>	Window start time.
<code>window_end_value</code>	Window end time.

Argument	Description
<code>-period <i>period_time</i></code>	Specifies a waveform window with a sampling period. If you specify an odd number for the window time, the <i>period_time</i> is ignored. If you specify an even number for the window time, the <i>period_time</i> is applied to the last window.

Description

This command affects only the output waveform file. It does not have any effect on any measure/extract or any other diagnostic command. If you specify multiple `set_probe_window` commands, the last one takes precedence. Multiple commands do not cause print windows to accumulate.

If the final *time_end* value is not specified it is assumed to be end. If multiple windows are specified, they cannot overlap. The window end time must be greater than the start time. The window times must be greater than 0.

The PrimeSim XA tool supports the `tstart` option of HSPICE, Eldo, and TI-SPICE, as well as the `outputstart` option of Spectre. `set_probe_window` can be used to set the print window.

Examples

Prints from 100ns to 300ns and 400ns to 1us. The output waveform files are saved into two files. One is *output_filename_split1.waveform file format* for window from 100ns to 300ns and the other is *output_filename_split2.waveform file format* for window from 400ns to 1us, respectively.

```
set_probe_window -split -window 100n 300n 400n 1u
```

Prints from 100ns to 300ns and 400ns to 1us. The output waveform files are saved into two files. One is *output_filename_split1.waveform file format* for the window from 100ns to 300ns and the other is *output_filename_split2.waveform file format* for the window from 400ns to 1us respectively.

```
set_probe_window -split 100n 300n 400n 1u
```

Prints from 100ns to 300ns and 400ns to 1us. The output waveform file is saved in one file.

```
set_probe_window -window 100n 300n 400n 1u
```

Prints from 100ns to 300ns and 400ns to 1us. The output waveform file is saved in one file.

```
set_probe_window 100n 300n 400n 1u
```

Prints from 100ns to 300ns and 400ns to the end of simulation time. The output waveform file is saved in one file.

```
set_probe_window 100n 300n 400n
```

Prints from 100ns to 300ns and 400ns to the end of simulation time. The output waveform file is saved in one file.

```
set_probe_window 100n 300n 400n end
```

Prints from 100ns to 300ns, 400ns to 450ns, 700ns to 750ns, 1000ns to 1050ns, 1300ns to 1350ns, and so on. The output waveform file is saved in one file.

```
set_probe_window -window 100n 300n 400n 450n -period 300n
```

set_ra_functional_resistor

Specifies the model name for the design resistors to be processed by the electromigration and voltage drop (EMIR) analysis flow.

Syntax

```
set_ra_functional_resistor -res_model model_name  
[-layer layer_name]  
[-map_w width_name_in_netlist]  
[-map_l length_name_in_netlist]  
[-port_names port1_name port2_name]
```

Arguments

Argument	Description
-res_model <i>model_name</i>	Specifies either a regular model or macro model. All design resistors that match the <i>model_name</i> are processed for reliability analysis.
-layer <i>layer_name</i>	Assigns a layer name for the design resistors. The default is to use the -res_model name as the layer name. All design resistors are assumed to have the same layer name.
-map_w <i>width_name_in_netlist</i>	Defines the instance parameter for the resistor width. The default name is <i>w</i> .
-map_l <i>length_name_in_netlist</i>	Defines the instance parameter for the resistor length. The default name is <i>l</i> .

Argument	Description
<code>-port_names port1_name port2_name</code>	By default the tool automatically takes the first two functional resistor terminals/ports to calculate the correct resistor size. You can specify this argument to define specific port names that take precedence over automatic port index recognition.

Description

Once you specify the model name of the resistors, the electromigration and voltage drop (EMIR) flow automatically creates a virtual SPF net to store the design resistors, generate the current through those resistors, and propagate the current information to Phase II simulation.

The following rules apply to the `set_ra_functional_resistor` command.

- You can specify only one model name per command.
- You can use one or more `set_ra_functional_resistor` commands to identify design resistors with a different model name.
- PrimeSim EMIR analysis requires a back-annotation setup. For EM analysis on design resistors, the same requirement applies.

Tcl procedure requirements for functional resistors are the same as for metals and vias. The only difference is that the resistor length is accessed through the RATCL variable `hs_rlength`.

Examples

Runs a signal net reliability analysis (SIGRA) simulation that includes a design resistor with the model name `rhsim_m`.

```
set_ra_option -sigra 1
set_ra_functional_resistor -res_model rhsim_m
load_ba_file -file spf_file
```

Runs a SIGRA simulation that includes design resistors with model names `rhsim_m` and `rhsim_p`.

```
set_ra_option -sigra 1
set_ra_functional_resistor -res_model rhsim_m
set_ra_functional_resistor -res_model rhsim_p
load_ba_file -file spf_file
```

Runs a SIGRA simulation that includes a design resistor with model name `rhsim_m` and a particular layer name, `design_res`, instead of the default layer name.

```
set_ra_option -sigra 1
set_ra_function_resistor -res_model rhsim_m -layer design_res
load_ba_file -file spf_file
```

Runs a SIGRA simulation that includes a design resistor with the model name `rhsim_m` and `rhsim_p`. The netlist uses `wp` and `wn` to represent width for the corresponding resistors and `ln` and `lp` to represent length.

```
set_ra_option -sigra 1
set_ra_function_resistor -res_model rhsim_m -map_w wn -map_l ln
set_ra_function_resistor -res_model rhsim_p -map_w wp -map_l lp
load_ba_file -file spf_file
```

set_ra_net

Description

Specifies the nets for electromigration and voltage drop (EMIR) analysis flow.

Syntax

```
set_ra_net [-selectnet net_names]
           [-skipnet net_names]
```

Arguments

Argument	Description
<code>-selectnet net_names</code>	Provides a list of net names to be analyzed in EMIR flow. You must specify the full hierarchical BA name, not the schematic one. The default is to consider all back-annotated nets for EMIR flow. When selecting nets for EMIR analysis, you can specify both explicit names or use wildcard characters. The net name must be based on the back-annotation net name, not the schematic name.
<code>-skipnet net_names</code>	Provides a list of EMIR net names to be skipped for the EMIR flow. Wildcard name matching is allowed. If net names are matched by both the <code>-selectnet</code> and <code>-skipnet</code> options, <code>-skipnet</code> has a higher priority. By default no nets are skipped.

Examples

Selects only the net that matches `vdd`.

```
set_ra_net -selectnet vdd*
```

Selects all nets, but skips `vdd` and `gnd` nets.

```
set_ra_net -selectnet * -skipnet vdd gnd
```

set_ra_net_type

Specifies the net types for power net reliability analysis (PWRA) and signal net reliability analysis (SIGRA) during PrimeSim electromigration and voltage drop (EMIR) flow.

Syntax

```
set_ra_net_type [-pwnet net_names]
                [-signet net_names]
```

Arguments

Argument	Description
<code>-pwnet net_names</code>	Provides a list of power net names to be analyzed by the EMIR flow. You must specify full hierarchical BA net names, not the schematic ones. Wildcard name matching is supported. By default the net type is determined automatically in the PrimeSim XA tool. When selecting PWRA nets, you can specify both explicit names or use wildcard characters. The net name must be based on the back-annotation net name, not the schematic name.
<code>-signet net_names</code>	Provides a list of signal net names to be analyzed by the EMIR flow. You must specify full hierarchical BA net names, not the schematic ones. Wildcard name matching is supported. By default the net type is determined automatically in the PrimeSim XA tool. When selecting SIGRA nets, you can specify both explicit names or use wildcard characters. The net name must be based on the back-annotation net name, not the schematic name.

Description

The `set_ra_net_type` command selects the type of net that goes into the EMIR flow. There are only two types of nets that can go into EMIR flow: power nets or signal nets. If a net is selected as both power and signal, then the power selection takes precedence.

When you do not specify a `set_ra_net_type` command, the PrimeSim XA tool uses internal ID methods to identify net types. When you specify one or more `set_ra_net_type` commands, the automatic identification is disabled, allowing you to control net types manually.

You can specify multiple `set_ra_net_type` commands. Only nets that are selected by `-pwnet` or `-signet` are selected for the EMIR; that is, the `set_ra_option -pwra 1` command triggers power net reliability analysis (PWRA) and the `set_ra_option -sigra 1` command triggers signal net reliability analysis (SIGRA)).

Examples

Defines `vdd` and `vddhd` as power nets.

```
set_ra_net_type -pwnet vdd vddhd
```

set_ra_option

Description

Activates PrimeSim electromigration and voltage drop (EMIR) flow and sets the options for phase I and phase II simulations.

Syntax

```
set_ra_option [-pwra enable_value]
[-sigra enable_value]
[-waveform_split size]
[-rap2auto enable_value]
[-ratau ratau_value]
[-ratcl ratcl_name1 ratcl_name2 ...]
[-twindow time_points]
[-presimtime time]
[-tmpdir directory_name]
[-rmin value]
[-sigra_filter_run 0|1]
[-bulk2c enable_value]
[-pp processes]
[-pp_config cdpl_config_file]
[-pp_nets net_names]
[-pp_transient_fractionfraction|0|1]
[-dp processes]
[-dp_config cdpl_config_file]
[-substrate2c enable_value]
[-substrate2c_itf_layer_name list_of_layers]
[-substrate2c_layer_name list_of_layers]
```

Arguments

Argument	Description
<code>-pwra enable_value</code>	Activates the PrimeSim power net EMIR analysis flow. The default is 0.

Argument	Description
<code>-sigra enable_value</code>	Activates the PrimeSim signal net EMIR analysis flow. The default is 0.
<code>-waveform_split_size</code>	Sets the split size for the <code>.rasim</code> file. The default split size is 2G, in unit of bytes. If the <code>.rasim</code> file size exceeds this threshold, then additional <code>.rasim</code> files are created with names <code>.rasim#</code>, such as <code>.rasim1</code>, <code>.rasim2</code> and so on. Note: Because the tool compresses the split waveform file, the size of the split waveform file might become smaller after splitting.
<code>-rap2auto enable_value</code>	Launches the phase II simulation run automatically. The default is 1. When you set this option to 0, the phase I simulation stops after the <code>.rasim</code> file is generated.
<code>-ratau ratau_value</code>	Sets the RATAU (simulation step size) value for Phase II simulation. The default is 10ps.
<code>-ratcl ratcl_name1 ratcl_name2 ...</code>	Reads the specified <code>ra_tcl</code> file with instructions for phase II simulation. You can specify multiple <code>ra_tcl</code> files.
<code>-twindow time_points</code>	Specifies to generate <code>.rasim</code> file data only within time windows defined by: <pre>-twindow tstart tstop {tstart tstop} [tstart]</pre> By default, the tool uses <code>tstart=0ns</code> and <code>tstop=end_of_transient</code> as the window. When you use the <code>-twindow</code> option, if you want a similar window to be used during phase II of reliability analysis, you must use the <code>swin</code> command in the <code>.ratcl</code> file. For more information on the <code>swin</code> command, see the <i>PrimeSim™ EMIR Reference Manual</i>.
<code>-presimtime time</code>	Sets a pre-simulation time for phase II simulation to start earlier than time windows that are specified by the <code>-twindow</code> option. This allows current values on parasitic network to settle down before the actual simulation. Default is 1 ns. Example: <pre>-twindow 500n 600n -presimtime 10n</pre> In the above example, phase II simulation starts at 490 ns (that is, 10 ns earlier than 500 ns) and uses currents that are saved between 490 ns to 500 ns for simulation which starts at 500 ns.

Argument	Description
<code>-tmpdir directory_name</code>	Specifies the directory name to store temporary files. The default is the current directory.
<code>-rmin value</code>	Keeps resistors larger than the specified value in phase II simulation. The unit is Ohm. The default is 0.
<code>-sigra_filter_run 0 1</code>	If set to 1, runs a pre-phase I signal net EMIR run when the tool only executes filtering of signal nets without saving the device currents in <code>.rasim</code> files. This run produces a filtered list of nets to be analyzed, based on the <code>iavmin</code> command. The device currents are saved only for those nets. The default is 0. For more information about this command, see the Running Pre-Phase I Net Filtering section in the <i>PrimeSim EMIR Reference Manual</i>.
<code>-bulk2c enable_value</code>	When <code>-bulk2c</code> set to 0 (default), bulk currents are collected for Phase II simulation. When set to 1, bulk currents are not collected for Phase II.
<code>-pp processes</code>	Specifies the number of processes to run for power/signal net analysis in phase I. For details, see the EMIR Pipeline Processing section in the <i>PrimeSim EMIR Reference Manual</i> .
<code>-pp_config cdpl_config_file</code>	Specifies the CDPL hosts configuration file for power/signal net analysis in phase 1. For details, see the EMIR Pipeline Processing section in the <i>PrimeSim EMIR Reference Manual</i> . See the <i>Common DP Library User Guide</i> for the <code>cdpl_config_file</code> syntax.
<code>-pp_nets net_names</code>	Specifies the power/signal net names to process. You cannot use wildcards in the names. For details, see the EMIR Pipeline Processing section in the <i>PrimeSim EMIR Reference Manual</i> .
<code>-pp_transient_fraction fraction 0 1</code>	Specifies when to split the <code>rasim</code> files for EMIR pipeline processing. The <code>rasim</code> files are saved when the fraction of the transient analysis is finished. You should set this argument should in the [0,1] range. It is used in pipeline processing to eliminate waiting until the ending of transient analysis in phase 1.
<code>-dp processes</code>	Specifies the number of processes to run for power/signal net EMIR analysis in parallel in phase II. For details, see the EMIR Pipeline Processing section in the <i>PrimeSim EMIR Reference Manual</i> .

Argument	Description
<code>-dp_config cdpl_config_file</code>	Specifies the CDPL hosts configuration file for power/signal net analysis in phase II. For details, see the EMIR Pipeline Processing section in the <i>PrimeSim EMIR Reference Manual</i> . See <i>Common DP Library User Guide</i> for the <code>cdpl_config_file</code> syntax.
<code>-substrate2c enable_value</code>	When set to 1, .rasim currents will not be collected from devices in the layers that are specified by the <code>-substrate2c_itf_layer_name</code> or <code>-substrate2c_layer_name</code> option. The capacitance of these devices will always be reported in the .drcap file.
<code>-substrate2c_itf_layer_name list_of_layers</code>	Specifies the list of substrate ITF layers whose device currents will be ignored. Use this option when the <code>-substrate2c</code> option is enabled. The <code>-substrate2c_itf_layer_name</code> and <code>-substrate2c_layer_name</code> options are mutually exclusive. You should specify either one based on whatever layer name is known or more convenient to specify.
<code>-substrate2c_layer_name list_of_layers</code>	Specifies the list of substrate layers whose device currents will be ignored. Use this option when the <code>-substrate2c</code> option is enabled. The <code>-substrate2c_itf_layer_name</code> and <code>-substrate2c_layer_name</code> options are mutually exclusive. You should specify either one based on whatever layer name is known or more convenient to specify.

Examples

The following example runs the power net EMIR analysis flow.

```
set_ra_option -pwra 1 -ratcl ratcl_file
```

The following example runs both power net and signal net EMIR analysis flows.

```
set_ra_option -pwra 1 -sigra 1 -ratcl ratcl_file
```

The following example runs both power net and signal net EMIR analysis flows and stops after phase I completes.

```
set_ra_option -pwra 1 -sigra 1 -ratcl ratcl_file -rap2auto 0
```

The following example collects `.rasim` data (currents and voltages) in the time interval 5-10ns and from 20ns until the end of transient simulation.

```
set_ra_option -twindow 5n 10n 20n
```

In the following example a maximum four pipeline process run concurrently for the `vdd`, `vddmi`, `vddhd`, and `vss` nets.

```
set_ra_option -pwra 1 -ratcl ra.tcl -pp 4 -pp_nets vdd vddmi vddhd vss
```

In the following example, only the `vdd` and `vss` nets are selected for power net reliability analysis. As a result, there is not any pipeline processing on the `vddmi` net.

```
set_ra_option -pwra 1 -ratcl ra.tcl -pp 3 -pp_nets vdd vss vddmi  
set_ra_net -selectnet vdd vss
```

In the following example there are at most four nets running concurrently in the phase II simulation.

```
set_ra_option -pwra 1 -ratcl ra.tcl -dp 4
```

The following example specifies the `-pp` and `-dp` arguments. As a result, there are a maximum of three pipeline process running concurrently in phase 1 and three distributed processes running in phase II.

```
set_ra_option -pwra 1 -ratcl ra.tcl -pp 3 -pp_nets vdd vddmi vss -dp 3
```

In the following example the `vss` net is skipped because `-skipnet` has priority over the `-selectnet` argument. As a result, only the `vdd` and `vddmi` nets are selected for power net reliability analysis. Even if `-pp` is set to four, there are at most two pipeline processes in phase 1.

```
set_ra_option -pwra 1 -ratcl ra.tcl -pp 4 -pp_nets vdd vss vddmi vddai  
set_ra_net -selectnet vdd vss vddmi  
set_ra_net -skipnet vss
```

In the following example, `.rasim` currents will not be collected from devices in the layer tap. The capacitance values of these devices will always be reported in the `.drcap` file.

```
set_ra_option -substrate2c 1 -substrate2c_layer_name tap
```

set_ra_pwnet_driver

Provides the driver pin information for the internal power net.

Syntax

```
set_ra_pwnet_driver -net net_name  
-driver_ipname instance_pin_names  
[-exclude_netpins 0|1]
```


Arguments

Argument	Description
<code>-net net_name</code>	Provides the back-annotated net name of an internal power net. You can specify only one net name per <code>set_ra_pwnet_driver</code> command. Note: Wildcard characters are not supported.
<code>-driver_ipname instance_pin_names</code>	Provides driver instance pins for the specified internal power net. Wildcard characters are supported.
<code>-exclude_netpins 0 1</code>	Specify one of the following values. • 0 (default) where net pins are considered as drivers. • 1 specifies to convert all the net pins into loads instead of drivers.

Description

By default, driver instance pins are found automatically by the PrimeSim XA tool. An output file `.pw_driver` is created that contains information about internal power nets and their drivers. If you specify the `set_ra_pwnet_driver` command, automatic detection is turned off for this net. You can specify multiple `set_ra_pwnet_driver` commands for the same net.

If the net has net pins (`*|P`), you should delete those pins if you want only the specified drivers to be used. A solution to delete the unwanted pins is to use the `-delete_netpin` option in the [load_ba_file](#) command.

Examples

Defines the driver pin of the `vss` net.

```
set_ra_pwnet_driver -net vss -driver_ipname xx/xx/clk2@2:D
```

Defines the driver pins of the `vreg` net using wildcards.

```
set_ra_pwnet_driver -net vreg -driver_ipname X_op_reg.Xdrv<*>.MNout*"
```

set_ra_pwnet_option

Lets you define the reference voltage and source external power supply node for the internal power net.

Syntax

```
set_ra_pwnet_option -net net_name
[-vref vref]
[-src source_pwnet_node]
```

Arguments

Argument	Description
-net net_name	Provides the back-annotated net name of an internal power net. You can specify only one net name per set_ra_pwnet_option command. Wildcard characters are supported.
-vref vref	Specifies the reference voltage.
-src source_pwnet_node	Specifies the source power net node (its netlist name) that is supplying power into the internal power net. This is a hierarchical node name starting from the top level of the netlist. Wildcard characters are not supported.

Description

By default the PrimeSim XA tool automatically identifies internal power nodes and selects appropriate reference voltages and source nodes for them. An output file .pw_driver is created that contains information about internal power nets and their drivers.

You can override the automatic identification of vref and source_pwnet_node with the set_ra_pwnet_option command. If you specify source_pwnet_node, the automatic driver ID selects only those drivers that connect to the specified node.

Examples

Defines the source node and reference voltage of the vdd_int net.

```
set_ra_pwnet_option -net vdd_int -src vdd -vref 0.9
```

set_ra_reuse

Reuses previous simulation results to make the EMIR analysis verification cycle more efficient.

Syntax

```
set_ra_reuse -rasim master_output_path
[-inst_pin_tol tol_value]
```

Arguments

Argument	Description
<code>-rasim master_output_path</code>	Points to the <code>.rasim</code> file location of the master run. The <code>master_output_path</code> is the output directory of the master run. For example, if you use the following command line option for the master run: <pre>xa top.sp -c cmd -o rundir1/xal</pre> The <code>master_output_path</code> path is set to <code>rundir1/xal</code>. It is used as a reference to the master run to get the path to <code>.rasim</code> files. This path must be different from output path in the reuse flow run.
<code>-inst_pin_tol tol_value</code>	Defines the allowable discrepancy between device pins in the master run and the new DSPF file. Valid value are in the range of 0-1. The default is 0. Consistency in instance pin names is checked between the master run <code>*.rasim</code> files and new DSPF databases. For example, suppose that in PWRA in the master run there are 1K instance pin connections to the VDD net. These instance (or device) pin currents are saved in the <code>*.rasim</code> file in the master run. If the new DSPF has VDD instance pin connections other than those defined in the master run, the simulation checks these connections. The checking is performed at the beginning of phase II simulation. The simulation errors out if the percentage of the discrepancy is greater than user-defined threshold. If the ratio of the number of mismatched device pins and the number of saved device pin currents in the <code>.rasim</code> file exceeds this threshold, the simulation stops. The output mismatch file is generated to the <code>*.pin_mismatch</code> file. The tolerance is calculated as: $\frac{((\text{number_pins_in_new_SPF_that_we_cannot_find_in_rasim}) + (\text{number_pins_in_rasim_that_we_cannot_find_in_orig_SPF}))}{\text{number_of_pins_in_orig_SPF}}$

Description

Reliability analysis often requires iterations after an initial simulation. These iterative results help you debug layout problems. When layout issues are identified and corrected, a new DSPF file is generated and used for the next iteration of simulation. The process of layout changes, generating new DSPF files, and iterative simulation runs might continue until all layout issues are fixed.

Transient analysis can take a significant amount of time in these simulations. To shorten the EMIR verification cycle, the `set_ra_reuse` command lets you reuse transient analysis results after the first EMIR run and skip transient analysis for subsequent EMIR runs.

Examples

Reuses the RASIM result from the master run located in the `output_master` directory.

```
set_ra_reuse -rasim output_master/xa
```

Note:

Inside the `./output_master` directory there must be an `xa-pwnet_name.rasim` file available.

set_rc_network_option

Description

Controls options related to RC optimization.

Syntax

```
set_rc_network_option [-mode mode_value]
[-tconst tconst_value]
[-short_small enable_value]
```

Arguments

Argument	Description
<code>-mode mode_value</code>	Controls whether to perform optimization for all parasitic RC networks. You can specify one of the following values. • 0 disables RC reduction and optimizations. • 1 enables RC optimizations. This is the default value. • 2 enables moderate RC reduction. • 3 enables conservative RC reduction.
<code>-tconst tconst_value</code>	Sets the time constant value used by the <code>-mode 2</code> RC heuristic algorithm. The nodes with a <code>tconst_value</code> greater than the value specified by this option are preserved. The default is 1e-13.

Argument	Description
<code>-short_small enable_value</code>	By default resistors with a very small resistance (less than 1e-5 for HSPICE netlists) are shorted and it is not possible to probe the current in those elements. When <code>-short_small</code> is set to 0, the very small resistors are not shorted. The current flowing in these elements can be probed.

set_rc_option

Controls RC settings for the SRAM commands, such as the shorting of unprotected zero voltage sources in the beginning of simulation.

Syntax

```
set_rc_option -short_0vsrc 1|0|on|off
```

Arguments

Argument	Description
<code>-short_0vsrc</code>	Controls whether to short and remove unprotected zero voltage sources in the beginning of simulation. You can specify one of the following values: 0 or <code>off</code> specifies not to apply shorting on the unprotected voltage sources. 1 or <code>on</code> shorts and removes unprotected zero voltage sources.

Description

Use the `set_rc_option` command to control RC settings for simulation. When the `-short_0vsrc` option is enabled, constant zero voltage sources are shorted and removed after the netlist database is set up in the beginning of simulation. Shorting these voltage sources results in better performance and reduced RC network with no penalty in accuracy. Voltage sources that are protected, explicitly elements that are probed (such as, `.probe I(VDD)`), or those that are accessed by other devices (such as, current control sources) are never removed. Voltage sources that are probed by wildcards will be removed if not otherwise protected. By default, shorting is off.

Examples

The following example shorts and removes unprotected zero voltage sources.

```
set_rc_option -short_0vsrc 1
```

The following example disables shorting on the unprotected zero voltage sources.

```
set_rc_option -short_0vsrc 0
```

set_resistor_option

Lets you control resistor optimization. You can specify multiple `set_resistor_option` commands in one simulation.

Syntax

```
set_resistor_option -rule rule_value  
[-min min_value]  
[-max max_value]  
[-process_negative negres_switch]  
[-report report_value]
```

Arguments

Argument	Description
-rule rule_value	You must specify one of the following optimization rules for resistor elements: 1 specifies to process resistors at the netlist parsing stage after solving the parameters. As a result, all voltage and current probes of the optimized resistors cannot be recovered for analysis during simulation. This rule has the advantage of using less memory. 2 has the advantage of recovering the optimized resistors for voltage and current probes. However, this rule can impact the performance of the simulation. Rule 1 is executed before rule 2, so you can apply both rules in one simulation. See Figure 9. When you specify multiple commands, only rule 1 can override rule 1, and rule 2 can override rule 2. It is recommended to specify the absolute minimum resistance value of rule 1 to be less than absolute minimum resistance value of rule 2, and the absolute maximum resistance value of rule 1 to be greater than the absolute maximum resistance value of rule 2. See Figure 10.
-min min_value	Specifies to short all resistors with an absolute resistance value below or equal to <code>min_value</code> (<code>abs(Resistance) <= min_value</code>).
-max max_value	Specifies the upper threshold value for the resistors to be kept. All resistors with absolute resistance value above <code>max_value</code> (<code>abs(Resistance) > max_value</code>) are treated as open circuit.

Argument	Description
<code>-process_negative</code> <code>negres_switch</code>	Controls the optimization of negative resistors for simulation. This is only valid as part of the <code>-rule 1</code> option. You must specify <code>-rule 1</code> if you use this option. It can be set to: <code>normal</code> (default) keeps negative resistors for optimization by the <code>-rule</code> and <code>-min</code> options. <code>keep</code> specifies to keep all negative resistors for simulation. <code>open</code> treats all negative resistors as open circuit for simulation. <code>short</code> specifies to short all negative resistors for simulation.
<code>-report report_value</code>	Reports the list of optimized resistors. Parasitic resistors from the back-annotation file are not reported. It can be set to: <code>0</code> (default) does not report any optimized resistors. <code>1</code> reports the list of optimized resistors to an output file with a <code>*.rlcignore</code> extension.

Description

This command optimizes resistors to improve performance for post-layout simulation. The PrimeSim XA tool supports two rules as previously described.

If you do not specify this command, the PrimeSim XA tool supports the following behavior based on the netlist format.

Default PrimeSim XA Behavior for Handling Resistors in Other Netlist Formats

Table 20 Handling Resistors in HSPICE Netlists

set_sim_level	Description
3-6	- Shorts all the resistors with value equal to 0 by <code>-rule 1</code> option. - Shorts all the resistors with value from <code>-RESMIN</code> to <code>RESMIN</code> by <code>-rule 2</code> option. - Treats all the resistors with value larger than or equal to <code>1e12</code>, or is smaller than <code>-1e12</code> as open circuit by <code>-rule 2</code> option.
7	- Shorts all the resistors with value equal to 0 by <code>-rule 1</code> option. - Sets all the resistors with value between 0 and <code>RESMIN</code> to <code>RESMIN</code>. - Sets all the resistors with value between <code>-RESMIN</code> and 0 to <code>-RESMIN</code>. - Treats all the resistors with value larger than or equal to <code>1e12</code>, or is smaller than <code>-1e12</code> as open circuit by <code>-rule 2</code> option.

Table 21 Handling Resistors in Eldo Netlists

set_sim_level	Description
3-6	- Shorts all the resistors with value equal to 0 by <code>-rule 1</code> option. - Shorts all the resistors with value from <code>-RSMALL</code> to <code>RSMALL</code> by <code>-rule 2</code> option. - Treats all the resistors with value larger than or equal to <code>1e12</code>, or is smaller than <code>-1e12</code> as open circuit by <code>-rule 2</code> option.
7	- Shorts all the resistors with value equal to 0 by <code>-rule 1</code> option. - Sets all the resistors with value between 0 and <code>RSMALL</code> to <code>RSMALL</code>. - Sets all the resistors with value between <code>-RSMALL</code> and 0 to <code>-RSMALL</code>. - Treats all the resistors with value larger than or equal to <code>1e12</code>, or is smaller than <code>-1e12</code> as open circuit by <code>-rule 2</code> option.

Table 22 Handling Resistors in Spectre Netlists

set_sim_level	Description
3-6	- Shorts all the resistors with value equal to 0 by <code>-rule 1</code> option. - Shorts all the resistors with value from <code>-rmin</code> to <code>rmin</code> by <code>-rule 2</code> option. - Treats all the resistors with value larger than or equal to <code>1e12</code>, or is smaller than <code>-1e12</code> as open circuit by <code>-rule 2</code> option.
7	- Shorts all the resistors with value equal to 0 by <code>-rule 1</code> option. - Sets all the resistors with value between 0 and <code>rmin</code> to <code>rmin</code>. - Sets all the resistors with value between <code>-rmin</code> and 0 to <code>-rmin</code>. - Treats all the resistors with value larger than or equal to <code>1e12</code>, or is smaller than <code>-1e12</code> as open circuit by <code>-rule 2</code> option.

SPF or SPEF Netlist Format (Back-Annotation)

The PrimeSim XA tool supports the [load_ba_file](#) command and [set_ba_option](#) command to read in the back-annotation file (SPF and SPEF) and control the resistor optimization.

The `-min_res` option in [load_ba_file](#) command and [load_ba_file](#) command set the lower threshold of the resistors in the back-annotation file to be kept. All resistors with absolute resistance value less than the value specified by `-min_res` option are shorted with `-rule 1` option. The default value is 0.01 ohm.

The `-max_res` option in [load_ba_file](#) command and [set_ba_option](#) command set the upper threshold of the resistors in the back-annotation file to be kept. All resistors with absolute resistance value greater than the value specified by the `-max_res` option are treated as open circuit with `-rule 1` option. The default value is `1e12` ohms.

The resistor optimization from the back-annotation file is affected by these commands in the following order of priority:

1. [load_ba_file](#)
2. [set_ba_option](#)
3. [set_resistor_option](#)

Figure 9 Simulation Flow of -rule 1 and -rule 2

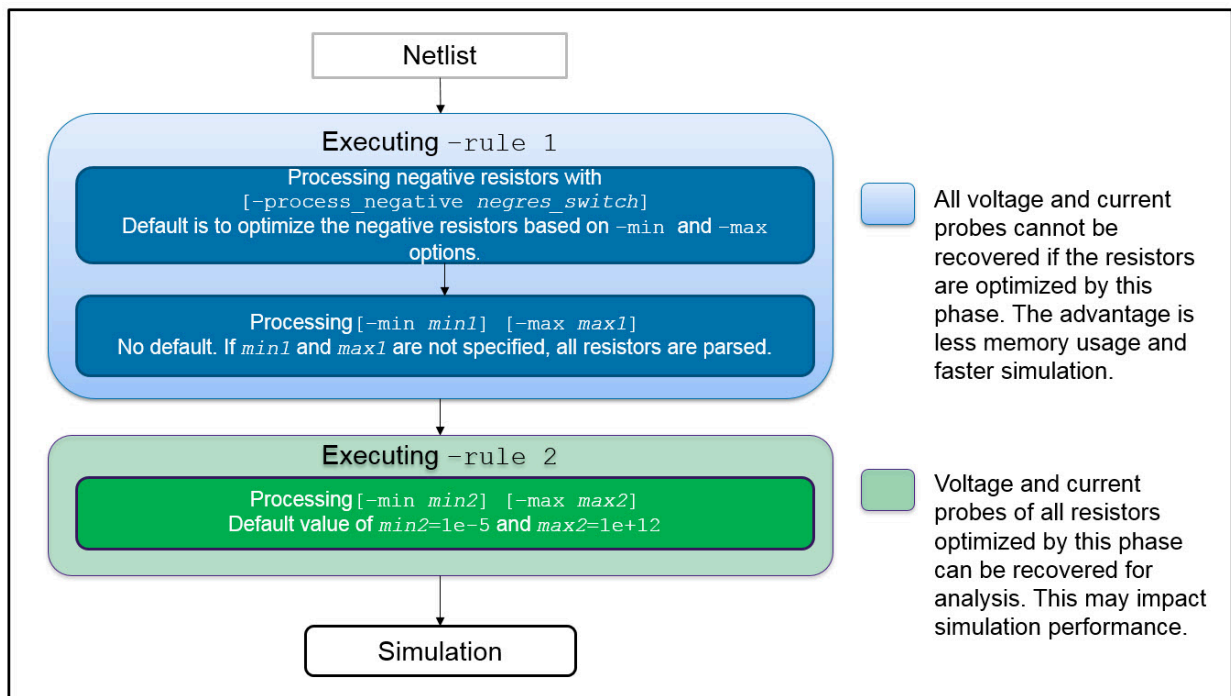
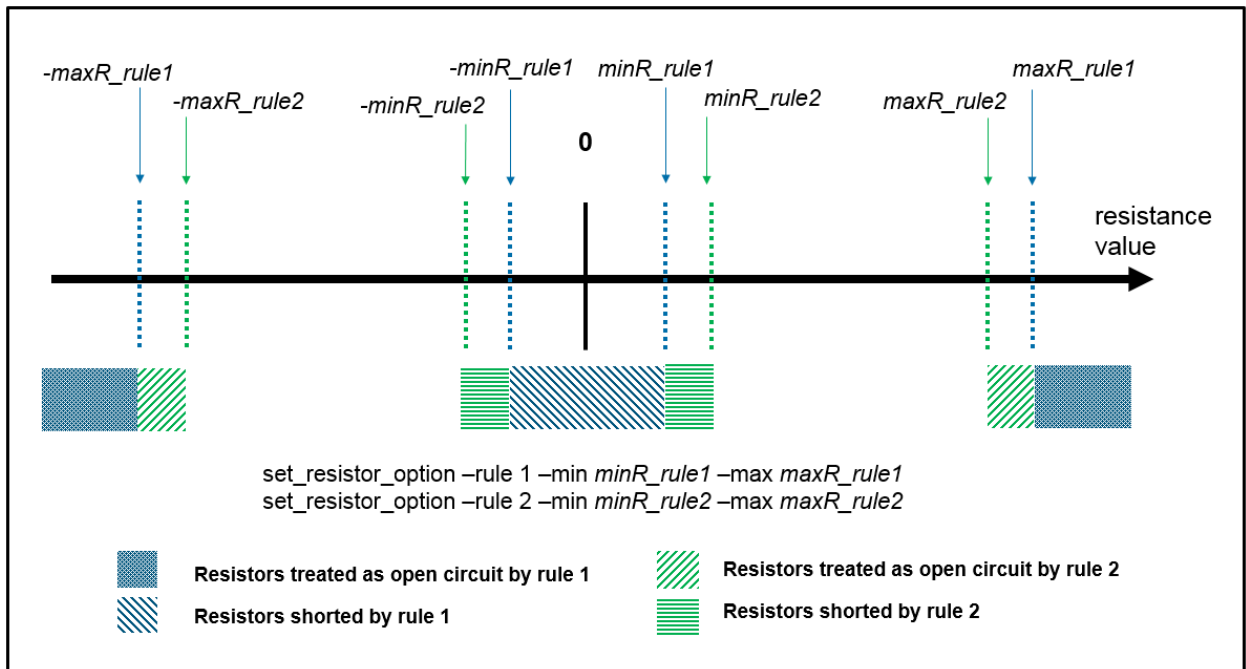


Figure 10 Using -rule 1 and -rule 2 Options in the Same Simulation



Examples

The following example shorts all the resistors with value between -15 ohms and 15 ohms with rule 1, and open all resistors with value greater than 1000 or less than -1000 with rule 1.

```
set_resistor_option -rule 1 -min 15 -max 1000
```

The following example shorts all the resistors with value less than 0.1 ohm with rule 1 and reports them in *.rlcignore file, and shorts all the resistors with value less than 1 ohm with rule 2 and reports them in *.rlcignore file.

```
set_resistor_option -rule 1 -min 0.1 -report 1  
set_resistor_option -rule 2 -min 1 -report 1
```

The following example shorts all the negative resistors with rule 1, and resistors with value less than 5 ohms with rule 2.

```
set_resistor_option -rule 1 -process_negative short  
set_resistor_option -rule 2 -min 5
```

See Also

- [load_ba_file](#)
- [set_ba_option](#)

set_restore_option

Description

Restarts the saved run at time 0, regardless of the saved time.

Syntax

```
set_restore_option -time 0
```

Arguments

Argument	Description
-time 0	Note that the PrimeSim XA tool does not do any time shift of the input stimuli, it just restarts the run at time 0.

Note:

The `set_restore_option` command is not available in a VCS PrimeSim AMS mixed-signal simulation.

set_rf_option

Controls if the critical RF components or blocks in a transceiver circuit are simulated more conservatively, including tighter circuit partitioning rules and simulator tolerances for the specified subcircuit and instances. By default, the command scales the tolerances by 0.2 for the specified sub-circuit or instances. The `set_rf_option` command should be used with the `set_circuit_transceiver` command.

Syntax

```
set_rf_option [-enable 0|1] -process 0|1 [-scale value] [-subckt name]  
[-inst name]
```

Arguments

Argument	Description
-enable 0 1	Sets to 1 to enable FinFET transceiver simulation. The default is 1.

Argument	Description
<code>-process 0 1</code>	Specifies to simulate design by process technology. • 0 (default): Runs FinFET simulation for 16nm designs and above. • 1: Runs FinFET simulation for 7nm designs and below.
<code>-scale value</code>	Scales the tolerances for the sub-circuit and instances by the specified value. This option tightens the tolerance for the sub-circuit and instances. The smaller the scale value, the tighter the tolerance for the simulator. The default value is 0.2, which is a conservative setting. The default value provides a good tradeoff between performance and accuracy and does not typically need to be set lower. Increasing the value will loosen the tolerance and may improve performance.
<code>-subckt name</code>	Specifies the name of the subcircuit.
<code>-inst name</code>	Specifies the name of the instances.

See Also

- [FinFET Transceiver Simulations](#)

set_robust_model

Controls simulation speed and model complexity. The command modifies Newton-Raphson (NR) simulation controls to resolve model discontinuities in 90 nm BSIM processes and before.

Use the `-accuracy` option to specify an accuracy level for performance and accuracy tradeoffs. All accuracy levels are intended for use with designs in 90 nm technology and larger nodes. A lower accuracy level produces faster performance but with less accuracy. The recommended level setting is 3 for the best combination of performance and accuracy. Note that you can use the `acc` alias for the `-accuracy` option.

Syntax

```
set_robust_model -accuracy 0|1|2|3|4
```

Arguments

Argument	Description
<code>-accuracy 0 1 2 3 4</code>	Specifies the performance and accuracy settings for the model robustness feature by modifying the NR simulation controls. Specify 1 for the fastest simulation, or 4 for the most accurate. The default is 0, meaning the model robustness feature is disabled.

Examples

The following example sets the accuracy level to 3. A value of 3 is recommended for analog circuit designs that contain PLLs, voltage regulators and level shifters.

```
set_robust_model -accuracy 3
```

set_sample_point

Use this command when you need to make very precise periodic measurements, such as in FFT applications.

Syntax

```
set_sample_point -period period_value  
[-twindow {start_time stop_time} start_time [stop_time]]
```

Arguments

Argument	Description
-period period_value	Sets the period between sampling points (the sampling frequency). If you specify multiple start_time and stop_time arguments, the period applies to all the -twindow values.
-twindow {start_time stop_time} start_time [stop_time]	Sets the start and stop time of the first sampling point.

Description

Very precise measurements such as those made during FFT analysis can be adversely affected by sampling the simulator output waveforms, if the sample points interpolate between time points solved by the simulator. This command forces the simulator to synchronize all partitions and solve each time point that is sampled during a post-processing measurement. This prevents any interpolation errors and maximizes the precision of a measurement.

Examples

Assume you are simulating a DAC and need to analyze the spectral output by computing an FFT. The following example starts the first sample at 10 microseconds and sets the sampling rate to 10 mega samples per second. That is, a sample point occurs at 10μ, 10.1μ, 10.2μ, 10.3μ, and so on.

```
set_sample_point -period 100n -twindow 10u
```

Assume you are simulating a DAC and need to analyze the spectral output by computing an FFT. In the following example, the first sample is at 10μ sampling at a rate of 10 Msps. The end of the sampling time is 20μ.

```
set_sample_point -period 100n -twindow 10u 20u
```

set_save_state

Saves a current state of the simulation to be restarted later with a restore command line option, such as `-auto_restore` or `-restore` and so on. See the Saving and Restoring Simulations section in *PrimeSim XA User Guide* for detailed usage information.

Syntax

```
set_save_state [-time t_val {t_val}] |  
[-tran_time_period time_period] |  
[-wall_time_period wall_time] |  
[-period percent_value%]  
[-keep_all 0|1]  
[-keep_last 0|1]  
[-save_on_kill enable_value]  
[-file file_name]  
[-dump_waveform 0|1]  
[-type op|image]
```

Arguments

Argument	Description
<code>-time t_val {t_val}</code>	Saves the simulation at the time or times you specify. For each <code>t_val</code> time you specify, the PrimeSim XA tool creates a file with the <code>.t_val#.ic</code> extension. The <code>dc</code> (for time 0) and <code>end</code> (for last transient point) keywords are valid values for <code>t_val</code> .
<code>-tran_time_period time_period</code>	Saves the simulation at the time period you specify. The image is saved in a file with a <code>.time.ic</code> extension.
<code>-wall_time_period wall_time</code>	Records checkpoints at the elapsed wall-clock period you specify, and saves the simulation with a <code>.save.ic</code> extension. The latest checkpoint overwrites the previous one. You must specify the wall time period in hours, and decimal points are accepted. When used with the <code>-save_on_kill</code> option, the image generated at the kill event overwrites the previous one.

Argument	Description
<code>-period percent_value%</code>	Specifies the percentage of the total transient simulation time (.TRAN ... TSTOP) to save. When used with image-based snapshot, the latest checkpoint overwrites the previous one, irrespectively of the <code>-keep_last</code> value. For example, if you specify a value of 25%, the simulation is saved when it reaches 25% of the total simulation time. The operating point (OP) state is stored by default in the following files: *.save2.ic, *.save2.ic.sup, *.save3.ic, and *.save3.ic.sup. Use the <code>-auto_restart</code> or <code>-restart</code> command line option to rerun the saved simulation. Note: If a simulation is interrupted before the specified percentage value is reached, it is not saved.
<code>-keep_all 0 1</code>	Sets to 1 to store all the simulation results saved in files *.{time}.ic (for operating states) or *.snapshot* (for image-based snapshots) at every specified <code>-period percent_value</code>. You can use this option to save an exact copy of the saved simulation results, but this might create disk overflow problems and is not recommended. The default is 0.
<code>-keep_last 0 1</code>	Sets to 1 to save the last saved simulation results in the *.save.ic or *.snapshot* files, depending on the type of the simulation state or snapshot to save is operating point (OP) or image-based. The default is 0.
<code>-save_on_kill enable_value</code>	Saves the simulation when run a UNIX <code>kill -15</code> command. You cannot use a <code>kill -9</code> command. The image is saved in a file with a <code>.save.ic</code> extension.
<code>-file file_name</code>	Specifies the names of the saved files. The saved files have <code>.ic</code> and <code>.ic.sup</code> extensions. If you specify multiple file names, the last one is used for all saved files. Note: The saved files are in the working directory, not in the output directory specified with the <code>xa -o</code> command line option.
<code>-dump_waveform 0 1</code>	Saves FSDB files at each periodic save time specified by the <code>-tran_time_period</code> or <code>-wall_time_period</code> option. The default is 0.

Argument	Description
-type op image	Specifies the type of the simulation state or snapshot to save. Specify one of the following: • <code>op</code> (Default): Generates a snapshot with operating point (OP) information. Use the OP-based snapshot when you want to make some modification to the netlist or simulation setup in the restarted run. PrimeSim XA performs an OP-like solution at the restarted point. • <code>image</code>: Generates an image-based snapshot. This type allows a seamless restart as though the run was completed by one single continuous job. Modification to netlist and simulation setup are not allowed. Other limitations in the compute environment may apply.

Description

The `set_save_state` command generates two files, one with a `.time.ic` extension and one with a `.time.ic.sup` extension for each saved time you specify with the `-time` or `-tran_time_period` option. The saved time uses scientific notation when the time value is less than 100us. When the time is equal to or greater than 100us, the time is a floating number.

When you specify the `-wall_time_period` option, two files are created with the `.save.ic` and `.save.ic.sup` extension. The files are overwritten at each wall time period. When you specify the `-save_on_kill` option, two files are created with the `.save.ic` and `.save.ic.sup` extension.

Note:

The `set_save_state` command is not available in a VCS PrimeSim AMS mixed-signal simulation.

Note:

Do not delete the `.ic` or `.ic.sup` files. You need both files to restore a saved simulation. For more information about restoring a saved simulation, see the Saving and Restoring Simulations section in *PrimeSim XA User Guide*.

The following rules apply to diagnostic commands and measurements:

- The diagnostic files (`.errz`, `.errt`, `.hotspot`, and CCK command outputs) must be written out and updated at each saved time point.
- The completed output measurements (`.power`, `.meas`, `.mt`) must be written out and updated at each saved time point.
- When restoring a simulation, if a measurement command starts before the restore time and finishes after the restore time, that measurement is ignored and a warning is issued.

The `set_save_state` command has the following restrictions:

- You cannot specify multiple `-time`, `-tran_time_period` and `-wall_time_period` options on a single command line.
- You cannot specify multiple `set_save_state` commands with different `-tran_time_period` or `-wall_time_period` values. If this is the case, only the last period or wall_time will be applied.
- You cannot specify the `set_save_state -tran_time_period` and `set_waveform_option -tran_time_period` commands together. Otherwise, the following warning message will be issued:

```
Warning: set_save_state waveform tran time splitting will overwrite  
set_waveform_option tran time splitting setting
```

- You can specify multiple values for the `-time` option, or multiple `set_save_state` commands with different `-time` values.
- You can combine the `-save_on_kill` option with the other options.

Example 1

The following example saves the simulation at 100 ns and at 1400 ns.

```
set_save_state -time 100n 1400n
```

Example 2

This example saves an image-based snapshot at 100 ns and 1400 ns.

```
set_save_state -time 100n 1400n -type image
```

Example 3

The following example saves the simulation every 200 ns.

```
set_save_state -tran_time_period 200n
```

Example 4

The following example saves an image-based snapshot periodically at every half hour. The latest generated snapshot overrides the previous one, so only one snapshot is kept.

```
set_save_state -wall_time_period .5 -type image
```

Example 5

The following example saves an image-based snapshot periodically at every hour, and additionally when or if there is a Unix “signal 15” interrupt event. The latest generated snapshot overrides previous one so only one snapshot is kept.

```
set_save_state -wall_time_period 1 -save_on_kill 1 -type image
```

Example 6

The following example saves the simulation at 10 ns, 20 ns, and every 200 ns.

```
set_save_state -tran_time_period 200n
set_save_state -time 10n 20n
```

Example 7

The following example saves the simulation at times 10 ns, 20 ns and 300 ns.

```
set_save_state -time 300n
set_save_state -time 10n 20n
```

Example 8

The following example saves the simulation every 100 ns and generate a waveform file every 100 ns. The waveform file contains the saved signals as:

```
prefix.1e-07.fsdb has all probed signals from 0 to 100ns.
prefix.2e-07.fsdb has all probed signals from 100ns to 200ns.
prefix.3e-07.fsdb has all probed signals from 200ns to 300ns.
set_save_state -tran_time_period 100n -dump_waveform 1
```

Example 9

The following example saves the simulation at every 30% of 5 ms.

```
xa test.sp -c test.tcl -o out/nointr -time 5ms
```

The `test.tcl` file contains:

```
set_save_state -period 30%
set_sim_level 7
```

After saving the OP for the simulation and rerunning with the `auto_restart` command line option the `out` directory contains the following two files with the default suffixes of `save2.ic(.sup)` and `save3.ic(.sup)`.

```
nointr.save2.ic
nointr.save2.ic.sup

nointr.save3.ic
nointr.save3.ic.sup
```

In this case, at every 30% of transient time, the OP is saved two files. The first saved OP at 30% of transient time is stored in `nointr.save2.ic` and the second saved OP at 60% of transient time is stored in `nointr.save3.ic`. The last saved OP at 90% of transient time is overridden by `nointr.save2.ic`.

Finally, the `nointr.save2.ic` file stores OP data at 90% of the transient time and the `nointr.save3.ic` file stores OP data at 60% of the transient time.

Example 10

The following example saves the simulation at every 30% of 5 ms and uses the `-keep_last 1` option.

```
%xa test.sp -c test.tcl -o out/nointr -time 5ms
```

The `test.tcl` file contains:

```
set_save_state -period 30% -keep_last 1
set_sim_level 7
```

After saving the OP for the simulation and rerunning with the `-auto_restart` command line option the `out` directory contains the following files.

```
nointr.save.ic
nointr.save.ic.sup
```

Example 11

The following example saves the simulation at every 10% of 5 ms and uses the `-keep_last 0` option.

```
% xa test.sp -c test.tcl -o out/nointr -time 5ms
```

The `test.tcl` file contains:

```
set_save_state -period 30% -keep_last 0
set_sim_level 7
```

After saving the OP for the simulation and running with the `-auto_restart` command line option, the `out` directory contains the following files according to the `-keep_last 0` option (0.0005 = 10% of 5 ms).

```
nointr.0.0005.ic
nointr.0.0005.ic.sup
nointr.0.001.ic
nointr.0.001.ic.sup
nointr.0.0015.ic
nointr.0.0015.ic.sup
nointr.0.002.ic
nointr.0.002.ic.sup
nointr.0.0025.ic
nointr.0.0025.ic.sup
nointr.0.003.ic
nointr.0.003.ic.sup
nointr.0.0035.ic
nointr.0.0035.ic.sup
nointr.0.004.ic
nointr.0.004.ic.sup
```

```
nointr.0.0045.ic
nointr.0.0045.ic.sup
```

set_sim_case

Controls case-sensitivity. Note that you must specify this command in the PrimeSim XA configuration file, not in the netlist.

Syntax

```
set_sim_case -case upper|lower|sensitive
```

Arguments

Argument	Description
-case upper lower sensitive	Specifies one of the following keywords: • <code>upper</code> to convert all the names to uppercase. • <code>lower</code> to convert all the names to lowercase. • <code>sensitive</code> to treat all names as case- sensitive. The default is based on the netlist format. For HSPICE and Eldo, the default is lowercase. For TI-SPICE, the default is uppercase. For Spectre and Verilog, the default is case-sensitive.

Description

The `set_sim_case` command lets you control the case-sensitivity of the PrimeSim XA tool for all the supported netlist formats. When the `-case upper|lower` argument is set, the PrimeSim XA tool is case- insensitive and converts all names to uppercase or lowercase, respectively. The `-case sensitive` argument enables the PrimeSim XA tool to be case-sensitive for all netlist formats.

Note:

The `temper` and `hertz` keywords are always case insensitive when evaluating expressions.

Examples

```
* Example 2 : HSPICE format
R1 port1 0 10
R2 porT1 0 10
r3 Port1 0 10

.probe v(*) i(r*)
```

If you specify `set_sim_case -case upper` or `-case lower`, the PrimeSim XA tool is case- insensitive and all 3 resistors are connected to the same node, `port1`. If you specify

set_sim_case -case sensitive, case-sensitivity is enabled and the 3 resistors are connected to 3 different nodes: port1, porT1, and Port1.

set_sim_hierid

Specifies the hierarchical separation character.

Syntax

```
set_sim_hierid -hierid sep_char [-add_spf sep_char]
```

Arguments

Argument	Description
-hierid sep_char	The legal hierarchical separation characters are: • Period (.) (default) • Colon (:) • Forward slash (/) • Caret (^)
-add_spf sep_char	This -add_spf option effectively replaces the SPF netlist delimiter with the set_sim_hierid delimiter. The replacement delimiter is applied in all included files. This option can be useful when you include a back-annotation netlist, as in a flat flow.

Description

If you specify both the Eldo .hier option and the set_sim_hierid command in the netlist file, the set_sim_hierid command takes precedence. The same precedence applies if the set_sim_hierid command is in the PrimeSim XA command file.

Examples

Sets the hierarchical separation character to "/".

```
set_sim_hierid /
```

Sets the hierarchical separation character to ":".

```
set_sim_hierid :
```

set_sim_level

Controls the speed and model complexity tradeoff.

Syntax

```
set_sim_level -level level [-inst instance_spec]
```

Arguments

Argument	Description
-level level	Specifies the simulation level. The supported values are: 1: Specifies a functional verification of digital circuits. Level 1 is not supported if PrimeSim XA is run with the <code>-ssl2</code> option. 2: Specifies a functional verification of digital circuits, more conservative than level 1. Level 2 is not supported if PrimeSim XA is run with the <code>-ssl2</code> option. 3 (default): Specifies a functional and timing verification of digital, memory, low-sensitivity analog, mixed-signal, and full-chip circuits. 4: Specifies a functional, timing and power verification of all circuits, especially for current or low voltage applications. 5: Specifies an accurate timing and power simulation of all circuits, and block characterization. 6: Specifies a SPICE-like accuracy for timing and power simulation of all circuits, cell characterization. 7: Specifies a setting for small designs (less than 1000 elements) or a small block in a large design, and device model verification.
-inst instance_spec	Specifies the instance information. See the Common Syntax Definitions section for details about the <code>instance_spec</code> argument.

Description

The `set_sim_level` command controls the simulator speed and model complexity tradeoff. This command can be applied to the entire netlist, or to specific subcircuits or instances. The default level is 3, if this command is not specified.

There is a new implementation of the `set_sim_level` command that can improve performance and accuracy for some types of simulations. For details about this new implementation, see the Running Simulator Version II section in *PrimeSim XA User Guide*.

Examples

The following example sets the simulation level to 5 on the entire netlist:

```
set_sim_level -level 5
```

The following example sets the simulation level on a specific instance:

```
set_sim_level -level 4 -inst xp11
```

The following example sets the simulation level on all instances of a particular subcircuit type:

```
set_sim_level -level 6 -subckt chgpump
```

set_sim_mode

Enables high performance simulation algorithms for analog and mixed signal designs. It is recommended that you use the `set_sim_mode` command along with the `set_sim_level` command. This command is equivalent to the `-sim_mode` command line option.

Syntax

```
set_sim_mode 0|1
```

Arguments

Argument	Description
0	Disables high performance simulation algorithms for analog and mixed signal designs. When specified, the default setting of the <code>set_sim_level</code> command is applied.
1	Enables high performance simulation algorithms for analog and mixed signal designs.

By default, running the simulation with the `set_sim_mode 1` command uses 2 cores. You can adjust the number of cores by specifying the `-mt` command line option or the `set_multi_core` command.

For SRAM simulations, the `set_sram_characterization` command should be used. The `set_sim_mode` and `set_sram_characterization` commands are mutually exclusive. When used together, the simulation run stops with an error message.

set_sim_stop

Controls when to stop the simulation when a violation is found and specifies tolerance criteria for violation reporting.

Syntax

You can include the `set_sim_stop` command in a PrimeSim XA command file, or inside a netlist file using the `xa_cmd` option.

```
set_sim_stop -check meas|vector [-name name] [-min low_threshold]
[-max high_threshold] \
[-stop 1|0] [-logic safe|violation]
```

or

```
.option xa_cmd="set_sim_stop -check meas|vector -name name \
[-min low_threshold] [-max high_threshold] [-stop 1|0] \
[-logic safe|violation]"
```

Arguments

Argument	Description
<code>-check</code> <code>meas vector</code>	Specifies one of the following: <code>meas</code>: Uses a <code>.MEASURE</code> statement for setting tolerance criteria. In the <code>.MEASURE</code> documentation, the measurement statement is identified using the <code>-name</code> argument. <code>vector</code>: Stops the simulation when a vector comparison error is found.
<code>-name</code>	(Optional) Specifies the name of the measurement statement whose results are used to check against the low threshold value and high threshold value specified with <code>-min</code> and <code>-max</code> options.
<code>-min</code> <code>low_threshold</code>	(Optional) The simulation stops when the measurement result is less than the specified <code>low_threshold</code> value. You must specify either the <code>-min</code> or <code>-max</code> option.
<code>-max</code> <code>high_threshold</code>	(Optional) The simulation stops when the measurement result is larger than the specified <code>high_threshold</code> value. You must specify either the <code>-min</code> or <code>-max</code> option.
<code>-stop 0 1</code>	(Optional) Stops the simulation when a violation is detected. Set the option to 0 if you want to continue the simulation until the end of transient analysis. The default is 1.
<code>-logic</code> <code>safe violation</code>	(Optional) Specifies one of the following values: <code>safe</code>(default): Reports a violation when the <code>meas</code> value is greater than the <code>max</code> value or smaller than the <code>min</code> value. <code>violation</code>: Reports a violation when the <code>meas</code> value falls between the <code>max</code> and <code>min</code> values.

Description

Use the `set_sim_stop` command to specify tolerance criteria for violation reporting. Use the `-stop` option to control whether simulation stops when a violation is reported or continues until the end of transient analysis. The tolerance criteria defined in a `.MEASURE` statement are used for checking against the other option settings, such as the low threshold value and high threshold value specified with `-min` and `-max` options.

Examples

The following example checks the value of the 'delay3' measurement value. If this measurement value is less than 500e-9 or more than 503e-9, the simulation will stop with a violation message.

```
set_sim_stop -check meas -name delay3 -min 500e-9 -max 503e-9
Warning: measure "delay1" rose to high threshold 5.03e-07 with
7.01616e-07 at 7.04629e-07
```


The following example checks the value of the 'vbg_max' measurement value. If this measurement value is more than 0.64, a violation message will be written to the log file and the simulation will continue.

```
set_sim_stop -check meas -name vbg_max -max 0.64 -stop 0
```

set_spectre_mc

Description

Specify a value of 0 to disable Monte Carlo analysis for Spectre format netlists. The default is 1. For more information, see Spectre Netlist Compatibility in *PrimeSim XA User Guide*.

Syntax

```
set_spectre_mc -enable 0|1
```

set_speed_scale

Dynamically scales the transient simulation engine tolerance to help tune performance and accuracy.

Syntax

```
set_speed_scale time_spec  
  
or  
  
set_speed_scale -node node_name node_spec  
  
or  
  
set_speed_scale -scale scale_value \  
    -inst inst1 [inst2 ...] -subckt def1 [def2 ...]
```

where

```
time_spec : time1 scale1 [time2 scale2 ...]  
node_spec : voltage1 scale1 [voltage2 scale2 ...]
```

Arguments

Argument	Description
time1 scale1 time2 scale2 ...	time can be any value greater than or equal to 0 and less than or equal to the .tran time. scale can be one of the following values: 0.5, 1, 2, 3, 4, or 5.

Argument	Description
<code>-node node_name</code> <code>voltage1 scale1</code> <code>voltage2 scale2 ...</code>	Specifies the node name to apply to the monitored voltage value. You cannot specify wildcard characters in the node names. <code>voltage</code> can be any voltage value. <code>scale</code> can be one of the following values: 0.5, 1, 2, 3, 4, or 5.
<code>-scale value</code>	Specifies the scaling factor for scaling the transient time. A scaling factor must be a value between 0.1 and 3. This option is mutually exclusive with <code>time_spec</code> and <code>node_spec</code> .
<code>-inst inst1_name</code> <code>[instance2_name ...]</code>	Specifies the name of the instance to be assigned with the scaled value during simulation. This option is mutually exclusive with <code>time_spec</code> and <code>node_spec</code> .
<code>-subckt def1_name</code> <code>[def2_name ...]</code>	Specifies the name of the subcircuit to be assigned with the scaled value during simulation. This option is mutually exclusive with <code>time_spec</code> and <code>node_spec</code> .

Description

The `set_speed_scale` command lets you manage a piecewise-defined performance and accuracy tradeoff for the entire simulation without having to break up your simulation into different segmented runs with the save and restore functionality. You can set scaling by either time windows or by voltage levels of a specified node.

In cases where different blocks require different levels of tolerances in the design, use the `-inst` and `-subckt` options. For example, you can specify some blocks in the design with smaller tolerances and some other less important blocks with larger tolerances.

A larger tolerance setting produces faster performance, but with less accuracy. Note that scaling is performed only on transient engine tolerances. Front-end optimizations, such as netlist reductions, modeling and simulation strategies and the DC engine, are unaffected by this command. The recommended setting is between 0.5 and 3.

You can use `set_speed_scale` with the [set_sram_characterization](#) or [set_sim_level](#) commands to provide an additional level of performance and accuracy refinement between the available level settings of these commands.

The following rules apply to the `set_speed_scale` command:

- You can specify only one `set_speed_scale` command at a time. Its effect is global to the entire design.
- You cannot mix the time scale and voltage syntax in one `set_speed_scale` command.
- The `-scale`, `-inst` and `-subckt` options are for the whole simulation time, and are mutually exclusive with `time_spec` and `node_spec`. If specified with `time_spec` or `node_spec` in one `set_speed_scale` command, an error message will be issued.

Examples

The following example runs dynamic scaling by signal voltage levels:

```
set_speed_scale -node vbusy 0.9 0.5 1 2
```

- If the `vbusy` node value is less than 0.9v, it simulates with default engine tolerance.
- If it is equal to or greater than 0.9v and less than 1v, the result is to tighten the accuracy with an engine tolerance scale of 0.5.
- If it is equal to or greater than 1v, the result is to speed up simulation with engine tolerance scale of 2.

The following example runs dynamic scaling by time windows:

```
set_speed_scale 0 2 1e-6 0.5 2e-6 1
```

- Up to but not including 1us, the simulation speeds up with a loosened tolerance scale of 2.
- Up to 1us but not including 2us, it tightens accuracy with an engine tolerance scale of 0.5.
- From 2us onward, it simulates with the default engine tolerance scale of 1.

The following example shows a performance and accuracy tuning refinement. It relaxes the accuracy of `-acc 5` for the [set_sram_characterization](#) command to gain faster performance by loosening the tolerance scale by 2 for the entire transient simulation. Performance is gained while retaining the tighter front-end optimization level of `-acc 5` of the [set_sram_characterization](#) command.

```
set_speed_scale 0 2  
set_sram_characterization -app power -acc 5
```

The following example sets a scaling factor of 1.5 and specifies the `test_logic` subcircuit for simulation.

```
set_speed_scale -scale 1.5 -subckt test_logic
```

set_sram_characterization

Specifies the performance/accuracy settings for simulating SRAM designs.

Syntax

```
set_sram_characterization
[[-enable] 0|1]
[-ver[sion] 0|1]
[-app[lication] timing|power]
[-acc[uracy] 1|2|3|4|5]
[-resistor_rule 1|2]
[-capacitor_rule 1|2]
[-snps_ip value]
```

Arguments

Argument	Description
-enable 0 1	Sets to enable or disable SRAM design simulation.
-version 0 1	Specifies the <code>-version</code> argument according to the process node of the design: • 0 is the recommended setting for designs that use planar process nodes. • 1 (default) is the recommended setting for designs that use FinFET process nodes. Note that setting <code>-version</code> to 1 enables multicore simulation with a default of two cores. You can adjust the number of cores used with the <code>-mt</code> command line option, or by adding <code>set_multi_core -core</code> to the configuration command file.
-application timing power	Specifies the SRAM simulation types: • <code>timing</code> for a timing simulation (default). • <code>power</code> for a power or leakage simulation. Note that use can use the <code>app</code> alias for this option. The name is case insensitive.
-accuracy 1 2 3 4 5	Specifies the performance/accuracy tradeoff. The default of 3 provides the best combination of performance/accuracy. Specify 1 for the fastest simulation, or 5 for the most accurate. Note that you can use the <code>acc</code> alias for this option.
-resistor_rule 1 2	You can specify one of the following values to control how the PrimeSim XA tool processes resistors: • 1 (default): to specify that resistor optimizations are done during parsing. • 2 to specify that resistor optimizations are done during front-end optimization. Specify this option when you want to protect the resistors that are probed.

Argument	Description
<code>-capacitor_rule 1 2</code>	You can specify one of the following values to control how the PrimeSim XA tool processes capacitors: • 1 (default) to specify that capacitors optimizations are done during parsing. • 2 to specify that capacitor optimizations are done during front-end optimization. Specify this option when you want to protect the capacitors that are probed.
<code>-snps_ip value</code>	Customizes internal tool command settings for specific SRAM designs and technologies of Synopsys IPs. Note: Use this argument only for the simulation of Synopsys memory IP designs.

Description

For details about how to use the `set_sram_characterization` command to simulate SRAM designs, see SRAM Design Simulation in *PrimeSim XA User Guide*.

Caution:

Do not use `set_sram_characterization` with the `set_sim_level` command in the same configuration file. This combination causes the PrimeSim XA tool to issue a warning and ignore the `set_sim_level` command.

Examples

Specifies an SRAM timing simulation with the default setting.

```
set_sram_characterization 1
```

Specifies an SRAM timing simulation with accuracy level 4.

```
set_sram_characterization -application timing -accuracy 4
```

Same as the previous example, but uses an abbreviated name.

```
set_sram_characterization 1 -app timing -acc 4
```

Specifies an SRAM power simulation with the default setting.

```
set_sram_characterization 1 -application power
```

Disables an SRAM simulation.

```
set_sram_characterization 0
```

set_ssl_version

Specifies the version of the [set_sim_level](#) command to be used during the simulation.

Syntax

```
set_ssl_version 1|2
```

Description

This command sets the global [set_sim_level](#) version to be used during the simulation. You can set it to:

- 1 (default) to specify the same behavior as the [set_sim_level](#) command. This is also the default if you do not specify a `set_ssl_version` command.
- 2 to specify the same behavior as the `-ssl2` command line option. For details about this option, see the Running Simulator Version II section in the *PrimeSim XA User Guide*.

Note:

The `-ssl2` command line option takes precedence over the `set_ssl_version 1` command.

set_struct_verilog_option

Controls whether to preserve backslash (\) characters only for buses or all signal nets in the structural Verilog netlists.

Syntax

```
set_struct_verilog_option -escape_id 0|1
```

Description

The command affects only the structural Verilog netlists. You can choose to preserve all backslash (\) characters in the signal names or those in the bus names.

- 0 (default): Preserves backslash (\) characters only for buses.
- 1: Preserve all backslash (\) characters as part of the identifier string.

set_synchronization_level

Controls the synchronization settings between the gate, channel or bulk of the elements.

Syntax

```
set_synchronization_level -level level_value
[-inst inst_name {inst_name}]
[-subckt subckt_name {subckt_name }]
```

Arguments

Argument	Description
-level level_value	Sets the synchronization level. By default, the set_synchronization_level value matches the specified set_sim_level value.
-inst inst_name {inst_name}	Applies the synchronization settings to the specified instances.
-subckt subckt_name {subckt_name}	Applies the synchronization settings to the specified subcircuits.

Description

You use the set_synchronization_level command with set_sim_level to speed up or to increase the accuracy of a simulation. For example, in some designs that use set_sim_level 5, you can get better performance, and close to the same accuracy, with set_synchronization_level 3.

set_synchronization_option

Provides flexibility to control the synchronization between blocks.

Syntax

```
set_synchronization_option -rule full | gate | never
[-inst inst_name {inst_name}]
[-subckt subckt_name {subckt_name}]
[-global_synch 0|1]
[-sync_from_nobreak 0|1|on|off]
```

Arguments

Argument	Description
<code>-rule full gate never</code>	Specifies whether to enable synchronization between different partitions. <code>full</code> specifies that if the gate, drain, source or bulk of a MOSFET are in different partitions, these partitions are evaluated at the same time points. Applying synchronization to all ports of the MOSFET might help to improve accuracy. <code>gate</code> specifies that If the gate of a MOSFET is in a different partition than the channel, those two partitions are evaluated at the same time points. Applying synchronization on the gate of this MOSFET might help to improve accuracy. <code>never</code> turns off the synchronization between different partitions. This is useful for speeding up the simulation when the signal is only fed forward from gate to the channel when <code>set_partition_option -rule</code> is applied.
<code>-inst inst_name {inst_name}</code>	Applies the synchronization rule to the specified instances.
<code>-subckt subckt_name {subckt_name}</code>	Applies the synchronization rule to the specified subcircuits.
<code>-global_synch 0 1</code>	Enables full synchronization. <code>0</code> (default): Disables full synchronization. <code>1</code>: Uses the fully synchronized engine to run the entire simulation on an analog circuit with <100K MOS devices for better performance and accuracy.
<code>-sync_from_nobreak 0 1 on off</code>	When set to <code>1</code> and <code>on</code>, the tool applies synchronization analog blocks through gates for better accuracy, while retaining the performance in the asynchronous part of the simulation.

Description

This command is often used with the [set_partition_option](#) command. When manual partitioning is applied to an instance, different partitions might need to be synchronized to improve the accuracy, or un-synchronized to speed up the simulation. You can apply this command globally or locally.

set_tolerance_level

Provides flexibility for tuning the PrimeSim XA tool relative tolerance level to the [set_sim_level](#) setting. The relative tolerance level controls the sensitivity of the simulator to small voltage changes.

Syntax

```
set_tolerance_level -level tolerance_level [instance_spec]
```

Arguments

Argument	Description
<code>-level</code> <code>tolerance_level</code>	The default relative tolerance levels settings are as follows: • 1 applies the same relative tolerance value used with set_sim_level 1. • 2 applies the same relative tolerance value used with set_sim_level 2. • 3 applies the same relative tolerance value used with set_sim_level 3. • 4 applies the same relative tolerance value used with set_sim_level 4. • 5 applies the same relative tolerance value used with set_sim_level 5. • 6 applies the same relative tolerance value used with set_sim_level 6. • 7 applies the same relative tolerance value used with set_sim_level 7. The following section describes the cases when a lower or higher <code>tolerance_value</code> can improve performance and accuracy.
<code>instance_spec</code>	See the Common Syntax Definitions section for details about the <code>instance_spec</code> argument.

Description

The `set_tolerance_level` command provides more flexibility to tune the PrimeSim XA tool to specific simulation requirements, similar to the [set_tolerance_option](#) command. It lets you adjust the relative tolerance level appropriate to the corresponding [set_sim_level](#) level.

The `set_tolerance_level` command is helpful when:

- A PrimeSim XA simulation is functionally correct at a lower [set_sim_level](#) setting, but a higher [set_sim_level](#) setting is needed to achieve the required accuracy. There are cases when you can use a lower [set_sim_level](#) setting with a higher `tolerance_level` to obtain the desired accuracy with better performance.

For example, if a data converter is functional at `set_sim_level=5` and higher, but the FFT requirements are only met at `set_sim_level=7`. You might find that `set_sim_level=5` and `set_tolerance_level 7` achieves the required accuracy with significantly better performance than `set_sim_level=7`.

- An PrimeSim XA simulation meets the accuracy requirements, but runs slowly. You can set `tolerance_level` to a lower value below the corresponding `set_sim_level` setting to gain better performance with some sacrifice in accuracy. A lower `tolerance_level` setting might be useful when another simulator gets good results with or close to its default settings, but the PrimeSim XA tool runs more slowly.

`set_tolerance_level` can control the relative tolerance level globally or locally. The most significant impact of the relative tolerance level is to adjust the time step of the simulator. There are secondary impacts on MOS lookup table granularity and RC reduction.

The default tolerance values for each `set_tolerance_option` setting are shown in Table 23.

Table 23 *Default Relative Tolerance Option Values*

Tolerance Level	Default set_tolerance_option Value
set_tolerance_level 1	-tol=400 (less accurate)
set_tolerance_level 2	-tol=250
set_tolerance_level 3	-tol=200
set_tolerance_level 4	-tol=150
set_tolerance_level 5	-tol=100
set_tolerance_level 6	-tol=50
set_tolerance_level 7	-tol=0.1 (very accurate)

Examples

This example applies a `set_sim_level` of 6 setting to the entire circuit. For the `a2d` subcircuit, the tolerance parameters are overwritten with a more conservative setting of `set_tolerance_level 7`.

```
set_sim_level 6
set_tolerance_level 7 -subckt a2d
```

set_tolerance_option

Controls the simulation tolerance independent of the [set_sim_level](#) command for more flexibility in tuning performance/accuracy.

Syntax

```
set_tolerance_option -tol tol_value
[-tol_rule 0|1]
[-inst inst_name {inst_name}]
[-subckt subckt_name {subckt_name}]
```

Arguments

Argument	Description
-tol tol_value	Specifies the tolerance value. The most significant impact of this value is to adjust the time step of the simulation. However, the specified tolerance value also has secondary impacts on the MOS look-up table granularity and RC reduction. Table 24 shows the default tolerance values.
-tol_rule 0 1	Applies the PrimeSim XA simulator heuristics as follows: • A value of 0 (default) applies the specified <i>tol_value</i> as a base setting, but the simulator heuristics might override this value in certain parts of the design. The value override used by the simulator heuristics is typically smaller than the user-specified <i>tol_value</i>. • A value of 1 applies the specified <i>tol_value</i> to the entire design. The simulator heuristics do not override the <i>tol_value</i>.
-inst inst_name {inst_name}	Instance names to which <code>set_tolerance_option</code> applies.
-subckt subckt_name {subckt_name}	Subcircuit names to which <code>set_tolerance_option</code> applies.

Description

By providing more flexibility in tuning a PrimeSim XA simulation to specific design requirements, `set_tolerance_option` lets you control the simulation tolerance independent of the [set_sim_level](#) command. You can apply `set_tolerance_option` globally or locally.

The tolerance value (`-tol`) controls the sensitivity of the simulation to small voltage changes. This value is arbitrary and is not based on a unit. Valid values are 0.1 to 800. Note that a value of 800 is extremely aggressive in favor of performance, and most circuits are not functional with this setting.

The default tolerance values for each [set_sim_level](#) setting are shown in [Table 24](#).

Table 24 *Default Relative Tolerance Values*

Accuracy Level	Default set_tolerance_option Value
set_sim_level 3	-tol=200 (less accurate)
set_sim_level 4	-tol=150
set_sim_level 5	-tol=100
set_sim_level 6	-tol=50
set_sim_level 7	-tol=0.1 (very accurate)

The `set_tolerance_option` command is helpful when:

- A PrimeSim XA simulation is functionally correct at a lower `set_sim_level` setting, but a higher `set_sim_level` setting is needed to achieve the required accuracy. There are cases when you can use a lower `set_sim_level` setting with a smaller `tol_value` to obtain the desired performance and accuracy.

For example, if a data converter is functional at `set_sim_level=5` and higher, but the FFT requirements are only met at `set_sim_level=7`, you might find that `set_sim_level=5` and `set_tolerance_option -tol 0.1` achieve the required accuracy with performance significantly better than `set_sim_level=7`.

- A PrimeSim XA simulation meets the accuracy requirements, but runs slowly. You can increase `tol_value` above the default `set_sim_level` setting to gain better performance with some sacrifice in accuracy. A higher `tol_value` setting might be useful when another simulator gets good results with or close to its default settings, but PrimeSim XA runs more slowly.

Examples

This example applies `set_sim_level=6` to the entire circuit, but overrides the tolerance parameters for the `a2d` subcircuit with a more conservative setting of `0.1`.

```
set_sim_level 6
set_tolerance_option -tol 0.1 -subckt a2d
```

set_va_view

Controls whether a SPICE subcircuit or a Verilog-A module definition (or “view”) is used for the entire netlist, or for an instance or subcircuit when both SPICE and Verilog-A definitions exist for that instance or subcircuit.

Syntax

```
set_va_view -view VA|va|SPICE|spice
[-inst inst_name {inst_name}]
[-subckt subckt_name {subckt_name}]
```

Arguments

Argument	Description
-view VA va	Specifies that Verilog-A module definitions have priority over SPICE subcircuit definitions.
SPICE spice	Specifies that SPICE subcircuit definitions have priority over Verilog-A module definitions.
-inst inst_name {inst_name}	Specifies a particular instance or set of instances the set_va_view command affects. See Common Syntax Definitions for more about specifying instance names.
-subckt subckt_name {subckt_name}	Specifies a particular subcircuit or set of subcircuits the set_va_view command affects. See Common Syntax Definitions for more about specifying subcircuit names.

Description

If your design has both SPICE subcircuit definitions and Verilog-A definitions, you can switch between SPICE and Verilog-A depending on the type of analysis you want to perform. This lets you achieve tradeoffs between the speed and accuracy of the simulation for particular definitions. Use the set_va_view command to perform module- and instance-based partitioning of the circuit definitions. You can switch to a Verilog-A module description for a single SPICE subcircuit, or set of subcircuits, within a netlist.

If multiple definitions or “views” of a SPICE subcircuit or Verilog-A module exist, the PrimeSim XA tool defaults to use the view of the parent netlist. If you set the entire netlist to use SPICE (set_va_view -view SPICE), then any subcircuit instance uses the subcircuit definition by default. If you set the entire netlist to use Verilog-A (set_va_view -view VA), then the Verilog-A module definitions is used by default; if the module definition does not exist then the subcircuit view is used.

If you specify set_va_view for a specific subcircuit or instance, then that subcircuit or instance uses the definition you specify (SPICE or Verilog-A), as shown in the following examples.

If set_va_view references a view that does not exist, a warning message appears and the other view is used. If neither a SPICE nor a Verilog-A definition exist for the subcircuit, the PrimeSim XA tool returns an error and the simulation is terminated.

Note:

If an Eldo instance is declared with a `y` element, the Verilog-A definition is used in all cases. It cannot be overridden with the `set_va_view` command.

Examples

The following example shows an HSPICE netlist that uses the SPICE view by default.

```
.hdl mymodule.va
.subckt mymodule a b
...
.ends mymodule
X1 1 2 mymodule
X2 3 4 mymodule
X3 5 6 mymodule
```

The following example sets the SPICE view for all subcircuits and modules in a netlist.

```
set_va_view SPICE
```

The following example sets the Verilog-A view for all subcircuits and modules in a netlist.

```
set_va_view VA
```

The following example uses the Verilog-A module definition for all instances of subcircuit *my_module*.

```
set_va_view VA -subckt mymodule
```

If the netlist contains the command in the following example, the Verilog-A module definition is used for subcircuit instance X2 of *my_module*.

```
set_va_view VA -inst X2
```

For details about how to include Verilog-A definitions in HSPICE, Spectre, or Eldo netlists, see the Netlist Syntax for Verilog-A in the PrimeSim XA Tool section in *PrimeSim XA User Guide*.

set_vector_char

Description

Overwrites the logic-high and logic-low voltages for the input vectors, or the bidirectional vectors when in input mode in the VEC file (without modifying the VEC file).

Note:

VCD and EVCD files are not supported with this command.

Syntax

```
set_vector_char [-vih h_voltage]
                [-vil l_voltage]
                [-node node_name {node_name}]
```

Arguments

Argument	Description
-vih h_voltage	Overwrites the logic-high voltage for input vectors or bidirectional vectors when in input mode. The default value is 3.3 volts.
-vil l_voltage	Overwrites the logic-low voltage for input vectors or bidirectional vectors when in input mode. The default value is 0.0 volts.
-node node_name {node_name}	Specifies the node names inside a vector file. The default is to match all nodes specified in the vector file.

Examples

Overwrites the logic-high and logic-low voltages defined in the vector file with 3.0 and 0.8 volts, respectively.

```
set_vector_char -vih 3.0 -vil 0.8
```

set_vector_option

Description

Enables additional control on the vector file to provide more flexibility.

Note:

Only the WDF, FSDB, OUT, and VPD formats support this feature.

Syntax

```
set_vector_option [-vec_mode mode]
                  [-define_x_state x_state_value]
                  [-check_u_state u_state_value]
                  [-check_z_state z_state_value]
                  [-check_method method_value]
                  [-precedence precedence_switch]
                  [-print_expected print_value]
                  [-node node_name {node_name}]
                  [-check_io io_switch]
```

Arguments

Argument	Description
<code>-vec_mode mode</code>	Specifies the mode to <code>xa</code> (default) for the HSPICE vector file.
<code>-define_x_state state_x_state_value</code>	Defines how to treat the X-state in VEC, VCD, and EVCD files. It can be set to: • 1 (default) sets to the logic-low voltage. • 2 sets to the logic-high voltage. • 3 sets to the mid-point of logic-high voltage and logic-low voltage. • 4 sets to the same voltage as the previous logic state.
<code>-check_u_state u_state_value</code>	Controls the checking of U-state (unknown state) in the VEC file against the actual simulation output. It can be set to: • 0 (default) sets to "Don't Care" or ignored. • 1 enables the checking of U-state. An error is reported if the simulation output does not match with the expected output for U-state as follows: Expected state is U-state, but the simulation output value is less than VOL or greater than VOH. Expected state is not U-state, but the simulation output value is between VOL and VOH.
<code>-check_z_state z_state_value</code>	Controls the checking of high impedance (Z) state in the VEC, VCD and EVCD files against the actual simulation output. It can be set to: • 0 (default) sets to "Don't Care" or ignored. • 1 enables the checking of Z-state. An error is reported if the simulation output does not match with the expected output for Z-state. The vector check with <code>-check_z_state 1</code> not only checks the node HiZ status, but also compares the node logic state (converted from its voltage) against the "EXPECTED VALUE" from the vector file. For example, if "L" is the expected value specified in the vector file, then a node needs to be in an HiZ state and also needs to be in logic-0 state in order to pass the vector check. If it is not in HiZ or not logic-0, the PrimeSim XA tools flags it as a vector error in the <code>.err</code> file.
<code>-check_method method_value</code>	Specifies the checking mechanism of the VCD and EVCD files. It can be set to: • 1 (default) enables continuous checking. • 2 enables strobed-based time point checking.

Argument	Description
<code>-precedence</code> <code>precedence_switch</code>	Determines the behavior of the PrimeSim XA tool when a voltage source in the netlist file and a vector input from a VEC, VCD or EVCD file apply to the same node. It can be set to: <code>error</code> (default) causes the PrimeSim XA tool to error out if. <code>vsrc</code> causes the PrimeSim XA tool to use the voltage source in the netlist.
<code>-print_expected print_value</code>	Controls the printing the waveform of the expected states of the output vectors in the waveform file. You can then compare the expected states with the actual simulation output. It can be set to: <code>0</code> (default) does not generate waveform of the expected states of the output vectors. <code>1</code> generates the waveform (with a <code>#e</code> extension) of the expected states of the output vectors.
<code>-node node_name {node_name}</code>	Applies only to <code>-print_expected</code> argument. If it is not specified, it applies to all the output vectors in the VEC, VCD, and EVCD files.
<code>-check_io message</code>	Determines the behavior of the PrimeSim XA tool when io statement is missing from the VEC file. It can be set to: <code>warn</code> (default) gives a warning and continues the simulation. All vectors are considered as input. <code>error</code> issues an error and stops the simulation.

Examples

The following example generates the expected output waveform in the waveform file for visual comparison with the actual simulation output waveform.

```
set_vector_option -print_expected 1
```

The following example generates the expected output waveform for the following signal names: `d[0]`, `d[1]`, `d[2]`, and `s[0]`.

```
set_vector_option -print_expected 1 -node d[0] d[1] d[2] s[0]
```

The following example instructs the PrimeSim XA tool to report the mismatch of the high impedance (Z) state and U-state between simulated node states and the states specified in the stimulus file.

```
set_vector_option -check_z_state 1 -check_u_state 1
```

set_waveform_option

Sets the output waveform options. You can specify the file format, file-split size, voltage resolution, current resolution and flush percentage.

Syntax

```
set_waveform_option [-grid_v grid_val]
[-grid_i grid_val]
[-compress_v compress_val]
[-compress_i compress_val]
[-compress_fsdb 0|1]
[-tres time_resolution value]
[-flush flush_time | flush_percentage_value%]
[-format format]
[-file split|merge]
[-size max_file_size_MB]
[-disk_full space_in_MB]
[-disk_full_wait_time hour|minute|percentage]
[-output_step step_size]
[-group 0 | 1]
[-parallel_dump 0|1|n]
[-ade_logfile enable_value]
[-wall_time_period time_in_hour]
[-tran_time_period time]
[-double_precision 0|1]
```

Arguments

Argument	Description
-grid_v grid_val	Controls the grid size of the output voltage signal. All output points are rounded to the nearest integer of the grid. If set to 0, the rounding is turned off.
-grid_i grid_val	Controls the grid size of the output current signal. All output points are rounded to the nearest integer of the grid. If set to 0, the rounding is turned off.
-compress_v compress_val	Controls the voltage compression. If set to 0, the compression is turned off. The default is 1e-6.
-compress_i compress_val	Controls the current signal compression. If set to 0, the compression is turned off.
-compress_fsdb 0 1	If set to 1, it compresses the output FSDB file to save the maximum amount of disk space. Use this setting when simulating a large design that creates a large output waveform file. The default is 0, which uses the existing compression algorithm.

Argument	Description
<code>-tres</code> <code>time_resolution_value</code>	Specifies the waveform file time resolution value. The default is 0 fs.
<code>-flush flush_time </code> <code>flush_percentage_value%</code>	By default, the PrimeSim XA tool writes the buffered data to the output file when the output file buffer is full. This argument forces periodic writes to the output file. If a <i>flush_time</i> is specified, the output file is written at an integer multiples of flush time. The flush time is in terms of the transient simulation time, not real time. The value is seconds. If a percentage is specified by using a % character, the file is written at an integer multiples of the percentage of the simulation transient end time. When neither <i>flush_time</i> nor <i>flush_percentage_value</i> values is set, the default is 10%, meaning that the tool writes the data to the output file every 10% of the simulation transient end time.
<code>-format</code> <code>[fsdb out wdf tr0 psf v</code> <code>pd print]</code>	Sets the output waveform file format. See Supported Waveform File Formats in the <i>PrimeSim XA User Guide</i> for more details. By default, PrimeSim XA supports FSDB version 6.1 (fsdb_6.1). The tool also supports two previous versions, which are 6.0 (fsdb_6.0) and 5.9 (fsdb_5.9), for backward compatibility. See the Supported FSDB Versions table in PrimeSim XA Release Notes for the version compatibility information for PrimeSim XA and FSDB. When you specify <code>-format psf</code>, the standalone PrimeSim XA tool dumps a binary PSF file, and PrimeSim XA mixed-signal simulation dumps an ASCII PSF file. When you specify <code>-format print</code> and the enable_print_statement 1 command together, the waveform file is not generated. Only the .print file is generated. You can also use the SPICE option <code>.option print</code>, instead of <code>set_waveform_option -format print</code>.
<code>-file [split merge]</code>	You can use the <code>-file</code> option to set the file generation mode during PrimeSim XA-VCS mixed-signal simulation. If you specify <code>split</code>, two output files are generated by the PrimeSim XA tool and VCS separately. If you specify <code>merge</code>, only one output file is generated. For more information about file splitting, see the Split column in Table 25.
<code>-size max_file_size_MB</code>	Sets the maximum file size before file splitting. You use this argument to force the splitting of the waveform output file to a more convenient size. The file size is specified in megabytes. The minimum file size for the formats that support file splitting is 10 Meg.

Argument	Description
<code>-disk_full space_in_MB</code>	Sets the minimum disk space remaining in the output file disk for writing waveforms. If the available disk space is less than twice the specified value, the PrimeSim XA tool issues an out-of-disk-space error and terminates the simulation. If the available disk space is less than the specified value and the PrimeSim XA tool is invoked with the <code>-intr</code> command-line option, the PrimeSim XA tool enters the interactive mode; the tool issues an out-of-disk-space warning and suspends the simulation. To resume the simulation and exit from the interactive mode, use the <code>icontinue_sim</code> command. The default is 100 M.
<code>-disk_full_wait_time hour minute percentage</code>	Specifies the waiting time (wall time) for disk checking if the available disk space is less than the value specified by the <code>-disk_full</code> option. The supported values are: <code>hour</code>: Specifies the waiting time in hours. For example: 1h. <code>minute</code>: Specifies the waiting time in minutes. For example: 5m. <code>percentage</code>: Specifies the waiting time in a percentage of wall time. For example: 10%. When specified, the tool checks the available disk space per minute until the specified waiting time is up. The tool issues a warning message about how many minutes left to wait every 5 minutes. When no waiting time is left, the tool terminates the simulation with an error message. This option must be used together with the <code>-disk_full</code> option.
<code>-output_step step_size</code>	When you set <code>-output_step</code> to a certain time value, such as 1ns or 100ps, the result format is in fixed time steps by every 1ns or 100ps, repetitively, to dump all probed waveform signals at every step. This capability works for all waveform formats. When you set both <code>-output_step</code> and <code>-tres</code>, the <code>-tres</code> argument is ignored and a warning message issued. Arguments, such as <code>-compress_v</code>, <code>-compress_i</code>, <code>-grid_v</code> and <code>-grid_i</code>, are still honored, as long as the exact data point requested by <code>-output_step</code> is there. Note: When <code>set_probe_window</code> is used together with <code>-output_step</code>, the first time step is the start time of <code>set_probe_window</code>. There is no default step size setting.

Argument	Description
<code>-group 0 1</code>	Creates a group file that includes the partitioned waveforms. The supported values are: • 0: The partitioned waveform files are not grouped. • 1 (default): If any command triggers waveform partitioning, the partitioned waveform files are grouped by either a virtual file or a group file. The PrimeSim XA tool generates a virtual file with the .vf.fsdb extension for waveform files in FSDB format. For WDF format, a group file is generated with the .grp extension. Note: The PrimeWave Design Environment WaveView tool is currently the only waveform viewer that supports group files.
<code>-parallel_dump 0 1 n</code>	Parallel waveform output writes the signals to the same number of output files as the number of cores in the simulation. This option supports only the FSDB and WDF formats. The output files have a mw#.fsdb or mw#.wdf extension. The PrimeSim XA tool determines which signals go into one output file or another, based on the number of cores. In addition, a group (*.grp) file is generated to represent the output files as a single result. The PrimeWave Design Environment WaveView tool can read the group file as single entry and display all the waveforms. The group file and split output files are all written to the same simulation output directory. If you specify a waveform output format other than .fsdb or .wdf, parallel output is disabled, in which case the PrimeSim XA tool issues a warning and uses serial waveform output instead. You can specify one of the following values: • 0 disables creating parallel files (default). • 1 specifies that the number of cores used to create parallel output files is equal to the number of core specified in the simulation (with <code>-mt</code> command line option), but parallel output is not forced. It happens only when the number of signals exceeds a certain threshold determined by the PrimeSim XA tool. • <i>n</i> is a value greater than 1 that specifies the number of cores used in parallel dumping is <i>n</i>, and forces the creation of parallel output files.
<code>-ade_logfile 0 1</code>	You can specify: • 1 to generate a log file for PrimeWave Design Environment WaveView to use in the PrimeSim XA-Cadence ADE interface. • 0 for not generating the log file (default).

Argument	Description
<code>-wall_time_period</code> <code>time_in_hour</code>	Specifies the period of wall time to partition and write waveform data to multiple files. You need to specify an integral value in hours. By default, the value is 24. The specified wall time is added to the prefix of the waveform files, like <code>xa.wt9h.fsdb</code> or <code>xa.wt18h.fsdb</code>. If the period is not specified in a whole number of hours, the PrimeSim XA tool rounds it to the nearest integer and gives a warning. This option is mutually exclusive with the <code>-tran_time_period</code> option.
<code>-tran_time_period</code> <i>time</i>	Specifies the period of transient time to partition and write waveform data to different files. You could specify the value with or without unit, like <code>2e-9</code>, <code>4n</code> or <code>0.1u</code>. By default, the transient time is in unit of second. Example: <pre>-tran_time_period 1e-7</pre> The specified transient time is added to the prefix of waveform file, like <code>xa.1e-7.fsdb</code> or <code>xa.2e-7.fsdb</code>. This option is mutually exclusive with the <code>-wall_time_period</code> option.
<code>-double_precision</code> 0 1	Sets to 1 to enable double precision in the writer library to keep more significant digits for generating more accurate waveforms. The default is 0.

Description

The `set_waveform_option` command provides options to control behaviors when generating output waveform files. You can specify the file format, file-split size, voltage resolution, current resolution, flush percentage, parallel waveform output, and so on.

Use the `-format` option to set the waveform output file format. When you specify file format using the `-wavefmt` command-line option or the `.opt post=format` or `.opt xa_cmd="set_waveform_option ..."` command, the tool honors the setting to specify the output waveform file format in the following precedence order from high to low:

1. The `-wavefmt` option on the command line
2. The `set_waveform_option` command in a command script file (the command script file is evaluated as if the file were used on the final line of the netlist)
3. The last `.opt post=format` or `.opt xa_cmd="set_waveform_option ..."` command that appears in the netlist

Use the `-compress_v` and `-compress_i` options to specify the resolution values for the slope-based lossy compression. Any data points that deviate from a straight line by less

than this lossy compression value are dropped. The default voltage resolution is 1uV, and the default current resolution is 1pA.

Use the `-grid_v` and `-grid_i` options to specify the resolution values for rounding lossy compression. Any data points that deviate from resolution grid are rounded off to the closest grid. The default voltage resolution is 1 uV, and the default current resolution is 1 pA.

When using the `compress_v` and `compress_i` options to specify resolution values, there is an impact on the results of `.meas` command and dynamic CCK analysis. This is because the compression is applied before the measurements or dynamic checks are done. Using the `grid_v` and `grid_i` options has no impact on the measurements or dynamic CCK analysis results; it only has an impact on the generated waveforms.

Use the file splitting options to write the waveform data into multiple waveform files, based on file size, signals, or probe window. In addition, use the `-wall_time_period` or `-tran_time_period` option to split waveforms with respect to transient time or wall time without saving the simulation state.

Note:

The `-tran_time_period` option of the `set_waveform_option` command is mutually exclusive with the `-tran_time_period` option of the `set_save_state` command. When both options are specified, the setting of the `set_save_state -tran_time_period` command takes precedence.

Use the `-group 1` option to generate a virtual file for grouping all the waveform files that are generated during waveform splitting. This virtual file is generated with a `vf.fsdb` extension for waveform files in FSDB format. If the waveform file format is WDF, a group file is generated with a `.grp` extension. You can then use a waveform viewer that supports FSDB format to load the virtual FSDB file for waveform display, such as nWave. A group file is supported only in PrimeWave Design Environment WaveView.

The PrimeSim XA tool provides various methods for waveform splitting. For more information, see the Setting Waveform Options section in *PrimeSim XA User Guide*.

Table 25 lists the PrimeSim XA default resolutions and the supported file-splitting features.

Table 25 Default Settings for Output File Formats (N/S means not supported)

Output Format	-compress_v	-compress_i	-grid_v	-grid_i	-tres	-flush	-size
fsdb	1 uV	1 pA	1 uV	1 pA	0 fs	Yes	Default no split
out	1 uV	1 pA	1 uV	1 pA	0 fs	Yes	Default no split
wdf	1 uV	1 pA	1 uV	1 pA	0 fs	Yes	Default no split

Table 25 *Default Settings for Output File Formats (N/S means not supported)*
(Continued)

Output Format	-compress_v	-compress_i	-grid_v	-grid_i	-tres	-flush	-size
tr0	1 uV	1 pA	N/S	N/S	0 fs	N/S	N/S
psf	1 uV	1 pA	N/S	N/S	0 fs	N/S	N/S
vpd	1 uV	1 pA	1 uV	1 pA	0 fs	Yes	Default no split

Examples

The following example sets the output format to `out`, a split size of 1000 megabytes, current grid to 1nA, voltage grid to 1nV, and writes to the waveform output file every 5% of the simulation time. The time resolution is set to a minimum of 1 ps. A single `xa.wdf.grp` waveform file is generated with three simulation results.

```
set_waveform_option -format out -size 1000 -grid_i 1n -grid_v 1n -tres 1p
-flush 5%
```

If a netlist includes:

```
.alter
.param clk_freq=30n
+ Cload=1p
+ vsupply=3.3
.param clk_freq=40n
+ Cload=10p
+ vsupply=3.5
.param clk_freq=25n
+ Cload=1p
+ vsupply=2.8
.option xa_cmd="set_waveform_option -format wdf -group 1"
```

The following example updates the waveform file after every 100 ns of simulation time.

```
set_waveform_option -flush 100ns
```

In the following example, the PrimeSim XA tool issues a warning when the disk space is less than two GB and stops when it is less than one GB.

```
set_waveform_option -disk_full 1000
```

The following example writes waveforms to multiple files in parallel by wall time of 2 hours. A virtual FSDB file is generated.

```
set_waveform_option -format fsdb -parallel_dump 1 -wall_time_period 2
```


The following example splits waveforms by transient time of 100 ns and generates a virtual FSDB file.

```
set_waveform_option -format fsdb -tran_time_period 100n -group 1
```

The following example partitions waveforms based on the maximum file size of 500 MB and wall time period of 24 hours:

```
set_waveform_option -size 500 -wall_time_period 24 -group 1
```

When simulation is complete, the following files are created:

- Waveform files: prefix.wt1d.fsdb, prefix.fsdb.2, prefix.wt2d.fsdb ...
- Virtual file: prefix.vf.fsdb

See Also

- [set_probe_window](#)
- [enable_print_statement](#)

set_waveform_sim_stat

Lets you display the CPU (or wall) time as a function of the transient simulation time in the same WaveView window. This command helps you detect bottlenecks when the CPU or wall time is increasing faster than the transient simulation time.

Syntax

```
set_waveform_sim_stat -type sim_stat_type {sim_stat_type}
```

Arguments

Argument	Description
<code>-type sim_stat_type {sim_stat_type}</code>	Specify one or more of the following options: • <code>cpu</code> dumps the CPU time versus the transient simulation time in the waveform file. • <code>wall</code> dumps the wall time versus the transient simulation time in the waveform file. • <code>vm</code> dumps the virtual memory usage versus the transient simulation time in the waveform file. • <code>pvm</code> dumps the peak virtual memory usage versus the transient simulation time in the waveform file. • <code>pm</code> dumps the peak physical memory (which resides in core memory) versus the transient simulation time in the waveform file. • <code>all</code> dumps the CPU and wall times and memory usage versus transient simulation time in the waveform file. It also dumps <code>cpu_load</code> and remaining time versus transient simulation time. • <code>cpu_load</code> dumps the CPU load in the waveform file • <code>remaining</code> dumps the estimated remaining time (as shown on stdout) is in the waveform file.

Description

The `set_waveform_sim_stat` command dumps the specified time type (either CPU time, wall time, or both, or memory) in the waveform output file on the Y axis (X axis being the usual transient simulation time). The signal labels on the Y axis are `CPU Time`, `Wall Time`, `Virtual Mem`, `Peak Virtual Mem` and `Physical Memory`. The Y-axis unit is seconds for all the times and is MB for the memory. This new wave is reported in the waveform output file in all the supported output file formats.

The CPU or wall time or memory value is reported every time it is updated in the standard output (or in the GRID or LSF log file).

You can specify multiple `set_waveform_sim_stat` commands.

Examples

Dumps the wall, CPU, and peak virtual memory as f(transient time) in the log file and as Y-axis values, while the X-axis shows the transient time.

```
set_waveform_sim_stat -type wall cpu pvm
```

Dumps the wall time=f(transient time) in the log file.

```
set_waveform_sim_state -type wall
```

set_wildcard_rule

Enables additional control on wildcard matching. This command provides the capability to match one or all levels of hierarchy in instance and node names for some commands when an asteroid symbol (*) is used as a wildcard character.

Syntax

```
set_wildcard_rule -match* match*_switch  
[-match_ic* matchic*_switch]  
[-node_alias node_alias_value]  
[-model_alias model_alias_value]
```

Arguments

Argument	Description
<code>-match* match*_switch</code>	Specifies * as a wildcard character to match all levels of hierarchy or one level of hierarchy. It can be specified as: • <code>all</code> (default) to match the * character to all levels of hierarchy. • <code>one</code> to match the * character to one level of hierarchy. Some commands support <code>-level</code> and <code>-limit</code> options. When using the commands with the <code>-limit</code> or <code>-level</code> option, it is recommended to use the with the <code>-match all</code> option. When the <code>-match all</code> option is specified along with the commands with the <code>-level</code> and <code>-limit</code> options, the * wildcard character matches to the number of hierarchy levels specified in those options. See the Understanding the Difference Between the -level and -limit Options section for more information.
<code>-match_ic* matchic*_switch</code>	Specifies * as a wildcard character to match all levels of hierarchy or one level of hierarchy for only the <code>ic</code> or <code>nodeset</code> related commands. It can be specified as: • <code>all</code> (default) to match the * character to all levels of hierarchy. • <code>one</code> to match the * character to one level of hierarchy.
<code>-node_alias node_alias_value</code>	Enables all node aliases to be reported in the waveform files. It can be set as: • <code>0</code> (default) does not report node aliases. • <code>1</code> reports node aliases.
<code>-model_alias model_alias_value</code>	Enables <code>.malias</code> model alias names to be matched. It can be set as: • <code>0</code> does not match the <code>.malias</code> model alias names. • <code>1</code> (default) matches the <code>.malias</code> model alias names.

Description

Sets the rules for wildcard matching of an asteroid symbol (*) as a wildcard character. Some commands support scoping options that allow wildcard matching for instance and node names. Subcircuit names do not support wildcard matching. Refer to each individual command for supported command scoping options and the [Command Scoping](#) section for more information.

The following table lists the options supported for each command.

Command	-match*	-node_alias
check_node_excess_rf	x	x
check_node_hotspot	x	x
check_node_quick_rf	x	x
check_node_zstate	x	x
check_timing_edge	x	x
check_timing_hold	x	x
check_timing_pulse_width	x	x
check_timing_setup	x	x
force_node_voltage	x	
load_operating_point	x	
map_ba_terminal	x	
print_pcm_state	x	
probe_waveform_current	x	x
probe_waveform_ixba	x	x
probe_waveform_logic	x	x
probe_waveform_pcm	x	x
probe_waveform_va	x	x
probe_waveform_voltage	x	x
release_node_voltage	x	
report_node_cap	x	

Command	-match*	-node_alias
report_operating_point	x	
report_power	x	
report_sim_activity	x	
set_ccap_level	x	
set_flash_option	x	
set_model_level	x	
set_model_option	x	
set_oscillator	x	
set_partition_option	x	
set_sim_level	x	
set_synchronization_level	x	
set_synchronization_option	x	
set_tolerance_level	x	
set_tolerance_option	x	
set_va_view		
skip_circuit_block	x	

The following table lists the options supported for each netlist statement.

Netlist Format	Netlist Statement	-match*	-match_ic*	-node_alias
HSPICE and Eldo	.probe	x		x
HSPICE and Eldo	.print	x		x
HSPICE	.lprint (see the Using the .LPROBE and .LPRINT Statements section in <i>PrimeSim XA User Guide</i> for more information)	x		x
Spectre	save	x		x
HSPICE and Eldo	.measure	x		x

Netlist Format	Netlist Statement	-match*	-match_ic*	-node_alias
HSPICE	.biaschk	x		x
Eldo	.setsoa	x		x
Eldo	.extract	x		
HSPICE and Eldo	.ic	x	x	x
HSPICE and Eldo	.nodeset	x	x	x

Examples

The following example sets the wildcard rule for the * wildcard character to match at one level of hierarchy only. Therefore, the `probe_waveform_voltage` command prints the waveforms of only the nodes under instance `x1`.

```
set_wildcard_rule -match* one
probe_waveform_voltage -v x1.*
```

The following example sets the wildcard rule for the * wildcard character to match all levels of hierarchy. The `probe_waveform_voltage` command does not print any waveforms because the command defaults to `-limit 3` and the hierarchy `x1.x2.x3.x4.x5` is beyond three levels of hierarchy.

```
set_wildcard_rule -match* all
probe_waveform_voltage -v x1.x2.x3.x4.x5.*
```

The following example sets the wildcard rule for the * wildcard character to match all levels of hierarchy that include the nodes at top level and the nodes 3 levels underneath the top hierarchy.

```
set_wildcard_rule -match* all
probe_waveform_voltage -v x1.*
```

In the following example, the PrimeSim XA tool defaults to probe only the voltage of nodes `x1.a2`, `x1.ao`, and `x1.aout`. Nodes `x1.ini` and `x1.out` are not probed because they are the hierarchical aliases for `ain` and `aout`, respectively. You can probe the hierarchical aliases if you specify the `set_wildcard_rule -node_alias 1` command.

```
x1 ain aout resnetwork

.subckt resnetwork ini out
r1 ini a2 10
r2 a2 ao 100
r3 a2 out 15
r4 ao aout 10
r5 aout 0 20
```

```
.ends
.probe v(x1.*)
```

set_zstate_option

Description

Provides more controls to the [check_node_zstate](#) command. See [Table 34](#) in the [check_node_zstate](#) command page for the conducting condition for each element.

Syntax

```
set_zstate_option -idsth mosfet_value
[-rule rule_value {rule_value}]
[-mos_diode 0|1]
[-mos_diode_vth mos_diode_vth_value]
[-mos_vth_rule 0|1]
[-mos_vth_scale scale_value]
[-vbeth bjt_value]
[-diode_vth value]
[-va_rule 0|1]
[-res_th res_th_value]
[-isrc_rule 0|1]
[-xdummy 0|1]
[-merge_subckt 0|1]
[-format format_switch]
```

Arguments

Argument	Description
-idsth mosfet_value	Specifies the threshold for a MOSFET to be considered conducting. When $I_{ds} > idsth_value$, it is considered conducting. The default is 1e-8 ampere.
-rule rule_value {rule_value}	Selects the rule of conducting for a MOSFET. The MOSFET is considered conducting if one of the rules is met. See check_node_zstate for the conducting conditions.
-mos_diode 0 1	When set to 1, the MOS is set as a MOS diode. A MOS diode is a MOSFET with drain terminals connected to gate terminals or source terminals connected to gate terminals. The default is 0. A MOS diode is considered conducting when the following conducting rules are met: • For NMOS: $V_{g/d} - V_s$ (or $V_{g/s} - V_d$) $> mos_diode_vth$ • For PMOS: $V_d - V_{g/s}$ (or $V_s - V_{g/d}$) $> mos_diode_vth$ Use the <code>-mos_diode_vth</code> option to define the <code>mos_diode_vth</code> value. For more information about conducting conditions, see check_node_zstate.

Argument	Description
<code>-mos_diode_vth</code> <code>mos_diode_vth_value</code>	Specifies the threshold for considering a MOS diode conducting. The default is 0.2 volt.
<code>-mos_vth_rule 0 1</code>	When set to 1, the substrate bias (Vsb) dependent threshold voltage (Vth) is used for the MOS conducting rule. The default is 0, meaning that the Vth for Vsb = 0V is used for the MOS conducting rule. For more information about conducting conditions, see Table 34 in the check_node_zstate command page.
<code>-mos_vth_scale</code> <code>scale_value</code>	Scales the vth value of MOS device model card parameters during the HiZ nodes determination process. <i>scale_value</i>: A positive real number as multiplication factor to the MOS vth used for HiZ check. The default is 1. HiZ check "vth" = "default vth" * <scale_val>
<code>-vbeth bjt_value</code>	Specifies the threshold for a BJT to be considered conducting. The default is 0.64 volt.
<code>-diode_vth value</code>	Specifies the threshold for a diode to be considered conducting. The default is 0.2 volt. When it is set to 0, a diode is considered not conducting. For more information about conducting conditions, see check_node_zstate .
<code>-va_rule 0 1</code>	Selects the reporting rule if there is no conducting path from the voltage source to the output nodes of Verilog-A. It can be set to: 0 (default): Reports the output node of Verilog-A 1: Does not report the output node of Verilog-A
<code>-res_th res_th_value</code>	Specifies the resistance threshold for a resistor to be considered conducting. A resistor is considered conducting if the resistance value is less than <i>resth_value</i> . By default, a resistor is always considered conducting.
<code>-isrc_rule 0 1</code>	Selects the rule of conducting for an independent current source. An independent current source is the current source with a value that is not depending on other sources. It can be set to: 0 (default): Non-conducting 1: Conducting

Argument	Description
<code>-xdummy 0 1</code>	Controls additional checking of the fanout devices on a high-impedance node. It can be set to: • 0 (default): Disables the checking and always reports all high-impedance nodes. • 1: Enables the checking and only reports the high-impedance node (and fanout devices) when it connects to at least one non-dummy device. A device is considered dummy if one of the conditions is met: ◦ If the source and gate terminals of a MOSFET are in high-impedance state ◦ If the drain and gate terminals of a MOSFET are in high-impedance state ◦ If the collector and base of a BJT are in high-impedance state ◦ If the emitter and base of a BJT are in high-impedance state
<code>-merge_subckt 0 1</code>	Controls the reporting due to subcircuit merging. It can be set to: • 0: Disables subcircuit merging and generates more comprehensive reporting • 1 (default): Enables subcircuit merging
<code>-format format_switch</code>	Specifies the format of the output file. The supported format can be one of the following: • <code>xa</code> (default): PrimeSim XA standard format • <code>cck</code>: PrimeSim XA CCK format

Examples

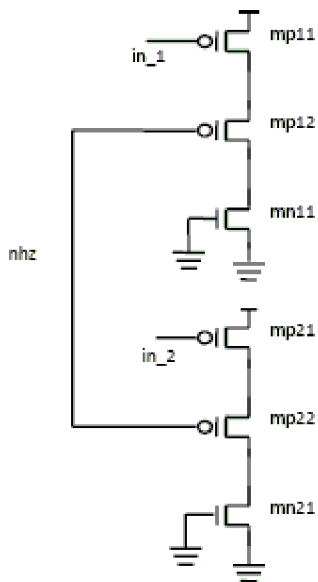
This following example shows how the `-xdummy` argument filters the high-impedance node and reports the violation to an output file named `hiz_out.errz`.

```
check_node_zstate -node * -title hiz_check1 -error_file hiz_out -xdummy 1
```

Note:

Node `nhz` is a high-impedance node. By default, it is always reported or when `-xdummy 0` is set.

When `-xdummy 1` is set, it enables the checking of the fanout devices of the node `nhz`. Node `nhz` is only reported if there is one non-dummy device. When the controlling nodes of `mp12` and `mp22` are logic-high, both `mp12` and `mp22` are off (dummy) so node `nhz` is not reported.



skip_circuit_block

Skips subcircuit instances without editing netlist files. The command can also replace the skipped instances with the estimated capacitive loads at the ports.

Syntax

```
skip_circuit_block [-load 0|1]
[-inst instance_name]
[-subckt subckt_name]
[-except_inst instance_name]
[-except_subckt subckt_name]
```

Arguments

Argument	Description
-load 0 1	• 0 (default) skips the specified instances as black boxes. • 1 skips the specified instances as gray boxes.
-inst instance_name	Specifies the relative path names of the circuit instances to be skipped.
-subckt subckt_name	Specifies the names of subcircuits whose instances are to be skipped.
-except_inst instance_name	Specifies the absolute path names of the circuit instances to be excluded from skipping.

Argument	Description
<code>-except_subckt</code> <code>subckt_name</code>	Specifies the names of subcircuits whose instances are to be excluded from skipping.

Description

When you use the `-load 1` argument, the blocks to be skipped are replaced with estimated capacitive loads to improve the simulation accuracy. These skipped blocks are called *gray boxes*.

When you use the default `-load 0` argument, the blocks are skipped without any load replacements. These skipped blocks are called *black boxes*.

Be careful how you use this command, because it can affect the circuit connectivity. It checks for floating nodes, gates, and dangling nodes after the circuit block has been removed and issues appropriate warnings.

Examples

Skips the `xcell` block as a black box.

```
skip_circuit_block -load 0 -inst xcell
```

Skips the `x1` instance as a black box inside the `cell` subcircuit.

```
skip_circuit_block -subckt cell -inst x1
```

Skips all instances of the `cell` subcircuit as gray boxes, except the `x1` instance inside the `xcell2` instance.

```
skip_circuit_block -load 1 -subckt cell -except_inst xcell2.x1
```

Skips all instances of the `cell` subcircuit as black boxes, except instances of the `optin` subcircuit inside `cell`.

```
skip_circuit_block -subckt cell -except_subckt optin
```

Skips the `xc1` instance as a black box, except the `x5` instance inside `xc1`.

```
skip_circuit_block -inst xc1 -except_inst xc1.x5
```

Skips all instances in the circuit as black boxes, except the instances of the `buffer` subcircuit, and the instances matching `xc*`.

```
skip_circuit_block -except_subckt buffer -except_inst xc*
```

Skips all instances of subcircuits matching `buf*` as black boxes, and all instances matching `xr*` as gray boxes.

```
skip_circuit_block -load 0 -subckt buf*
skip_circuit_block -load 1 -inst xr*
```

source

The `source` command is a native TCL command, not a PrimeSim XA-specific command. You use it to include other command script files.

Syntax

```
source file_name
```

Description

This command reads and references another command script file. Use it with the `-c` command line option, not the `.opt xa_cmd` option.

Examples

```
#Test case-specific file
set_sim_level 4
#Load common settings
source xa_common_settings.tcl
```

vcd2vec

Converts VCD format into VEC format.

Syntax

```
vcd2vec [[-d] -nvcd vcd_file -nsig sig_file \
        [-nvec vec_file] [-start start_time -stop stop_time]] \
        [[-d] -nevcd evcd_file [-nsig sig_file] \
        [-nvec vec_file] [-start start_time -stop stop_time]]
```

Arguments

Argument	Description
-d	Enables debug mode.
-nvcd vcd_file	Specifies the input VCD file.
-nsig sig_file	Specifies the signal file.
-nvec vec_file	Specifies the output VEC file.
-nevcd evcd_file	Specifies the input EVCD file.

Argument	Description
<code>-start <i>start_time</i></code>	Sets the start time for conversion.
<code>-stop <i>stop_time</i></code>	Sets the stop time for conversion.

3

PrimeSim XA Interactive Commands

This chapter provides the syntax for all of the PrimeSim XA interactive and DC interactive mode commands, as described in the following sections:

- [PrimeSim XA Interactive Commands](#)
- [DC Interactive Mode Commands](#)

PrimeSim XA Interactive Commands

This section describes the following interactive commands:

- [alias](#)
- [icheck_node_zstate](#)
- [iclose_log](#)
- [icontinue_sim](#)
- [idelete_break_point](#)
- [iforce_node_voltage](#)
- [ilist_break_point](#)
- [ilist_force_node](#)
- [imatch_elem](#)
- [imatch_node](#)
- [iopen_log](#)
- [iprint_connectivity](#)
- [iprint_dcpath](#)
- [iprint_flash_cell](#)

- [iprint_elem_info](#)
- [iprint_exi](#)
- [iprint_help](#)
- [iprint_node_info](#)
- [iprint_pcm_state](#)
- [iprint_subckt](#)
- [iprint_time](#)
- [iprint_tree](#)
- [iprobe_waveform_current](#)
- [iprobe_waveform_va](#)
- [iprobe_waveform_voltage](#)
- [iquit_sim](#)
- [irelease_node_voltage](#)
- [ireport_node_cap](#)
- [ireport_operating_point](#)
- [isearch_node](#)
- [iset_break_point](#)
- [iset_diagnostic_option](#)
- [iset_env](#)
- [iset_interactive_option](#)
- [iset_interactive_stop](#)
- [iset_save_state](#)
- [iset_waveform_option](#)
- [iset_zstate_option](#)
- [iset_speed_scale](#)

Note:

The following PrimeSim XA interactive commands are not available in a VCS PrimeSim AMS mixed-signal simulation.

- `icontinue_sim`
- `idelete_break_point`
- `ilist_break_point`
- `iset_break_point`
- `iset_diagnostic_option`
- `iset_interactive_stop`
- `iset_save_state`

alias

Description

Creates an alias name for the interactive commands. When arguments are specified, an alias is defined for each *alias_name* for whose *actual_name* is given.

Syntax

```
alias alias_name actual_name
```

Arguments

Argument	Description
<i>alias_name</i>	Creates an alias for an interactive command.
<i>actual_name</i>	Defines the interactive command.

Examples

Creates an alias `pns` for the `iprint_node_info` command.

```
alias pns iprint_node_info
```

Creates an alias `ipe` for the `iprint_elem_info` command with the argument `-index`.

```
alias ipe iprint_elem_info -index
```

icheck_node_zstate

Performs a high-impedance node check in interactive mode.

Syntax

```
icheck_node_zstate -node node_name {node_name}  
[-fanout <0|1|2>]  
[-rule rule_value {rule_value}]  
[-subckt subckt_name {subckt_name}]  
[-except_subckt subckt_name {subckt_name}]  
[-diode_vth value]  
[-idsth ids_value]  
[-vbeth vbeth_value]  
[-except_node node {node}]  
[-file file_name]  
[-report report_value {report_value}]
```

Arguments

Argument	Description
<code>-node node_name {node_name}</code>	Specifies the node names at which the high-impedance state is checked. The node name can be a single node or a node name with a wildcard character that represents a group of node names. The rule of asterisk (*) character is controlled by set_wildcard_rule .
<code>-fanout <0 1 2></code>	Selects the type of nodes for high-impedance checking: • A value of 0 enables all nodes. • A value of 1 enables nodes that have a direct connection to a transistor gate. • A value of 2 enables all nodes that have a direct connection to a transistor bulk. If you do not use this option, the default is 0.
<code>-rule rule_value {rule_value}</code>	Selects the rule of conducting for a MOSFET. The MOSFET is considered conducting if one of the rules is met. Other rules are as follows: NMOS: • $V_{gs} > V_{th}$ (-rule 1) • $I_{ds} > idsth$ (-rule 2) • $V_g > VDD - 0.1$ (-rule 3) PMOS: • $V_{gs} < V_{th}$ (-rule 1) • $I_{ds} > idsth$ (-rule 2) • $V_g < 0.1$ (-rule 3) The default is 1 and 2.
<code>-subckt subckt_name {subckt_name}</code>	Specifies the subcircuits to be checked. Wildcard characters are allowed in subcircuit names.
<code>-except_subckt subckt_name {subckt_name}</code>	Specifies the subcircuits to be excluded from high-impedance checking. This argument should be used only when <code>-node node_name</code> contains wildcard characters. Wildcard characters are allowed in subcircuit names

Argument	Description
<code>-diode_vth value</code>	Sets the forward bias threshold for diodes. A diode with $v(a,c)$ greater than <i>value</i> is considered conducting for a high impedance check. A value less than 0 causes diodes to be always considered non-conducting. The default value is 0.2V.
<code>-idsth ids_value</code>	Specifies the threshold for a MOSFET to be considered conducting when <i>ids</i> exceeds <i>value_1</i> . If not specified, the default is 1e-8 ampere. See the following Description section for more information about the conducting criteria.
<code>-vbeth vbeth_value</code>	Specifies the threshold for BJT to be considered conducting when <i>Vbe</i> exceeds <i>value_2</i> . If not specified, the default is 0.64 volt. See the following Description section for more information about the conducting criteria.
<code>-except_node node_name {node_name}</code>	Specifies the nodes to be excluded from high-impedance checking. This argument should be used only when <code>-node node_name</code> contains wildcard characters.
<code>-file file_name</code>	If you specify a file name the output is written to that file instead of standard output. If you specify an existing file name, that file is overwritten.
<code>-report report_value {report_value}</code>	You can specify the following keywords: <code>node</code> lists Hi-z nodes. <code>channel</code> lists all channel-connected devices for the node. <code>gate</code> lists all gate-connected devices for the node. The default is <code>node</code>.

Description

This interactive command enables the PrimeSim XA tool to diagnose specified nodes staying in a high-impedance (floating) state.

A node stays in a high-impedance state if there is no conducting path from any voltage source to the node. A conducting path consists of conducting elements. An element is conducting if that specific element meets the following criteria:

Device	Rule
NMOS	$V_{gs} > V_{th}$ (rule=1) $I_{ds} > idsth$ (rule=2) $V_g > VDD - 0.1$ (rule=3)

Device	Rule
PMOS	• $V_{gs} < V_{th}$ (rule=1) • $I_{ds} > I_{dsth}$ (rule=2) • $V_g < 0.1$ (rule=3) For more information about the rule values, see iset_zstate_option.
NPN	$V_{be} > v_{beth}$
PNP	$V_{be} < -v_{beth}$
Diode	Forward-biased.
Resistor	Always conducting.
Inductor	Always conducting.
Capacitor	Never conducting.
Verilog-A	Never conducting.
Controlled sources	Always conducting.

To diagnose if a high impedance condition is harmful, you need to know which gates it is driving. To diagnose how to fix the high impedance condition, the channel-connected elements must be reported. The voltage on the high impedance nodes is always reported. The hierarchical subcircuit names are always be printed when you specify `-report gate` and `channel`. The MOS terminal voltages are also printed.

iclose_log

Description

Closes the interactive mode log file that was opened by the [iopen_log](#) command. The log file contains all records of interactive mode commands and the results reported by the commands entered between `iopen_log` and `iclose_log`.

Syntax

```
iclose_log
```

icontinue_sim

Description

Continues the transient simulation from the initial stop point to the following stop point. If `time` and `unit` are specified, the simulation stops and enters the interactive mode at

`time t+(time) (unit)`. The `-to` option specifies the absolute time to which the simulation proceeds, then enters interactive mode.

Syntax

```
icontinue_sim -i time[unit] [-to time[unit]]
```

Arguments

Argument	Description
<code>-i time[unit]</code>	Stops the simulation and enters the interactive mode at time <code>t+(time) (unit)</code> <code>t</code> is the current simulation time. If <code>t+(time) (unit)</code> is less than the simulation stop time, the simulation continues until <code>t+(time) (unit)</code>, then enters the interactive mode at <code>t+(time) (unit)</code>. White space is not allowed between <code>time</code> and <code>unit</code>. If <code>time[unit]</code> is not specified, the simulation continues until it is completed, or until the <code>ctrl-C</code> is entered. The default for <code>unit</code> is seconds, if not specified.
<code>-to time[unit]</code>	Specifies the absolute time to which the simulation proceeds, then enters interactive mode. For example, if you specify <code>icontinue_sim -to 11ns</code> at time 9ns, the transient simulation runs to 11ns. This command issues a warning, and is ignored, if the time specified by <code>-to</code> is less than the current time.

Note:

The `icontinue_sim` command is not available in a VCS PrimeSim AMS mixed-signal simulation.

Examples

Continues the simulation for another 10ns.

```
XA> icontinue_sim 10n
```

Runs the simulation to 10ns and returns to interactive mode, assuming current time is less than 10ns.

```
XA> icontinue_sim -to 10n
```

idelete_break_point

Description

Removes the stop points.

Syntax

```
idelete_break_point -point all|stop_point1 [stop_point2 ... stop_pointn]
```

Arguments

Argument	Description
-point all	Removes all the stop points.
-point stop_point1 stop_point2... stop_pointn	Removes the specified stop points.

Note:

The `idelete_break_point` command is not available in a VCS PrimeSim AMS mixed-signal simulation.

Examples

```
XA> ilist_break_point
```

```
1: break at time: 1 ns
2: break at time: 10 ns
```

```
XA> idelete_break_point 2
```

```
XA> ilist_break_point
```

```
1: break at time: 1 ns
```

The following example removes stop point 2.

```
XA> idelete_break_point -point 2
```

See Also

- [ilist_break_point](#)

iforce_node_voltage

Description

Forces the specified nodes to stay at the specified constant voltage. The node voltage stays at the same value from the current time until either the end of simulation or when the constant node voltage status is released by [irelease_node_voltage](#).

Syntax

```
iforce_node_voltage -node node_name {node_name}  
-voltage voltage_value
```

```
[-slope t_value]  
[-time time_val]
```

Arguments

Argument	Description
<code>-node <i>node_name</i></code> <code>{<i>node_name</i>}</code>	Specifies the node name to force.
<code>-voltage <i>voltage_value</i></code>	Specifies the voltage value to set.
<code>-slope <i>t_value</i></code>	Forces the voltage with a ramp of <i>t_value</i> (in seconds per volt). The default is 1ps and must be positive a positive value.
<code>-time <i>time_val</i></code>	Specifies the time when the nodes are forced. The default is the current break point.

Examples

The `cn` node is forced at 2 V starting at the current time. It remains at this value until the end of the simulation unless you use the `irelease_node_voltage` command for the same node.

```
XA> iforce_node_voltage -node cn -voltage 2
```

ilist_break_point

Description

Lists the existing stop points.

Syntax

```
ilist_break_point -list [number]
```

Arguments

Argument	Description
<code>-list <i>number</i></code>	Lists the last <i>number</i> of the stop points. If <i>not</i> specified, lists all the stop points.

Note:

The `ilist_break_point` command is not available in a VCS PrimeSim AMS mixed-signal simulation.

Examples

Lists the existing stop points.

```
XA> ilist_break_point -at 1n
XA> iset_break_point -at 10n
XA> ilist_break_point

1: break at time: 1 ns
2: break at time: 10 ns

XA> ilist_break_point -list
```

ilist_force_node

Lists all nodes you specified with [force_node_voltage](#).

Syntax

```
ilist_force_node -file file_name
```

Arguments

Argument	Description
<code>-file <i>file_name</i></code>	Specifies the file name that contains the list of forced nodes.

Description

If you specify the `-file` argument, the forced node list is written to the specified file. Otherwise the list is echoed to the standard output.

Note that a node forced with [force_node_voltage](#) is not listed in the output of this command unless the simulation time advanced since the node was forced.

Examples

```
XA> iforce_node_voltage xbuffer.pd_d -v 1.2
XA> iforce_node_voltage xbuffer.sigi -v 0
XA> ilist_force_node

XA> iconf 1n
XA> ilist_force_node
xbuffer.pd_d (4) = 1.2
xbuffer.sigi (3) = 0
```

imatch_elem

Description

Prints a list of the element indexes and hierarchical element names that match the specified pattern.

Syntax

```
imatch_elem -pattern pattern ...
```

Arguments

Argument	Description
<i>pattern</i>	Patterns can contain the * wildcard character. The printed list shows all matches to the specified pattern. You can set wildcard matching for the hierarchy with iset_interactive_option .

Examples

In this example, the *x1* pattern matches 3 elements.

```
XA> imatch_elem *x1*
5 x1.r1
6 x1.r2
7 x1.x1.r1
```

See Also

- [iset_interactive_option](#)

imatch_node

Prints a list of the node indexes and node names that match the specified pattern.

Syntax

```
imatch_node -pattern pattern ...
[-limit level]
[-port enable_value]
```


Arguments

Argument	Description
<i>pattern</i>	Patterns can contain the * wildcard character. The printed list shows all matches to the specified pattern. You can set wildcard matching for the hierarchy with iset_interactive_option .
<code>-limit level</code>	Specifies wildcard matching only down to the specified hierarchical depth. The default hierarchical limit level is 3.
<code>-port enable_value</code>	Enables or disables the matching of subcircuit port names. The default value of is taken from the iset_interactive_option command.

Description

`imatch_node` prints a list of matched nodes. The PrimeSim XA tool first reports the value of `-limit` applied, then lists the node index and node name matched with one item per line. Finally, the PrimeSim XA tool reports the total number of matched nodes.

The [iset_interactive_option](#) command settings apply to this command. The "*" wildcard can be set to match or not to match the hierarchical delimiter. The "*" wildcard only matches primary node names, unless you specify the `-port` argument. When you specify `-port`, the "*" wildcard also matches alias node names.

Examples

In this example, the `*a` pattern matches 3 nodes.

```
XA> imatch_node *a
1 a
2 x1.a
3 x1.fa
pattern *a matched 3 nodes
```

This example finds all nodes ending in `a` at the top-level of the netlist, level 0.

```
XA> imatch_node *a -limit 0
1 a
pattern *a matched 1 nodes
```

This example finds all nodes ending in `out` down to the hierarchical depth of 4 and also reports ports.

```
XA> imatch_node *out -limit 4 -port 1

15 x0.x1.x2.aout
15 x0.x1.x2.x3a.out
```

```
16 x0.x1.x2.bout
16 x0.x1.x2.x3b.out
```

See Also

- [iset_interactive_option](#)

iopen_log

Description

Opens the interactive mode *logfile_name*, which contains the record of the interactive mode commands and the results reported by these commands until the log file is closed by the [iclose_log](#) command. Only one log file can be opened at one time.

Syntax

```
iopen_log -file logfile_name [-mode append|write]
```

Arguments

Argument	Description
-file logfile_name	Specifies the interactive mode log file name.
-mode append	Specifies that contents are appended to pre-existing files using the same file name.
-mode write	Specifies that contents overwrite pre-existing files using the same file name. Default is <code>-mode write</code> .

Examples

This example opens the log file named `logfile`.

```
XA> iopen_log -file logfile
```

See Also

- [iclose_log](#)

iprint_connectivity

Description

Prints the detailed node connectivity information for the given node names or indices. The elements are categorized into channel-connected, gate-connected, and other elements.

or

Syntax

```
iprint_connectivity -node node_name {node_name}
[-print gc|cc|o|all]
[-on current_value]
[-file_format text|csv]
[-except_inst inst_name {inst_name}]
[-except_subckt subckt_name {subckt_name}]
[-xprobe 0|1]
[-short_resistor resistor_value]
[-file file_name]
[-file_append file_name]
iprint_connectivity -index node_index {node_index}
[-print gc|cc|o|all]
[-on current_value]
[-file_format text|csv]
[-except_inst inst_name {inst_name}]
[-except_subckt subckt_name {subckt_name}]
[-xprobe 0|1]
[-short_resistor resistor_value]
[-file file_name]
[-file_append file_name]
```

Arguments

Argument	Description
-node node_name {node_name}	Specifies the node name, which can contain wildcard characters.
-print gc cc o all	Specifies to selectively print the gate-connected, channel-connected, other, or all elements. If not specified, the default prints gate-connected and channel-connected elements.
-index node_index	Specifies node index.
-on current_value	Prints only those instances that have a current exceeding the specified <i>current_value</i> , and also reports bulk current.
[-file_format text csv]	The default file format is <i>text</i> (existing format). You can also specify <i>csv</i> to create a file in Comma Separated Format, with a <i>.csv</i> extension. You can only use this argument together with <i>-file</i> or <i>-file_append</i> argument. If you use it with <i>-file_append</i> , the existing file must be in CSV format,
-except_inst inst_name {inst_name}	Specifies the instances to be ignored. The default is not to ignore any instances.

Argument	Description
<code>-except_subckt subckt_name</code> <code>{subckt_name}</code>	Specifies the subcircuits to be ignored. The default is not to ignore any subcircuits.
<code>-xprobe 0 1</code>	When set to 1, the current is also calculated in the instance ports by summing up the currents for each instance belonging to the instance. You can only use this argument with the <code>-on</code> and <code>-file</code> arguments. The default is 0.
<code>-short_resistor resistor_value</code>	When a resistor connected to the net has a value less than <code>resistor_value</code> , the connectivity at the other port of the resistor is reported.
<code>-file file_name</code>	Writes output only to the specified file. If the file exists it is overwritten. The file is located in the directory specified by the <code>-o</code> command line argument.
<code>-file_append file_name</code>	Writes output only to the specified file. If the file exists this argument appends the output to the existing file, otherwise it creates the specified file. The file is located in the directory specified by the <code>-o</code> command line argument.

Examples

```
iprint_connectivity -node xspine_0.xdqr_0.xl850.osc_2
iprint_connectivity xspine_0.xdqr_0.xl850.osc_2 -print cc
iprint_connectivity -index 3
```

If you specify:

```
iprint_connectivity -node vdd -on 0 -file vdd_curr -file_format csv
```

This command reports all devices connected to vdd in the `vdd_curr.csv` file:

```
Node=vdd,
Channel-connected,
x1,mp1,Ig,1.69508e-08u
x1,x1,x2,mp1,Is,2.70874e-06u
x1,x1,x3,mp1,Ig,5.65257e-08u
x1,x1,x1,mp1,Is,1.95718e-06u
x1,x1,x2,mp1,Is,2.9166e-06u
x1,x2,x3,mp1,Ig,5.63572e-08u
x1,x2,x1,mp1,Is,1.9574e-06u
x1,x3,mp1,Is,7.94681e-07u
x1,x4,mp1,Ig,2.07707e-08u
x1,x5,mp1,Is,7.94681e-07u
x1,x6,mp1,Ig,2.07707e-08u
x1,x7,mp1,Is,6.89369e-07u
```

Chapter 3: PrimeSim XA Interactive Commands

PrimeSim XA Interactive Commands

```
x1,x8,mp1,Is,3.03595e-06u
x1,x9,mp1,Is,7.19612e-07u
x2,mp1,Ig,1.69505e-08u
x2,x1,x2,mp1,Is,2.70874e-06u
x2,x1,x2,mp2,Ib,2.70874e-15u
...
```

If you specify:

```
iprint_connectivity -node vdd -on 1e-6 -file vdd_curr -file_format csv
-xprobe 1
```

This command reports all devices connected to vdd that have a current value greater than 1uA in the `vdd_curr.csv` file:

```
x1,,,,,0.014712663984
x1,x0,,,0.0147117659165
x1,x0,x34,,0.010773832
x1,x0,x34,m4,0.010558004
x1,x0,x34,m3,0.000215828
x1,x0,x41,,0.003750335
x1,x0,x41,m3,0.003750335
...
```

If you specify

```
iprint_connectivity -node vdd -on 0 -file vdd_curr -file_format csv
-exception_inst x1.x1
```

This command reports all devices connected to vdd, except the devices in `x1.x1` instance, in `vdd_curr.csv` file:

```
Node=vdd,
Channel-connected,
x1.mp1, Ig,1.69508e-08u
x1.x2.x2.mp1, Is,2.9166e-06u
x1.x2.x3.mp1, Ig,5.63572e-08u
x1.x2.x1.mp1, Is,1.9574e-06u
x1.x3.mp1, Is,7.94681e-07u
x1.x4.mp1, Ig,2.07707e-08u
x1.x5.mp1, Is,7.94681e-07u
x1.x6.mp1, Ig,2.07707e-08u
x1.x7.mp1, Is,6.89369e-07u
x1.x8.mp1, Is,3.03595e-06u
x1.x9.mp1, Is,7.19612e-07u
x2.mp1, Ig,1.69505e-08u
x2.x1.x2.mp1, Is,2.70874e-06u
x2.x1.x2.mp2, Ib,2.70874e-15u
...
```

See Also

- [iprint_node_info](#)

iprint_dcpath

Finds and prints the DC path information.

or

Syntax

```
iprint_dcpath -ith ival [-node node_name {node_name}]  
[-at tval {tval}]  
[-file file_name]  
iprint_dcpath -ith ival [-node node_name {node_name}]  
[-period period_value  
[-start start_time]  
[-end end_time]]  
[-file file_name]
```

Arguments

Argument	Description
<code>ith ival</code>	Sets the current threshold value. The default is 50uA.
<code>-node node_name {node_name}</code>	Specifies the terminal node names of the DC current path. The DC path search starts from any node specified in this node list and ends when it reaches either another node in the list or a DC voltage source node. If you do not specify a node, the PrimeSim XA tool reports DC current paths between any pair of voltage source nodes.
<code>-at tval {tval}</code>	Specifies the specific time points at which the path checking occurs.
<code>-period period_value</code>	When you specify the <code>-period</code> argument the DC path search occurs at integer multiples of <code>period_value</code> beginning at the current time if you do not use the <code>-start</code> argument.
<code>-start start_time</code>	Specifies the first time that periodic path checking occurs. If you do not use <code>-start</code> periodic checking starts at the current time. You can only use <code>-start</code> with the <code>-period</code> argument.
<code>-end end_time</code>	Specifies that no path checking occurs past the <code>end_time</code> . If you do not use <code>-end</code> path checking occurs until the end of the simulation. You can only use <code>-end</code> with the <code>-period</code> argument.
<code>-file file_name</code>	Writes the DC path report to the specified file.

Description

The `iprint_dcpath` command searches for and reports DC paths. The DC path search starts from any specified node and ends when the search reaches either another node in the list or a DC voltage source node. If you do not specify a node, the PrimeSim XA tool reports the DC current paths between any pair of voltage source nodes. The path is only reported through MOS and resistor elements.

Examples

```
XA> iprint_dcpath -ith 1e-6

path 1 from vdd to 0 @ 1.04e-07s

xmos.mn (NMOS: lds=3.33804e-06
  drain vdd 3
  source xmos.n1 0.0530892
  gate g 0.6
xmos.mn2 (NMOS: lds=3.33793e-06)
  source 0 0
  drain xmos.n1 0.0530892
```

iprint_flash_cell

Prints information for flash core cell elements.

Syntax

```
iprint_flash_cell -dvth value -inst inst_name
{inst_name}
[-file filename]
[-save save_filename]
```

Arguments

Argument	Description
-dvth value	Specifies the minimum vth change for cells to be printed.
-inst inst_name	Specifies the instance names. You can use an asterisk (*) wildcard character to represent all instances matching the pattern.
-file filename	Specifies a file name to which the command output is written.
-save save_filename	Specifies a file name to which a command file is written with the configuration command to initialize the cells to their current state. You can include this file to run subsequent simulations.

Description

`iprint_flash_cell` identifies and reports the flash core cell instances from specified instance names having a threshold voltage shift (`-dvth`) with either of the following parameters:

- Greater than or equal to the specified value if it is positive or zero.
- Less than or equal to the specified value if it is negative or zero.

The reported `Vth` value is the current threshold voltage of the cell. The `dVth` value is the change in threshold voltage and the `delvto` value is the initial change in threshold voltage for the cell. The `delvto` value should correspond to the `delvto` value from the instance parameter.

Examples

```
XA> iprint_flash_cell -dvth 0 -inst *
XF0.MCELL, Vth=-1.0986, dVth=-2.8487, delvto=0.5
XF1.MCELL, Vth=-0.74198, dVth=-2.4921, delvto=-0
XF2.MCELL, Vth=-0.92028, dVth=-2.6704, delvto=0.25
```

iprint_elem_info

Prints the detailed element information for the given element names at the specific time of activation.

or

Syntax

```
iprint_elem_info -elem element_name {element_name}
[-report brief]
[-file file_name]
iprint_elem_info -index elem_index {elem_index}
[-report brief]
[-file file_name]
```

Arguments

Argument	Description
<code>-elem element_name</code>	Specifies the element name, which can include wildcard characters.
<code>-index elem_indexn</code>	Specifies element index.
<code>-report brief</code>	Provides a brief version of the output that reports only the element terminal connectivity, voltage, and currents. Other information is suppressed.

Argument	Description
<code>-file file_name</code>	Writes <code>iprint_elem_info</code> output to the specified file.

Description

You can also provide a subcircuit instance as an element. In this case, the PrimeSim XA tool prints the subcircuit name, the list of its ports, and the voltages on each port. The detailed element information includes:

- Element name, element type, and model name
- Element terminal connectivity
- Element parameters and values
- Element terminal voltages

MOSFET-specific information:

- MOS logic state (`ON/OFF`)
- MOS effective length and width (`Leff` and `Weff`)
- MOS conductance (`gds` and `gm`)
- MOS threshold voltage (`Vth`)
- MOS Voltage-dependent diode capacitance (`cbs` and `cbd`)
- MOS Voltage-dependent gate capacitance (`cgs`, `cgb`, and `cgd`)
- MOS I_{ds} current (`Ids`)

Examples

Prints information for element `x1.x2.m1`.

```
XA> iprint_elem_info x1.xi@1477.mpt1
```

```
Elem=X1.XI@1477.MPT1 (203) Type=PMOS Model=PENH
D=X1.X001 (232) G=X1.N@1514 (130) S=VDD (14) B=VDD (14)
  Vd=0.0108579 Vg=1.69226 Vs=1.69213 Vb=1.69213
Weff=11.9178u Leff=0.205123u PD=13.4u PS=13.4u AD=4.2u^2 AS=4.2u^2
Vt=-0.73418 OFF
Ids=-2.7808e-07u
gds=2.86161e-13 gm=6.111e-12
cgs=3.03919f cgd=3.03903f cgb=4.59196f cbs=6.99465f cbd=4.48262f
id=0.105008u ig=-0.0446823u is=0.000726083u ib=-0.0610515u
```

```
XA> iprint_elem_info X1.RI@5234
```

```
Elem=X1.RI@5234 (161) Type=R
```

```
N1=X1.N@5446 (195) N2=X1.N@5445 (194)
R=33
Vn1=1.69267 Vn2=1.69267
I=0.0036124u
```

```
iprint_elem_info -elem x1.x2.m1
```

Prints information for element indices 1, 3, and 2.

```
iprint_elem_info -index 1 3 2
```

Prints subcircuit information and port voltages for a subcircuit instance.

```
XA> iprint_elem_info x1
Elem=x1 Type=subckt subckt=mysub1
PORT 1 port=a - 11 (1) v=1
PORT 2 port=b - 12 (2) v=1
PORT 3 port=c - 13 (3) v=2.65
PORT 4 port=d - 14 (4) v=1
```

See Also

- [iprint_connectivity](#)
- [iprint_node_info](#)

iprint_exl

Prints elements with excessive currents.

Syntax

```
iprint_exl -inst inst_name {inst_name}
[-ith ivalue]
[-file file_name]
[-report report_value {report_value}]
```

Arguments

Argument	Description
-inst <i>inst_name</i>	Specifies the hierarchical element names to be searched. You can use wildcard characters (*) in the instance names. The set_wildcard_rule conventions apply.
-ith <i>ivalue</i>	Specifies the current threshold that must be exceeded for the element to be reported. The default value is 50uA.
-file <i>file_name</i>	Specifies the output file name.

Argument	Description
<code>-report report_value</code> <code>{report_value}</code>	<i>report_value</i> can be: <code>subname</code> to specify a list of subcircuit names of the hierarchical instance components to be printed in the output. <code>brief</code> to report only element terminal connectivity, voltages, and currents. Other information is suppressed.

Description

The `iprint_exi` command reports the current through any device terminal that exceeds the threshold. The following device types are checked and reported:

- MOS
- Resistor
- BJT
- Diode

Examples

The output format for elements found that exceed the current threshold is the same as [iprint_elem_info](#). If you specify a list of subcircuit names with the `-report` argument, the hierarchical element instance names have the following general format:

Elem=X0.X1.X2...Xn.modelname

Subckt: X0=x0name X1=X1name X2=x2name ... Xn=Xnname

For example:

```
XA> iprint_exi -ith 1n xmos.m* -report subname
Elem=Xmos.mn (6) Type=NMOS Model=nch.7
D=vdd (5) G=g (3) S=Xmos.n1 (16) B=vb (4)
Vd=3 Vg=1.81293e-90 Vs=-0.00837316 Vb=1.81293e-90
M=1
Weff=2.016u Leff=0.948223u PD=2.365u PS=2.365u AD=0.351792u^2
AS=0.351792u^2 SA=0u SB=0u
Vt=0.19898 OFF
Ids=0.26352u
gds=5.38557e-10 gm=0
cgs=4.30157f cgd=0.416708f cgb=3.65854f cbs=1.20401f cbd=0.487088f
id=0.26352u ig=-0.0663667u is=-0.00752308u ib=-0.189631u
Subckt: Xmos=mymos
```

iprint_help

Description

Displays the syntax and a brief description of the specified interactive commands.

Syntax

```
iprint_help -cmd command_name1 ... command_namen
```

Arguments

Argument	Description
<i>command_namen</i>	Specifies the interactive commands to be displayed on the screen.

Examples

```
iprint_help iprint_node_info
```

This example prints information about the `iprint_node_info` command.

iprint_node_info

Description

Prints the node voltage, node index, and simulation time for the given node names. Each value is evaluated at the current simulation time, when the node voltage is last updated.

or

Syntax

```
iprint_node_info -node node_name {node_name}  
iprint_node_info -index node_index {node_index}
```

Arguments

Argument	Description
<code>-node <i>node_name</i></code>	Specifies node name. Supports wildcard characters.
<code>-index <i>node_index</i></code>	Specifies node index.

Examples

Prints information for node `xalu3.xlatch2.q`.

```
XA> iprint_node_info x1.sout

Node=X1.SOUT (225)
V=1.69417 V dV/dt=-0.00244323 V/ns t=1 ns
```

```
XA> iprint_node_info -node xalu3.xlatch2.q
```

Prints node information for node indices 1, 3, and 2.

```
XA> iprint_node_info -index 1 3 2
```

See Also

- [iprint_connectivity](#)
- [iprint_elem_info](#)

iprint_pcm_state

Description

Prints the state of Phase Change Memory (PCM) cells so you can inspect them in interactive mode.

Syntax

```
iprint_pcm_state inst_name
```

Arguments

Argument	Description
<i>inst_name</i>	Selects PCM cells to be printed interactively. You can specify a wildcard character in the <i>inst_name</i> .

Examples

```
XA> iprint_pcm_state x1.x2.*
x1.x2.r1 0
x1.x2.r2 0
x1.x2.r3 1
```

iprint_subckt

Description

Prints the list of hierarchical instance names for all instances of the specified subcircuit. Note that this command does not support wildcard characters.

Syntax

```
iprint_subckt subckt_name [-file file_name]
```

Arguments

Argument	Description
<i>subckt_name</i>	Specifies the subcircuit name.
<i>-file file_name</i>	Writes command output to the specified file.

Examples

```
XA> iprint_subckt nand2
```

```
x1.x2.x3.xnand1  
x1.x2.x3.xnand2  
x1.x5.xnand1
```

iprint_time

Description

Prints the current simulation time.

Syntax

```
iprint_time
```

Arguments

Argument	Description
N/A	

Examples

```
iprint_time
```

iprint_tree

Description

Prints information about the hierarchical instance tree. Only subcircuit instances are displayed. If the instance list is omitted, it is assumed to be an asterisk (*). The output can also be dumped to a file.

Syntax

```
iprint_tree -inst inst_list
[-limit val]
[-a enable_value]
[-def enable_value]
[-file filename]
```

Arguments

Argument	Description
-inst <i>inst_list</i>	Specifies an instance list. The underlying hierarchy of instances matching the pattern is printed.
-limit <i>val</i>	Specifies that the hierarchical depth to which the tree is printed. If not specified, this setting defaults to 3.
-a <i>enable_value</i>	Specifies that the full hierarchical name of each instance is displayed.
-def <i>enable_value</i>	Specifies that the subcircuit name is also displayed.
-file <i>filename</i>	Specifies that the command output is also written to the named file.

Examples

```
XA > iprint_tree -limit 0 -a 1 -def 1
x1 (dco_xtl)
x2 (dco)
```

```
XA > iprint_tree x1* -limit 1 -a 1 -def 1
x1 (main)
x1.xadd_204_u1 (muxi21x4)
x1.xadd_204_u7 (nor2x3)
```

iprobe_waveform_current

Creates device current waveform output.

Syntax

```
iprobe_waveform_current [[-i|i1] instance_name {instance_name}]
  [-in instance_name {instance_name}]
  [-iall instance_name {instance_name}]
  [-subckt subckt_name]
  [-limit level]
  [-delete enable_value]
  [-isub | subckt_instance_name.port {subckt_instance_name.port}]
```

Arguments

Argument	Description
<code>-i i1 instance_name {instance_name}</code>	Specifies current through terminal 1. If you do not specify <code>-i i1</code> , the instance name must be the first argument. You can use wildcard characters in the instance name. Note that option applies only to instances not specified by <code>-subckt</code> .
<code>-in instance_name {instance_name}</code>	Specifies current through terminal <i>n</i> , where <i>n</i> is a positive integer. You can use wildcard characters in the instance name. Note that option applies only to instances not specified by <code>-subckt</code> .
<code>-iall instance_name {instance_name}</code>	Specifies current through all terminals. You can use wildcard characters in the instance name.
<code>-subckt subckt_name</code>	Specifies the subcircuit name of the specified instances to be probed.
<code>-limit level</code>	Specifies the hierarchy level. The default is 3.
<code>-delete enable_value</code>	If enabled, deletes the specified current probe instead of adding a current probe. the default off.
<code>-isub subckt_instance_name. port {subckt_instance_name .port}</code>	<code>-isub</code> or <code>-x</code> probe returns the total current flowing into the subcircuit port. The <i>subckt_instance</i> is a subcircuit instance name, and <i>port</i> is a name of the subcircuit port. Positive values indicate the current is flowing into the subcircuit port. Negative values indicate the current is flowing out of the subcircuit port.

Description

During interactive debugging, it is sometimes necessary to plot additional device currents for viewing that were not specified for printing in the netlist, and to remove other device currents. This command supports device current probing in interactive mode.

Examples

In order to add/delete device current probes, you need to predefine the scope for possible device current probes with the `iprobe_waveform_current` command in the configuration file. Then, during the interactive mode, you can add the device current probes that you defined in the configuration file. You can delete the ones defined from the predefined scope and the ones defined in the netlist. For example, if you want to probe and delete some of the device currents under the `xd` sub-instance during the simulation, specify the following scope in the configuration file:

```
iprobe_waveform_current xd*
```

In interactive mode, you can probe any device current under that scope. For example:

```
iprobe_waveform_current xd.xcell10.*
```

Adds current probes of device matching `xd.xcell10.*`, such as `i(xd.xcell11.r1)` and `i(xd.xcell11.mn1)`.

```
iprobe_waveform_current -iall xd.xcell11.*
```

Adds current probes of devices matching `xd.xcell11.*`, such as `i1(xd.xcell11.r1)`, `i2(xd.xcell11.r1)`, `i1(xd.xcell11.mn1)`, `i2(xd.xcell11.mn1)`, `i3(xd.xcell11.mn1)`, and `i4(xd.xcell11.mn1)`.

Note:

You need to set the scope using the `-iall` option in the configuration file.

```
iprobe_waveform_current xd.* -limit 2
```

Adds current probes of devices under `xd.*` below two hierarchical levels.

```
iprobe_waveform_current * -subckt subcell
```

Adds current probes of all devices under the `subcell` subcircuit.

You can also delete the specified current probe with:

```
iprobe_waveform_current xd.xcell10.* -delete 1
```

Deletes all device current probes under `xd.xcell10`.

```
iprobe_waveform_current xd.xcell10.m1i23  
-delete 1
```

Deletes the `xd.xcell10.m1i23` device current probe.

If you specify:

```
iprobe_waveform_current * -delete 1
```

This command deletes all the current probes. It includes all the interactive current probes (the probes added interactively) and the normal current probes.

iprobe_waveform_va

Probes the values of Verilog-A variables or parameters, branch voltages, branch currents and writes them to the plot file during interactive mode.

Syntax

```
iprobe_waveform_va variable_name [[-var] {variable_name}] \
    [-subckt subckt_name] [-limit limit_val] \
    [-branch_voltage named_branch_list] \
    [-branch_current named_branch_list]
```

Arguments

Argument	Description
<code>-var {variable_name}</code>	Specifies the hierarchical path to the Verilog-A variable.
<code>-subckt subckt_name</code>	Specifies to probe the variables in all instances of the specified subcircuit. If you specify <code>-subckt</code> the <code>variable_name</code> is local to the subcircuit.
<code>-limit limit_val</code>	Specifies the hierarchy level down to which values of Verilog-A variables and parameters are probed when you use wildcard characters. A value of 0 specifies the top level. The default for <code>limit_val</code> is 3. When you specify <code>-subckt</code> , <code>limit_val</code> is relative to the scope of <code>subckt_name</code> .
<code>-branch_voltage named_branch_list</code>	Specifies the hierarchical path to the named Verilog-A branch for which branch voltage is probed.
<code>-branch_current named_branch_list</code>	Specifies the hierarchical path to the named Verilog-A branch for which branch current is probed.

Description

The `iprobe_waveform_va` command:

- Probes the value of Verilog-A variables or parameters, branch voltages, branch currents and writes them to the plot file during interactive mode. It does not probe Verilog-A ports or electrical nodes. Use the [iprobe_waveform_voltage](#) command to probe electrical signals.
- Scopes only to Verilog-A instances/modules. It does not print SPICE MOS model parameters.

The output to the waveform file is the hierarchical path and the variable name. You must predefine the scope in the configuration file with the `iprobe_waveform_va` command.

There are no signal access functions such as `v()` or `i()` for the voltage and current signals. The separator between the hierarchical path and the variable name is the hierarchical separator for the simulation as defined by the [set_sim_hierid](#) command or the `.hier` command (Eldo® format only).

The PrimeSim XA tool supported wildcard character (`*`) can be used in the *variable_name* identifier. The wildcard character can be used to match one level, or all levels, of the hierarchy.

Examples

Probes the variable count in the `x1` module. The variable appears in the waveform file as `x1.count`.

```
iprobe_waveform_va -var x1.count
```

Probes all variables and parameters in the `x1` module.

```
iprobe_waveform_va -var x1.*
```

Probes all variables and parameters in all instances of `mymodule`.

```
iprobe_waveform_va * -subckt mymodule
```

Probes the branch voltage of `b1` inside `x1` module.

```
iprobe_waveform_va -branch_voltage x1.b1
```

Probes the branch current of `b1` inside `x1` module.

```
iprobe_waveform_va -branch_current x1.b1
```

See Also

- [iprobe_waveform_current](#)
- [iprobe_waveform_voltage](#)

iprobe_waveform_voltage

Creates a voltage waveform output.

Syntax

```
iprobe_waveform_voltage -v node_name {node_name}  
[-vn instance_name {instance_name}]  
[-vall instance_name {instance_name}]  
[-subckt subckt_name]
```

```
[-limit level]
[-port enable_value] [-index index {index}]
[-delete enable_value]
```

Arguments

Argument	Description
<code>-v node_name{ node_name}</code>	Specifies the node name at which the voltage is probed.
<code>-vn instance_name</code>	Specifies the name of the instance at which the voltage of terminal <i>n</i> is probed, where <i>n</i> is a positive integer. Note this argument applies only to instances not specified by <code>-subckt</code> .
<code>-vall instance_name</code>	Specifies the name of the instance at which the voltage of all terminals are probed. Note this argument applies only to instances not specified by <code>-subckt</code> .
<code>-subckt subckt_name</code>	Probes the nodes in all instances of the named subcircuit. If you specify this argument, the nodes or instances should be in the subcircuit instance hierarchy level.
<code>-limit level</code>	Specifies the hierarchy level down to which the voltage is probed. When <code>-subckt</code> is specified, the <code>-limit level</code> is relative to where the particular node is located in the hierarchy. A value of 0 specifies the top level of the subcircuit. The default for <i>level</i> is 3.
<code>-port enable_value</code>	If you specify an <code>enable_value</code> value of 1, a wildcard character matches the subcircuit port names.
<code>-delete enable_value</code>	If enabled, removes the specified signals from the plot list.
<code>-index index {index}</code>	Writes the signals that match the specified indexes to the plot file.

Description

Probes the voltage on a node or on the pin of a primitive instance. The voltage waveform is written to the output file in the format specified by the `post` option in the netlist. Note that only the FSDB and WDF formats support adding new waveforms to the file on the fly.

You can use wildcards (*) with the `-v`, `-vn`, and `-vall` arguments. When used with `-v`, the port alias matching is controlled by the `-port` argument or the [iset_interactive_option](#) setting. The `-port` option specified with the command takes precedence. The [set_wildcard_rule](#) setting also applies.

Probes specified by this command are in addition to the `.probe` statement in the HSPICE or ELDO netlist files or the `save` statements in the Spectre netlist files.

Long simulations, or simulations where some nodes have a high level of activity, can produce very large waveform files. To minimize waveform file loading time in these files, you can direct signals to separate waveform files and keep file sizes smaller.

If you use PrimeWave Design Environment WaveView to display the waveform file, you must close and reopen the file to be able to see the voltages you have added.

Examples

Adds probes for all nodes in instance `x1`. Wildcard matching is influenced by [set_wildcard_rule](#) and [iset_interactive_option](#).

```
XA> iprobe_waveform_voltage x1.*
```

Deletes all voltage probes.

```
XA> iprobe_waveform_voltage * -delete 1
```

Adds voltage probes for all nodes in instance `x1` down to level 2 of the hierarchy.

```
XA> iprobe_waveform_voltage x1.* -level 2
```

iquit_sim

Terminates the simulation.

Syntax

```
iquit_sim
```

Arguments

Argument	Description
N/A	

Examples

This example terminates the simulation.

```
iquit_sim
```

irelease_node_voltage

Description

Releases the node voltages from the values fixed by [iforce_node_voltage](#). When you specify this command, the simulation results determine the node voltages.

Note:

Nodes of voltage sources, Verilog-A outputs, and nodes that have been optimized out cannot be forced/released.

Syntax

```
irelease_node_voltage -node node_name {node_name}  
    [-time time_val] [-soft time_value]
```

Arguments

Argument	Description
-node <i>node_name</i> { <i>node_name</i> }	Releases voltages from the specified nodes.
-time <i>time_val</i>	Specifies the time when the nodes are released. The default is the current time.
-soft <i>time_value</i>	When a node has been forced and then is released, unexpected spikes can occur on that node because the node is suddenly driven/loaded again. This argument adds a resistive path to the node to take over the current when the node is released. The user-defined <i>time_value</i> specifies the time needed by the node to stabilize again after it has been released.

Examples

The `cn` signal previously forced to a given value is released at the current time. The simulation results determine the `cn` voltage value until the end of the run unless a new [iforce_node_voltage](#) command is specified.

```
xa> irelease_node_voltage cn
```

ireport_node_cap

Reports capacitance information for the specified nodes.

Syntax

```
ireport_node_cap -node node_name {node_name}  
    [-group group_name]  
    [-limit limit_value]  
    [-report basic|detail]
```

Arguments

Argument	Description
<code>-node node_name {node_name}</code>	Reports capacitance for the node names you specify. You can use wildcard characters in the node names.
<code>-group group_name</code>	Creates a group name for the nodes you specify with <code>-node</code> . If you specify this option, all nodes for a <code>report_node_cap</code> command are grouped together. The PrimeSim XA tool reports the capacitance information based on this group. Use this option only for a flat, postlayout design.
<code>-limit limit_value</code>	Specifies the hierarchy level down to which the PrimeSim XA tool reports the capacitance information. The default is 3.
<code>-report basic detail</code>	Specifies the type of report to print. Use the <code>basic</code> keyword (the default) to print only the basic capacitance information. Use the <code>detail</code> keyword to print a detailed report.

Description

The reported node capacitance information includes total node capacitance, wire capacitance, gate capacitance of a MOSFET, and junction capacitance of a MOSFET. You can specify multiple commands in a simulation. The PrimeSim XA tool processes each command separately.

`report_node_cap` outputs the capacitance information in a `*.cap#` file.

In a prelayout design, capacitance is reported as:

$$C_{total} = C_{gate} + C_{junction} + C_{wire}$$

$$C_{gate} = C_{gd} + C_{gs} + C_{gb}$$

$$C_{junction} = C_{db} + C_{sb}$$

$$C_{wire} = C_{design}$$

In postlayout design, the PrimeSim XA tool expands a single node from the prelayout design into multiple nodes because of the RC parasitics. The postlayout flow is divided into 2 scenarios:

- A back-annotated postlayout.
- A flat postlayout.

In the back-annotated postlayout flow, you can trace the connectivity back to the prelayout design with the information from the prelayout netlist and the back-annotation file:

```
*|NET na 0.00458507PF <-- Net Capacitance (CBAnet)
```

```
*|I (x02/mp:GATE x02/mp GATE I 4.8e-16 22.75 3.25) // $llx=22.55
$lly=3.25 $urx=22.95 $ury=3.25 $lvl=5

*|I (x02/mn:GATE x02/mn GATE I 2.4e-16 22.75 1.05) // $llx=22.55 $lly=0.6
$urx=22.95 $ury=1.05 $lvl=4

*|I (x01/mp:DRN x01/mp DRN B 0 8.45 3.25) // $llx=8.45 $lly=2.65 $urx=9.2
$ury=3.85 $lvl=7

*|I (x01/mn:DRN x01/mn DRN B 0 8.45 1.05) // $llx=8.45 $lly=0.75 $urx=9.2
$ury=1.35 $lvl=6

*|S (na:1 22.75 2.65) // $llx=22.55 $lly=2.65 $urx=22.95 $ury=2.65 $lvl=3
*|S (na:2 22.75 3.925) // $llx=22.55 $lly=3.85 $urx=22.95 $ury=4 $lvl=5

Cg1 na:1 0 4.33011e-17
Cg2 na:2 0 3.99892e-17

...

R158 na:1 x02/mp:GATE 4.8 $l=0.6 $w=0.4 $lvl=5
R159 na:1 na:3 30 $l=0.8 $w=0.4 $lvl=3
R160 x02/mp:GATE na:2 5.016 $l=0.675 $w=0.4 $lvl=5
R161 na:3 na:4 16.875 $l=0.5 $w=0.4 $lvl=3
R162 na:3 na:5 18.75 $l=0.5 $w=0.4 $lvl=3
R163 na:3 na:8 3.96 $a=0.04 $lvl=10
R164 na:4 na:6 3.96 $a=0.04 $lvl=1

...

R186 na:18 x01/mn:DRN 5.38888 $a=0.09 $lvl=12
R187 na:19 na:20 0.992002 $l=0.8 $w=0.5 $lvl=2
R188 na:19 na:21 13.1234 $l=6.45 $w=0.3 $lvl=2
R189 na:20 na:24 0.744002 $l=0.6 $w=0.5 $lvl=2
R190 na:20 x01/mp:DRN 5.38888 $a=0.09 $lvl=12

...
```

In back-annotated post-layout flow, the capacitance information is reported as:

$C_{total} = C_{gate} + C_{junction} + C_{wire}$

$$C_{gate} = C_{gd} + C_{gs} + C_{gb}$$

$$C_{junction} = C_{db} + C_{sb}$$

$$C_{wire} = C_{BA_{net}} + C_{design}$$

Based on the previous back-annotation file example:

$$C_{total} = C_{gate} + C_{junction} + C_{wire}$$

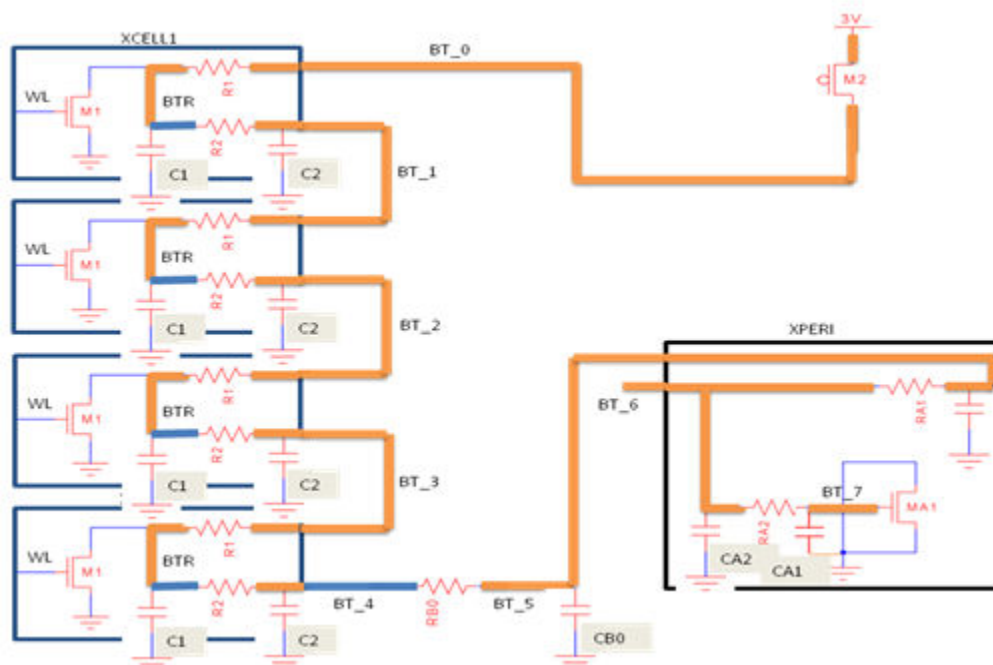
$$C_{gate} = C_{gate}(x02/mp) + C_{gate}(x02/mn)$$

$$C_{junction} = C_{junction}(x01/mp) + C_{junction}(x01/mn)$$

$$C_{wire} = C_{BA_{net}} (0.00458507PF) + C_{design}$$

In a flat postlayout flow, the PrimeSim XA tool treats all nodes as unique and independent. To accurately calculate the capacitance information, you need to group the nodes and then run `report_node_cap`. For example, see [Figure 11](#).

Figure 11 Flat Postlayout Flow Example



In [Figure 11](#), a prelayout node, `BT`, has been expanded into different nodes in the postlayout. Because the postlayout netlist is flat, and there is no trace of connectivity from the prelayout netlist and back-annotation flow, all nodes are treated as unique and independent.

To accurately report the capacitance information, you need to tell the PrimeSim XA tool which nodes can be grouped together for `report_node_cap` command to calculate the capacitance information. To group a list of nodes into one group, use `-group` argument and list the names of nodes to be grouped: `BT_0`, `BT_1`, `BT_2`, `BT_3`, `BT_4`, `BT_5`, `BT_6`, `XPERI.BT_7`, `XCELL1.BTR`, `XCELL2.BTR`, `XCELL3.BTR`, and `XCELL4.BTR` and do the following steps:

1. Specify the following `report_node_cap` command.

```
report_node_cap -node BT_? XPERI.BT_7 XCELL?.BTR -group BT
```

This command groups all the specified node names into one group named `BT`.

2. Assuming all parasitic capacitor has a value of 1 fF, the PrimeSim XA tool calculates the capacitance information is as:

```
Ctotal = Cgate + Cjunction + Cwire
Cgate = Cgate(XPERI.MA1)
Cjunction = Cjunction(M2) + Cjunction(XCELL1.M1) + Cjunction(XCELL2.M1)
+ Cjunction(XCELL3.M1) + Cjunction(XCELL4.M1) + Cjunction(XPERI.MA1)
Cwire = (12 * 1 fF) + Cdesign
Cdesign = 0 pF
```

ireport_operating_point

Description

Writes the circuit operating point at the current time to the specified file.

Syntax

```
ireport_operating_point -file filename
[ -type ic | nodeset ]
[-node node_name {node_name}]
```

Arguments

Argument	Description
-file file_name	Specifies the report output file name.
-type ic nodeset	Specifies if the file is a .ic or a .nodeset file. The default is .ic .

Argument	Description
<code>-node node_name</code> <code>{node_name}</code>	Specifies that only those nodes matching the pattern are written to the file. You can specify wildcards in the node names. The wildcard match behavior is determined by the alias matching rules. See the iset_interactive_option <code>-port.</code> description.

Examples

Writes the `op_100us.ic` file with `.ic` statements for the default nodes available in the database.

```
XA> ireport_op op_100us.ic
```

Writes the `op_100us.nodeset` file with `.nodeset` statement for the default nodes available in the database.

```
XA> ireport_op op_200us.nodeset -type nodeset
```

Writes the `file1` file with `.ic` for those nodes matching the `add*` pattern from the default node set.

```
XA> ireport_op file1 -node add*
```

isearch_node

Searches nodes in the netlist and reports various attributes.

Syntax

```
isearch_node -v [voltage_value] |-dv [dv_value] |  
-dt [dt_value] |-conn [conn_value]
```

Arguments

Argument	Description
<code>-v voltage_value</code>	Specifies the maximum voltage. Any node with an absolute value higher than the specified <code>voltage_value</code> is reported. If you do not specify a value, the PrimeSim XA tool reports the node with the highest voltage absolute value. The <code>voltage_value</code> unit is volts.
<code>-dv dv_value</code>	Specifies the maximum voltage change. Any node with a voltage change larger than the specified <code>dv_value</code> is reported. If you do not specify a value, the PrimeSim XA tool reports the node with the highest voltage change. The <code>dv_value</code> unit is seconds.

Argument	Description
<code>-dt dt_value</code>	Specifies the maximum time step. Any node with a time step smaller than the specified <code>dt_value</code> is reported. If you do not specify a value, the PrimeSim XA tool reports the node with the smallest time step value. The <code>dt_value</code> unit is seconds.
<code>-conn conn_value</code>	Returns the list of nodes that have more connections than the specified value. If you do not specify a value, it lists the most connected node in the circuit.

Description

This command searches the nodes in the netlist and reports nodes with:

- The highest voltage.
- All nodes with a voltage that exceeds the specified value.
- The highest voltage change.
- All nodes with a voltage change that exceeds the specified value.
- The nodes with the minimum time step.
- All nodes with a time step less than the specified value.
- The most connected node in the netlist.

`isearch_node` only counts connections to elements. A connection to a dangling subcircuit port does not count as a connection. In the following example, node `c` is connected only once to the `vc` voltage source.

```
.subckt dangling a
.ends
vc c 0 dc=1
xc c dangling
xc2 c dangling
```

The connected node output provides the name of the subcircuit that contains the node, the primary node name and node index, the number of connections, and a flag to indicate if a voltage source is connected to the node. For example:

```
XA > isearch_node -conn
Highest connectivity node: Subckt=con20 Node=a (1) #Conn=21 Vsrc=0

XA > isearch_node -conn 10
High connectivity node: Subckt=TLC Node=0 (0) #Conn=14 Vsrc=1
```

```
High connectivity node: Subckt=TLC Node=a (1) #Conn=10 Vsrc=1
High connectivity node: Subckt=con20 Node=a (1) #Conn=21 Vsrc=0
High connectivity node: Subckt=con20 Node=c (3) #Conn=21 Vsrc=0
```

Examples

Prints the node with the maximum voltage.

```
isearch_node -v
Maximum V=4.567 sy nofr vcc_int
```

Reports all nodes with a voltage absolute value greater than 2.5V.

```
XA> isearch_node -v 2.5
V=3 at node x8.qb
V=3 at node x8.CKn
V=3 at node vcc
V=3 at node d[2]
```

Reports the node with the largest voltage change.

```
XA> isearch_node -dv
Maximum DV=0.276185 at node x2.x1.n2
```

Reports all nodes with a voltage change greater than 1mV.

```
XA> isearch_node -dv 1e-3
DV=0.00243276 at node x3.x9.n1
DV=0.00230895 at node x3.x8.n1
```

Reports the node with the minimum time step.

```
XA> isearch_node -dt
Minimum time_step=33ps at node x4.x1.n2 , DV=0.272122
```

Reports the nodes with a time step smaller than 2000ps.

```
XA> isearch_node -dt 2000p
time_step=1934ps at node x3.x8.n1, DV=0.00230895
time_step=1934ps at node x1.x9.n1, DV=0.00144889
```

Reports the most connected node in the circuit.

```
XA> isearch_node -conn
Highest connectivity node: Subckt=con20 Node=a (1) #Conn=21 Vsrc=0
```

Reports all nodes that have 10 or more connections in the circuit.

```
XA> isearch_node -conn 10
High connectivity node: Subckt=TLC Node=0 (0) #Conn=14 Vsrc=1
High connectivity node: Subckt=TLC Node=a (1) #Conn=10 Vsrc=1
High connectivity node: Subckt=con20 Node=a (1) #Conn=21 Vsrc=0
High connectivity node: Subckt=con20 Node=c (3) #Conn=21 Vsrc=0
```

iset_break_point

Description

Pauses the simulation at the specified time.

Syntax

```
iset_break_point -at time[unit]
```

Arguments

Argument	Description
<i>time[unit]</i>	Specifies the time and unit at which the simulation is paused. White space is not allowed between <i>time</i> and <i>unit</i> . The default for <i>unit</i> is second, if it is not specified.

Note:

The `iset_break_point` command is not available in a VCS PrimeSim AMS mixed-signal simulation.

Examples

This example pauses the simulation at 10ns.

```
iset_break_point -at 10n
```

iset_diagnostic_option

Description

Lets you output the power report up to the current simulation time if you want to abort the simulation in progress.

Syntax

```
iset_diagnostic_option -report_power enable_value
```

Arguments

Argument	Description
<code>-report_power enable_value</code>	This command only works with the report_power command.

Note:

The `iset_diagnostic_option` command is not available in a VCS PrimeSim AMS mixed-signal simulation.

Examples

The PrimeSim XA tool completes the `vdd_1u_2u` power report using the 1.5us as the window end time. the PrimeSim XA tool needs to report the time value that was used to close the window for the calculation.

```
report_power -label vdd_1u_2u -by_node vdd -from 1u -to 2u
xa net.sp -c cmd -intr 1u
XA> iprint_time
1us
XA> cont 500n
XA> iprint_time
1.5us
XA> iset_diagnostic_option -report_power 1
XA> iquit
```

iset_env

Description

Use this command when you want to see the entire result of your interactive command at once.

Syntax

```
iset_env [-filter|-filter output_filter]
```

Arguments

Argument	Description
<code>-filter</code>	Removes the page-by-page display of interactive results that requires using the space key.
<code>-filter <i>output_filter</i></code>	Sets the output filter command, which can be filter for paging through the text, such as <code>more</code> , <code>less</code> , and so on. The default output filter command is <code>more</code> .

iset_interactive_option

Description

Controls the wildcard matching behavior in interactive mode for hierarchy and signal alias names. In interactive mode, this command overrides the wildcard-matching hierarchy set by the `set_wildcard_rule` command.

Syntax

```
iset_interactive_option -match* one|all  
[-port enable_value]  
[-tcl enable_value]  
[-tclbuf enable_value]
```

Arguments

Argument	Description
-match* one all	Specifies whether the * wildcard character matches the hierarchical delimiter for one or all levels of hierarchy.
-port enable_value	Specifies whether the * wildcard character matches the signal alias names.
-tcl enable_value	Specifies whether the full Tcl interpreter is enabled. This option is the same as the -tcl command line argument. For more information about Tcl, see Enabling Tcl Mode .
-tclbuf enable_value	Specifies if data from interactive mode commands is put in Tcl memory. Set -tclbuf 0 to avoid Tcl memory overflow in large circuits (default). To run Tcl scripts and enable data dumping, you need to set -tclbuf 1.

Note:

The `iset_interactive_option` command is not available in a VCS PrimeSim AMS mixed-signal simulation.

Examples

This example matches 3 nodes with the `x1.*` pattern.

```
XA> imatch_node x1.*  
7 X1.g  
8 X1.ha  
9 X1.x2.b1
```


This example matches 2 nodes with the `x1.*` pattern.

```
XA> iset_interactive_option -match* one
XA> imatch_node x1.*
7 X1.g
8 X1.ha
```

This example matches 8 nodes with the `x1.*` pattern.

```
XA> iset_interactive_option -match* one -port 1
XA> imatch_node x1.*
0 X1.6
0 X1.0
1 X1.a
1 X1.3
2 X1.4
3 X1.5
7 X1.g
8 X1.ha
```

iset_interactive_stop

Enables the node voltage monitoring/checking capability.

Syntax

```
iset_interactive_stop -check v(node_name) {v(node_name) }
[-max vmax]
[-min vmin]
[-twindow tstart tstop {tstart tstop}]
[-cmf script_file_name]
[-check_meas measurement_name]
[-at time_value]
```

Arguments

Argument	Description
-check v(<i>node_name</i>) {v(<i>node_name</i>) }	Specifies the voltage signals to be monitored during transient simulation.
-max <i>vmax</i>	If any monitored signal rises above the upper limit you specify, the PrimeSim XA tool interrupts the transient simulation and enters interactive mode to facilitate debugging.

Argument	Description
<code>-min vmin</code>	If any monitored signal falls below the lower limit you specify, the PrimeSim XA tool interrupts the transient simulation and enters interactive mode to facilitate debugging. Note that for monitored signals you should specify at least one <code>-max</code> or <code>-min</code> argument.
<code>-twindow tstart tstop {tstart tstop}</code>	Specifies the time periods for the signals to be monitored in the simulation.
<code>-cmf script_file_name</code>	Specifies the script file to be run when the PrimeSim XA tool enters interactive mode due to a violation at the upper or lower limits of <code>-max</code> or <code>-min</code> . Note that the commands in the script file must be interactive commands.
<code>-check_meas measurement_name</code>	When specify a measurement name and the measurement fails to meet the <code>-min</code> or <code>-max</code> values, the simulation stops in interactive mode.
<code>-at time_value</code>	Specifies the time to enter interactive mode.

Description

If any monitored signal rises above the upper limit or falls below lower limit, the PrimeSim XA tool interrupts the transient simulation and enters interactive mode. You can also use this command to specify a script file to run when entering interactive command due to either an upper/lower limit violation.

Examples

Continues the transient simulation and stops when `v(net1)` is greater than 1.2V and applies the commands in `cmd_file`.

```
XA> iset_interactive_stop -check v(net1) -max 1.2 -cmf cmd_file
XA> cont
```

iset_save_state

Description

Saves a simulation at the specified time point.

Syntax

```
iset_save_state -time time_value
```

Arguments

Argument	Description
<code>-time time_value</code>	Specifies the time point at which the state is saved to restore a simulation. If you do not specify a <code>-time</code> value, the simulation state is saved at the current time point.

Note:

The `iset_save_state` command is not available in a VCS PrimeSim AMS mixed-signal simulation.

iset_waveform_option

Description

Controls the waveform flush operation in interactive mode.

Syntax

`iset_waveform_option [-flush_auto on|off] | [-flush_now]`

Arguments

Argument	Description
<code>-flush_auto on off</code>	Resets the interactive mode automatic flush rule.
<code>-flush_now</code>	Flushes the waveform at the current simulation stop time.

iset_zstate_option

Sets the conducting rules for the [icheck_node_zstate](#) command.

Syntax

`iset_zstate_option [-rule rule_value {rule_value}]`
`[-diode_vh value]`
`[-idsth ids_value]`
`[-vbeth vb_value]`
`[-report report_value {report_value}]`

Arguments

Argument	Description
<code>-rule rule_value</code> <code>{rule_value}</code>	Selects the rule of conducting for a MOSFET. The MOSFET is considered conducting if one of the rules is met. Other rules are as follows: NMOS: • $V_{gs} > V_{th}$ (rule=1) • $I_{ds} > I_{dsth}$ (rule=2) • $V_g > V_{DD}-0.1$ (rule=3) PMOS: • $V_{gs} < V_{th}$ (rule=1) • $I_{ds} > I_{dsth}$ (rule=2) • $V_g < 0.1$ (rule=3) The default is 1 and 2.
<code>-diode_vth value</code>	Specifies the threshold for a diode to be considered conducting when V_{ac} exceeds the specified value. If you specify a value less than 0, diode is considered as not conducting. The default is 0.2V.
<code>-idsth ids_value</code>	Specifies the threshold for a MOSFET to be considered conducting when I_{ds} exceeds <i>value_1</i> . If not specified, the default is 1e-8 ampere. See the following Description section for more information about the conducting criteria.
<code>-vbeth vb_value</code>	Specifies the threshold for BJT to be considered conducting when V_{be} exceeds <i>value_2</i> . If not specified, the default is 0.64 volt. See the following Description section for more information about the conducting criteria.
<code>-report report_value</code> <code>{report_value}</code>	You can specify the following keywords: • <code>node</code> lists high impedance nodes. • <code>channel</code> lists all channel-connected devices for the node. • <code>gate</code> lists all gate-connected devices for the node. The default is <code>node</code> .

Description

This interactive command specifies the conducting rules so that the PrimeSim XA tool can diagnose specified nodes staying in a high-impedance (floating) state.

A node stays in a high-impedance state if there is no conducting path from any voltage source to the node. A conducting path consists of conducting elements. An element is conducting if that specific element meets the following criteria:

Device	Rule
NMOS	• $V_{gs} > V_{th}$ (-rule 1) • $I_{ds} > I_{dsth}$ (-rule 2) • $V_g > VDD - 0.1$ (-rule 3)
PMOS	• $V_{gs} < V_{th}$ (-rule 1) • $I_{ds} > I_{dsth}$ (-rule 2) • $V_g < 0.1$ (-rule 3)
NPN	$V_{be} > v_{beth}$
PNP	$V_{be} < -v_{beth}$
Diode	Forward-biased.
Resistor	Always conducting.
Inductor	Always conducting.
Capacitor	Never conducting.
Verilog-A	Never conducting.
Controlled sources	Always conducting.

iset_speed_scale

Dynamically scales the transient simulation engine tolerance in interactive mode.

Syntax

```
iset_speed_scale time_spec
```

where

```
time_spec : time1 scale1 [time2 scale2 ...]
```

Arguments

Argument	Description
<code>time1 scale1 time scale2 ...</code>	<i>time</i> can be any value greater than or equal to 0 and less than or equal to the <code>.tran</code> time. <i>scale</i> can be one of the following values: 0.5, 1, 2, 3, 4, or 5.

Examples

The following example demonstrates how to use the `iset_speed_scale` command to dynamically adjust simulation speed for different time intervals within a simulation.

```
XA> iset_speed_scale 0n 2 30n 5 1u 3
```

DC Interactive Mode Commands

DC interactive commands provide debugging tools to help you discover or identify issues in DC iterations. To enter DC interactive mode, use the `-dcintr` command line option. For example:

```
xa ... -dcintr iteration [-intr trans_time]
```

Where *iteration* specifies the number of DC iterations to run up to the specified iteration value and *trans_time* (optional) specifies the number of nanoseconds to run transient simulation, if desired.

The DC interactive mode commands are:

- [exit](#)
- [iclose_log](#)
- [icontinue_dc](#)
- [idelete_node_ic](#)
- [imatch_elem](#)
- [imatch_node](#)
- [iopen_log](#)
- [iprint_connectivity](#)
- [iprint_elem_info](#)
- [iprint_exi](#)
- [iprint_help](#)
- [iprint_node_info](#)
- [isearch_node](#)
- [iset_node_ic](#)

exit

Description

Stops and exits the simulation.

Syntax

```
exit
```

iclose_log

Description

Closes the interactive mode log file that was opened by the [iopen_log](#) command. The log file contains all records of interactive mode commands and the results reported by the commands entered between `iopen_log` and `iclose_log`.

Syntax

```
iclose_log
```

See Also

- [iopen_log](#)
-

icontinue_dc

Description

Continues a DC simulation until it completes, stops at a specified iteration, or is interrupted by Ctrl-C.

Syntax

```
icontinue_dc [-iter addl_iteration] [-to iteration]  
[-to end]
```

Arguments

Argument	Description
<code>-iter <i>addl_iteration</i></code>	Specifies the number of additional DC iterations to run based on the current DC iteration.
<code>-to <i>iteration</i></code>	Runs the DC simulation until it reaches the specified iteration.

Argument	Description
<code>-to end</code>	Runs the DC simulation until it reaches the DC converged point.

Examples

Continues DC analysis for 20 additional iterations.

```
XA DC> icontinue_dc -iter 20
```

idelete_node_ic

Releases initial condition values.

or

Syntax

```
idelete_node_ic [-node] node_name  
idelete_node_ic -index node_index
```

Arguments

Argument	Description
<code>-node node_name</code>	Releases the initial condition value from the specifies node. You can use a wildcard character in the node name.
<code>-index node_index</code>	Specifies the node index.

Description

This command releases the initial condition value from the .ic statement or [iset_node_ic](#) statement at DC interactive mode. This command does not work on voltage source nodes or vector files.

Examples

```
XA DC> idelete_node_ic -node n2 n5
```

imatch_elem

Description

Prints a list of the element indexes and hierarchical element names that match the specified pattern.

Syntax

```
imatch_elem -pattern pattern ...
```

Arguments

Argument	Description
<i>pattern</i>	Patterns can contain the * wildcard character. The printed list shows all matches to the specified pattern. You can set wildcard matching for the hierarchy with iset_interactive_option .

Examples

In this example, the `*x1*` pattern matches 3 elements.

```
XA DC> imatch_elem *x1*
5 x1.r1
6 x1.r2
7 x1.x1.r1
```

See Also

- [iset_interactive_option](#)

imatch_node

Prints a list of the node indexes and node names that match the specified pattern.

Syntax

```
imatch_node -pattern pattern ...
[-limit level]
[-port enable_value]
```

Arguments

Argument	Description
<i>pattern</i>	Patterns can contain the * wildcard character. The printed list shows all matches to the specified pattern. You can set wildcard matching for the hierarchy with iset_interactive_option .
<code>-limit level</code>	Specifies wildcard matching only down to the specified hierarchical depth. The default hierarchical limit level is 3.
<code>-port enable_value</code>	Enables or disables the matching of subcircuit port names. The default value of is taken from the iset_interactive_option command.

Description

`imatch_node` prints a list of matched nodes. The PrimeSim XA tool first reports the value of `-limit` applied, then lists the node index and node name matched with one item per line. Finally, the PrimeSim XA tool reports the total number of matched nodes.

The [iset_interactive_option](#) command settings apply to this command. The "*" wildcard can be set to match or not to match the hierarchical delimiter. The "*" wildcard only matches primary node names, unless you specify the `-port` argument. When you specify `-port`, the "*" wildcard also matches alias node names.

Examples

In this example, the `*a` pattern matches 3 nodes.

```
XA DC> imatch_node *a
1 a
2 x1.a
3 x1.fa
pattern *a matched 3 nodes
```

This example finds all nodes ending in `a` at the top-level of the netlist, level 0.

```
XA DC> imatch_node *a -limit 0
imatch_node using -limit 0
1 a
pattern *a matched 1 nodes
```

This example finds all nodes ending in `out` down to the hierarchical depth of 4 and also reports ports.

```
XA DC> imatch_node *out -limit 4 -port 1
15 x0.x1.x2.aout
20 x0.x1.x2.x3a.out
```

```
35 x0.x1.x2.bout
48 x0.x1.x2.x3b.out
```

See Also

- [iset_interactive_option](#)

iopen_log

Description

Opens the interactive mode *logfile_name*, which contains the record of the interactive mode commands and the results reported by these commands until the log file is closed by the [iclose_log](#) command. Only one log file can be opened at one time.

Syntax

```
iopen_log -file logfile_name [-mode append|write]
```

Arguments

Argument	Description
-file logfile_name	Specifies the interactive mode log file name.
-mode append	Specifies that contents are appended to pre-existing files using the same file name.
-mode write	Specifies that contents overwrite pre-existing files using the same file name. Default is <code>-mode write</code> .

Examples

This example opens the log file named `logfile`.

```
XA DC> iopen_log -file logfile
```

See Also

- [iclose_log](#)

iprint_connectivity

Description

Prints the detailed node connectivity information for the given node names or indices. The elements are categorized into channel-connected, gate-connected, and other elements.

or

Syntax

```
iprint_connectivity -node node_name {node_name}  
[-print gc|cc|o|all]  
[-on current_value]  
[-file file_name]  
[-file_append file_name]  
iprint_connectivity -index node_index {node_index}  
[-print gc|cc|o|all]  
[-on current_value]  
[-file file_name]  
[-file_append file_name]
```

Arguments

Argument	Description
-node node_name {node_name}	Specifies the node name, which can contain wildcard characters.
-print gc cc o all	Specifies to selectively print the gate-connected, channel-connected, other, or all elements. If not specified, the default prints gate-connected and channel-connected elements.
-index node_index	Specifies node index.
-on current_value	Prints only those instances that have a current exceeding the specified <i>current_value</i> .
-file file_name	Writes output only to the specified file. If the file exists it is overwritten. The file is located in the directory specified by the -o command line argument.
-file_append file_name	Writes output only to the specified file. If the file exists this argument appends the output to the existing file, otherwise it creates the specified file. The file is located in the directory specified by the -o command line argument.

Examples

```
XA DC> iprint_connectivity -node xspine_0.xdqr_0.xl850.osc_2  
XA DC> iprint_connectivity xspine_0.xdqr_0.xl850.osc_2 -print cc  
XA DC> iprint_connectivity -index 3
```

See Also

- [iprint_node_info](#)

iprint_elem_info

Prints the detailed element information for the given element names at the specific time of activation.

or

Syntax

```
iprint_elem_info -elem element_name {element_name}  
[-report brief]  
[-file file_name]  
iprint_elem_info -index elem_index {elem_index}  
[-report brief]  
[-file file_name]
```

Arguments

Argument	Description
-elem <i>element_name</i>	Specifies the element name, which can include wildcard characters.
-index <i>elem_indexn</i>	Specifies element index.
-report brief	Provides a brief version of the output that reports only the element terminal connectivity, voltage, and currents. Other information is suppressed.
-file <i>file_name</i>	Writes <code>iprint_elem_info</code> output to the specified file.

Description

You can also provide a subcircuit instance as an element. In this case, the PrimeSim XA tool prints the subcircuit name, the list of its ports, and the voltages on each port. The detailed element information includes:

- Element name, element type, and model name
- Element terminal connectivity
- Element parameters and values
- Element terminal voltages

MOSFET-specific information:

- MOS logic state (ON/OFF)
- MOS effective length and width (L_{eff} and W_{eff})
- MOS conductance (g_{ds} and g_m)
- MOS threshold voltage (V_{th})
- MOS voltage-dependent diode capacitance (c_{bs} and c_{bd})
- MOS voltage-dependent gate capacitance (c_{gs} , c_{gb} , and c_{gd})
- MOS I_{ds} current (I_{ds})

Examples

Prints information for element `x1.x2.m1`.

```
XA DC> iprint_elem_info x1.xi@1477.mpt1
Elem=X1.XI@1477.MPT1 (203) Type=PMOS Model=PENH
D=X1.X001 (232) G=X1.N@1514 (130) S=VDD (14) B=VDD (14)
Vd=0.0108579 Vg=1.69226 Vs=1.69213 Vb=1.69213
Weff=11.9178u Leff=0.205123u PD=13.4u PS=13.4u AD=4.2u^2 AS=4.2u^2
Vt=-0.73418 OFF
Ids=-2.7808e-07u
gds=2.86161e-13 gm=6.111e-12
cgs=3.03919f cgd=3.03903f cgb=4.59196f cbs=6.99465f cbd=4.48262f
id=0.105008u ig=-0.0446823u is=0.000726083u ib=-0.0610515u
```

```
XA DC> iprint_elem_info X1.RI@5234
Elem=X1.RI@5234 (161) Type=R
N1=X1.N@5446 (195) N2=X1.N@5445 (194)
R=33
Vn1=1.69267 Vn2=1.69267
I=0.0036124u
```

```
XA DC> iprint_elem_info -elem x1.x2.m1
```

Prints information for element indices 1, 3, and 2.

```
XA DC> iprint_elem_info -index 1 3 2
```

Prints subcircuit information and port voltages for a subcircuit instance.

```
XA DC> iprint_elem_info x1
Elem=x1 Type=subckt subckt=mysubl
PORT 1 port=a - 11 (1) v=1
PORT 2 port=b - 12 (2) v=1
PORT 3 port=c - 13 (3) v=2.65
PORT 4 port=d - 14 (4) v=1
```

See Also

- [iprint_connectivity](#)
- [iprint_node_info](#)

iprint_exi

Prints elements with excessive currents.

Syntax

```
iprint_exi -inst inst_name {inst_name}  
[-ith ivalue]  
[-file file_name]  
[-report report_value {report_value}]
```

Arguments

Argument	Description
<code>-inst <i>inst_name</i></code>	Specifies the hierarchical element names to be searched. You can use wildcard characters (*) in the instance names. The set_wildcard_rule conventions apply.
<code>-ith <i>ivalue</i></code>	Specifies the current threshold that must be exceeded for the element to be reported. The default value is 50uA.
<code>-file <i>file_name</i></code>	Specifies the output file name.
<code>-report <i>report_value</i></code> <code>{<i>report_value</i>}</code>	<i>report_value</i> can be: • <code>subname</code> to specify a list of subcircuit names of the hierarchical instance components to be printed in the output. • <code>brief</code> to report only element terminal connectivity, voltages, and currents. Other information is suppressed.

Description

The `iprint_exi` command reports the current through any device terminal that exceeds the threshold. The following device types are checked and reported:

- MOS
- Resistor

- BJT
- Diode

Examples

The output format for elements found that exceed the current threshold is the same as the `iprint_elem_info` command. If you specify a list of subcircuit names with the `-report` argument, the hierarchical element instance names have the following general format:

```
Elem=X0.X1.X2...Xn.modelname  
  
Subckt: X0=x0name X1=X1name X2=x2name ... Xn=Xnname
```

For example:

```
XA DC> iprint_exe -ith 1n xmos.m* -report subname  
Elem=Xmos.mn (6) Type=NMOS Model=nch.7  
D=vdd (5) G=g (3) S=Xmos.n1 (16) B=vb (4)  
Vd=3 Vg=1.81293e-90 Vs=-0.00837316 Vb=1.81293e-90  
M=1  
Weff=2.016u Leff=0.948223u PD=2.365u PS=2.365u AD=0.351792u^2  
AS=0.351792u^2 SA=0u SB=0u  
Vt=0.19898 OFF  
Ids=0.26352u  
gds=5.38557e-10 gm=0  
cgs=4.30157f cgd=0.416708f cgb=3.65854f cbs=1.20401f cbd=0.487088f  
id=0.26352u ig=-0.0663667u is=-0.00752308u ib=-0.189631u  
Subckt: Xmos=mymos
```

iprint_help

Description

Displays the syntax and a brief description of the specified interactive commands.

Syntax

```
iprint_help -cmd command_name1 ... command_namen
```

Arguments

Argument	Description
<i>command_namen</i>	Specifies the interactive commands to be displayed on the screen.

Examples

This example prints information about the `iprint_node_info` command.


```
XA DC> iprint_help iprint_node_info
```

iprint_node_info

Description

Prints the node voltage, node index, iteration, and initial conditions (if any) for the given node names. Each value is evaluated at the current iteration, when the node voltage is last updated.

or

Syntax

```
iprint_node_info -node node_name {node_name}  
iprint_node_info -index node_index {node_index}
```

Arguments

Argument	Description
<code>-node <i>node_name</i></code>	Specifies node name. Supports wildcard characters.
<code>-index <i>node_index</i></code>	Specifies node index.

Examples

Prints information for node `xalu3.xlatch2.q`.

```
XA DC> iprint_node_info x1.sout
```

```
Node=X1.SOUT (225)  
V=1.69417 V, dV=-0.00244323, iter=1, ic=0.5 V
```

```
XA DC> iprint_node_info -node xalu3.xlatch2.q
```

Prints node information for node indices 1, 3 and 2.

```
XA DC> iprint_node_info -index 1 3 2
```

See Also

- [iprint_connectivity](#)
- [iprint_elem_info](#)

isearch_node

Searches nodes in the netlist and reports various attributes.

Syntax

```
isearch_node -v [voltage_value] | -dv [dv_value] |  
-conn [conn_value]  
-hiz enable_value
```

Arguments

Argument	Description
-v <i>voltage_value</i>	Specifies the maximum voltage. Any node with an absolute value higher than the specified <i>voltage_value</i> is reported. If you do not specify a value, the PrimeSim XA tool reports the node with the highest voltage absolute value. The <i>voltage_value</i> unit is volts.
-dv <i>dv_value</i>	Specifies the maximum voltage change. Any node with a voltage change larger than the specified <i>dv_value</i> is reported. If you do not specify a value, the PrimeSim XA tool reports the node with the highest voltage change. The <i>dv_value</i> unit is seconds.
-conn <i>conn_value</i>	Returns the list of nodes that have more connections than the specified value. If you do not specify a value, it lists the most connected node in the circuit.
-hiz <i>enable_value</i>	When set to '0' prints all hiz nodes (default). When set to '1' prints all hiz nodes with fanouts.

Description

This command searches the nodes in the netlist and reports nodes with:

- The highest voltage.
- All nodes with a voltage that exceeds the specified value.
- The highest voltage change.
- All nodes with a voltage change that exceeds the specified value.
- High impedance nodes.
- The most connected node in the netlist.

`isearch_node` only counts connections to elements. A connection to a dangling subcircuit port does not count as a connection. In the following example, node `c` is connected only once to the `vc` voltage source.

```
.subckt dangling a
.ends
vc c 0 dc=1
xc c dangling
xc2 c dangling
```

The connected node output provides the name of the subcircuit that contains the node, the primary node name and node index, the number of connections, and a flag to indicate if a voltage source is connected to the node. For example:

```
XA DC> isearch_node -conn
Highest connectivity node: Subckt=con20 Node=a (1) #Conn=21 Vsrc=0
```

```
XA DC> isearch_node -conn 10
High connectivity node: Subckt=TLC Node=0 (0) #Conn=14 Vsrc=1
High connectivity node: Subckt=TLC Node=a (1) #Conn=10 Vsrc=1
High connectivity node: Subckt=con20 Node=a (1) #Conn=21 Vsrc=0
High connectivity node: Subckt=con20 Node=c (3) #Conn=21 Vsrc=0
```

Examples

Prints the node with the maximum voltage.

```
XA DC> isearch_node -v
Maximum V=4.567 sy nofr vcc_int
```

Reports all nodes with a voltage absolute value greater than 2.5V.

```
XA DC> isearch_node -v 2.5
V=3 at node x8.qjb
V=3 at node x8.CKn
V=3 at node vcc
V=3 at node d[2]
```

Reports the node with the largest voltage change.

```
XA DC> isearch_node -dv
Maximum DV=0.276185 at node x2.x1.n2
```

Reports all nodes with a voltage change greater than 1mV.

```
XA DC> isearch_node -dv 1e-3
DV=0.00243276 at node x3.x9.n1
DV=0.00230895 at node x3.x8.n1
```

Reports the most connected node in the circuit.

```
XA DC> isearch_node -conn
Highest connectivity node: Subckt=con20 Node=a (1) #Conn=21 Vsrc=0
```

Reports all nodes that have 10 or more connections in the circuit.

```
XA DC> isearch_node -conn 10
High connectivity node: Subckt=TLC Node=0 (0) #Conn=14 Vsrc=1
High connectivity node: Subckt=TLC Node=a (1) #Conn=10 Vsrc=1
High connectivity node: Subckt=con20 Node=a (1) #Conn=21 Vsrc=0
High connectivity node: Subckt=con20 Node=c (3) #Conn=21 Vsrc=0
```

iset_node_ic

Changes initial condition values.

or

Syntax

```
iset_node_ic [-node] node_name -val value
-index node_index -val value
```

Arguments

Argument	Description
-node node_name	Applies the initial conditions value to the specified nodes. You can use a wildcard character in the node name.
-val value	Specifies the initial conditions value.
-index node_index	Specifies the node index.

Description

This command changes initial condition value for the specified nodes. The specified nodes stay at the specified constant voltage. The node voltage stays at the same value from the current iteration until either DC convergence or when the constant node voltage status is released by the [idelete_node_ic](#) command. In the interactive mode, the this command does not work on voltage source nodes or vector files.

Examples

The voltage at the `pump` node stays at 6.5V from the current iteration on.

```
XA DC> iset_node_ic -node pump -val 6.5
```

4

PrimeSim XA Dynamic CCK Commands

This chapter provides syntax for the PrimeSim XA dynamic CCK (Circuit Check) commands.

The CCK commands are:

- `cck_analog_pdown`
- `cck_excess_ipath`
- `cck_disable`
- `cck_post`
- `cck_resistive_path_report`
- `cck_signal`
- `cck_soa`
- `cck_substrate`
- `cck_toggle_count`
- `check_measure`
- `check_node_excess_rf`
- `check_node_hotspot`
- `check_node_quick_rf`
- `check_node_zstate`
- `check_timing_edge`
- `check_timing_hold`
- `check_timing_post`
- `check_timing_pulse_width`

- [check_timing_setup](#)
- [set_report_option](#)

cck_analog_pdown

Detects unwanted currents due to floating MOS gates.

Syntax

```
cck_analog_pdown [-label] lname
                  [when_to_check]
                  [path]
                  [mos_control]
                  [bjt_control]
                  [diode_control]
                  [-numv numv|all]
                  [-rule 0|1|2|3|4]
                  [-HiZgate 0|1|2]
                  [-idsth hizdev1 value]
                  [-HiZvdd value]
                  [-except_subckt scoping_expression]
                  [-except_inst scoping_expression]
                  [-merge_subckt 0|1]
                  [-file merge|split]
                  [-HiZpath optimized|hizdev1]
                  [-max_length number]
                  [-max_hiz_length number]
                  [-report report_type]
                  [-constraint none|monotonic]
                  [-vtol value]
                  [-filteralert 0|1]
                  [-time_prec <prec><unit>]
                  [-voltage_prec <prec><unit>]
                  [-current_prec <prec><unit>]
                  [-numeric_prec <prec><unit>]
```

where

```
when_to_check ::= twindow_periodical_times | specific_times
twindow_periodical_times ::= [-twindow {tstart tstop ...} tstart
                             [tstop]]
                             -tstep tstep_val
specific_times ::= -at at_time {at_time}

path ::= [-from_node nname {nname}]
        [-to_node nname {nname}]
to_node ::= -to_node node_pattern {node_pattern}
node_supply ::= -match rule vsrc | none
mos_control ::= [-idsth ids_val]
               [-dvt dvt_val]
```

```

                                [-model_dvt mname mdvt {mname mdvt}]
bjt_control:= [-vbeth vbe_val]
diode_control:= [-diode_vth vth_val|-diode_ith ith_val]
scope      := [inst][subckt][except_i][except_s]
    inst    := -inst inst_pattern {inst_pattern}
    subckt  := -subckt subckt_name {subckt_name}
    except_i:= -except_inst inst_pattern {inst_pattern}
    except_s:= -except_subckt subckt_name[.inst_pattern]
                {subckt_name[.inst_pattern]}

```

Arguments

Argument	Description
-label	Specifies the label name that appears in the report file, which makes it easier to search the report.
-twindow {tstart1 tstop1...} tstart [tstop]	Performs the check within the specified time windows. The <i>tstart</i> and <i>tstop</i> values must come in pairs, except for the last specified window. This means when <i>tstop</i> is not specified, it is assumed to be the end of the simulation. The last <i>tstop</i> can also be the <code>end</code> or <code>END</code> keyword. You must use this option along with <code>-tstep</code> to perform discrete checking. Continuous checking is not supported in <code>cck_analog_pdown</code> .
-inst inst_pattern {inst_pattern}	Scopes to the named instances. You can use wildcards in instance names as specified by the <code>set_wildcard_rule</code> command.
-subckt subckt_name {subckt_name}	Scopes to instances of the named subcircuits.
-except_inst inst_pattern {inst_pattern}	Excludes instances from the <code>cck_analog_pdown</code> command scope. Any instance whose hierarchical instance name matches the pattern is excluded from the scope. Wildcards are supported.
-except_subckt sub_name[.inst_pattern] {sub_name} [.inst_pattern]	Excludes all instances of the named subcircuits from the <code>cck_analog_pdown</code> command scope. Any subcircuit whose name matches the pattern is excluded from the scope. You can also exclude only the named instances of the subcircuit. Wildcards are not supported in the subcircuit names, but are supported in the <i>inst_pattern</i> . Use the following syntax to exclude multiple subcircuits in the same <code>cck_analog_pdown</code> command statement: <code>cck_analog_pdown...-except_subckt sub_a sub_b sub_c</code> Where <i>sub_a</i> , <i>sub_c</i> and <i>sub_c</i> are the explicit subcircuit names to exclude.
-tstep tstep_val	Performs a discrete check at the time points that are integer multiples of <i>tstep_val</i> . You must use this option with the <code>-twindow</code> option. Otherwise, PrimeSim XA generates an error message and stops the simulation.

Argument	Description
<code>-at at_time {at_time}</code>	Performs a discrete check at the specified time. The <code>-tstep</code> and <code>-at</code> options are mutually exclusive. If both options are specified, PrimeSim XA generates an error message and stops the simulation.
<code>-from_node nname {nname}</code>	Specifies the starting nodes of a path to be checked, which avoids a long runtime due to many paths. Wildcards are supported. When you use a wildcard character (*), it only matches the nodes that have a direct connection to the voltage source.
<code>-to_node nname {nname}</code>	Specifies the ending nodes of a path to be checked, which avoids a long runtime due to many paths. Wildcards are supported. When you use a wildcard character (*), it only matches the nodes that directly connect to the voltage source.
<code>-match_rule vsrc none</code>	When set to <code>vsrc</code> (default), a wildcard character in <code>-from_node</code> or <code>-to_node</code> only matches a constant voltage source node. When set to <code>none</code> , a wildcard character in <code>-from_node</code> or <code>-to_node</code> can match to any circuit node name.
<code>-idsth ids_val</code>	Specifies the I_{ds} threshold for a MOS to be considered conducting. The default is 1e-8 A, which is the same as the <code>set_zstate_option</code> command.
<code>-dvt dvt_val</code>	Specifies the delta V_t threshold for a MOS to be considered conducting. The default is 0.
<code>-model_dvt mname mdvt {mname mdvt}</code>	Specifies the pairs of model name and its delta V_t threshold value. The <code>cck_analog_pdown</code> command uses its related <code>dvt</code> value of defined model to apply to the checking rule for a MOS.
<code>-vbeth vbe_val</code>	Specifies the V_{be} threshold for a BJT to be considered conducting. The default is 0.64. For more information, see Table 26 and Table 27 .
<code>-diode_vth vth_val</code>	Specifies the forward bias threshold for diodes. To be compatible with the conducting criteria of the <code>set_zstate_option</code> command, you must set this value to 0.2.
<code>-diode_ith ith_val</code>	Specifies the diode current threshold to be considered conducting. The default is 1e-8 A.
<code>-numv numv</code>	Limits the maximum number of total violations reported. The default is 300. To print all violations, specify the <code>all</code> keyword.
<code>-rule 0 1 2 3 4</code>	Specifies the conducting rule for analog power-down checking. For MOSFET devices, see Table 26 . For other devices see Table 27 . The default rule is 1.

Argument	Description
<code>-HiZgate 0 1 2</code>	Specifies how the PrimeSim XA tool uses the voltage of HiZ gate node. If you specify 0, based on the simulated circuit state, the PrimeSim XA tool uses the voltage of the HiZ gate node for MOSFET Ids (MOSFET drain-to-source current) calculation to search DC paths. If you specify 1, the PrimeSim XA tool re-evaluates the Ids values of the HiZ gate driven MOSFET based on the following device terminal conditions: - For P-MOSFET: Gate terminal with 50% of its maximum supply voltage, drain (or source) terminal with its maximum supply voltage, and source (or drain) with 50% of its maximum supply voltage. - For N-MOSFET: Gate terminal with 50% of its maximum supply voltage, drain (or source) terminal with zero, and source (or drain) with 50% of its maximum supply voltage. Note: To control and adjust 50% of maximum supply, use the <code>-HiZvdd</code> argument. The <code>-HiZgate 1</code> argument is effective only when <code>-rule 2 3</code> is set. If you specify 2, the PrimeSim XA tool uses the actual port voltages for MOSFET Ids calculation except gate terminal that uses 50% of its maximum supply voltage.
<code>-idsth_hizdev1 <value></code>	Specifies a separate current threshold for paths that contain HiZ-gated MOSFETs. Use this argument with the <code>-HiZpath hizdev1</code> option. Example: The following command performs regular ids check with a 5u threshold and further apply the 1u threshold to those hizdev1 paths. <pre>cck_analog_pdown -label pdown -idsth 5u \-from_node vdd -to_node vss \-HiZpath hizdev1 -idsth_hizdev1 1u</pre>
<code>-HiZvdd <value></code>	If you specify the desired supply value using the <code>-HiZvdd</code>, then 50% of specified value is applied to the MOSFET gate terminal, instead of the drain (or source) terminal for the Ids calculation. Only valid when <code>-HiZgate 1</code> and <code>-rule 2 3</code> are set. By default, the 50% of the maximum supply value of the MOSFET gate terminal will be applied to its gate terminal for Ids calculation, however if user specifies desired supply value through <code>-HiZvdd</code> option then the 50% of specified value will be applied to the MOSFET gate terminal for Ids calculation instead.
<code>-except_subckt</code>	Specifies the subcircuits that should be excluded from reporting. If required, you can specify the exceptions.

Argument	Description
<code>-except_inst</code>	Specifies the instances that should be excluded from reporting. If required, you can specify the exceptions.
<code>-merge_subckt 0 1</code>	Enables or disables subckt merging optimization. When subckt merging optimization is enabled, the command may not be able to report paths residing within the merged subckts such that the reporting can be less comprehensive than disabling the capability. If you set the <code>-merge_subckt</code> option to: 0: Subckt merging is disabled. 1 (default): Subckt merging is enabled.
<code>-file merge split]</code>	Enables file merging or splitting. When set to <code>merge</code>, all outputs are merged into one output file named <code>prefix.cckpdown</code>. When set to <code>split</code> (the default), all the outputs are split into different files named <code>prefix.cckpdown.label</code>.
<code>-HiZpath optimized hizdev1</code>	Controls how XA reports paths that involve HiZ gate devices within a network. You can use the arguments <code>optimized</code> and <code>hizdev1</code> independently. <code>optimized</code>: Reports only one path within the network for a network formed by HiZ-gated devices. <code>hizdev1</code>: Reports a path only when at least one MOSFET in the path has an HiZ gate. This option applies to all <code>-rule</code> settings.
<code>-max_length</code>	Reports a path only when the number of devices in the path is less than or equal to the specified value. For example, if there are 100 paths with 75 devices in each path and the <code>-max_length</code> option is set to 10, no path is reported. By default, the value is set to 50.
<code>-max_hiz_length</code>	Reports a path only when the number of Hiz devices in the path is less than or equal to the specified value.
<code>-report report_type {report_type}</code>	Specifies which information to report for each device. The supported values are: <code>subinfo</code>: Reports the subcircuit hierarchy information. <code>type</code>: Prints the path type for each violation. <code>IPath</code>: When all devices on the path are conductive. <code>ZPath</code>: When at least one device on the path has an HiZ gate.
<code>-constraint none monotonic</code>	Specifies constraint for all rules with condition V(from) - V(to). See the following Note section.
<code>-vtol value</code>	Used for <code>-constraint monotonic</code> . The default value is 0.02V. For more information, see the following Note section.

Argument	Description
<code>-filteralert 0 1</code>	Specifies how the PrimeSim XA tool reacts when the specified starting or ending node does not exist. When you set the <code>-filteralert</code> option to 1, the PrimeSim XA tool errors out when the starting or ending node that is specified with the <code>-from_node</code> or <code>-to_node</code> does not exist. The default is 0.
<code>-time_prec <prec><unit></code>	Controls precision and data unit when reporting violation time. <code>prec</code>: Specifies the number of digits after the decimal. The default is 4. <code>unit</code>: Specifies the unit for the violation time. When not specified, the tool automatically assigns a unit based on the value scale. The supported units are: <code>x</code>: Prints violation time data with no unit. <code>m</code>: Prints violation time data in milli unit. <code>u</code>: Prints violation time data in micro unit. <code>n</code>: Prints violation time data in nano unit. <code>p</code>: Prints violation time data in pico unit. <code>f</code>: Prints violation time data in fento unit. Example: The following command reports violation time for violated devices, in nanosecond with 6 digits after the decimal. <pre>cck_analog_pdown -time_prec 6n ...</pre>
<code>-voltage_prec <prec><unit></code>	Controls precision and data unit when reporting voltage data. <code>prec</code>: Specifies the number of digits after the decimal. The default is 3. <code>unit</code>: Specifies the unit for the voltage data. When not specified, the tool automatically assigns a unit based on the value scale. The supported units are: <code>x</code>: Prints voltage values with no unit. <code>m</code>: Prints voltage values in milli unit. <code>u</code>: Prints voltage values in micro unit. <code>n</code>: Prints voltage values in nano unit. <code>p</code>: Prints voltage values in pico unit. <code>f</code>: Prints voltage values in fento unit. Example: The following command reports voltage values for violated devices, with 6 digits after the decimal without any unit. <pre>cck_analog_pdown -voltage_prec 6x ...</pre>

Argument	Description
<code>-current_prec <prec><unit></code>	Prints current data and specifies the following: • <code>prec</code>: Specifies the number of digits after the decimal. The default is 3. • <code>unit</code>: Specifies the unit for the current data. When not specified, the tool automatically assigns a unit based on the value scale. The supported units are: ◦ <code>x</code>: Prints current values with no unit. ◦ <code>m</code>: Prints current values in milli unit. ◦ <code>u</code>: Prints current values in micro unit. ◦ <code>n</code>: Prints current values in nano unit. ◦ <code>p</code>: Prints current values in pico unit. ◦ <code>f</code>: Prints current values in fento unit.
<code>-numeric_prec <prec><unit></code>	Controls precision and data unit when reporting numeric data. • <code>prec</code>: Specifies the number of digits after the decimal. The default is 3. • <code>unit</code>: Specifies the unit for the numeric data. When not specified, the tool automatically assigns a unit based on the value scale. The supported units are: ◦ <code>x</code>: Prints numeric data with no unit. ◦ <code>m</code>: Prints numeric data in milli unit. ◦ <code>u</code>: Prints numeric data in micro unit. ◦ <code>n</code>: Prints numeric data in nano unit. ◦ <code>p</code>: Prints numeric data in pico unit. ◦ <code>f</code>: Prints numeric data in fento unit.

Note:

In a `cck_analog_pdown` check, the PrimeSim XA tool needs to check if the gate or base nodes of the transistors are HiZ nodes. This HiZ node check has its own rules for conducting elements, which are described in the [check_node_zstate](#) and [set_zstate_option](#) commands.

For conducting rules 0 and 1 of MOSFET devices, `Ids` is taken from the simulation. For conducting rules 2 and 3, when the `-HiZgate 1` is specified, `Ids` is taken from model evaluation with the following conditions:

PMOS: $V_g = V_{DD}/2$, $V_d = V_{DD}$, $V_s = V_{DD} 2$

NMOS: $V_g = V_{DD}/2$, $V_d = V_{DD}/2$, $V_s = 0$

However, when `-HiZgate 1` is not specified, `Ids` for conducting rules 2 and 3 is still taken from the simulation.

When the option `-constraint` is specified, the condition $V(\text{from}) \geq V(\text{to})$ in any rule for MOSFET devices is modified as follows:

`-constraint non`: Removes the condition $V(\text{from}) \geq V(\text{to})$

`-constraint monotonic`: Changes to $V(\text{from}) \geq V(\text{to}) - v_{tol}$

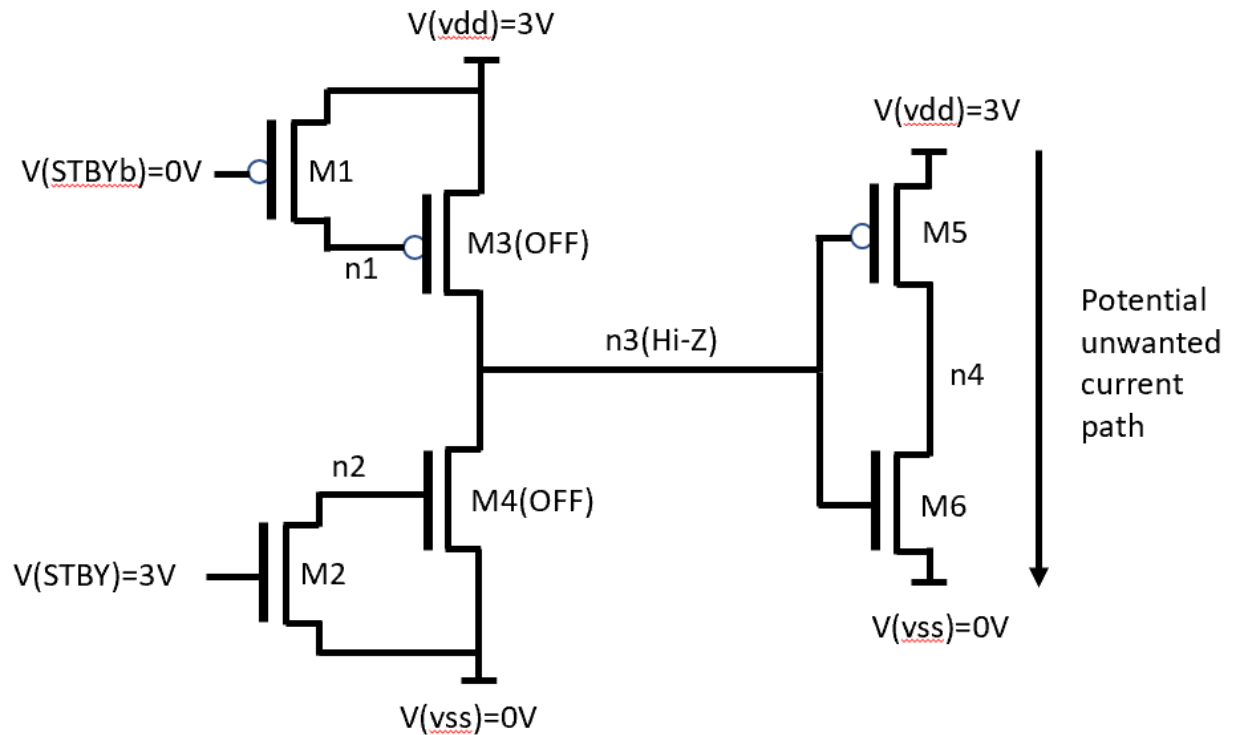
Due to the embedded PrimeSim XA RC optimization techniques, some resistors in the RC network are trimmed, which impacts the quality of the report generated by the dynamic CCK path analysis commands such as `cck_analog_pdown` and `cck_excess_ipath` as the expected resistors along the reported path are unavailable. You can apply the `cck_resistive_path_report` command together with the dynamic CCK path analysis commands to retain the original resistors and to ensure quality of the report.

Description

Unwanted currents in standby mode are typically checked by simulating the circuit in standby mode and observing the supply current. `cck_excess_ipath` can help with this task. However, unwanted currents due to floating MOS gates cannot be reliably predicted because the voltage values of floating nodes cannot be easily simulated (vector-dependence, very large time constant, and so on). [Figure 12](#) illustrates a situation that typically leads to unwanted current on silicon, which may not be observed in transient simulation. Whether or not the unwanted current through M5 and M6 can be observed depends on the voltage of n3, which cannot be easily simulated. In general, designers want to avoid this type of situation.

The following keywords are used in the `cck_analog_pdown` output file:

- `gateIsHiZ` for a MOS element whose gate is a Hi-Z node
- `baseIsHiZ` for a BJT element whose base is a Hi-Z node
- `conducting` for a MOS element whose gate is not a Hi-Z node



Device	NMOS	PMOS
-rule=0	(Ids >=idsth Vg - min(Vd, Vs) >= model->vth - dvt gate node is a HiZ node) && v(from) >= v(to)	(Ids >=idsth max(Vd, Vs) - Vg >= model->vth-dvt gate node is a HiZ node) && v(from) >= v(to)
-rule=1	Gate is a HiZ node ((Ids >=idsth Vg - min(Vd, Vs) >= model->vth - dvt) && v(from) >= v(to))	Gate is a HiZ node ((Ids >=idsth max(Vd, Vs) - Vg >= model->vth - dvt) && v(from) >= v(to))
-rule=2	Gate is a HiZ node && Ids >=idsth && v(from) >= v(to)	

Table 26 *cck_analog_pdown Conducting Criteria for MOSFET Devices (Continued)*

Device	NMOS	PMOS
-rule=3	(Gate is a HiZ node && Ids>=idsth && v(from) >= v(to)) (Gate is not a HiZ node && (Ids>=idsth Vg- min(Vd, Vs) >= model->vth - dvt) && (v(from) >= v(to)))	(Gate is a HiZ node && Ids >= Idsth && v(from) >= v(to)) (Gate is not a HiZ node && (Ids >= idsth max(Vd, Vs) - Vg >= model->vth - dvt) && (v(from) >= v(to)))
-rule=4	Gate is a HiZ node Vg- min(Vd, Vs) >= model->vth - dvt	Gate is a HiZ node max(Vd, Vs) - Vg >= model->vth -dvt

Table 27 *cck_analog_pdown Conducting Criteria for non-MOSFET Devices*

Device	Rule
BJT(NPN)	(Vbe >= Vbeth Base node is a HiZ node) && v(from) >= v(to)
BJT(PNP)	(Veb >= Vbeth Base node is a HiZ node) && v(from) >= v(to)
Diode	Diode (I(diode) > diode_ith V(a,c) >= diode_vth) && (v(from) >= v(to))
Resistor	I > 0 && v(from) >= v(to)
Other	Never conducting.

Examples

In the following example, the `cck_analog_pdown` command applies power down conducting rule (`-rule 2`) to the MOSFET and re-calculates IDs on the HiZ gate MOSFET without changing the state of the HiZ node (`-HiZgate 1`):

```
cck_analog_pdown -label test -at 120n -from_node vddd -to_node vssd \
  -rule 2 -HiZgate 1 -except_subckt *ram1*
```

cck_excess_ipath

Checks for excessive current paths from the specified starting nodes to the specified ending nodes. You can specify multiple `cck_excess_ipath` commands in a single simulation. Along the current path, PrimeSim XA can support resistor, inductor, diode, MOSFET, and bipolar devices.

Syntax

```
cck_excess_ipath -label lname
                 [scope]
                 [-ith value]
                 [-ith2 value]
                 [-tth value]
                 [twindow]
                 [from_node]
                 [to_node]
                 [node_supply]
                 [node]
                 [-at time {time}]
                 [-file merge|split]
                 [-capacitor_rule rule]
                 [-c_threshold threshold]
                 [-constraint value]
                 [-max_length value]
                 [-limit level]
                 [-filter bus]

scope== [inst][except_i]
inst== -inst inst_pattern {inst_pattern}
except_i== -except_inst inst_pattern {inst_pattern}
           -except_subckt subckt_name {subckt_name}...
twindow== [-twindow {tstart tstop ...} tstart [tstop]]
           [-tstep tstep_val]
           [-tdelay tdelay_val]
from_node== -from_node node_pattern {node_pattern}
to_node== -to_node node_pattern {node_pattern}
node_supply== -match_rule vsrc | none
capacitor_rule== 0|1|no|yes|false|true
c_threshold== value
```

Arguments

Argument	Description
-label label_name	Specifies the label name that appears in the report file, which makes it easier to search the report.
-inst inst_pattern {inst_pattern}	Scopes to the named instances. You can use wildcard characters in the instance names according to the rules specified by the set_wildcard_rule command.
-except_inst inst_pattern {inst_pattern}	Excludes instances from the command scope. Any instance whose hierarchical instance name matches the pattern is excluded. You can use wildcard characters in the instance names according to the rules specified by the set_wildcard_rule command.

Argument	Description
<code>-except_subckt subckt_name[.inst_pattern] ...</code>	Specifies the subcircuits that should be excluded from reporting. If required, you can specify the exceptions. Multiple subckt names are supported.
<code>-ith value</code>	Specifies the current threshold of an excessive current path. The elements in a path must have a current value that exceeds the specified value. For a violation to be reported, this option takes absolute value. The default is 50 uA or 5.0e-5.
<code>-ith2 value</code>	Specifies the threshold current for an additional remark on the DC path generated by <code>-ith</code> . If MOSFET devices are on the DC path and if their <i>I</i> _{ds} current exceeds the specified (or default) <code>-ith2</code> value, the command will print an additional remark under the device <i>Status</i> section (<i>I</i> > <i>ith2</i>). The default <i>ith2</i> value is 1e-3A.
<code>-tth value</code>	Specifies the time duration of an excessive current path. For a violation to be reported, the current in an excessive current path must last longer than the specified value. The default is 5 ns or 5.0e-9.
<code>-twindow {tstart tstop ...} tstart [tstop]</code>	Instructs PrimeSim XA to perform the check within the time window defined by <code>tstart tstop {tstart tstop}</code> . The <code>tstart</code> and <code>tstop</code> must come in pairs, except for the final window which if <code>tstop</code> is not specified, it is assumed to be the end of the simulation. The final <code>tstop</code> can also be the <code>end</code> or <code>END</code> keyword.
<code>-tstep tstep_val</code>	If this argument is specified, the <code>cck_excess_ipath</code> check is performed at discrete time points, which are integer multiples of <code>tstep_val</code> . You can use this argument in conjunction with the <code>-twindow</code> argument. When you specify this argument, the <code>-tth</code> argument is ignored.
<code>-tdelay tdelay_val</code>	Performs checking at each time defined as <code>t+tdelay_val</code> . " <i>t</i> " is the time an input voltage source changes to a new voltage level. You can use this option with <code>-twindow</code> .

Argument	Description
<code>-from_node node_pattern</code> <code>{node_pattern}</code>	Specifies the starting nodes of a path to be checked, which avoids long simulation times. Wildcard characters in the node names are supported for nodes that have a direct connection to the voltage source. For discrete mode: • If <code>-from_node</code> is applied with a wildcard expression, look for matched Vsrc nodes through the wildcard expression, and only check the paths that have $v(\text{from}) > v(\text{to})$. • If <code>-from_node</code> is applied without a wildcard, check paths between those specified from nodes and $v(\text{from}) > v(\text{to})$. • If no <code>-from_node</code> is applied, check the paths between DC Vsrc nodes and $v(\text{from}) > v(\text{to})$. For continuous mode: • If <code>-from_node</code> is applied with a wildcard expression, look for matched Vsrc nodes through the wildcard expression, and report paths from <code>-from nodes</code> to <code>-to nodes</code> and those paths that <code>-from nodes</code> and <code>-to nodes</code> that are DC Vsrc nodes. Paths that have $v(\text{from}) \leq v(\text{to})$ are skipped. • If <code>-from_node</code> is applied without a wildcard, check paths between specified from/to nodes and those paths that <code>-from node</code> and <code>-to node</code> are DC Vsrc nodes. Paths that have $v(\text{from}) \leq v(\text{to})$ are skipped. • If no <code>-from_node</code> is applied, check the paths between DC Vsrc nodes. Paths that have $v(\text{from}) \leq v(\text{to})$ are skipped.
<code>-to_node node_pattern</code> <code>{node_pattern}</code>	Specifies the ending nodes of a path to be checked, which avoids long simulation times. Wildcard characters in the node names are supported for nodes that have a direct connection to the voltage source. For discrete mode: • If <code>-to_node</code> is applied with a wildcard expression, look for matched Vsrc nodes through the wildcard expression, and only check the paths that have $v(\text{from}) > v(\text{to})$. • If <code>-to_node</code> is applied without a wildcard, check paths between those specified from nodes and $v(\text{from}) > v(\text{to})$. • If no <code>-to_node</code> is applied, check the paths between DC Vsrc nodes and $v(\text{from}) > v(\text{to})$. For continuous mode: • If <code>-to_node</code> is applied with a wildcard expression, look for matched Vsrc nodes through the wildcard expression, and report paths from <code>-from nodes</code> to <code>-to nodes</code> and those paths that <code>-from nodes</code> and <code>-to nodes</code> that are DC Vsrc nodes. Paths that have $v(\text{from}) \leq v(\text{to})$ are skipped. • If <code>-to_node</code> is applied without a wildcard, check paths between specified from/to nodes and those paths that <code>-from node</code> and <code>-to node</code> are DC Vsrc nodes. Paths that have $v(\text{from}) \leq v(\text{to})$ are skipped. • If no <code>-to_node</code> is applied, check the paths between DC Vsrc nodes. Paths that have $v(\text{from}) \leq v(\text{to})$ are skipped.

Argument	Description
<code>-match_rule vsrc none</code>	When set to <code>vsrc</code> (default), a wildcard character in <code>-from_node</code> or <code>-to_node</code> only matches a constant voltage source node. When set to <code>none</code> , a wildcard character in <code>-from_node</code> or <code>-to_node</code> can match to any circuit node name.
<code>-node node_pattern {node_pattern}</code>	If you specify the <code>-node</code> argument, the following rules apply. - If you do not specify the <code>-from_node</code> and <code>-to_node</code> arguments, the CircuitCheck tool picks up the DC/variable (PWL,PULSE,SIN and so on) V-source nodes as the default <code>-from_node</code> and <code>-to_node</code> list, automatically. If user you apply the <code>-node</code> argument, the specified nodes are added to both the <code>-from_node</code> and <code>-to_node</code> list. - If you apply the <code>-from_node</code> and <code>-to_node</code> arguments, the CircuitCheck tool sets the specified nodes to <code>-from_node</code> and <code>-to_node</code> list respectively. If you also apply the <code>-node</code> argument, the specified nodes are added to both the <code>-from_node</code> and <code>-to_node</code> list accordingly.
<code>-at time {time}</code>	Performs discrete checking at the specified time points. When you specify this argument, the output from the measurement is redirected to another file (<code>.cckexipath_label</code>) where <code>label</code> is the label name of the statement.
<code>-file merge split</code>	Enables file merging or splitting. When set to <code>merge</code> (default), all outputs are merged into one output file (named <code>prefix.cckexipath</code>). When set to <code>split</code>, all the outputs are split into different files (named <code>prefix.cckexipath_label</code>).
<code>-capacitor_rule 0 1 no yes false true</code>	Specifies whether or not to report excessive currents through capacitors. When <code>-capacitor_rule</code> is set to 1, excessive current paths through capacitors greater than 1pF are reported. To change the capacitor threshold from 1pF to another value, use the <code>-c_threshold</code> option. When set to 0 (default), excessive currents through capacitors are not reported. Note: The <code>-capacitor_rule</code> and <code>-c_threshold</code> options only apply to the <code>cck_excess_ipath</code> command line they are used in. If there are multiple <code>cck_excess_ipath</code> commands, some may contain a combination of <code>-capacitor_rule</code> and <code>-c_threshold</code> options or no options at all.

Argument	Description
<code>-c_threshold value</code>	Specifies the capacitors threshold value. Default is 1pF. Note: If you specify the <code>-c_threshold</code> option and if the <code>-capacitor_rule</code> option is not present or is set to 0 false no, the <code>-c_threshold</code> option is ignored. A warning message is issued and a log file with the error message is generated. Note: The <code>-capacitor_rule</code> and <code>-c_threshold</code> options only apply to the <code>cck_excess_ipath</code> command line they are used in. If there are multiple <code>cck_excess_ipath</code> commands, some may contain a combination of <code>-capacitor_rule</code> and <code>-c_threshold</code> options or no options at all.
<code>-constraint value</code>	Can be: <code>none</code> (default) to specify that devices are conducting when device current is larger than the <code>-ith</code> value. <code>monotonic</code> to specify that devices are conducting when device current is larger than <code>-ith</code> and $V(\text{from}) \geq V(\text{to})$.
<code>-max_length value</code>	Specifies the value of that limits the maximum number of devices in an excess current path. You can specify a value of 1 to indicate an unlimited path length.
<code>-limit level</code>	Specifies that all devices in the reported path must be at or below the hierarchical depth of the specified <code>level</code> . There is no default setting.
<code>-filter bus</code>	Consolidates path reporting associated with instances of same bus notation cluster.

Description

If you do not specify `-from_node` and `-to_node`, `cck_excess_ipath` searches the DC paths with all combinations of all constant voltage sources (VSRCs). To avoid a long simulation time, specify the starting nodes and ending nodes.

At the end of the simulation, violations are reported to an output file, `*.cckexipath`.

Examples

The following example monitors the paths from `vdd` or `xam.d1` to `gnd` between 10 –100 ns. If all the elements in these paths have currents greater than 10 uA, and last for more than 0.6 ns, a violation is reported.

```
cck_excess_ipath -label test1 -ith 10u -tth 0.6n -from_node vdd xam.d1
                -to_node gnd -twindow 10n 100n
```

Note:

Due to the embedded PrimeSim XA RC optimization techniques, some resistors in the RC network are trimmed. This impacts the quality of the report generated by the dynamic CCK path analysis commands, such as [cck_analog_pdown](#) and [cck_excess_ipath](#) as the expected resistors along the reported path are unavailable.

The [cck_resistive_path_report](#) command can be applied together with the dynamic CCK path analysis commands to retain the original resistors and to ensure quality of the report.

Monitors the paths from `vdd` to `gnd` between 10 ns and 100 ns. If all the elements in these paths have currents greater than 10uA, and last for more than 0.6 ns, a violation is reported. The violation is also reported if the current path flows through capacitors greater than 1pF.

```
cck_excess_ipath -label test1 -ith 10u -tth 0.6n -from_node vdd -to_node gnd -twindow 10n 100n -capacitor_rule 1
```

Monitors the paths from `vdd` to `vss`. If all the elements in these paths have currents greater than 10 uA, and last for more than 1 ns, a violation is reported. The violation is also reported if the current path flows through capacitors greater than 100pF.

```
cck_excess_ipath -label test2 -ith 10u -tth 1n -from_node vdd -to_node vss -capacitor_rule 1 -c_threshold 100e-15
```

cck_disable

Disables dynamic CCK checks performed by the `cck_soa` or `cck_signal` command.

Syntax

```
cck_disable <cck_soa | cck_signal> [-label label_name] \
    [-subckt subckt_name] [-inst inst_name] \
    [-twindows t_start t_end]
```

Arguments

Argument	Description
<code>cck_soa cck_signal</code>	Disables CCK checks performed by <code>cck_soa</code> or <code>cck_signal</code> .
<code>-label label_name]</code>	Specifies labels to be excluded from CCK checks.
<code>-subckt subckt_name</code>	Specifies subcircuits to be excluded from CCK checks.
<code>-inst inst_name</code>	Specifies instances to be excluded from CCK checks.

Argument	Description
<code>-twindow t_start t_end</code>	Specifies time windows to be excluded from CCK checks.

When both the `-subckt` and `-inst` options are specified, the tool disables checks on the devices that meet either of the argument settings. For example, when the following command is set:

```
cck_disable cck_soa -subckt s1 s2 -inst d1 d2
```

The tool disables checks on the following devices:

- d1 under subckt s1
- d2 under subckt s1
- d1 under subckt s2
- d2 under subckt s2

The `-label`, `-subckt`, `-inst` and `-twindow` options are mutually inclusive. For example, when the following command is set:

```
cck_disable -cck_soa -label test1 -inst m1
```

The tool disables checks on the device m1 for the `cck_soa` command with the test1 label. All other checks remain for the device m1.

If multiple `cck_disable` commands are specified for the same CCK command, the tool disables checks based on the combination of the command settings.

Examples

The following example disables `cck_soa` command checking.

```
cck_disable cck_soa
```

The following example disables checks for the `cck_soa` command with labels test1 or test2.

```
cck_disable cck_soa -label test1 test2
```

The following example disables checks on device x1.m1 for the `cck_soa` command with label test1.

```
cck_disable cck_soa -label test1 -inst x1.m1
```

The following example disables checks on the devices under subcircuits inv and adder during the simulation time between 10n and 20n.

```
cck_disable cck_soa -subckt inv adder -twindow 10n 20n
```

cck_post

Post processes results from the [cck_soa](#) and the [cck_signal](#) commands. [cck_soa](#) and [cck_signal](#) commands support voltage and current checking.

Syntax

```
cck_post -switch enable_value [-waveform file_name]
```

```
enable_value ::= 1|0|on|off|yes|no|true|false
```

Arguments

Argument	Description
<i>enable_value</i>	Enables/disables postprocessing.
<i>-waveform file_name</i>	Reads the waveform file and runs postprocessing. If the waveform file does not exist, PrimeSim XA issues an error message. Note that split waveforms are not supported for cck_soa and the cck_signal postprocessing.

Description

The [cck_post](#) command runs all the diagnostic circuit checks performed at the end of the simulation based on a waveform file. It supports the postprocessing results from the [cck_soa](#) and the [cck_signal](#) commands.

Note:

The [cck_post](#) command supports on the FSDB and WDF output formats.

PrimeSim XA supports postprocessing CCK results using a two-pass or single-pass method:

- In the two-pass method, the first pass dumps the signals into the waveform file using the probing commands for voltage and current checking, [probe_waveform_voltage](#) and [probe_waveform_current](#). For example:

```
probe_waveform_voltage -vall * -limit 100
probe_waveform_current -iall * -limit 100
```

If you know the specific signals that you want to use for postprocessing, you can limit the probing to only those signals. During postprocessing, if a signal called by the [cck_soa](#) or the [cck_signal](#) command is not found, a warning is issued.

- In the single-pass method, PrimeSim XA automatically dumps only the necessary signals in the waveform file based on the scope specified by the `cck_soa` and the `cck_signal` commands. This method generates a smaller waveform file. For example:

```
cck_post ...  
  
cck_soa -label test1 -check vgs ...  
  
cck_soa -label test2 -check id ...  
  
cck_signal -label test1 -check vgs ...  
  
cck_signal -label test2 -check i1 ...
```

Examples

```
cck_post -switch 1
```

The following example enables the second-pass of postprocessing by reading in the `test.fsdb` waveform file.

```
cck_post -switch 1 -waveform test.fsdb
```

cck_resistive_path_report

Due to embedded RC optimization methods some resistors in RC network are trimmed. This impacts the quality of the report generated by the dynamic CCK path analysis commands, [cck_analog_pdown](#) and [cck_excess_ipath](#) as the expected resistors along the reported path are not available. The `cck_resistive_path_report` command is used to retain the original resistors thereby ensuring that there is no impact on the performance.

Syntax

```
cck_resistive_path_report -level <value>
```


Arguments

Argument	Description
<code>-level <value></code>	Specify one of the following value: • 0: To skip current probing on resistive nodes and their connected resistors along the path. The default is 0. • 1: To enable current probing on resistive nodes and related resistors. The RC optimization algorithms are applied to the resistive networks monitored by the two CCK commands. Due to the RC optimization, some resistors might be missing from the report. • 2: To enable current probing on resistive nodes and related resistors. In this case the RC optimization algorithms are applied to a lesser extent than the previous setting (<code>-level 1</code>), to the resistive network monitored by the two CCK commands. Due to the RC optimization, some resistors in the power net, might be missing from the report. • 3: To enable current probing on resistive nodes and related resistors. In this case the RC optimization algorithms are applied to a lesser extent than the previous setting (<code>-level 2</code>), to the resistive network monitored by the two CCK commands. All the resistors in the path are retained.

Description

In the following example,

```
cck_resistive_path_report -level 1
```

RC optimization is applied to the to the resistive network monitored by the two CCK commands, [cck_analog_pdown](#) and [cck_excess_ipath](#). The two commands will report the resistive network as a reported path, but some resistors will be missed from the report.

cck_signal

Checks the state of an arbitrary collection of signals that can be part of the checks for node and instance probes. The `cck_signal` command also supports current signals.

Syntax

```
cck_signal -label lname
           [scope]
           [-message "string"]
           [-numv num_val]
           constraint
           [twindow]
           [-flush auto|auto2]
           [-limit level]
           [-filterAlert 0|1]
```

```

[-portalert enable_value]
[-port 0|1]
[-numvd num_value]
[-sort sort_order]
[-report conn|subinfo]
[-intr interactive_mode]
[-avg 0|1]
[-integ 0|1]
[-time_factor ]

scope      ::= [inst][subckt][except_i][except_s]
inst       ::= -inst inst_pattern {inst_pattern}
subckt     ::= -subckt subckt_name {subckt_name}
except_i   ::= -except_inst inst_pattern {inst_pattern}
except_s   ::= -except_subckt subckt_name [.inst_pattern]
           {subckt_name [.inst_pattern]}

constraint ::= real_value_check|logic_check
real_value_check ::= -check real_expression
                  -min min_value
                  -max max_value [-tth value]
logic_check  ::= -check logic_expression
                  [-logic safe|violation]
                  [-tth value]
logic_check  ::= -check logic_expression
                  -avg 1 | -integ 1
                  -min min_value
                  -max max_value

twindow     ::= [-twindow {tstart tstop} tstart [tstop]]
                  [-tstep tstep_val]
tstop       ::= time_value|end|END
report      ::= -report report_type
report_type ::= operand
interactive_mode ::= 0|1|yes|no|true|false
sort_order  ::= minimum|maximum|longest
num_value   ::= all|boundary|0...n

```

Arguments

Argument	Description
-label label_name	Specifies the label name that appears in the report file, which makes it easier to search the report. If on-the-fly reporting is enabled, the label forms part of the report file name.
-message "string"	Adds a message in the violation header to explain the purpose of the violation check. It is a sentence written enclosed in double quotes. You can put keywords in the sentence that help them to sort the violations.

Argument	Description
<code>-inst inst_pattern</code> <code>{inst_pattern}</code>	Scopes to the named instances. You can use wildcards in instance names as specified by the <code>set_wildcard_rule</code> command.
<code>-subckt subckt_name</code> <code>{subckt_name}</code>	Scopes to instances of the named subcircuits.
<code>-except_inst</code> <code>inst_pattern</code> <code>{inst_pattern}</code>	Excludes instances from the <code>cck_signal</code> command scope. Any instance whose hierarchical instance name matches the pattern is excluded from the scope. Wildcards are supported.
<code>-except_subckt</code> <code>sub_name[.inst_pattern]</code> <code>{sub_name}</code> <code>[.inst_pattern]</code>	Excludes all instances of the named subcircuits from the <code>cck_signal</code> command scope. Any subcircuit whose name matches the pattern is excluded from the scope. You can also exclude only the named instances of the subcircuit. Wildcards are not supported in the subcircuit names, but are supported in the <code>inst_pattern</code> . Use the following syntax to exclude multiple subcircuits in the same <code>cck_signal</code> command statement: <code>cck_signal...-except_subckt sub_a sub_b sub_c</code> Where <code>sub_a</code> , <code>sub_c</code> and <code>sub_c</code> are the explicit subcircuit names to exclude.
<code>-numv num_val</code>	Limits the maximum number of total violations reported. The default is 300. To print all violations, use the <code>all</code> keyword to print all violations. To turn off violation reporting, specify 0.

Argument	Description
<code>-check real_expression</code> <code>-min min_value -max max_value [-tth tval] logic_expression [-logic safe violation] [-tth tval]</code>	Specifies a constraint expression, which can be a logic expression or a real-value expression. - A real-value expression can also be an arithmetic expression. If the expression contains a logical operator, it is considered to be a logic expression. A constraint that is a real-value expression is reported as a violation if it is outside the safe operating range specified by <code>-min</code> and <code>-max</code> arguments. - A logic expression that evaluates to logical false (0) for a period longer than <code>tval</code> is reported as a violation. The logical expression uses the perspective of the defining safe operating area. You can change it with the <code>-logic</code> option. The default for <code>-logic</code> is <code>safe</code>. For information about the <code>safe</code> and <code>violation</code> keywords, see Table 28. To conform to Tcl conventions, enclose the constraint expression in double quotes or braces if it contains any white space. This expression is defined as a single group for the Tcl parser. This means that it should not contain white space or be grouped with the Tcl grouping operators, double quotes (""), or braces ({}). You can construct the expression with a single or a group of constraint functions (see Table 29) with either arithmetic operators (+, -, *, and /) or logical operators (>, <, >=, <=, &&, , ==, and !=). Note: The <code>-check</code> expression, and the <code>-min</code> and <code>-max</code> arguments all support local instance parameters.
<code>-min min_value</code>	
<code>-max max_value</code>	
<code>-tth</code>	
<code>-logic safe violation</code>	
<code>-twindow {tstart tstop}</code> <code>tstart tstop -tstep tstep_val</code>	If you specify this argument, the tool performs the <code>cck_signal</code> check within the time window defined by <code>tstart tstop {tstart tstop}</code>. The <code>tstart</code> and <code>tstop</code> values must come in pairs, except for the final window where: if <code>tstop</code> is not specified, it is assumed to be the end of the simulation. The final <code>tstop</code> can also be the <code>end</code> or <code>END</code> keyword. The default is from <code>tstart=0</code> to <code>tstop=end</code>. The <code>-tstep</code> argument must be used with the <code>-twindow</code> argument, except when the <code>-twindow</code> argument has no explicit stop time. For example: <pre>-twindow 10n 20n 50n</pre>

Argument	Description
<code>-tstep tstep_val</code>	When specified, the <code>cck_signal</code> check is performed at discrete time points, which are integer multiples of <code>tstep_val</code> . You can use this argument with the <code>-twindow</code> argument. When you specify this argument, the <code>-tth</code> argument is ignored.
<code>-flush auto auto2</code>	Auto-flushing is enabled by providing the <code>auto2</code> keyword, which specifies to report violations according to the following criteria: • Report violations as they occur. • Report violations if the violation duration satisfies the time period specified by the <code>cck_signal</code> command constraint expression.
<code>-filterAlert 0 1</code>	Issues the <code>zero scoping error</code> message if there are no devices that match with the user-specified scoping rule. This is caused by an unintended typo or the exclusion of the matched device by the <code>-except_inst</code> or the <code>-except_subckt</code> expression in the same <code>cck_signal</code> statement. By default, the <code>-filterAlert</code> option is set to 1, meaning that such a scenario is considered an error. The tool displays the <code>zero scoping error</code> message and terminates the simulation. If <code>-filterAlert</code> is set to 0, only the <code>zero scoping error</code> message is displayed and the simulation is not terminated.
<code>-limit level</code>	Specifies the hierarchy level to which the scope is applied. When used with the <code>-subckt</code> option, the <code>-limit</code> option is relative to where the subcircuit is located in the hierarchy. A value of 0 specifies the top level of the subcircuit. The default is for all levels. This option is effective only when scoping includes wildcard characters.
<code>-portalert enable_value</code>	Displays a warning or an error message if the verilog-A port specified in the constraint expression is not matched. • If you specify 1, an error message is displayed and the simulation is terminated. • If you specify 0, a warning message is displayed and the simulation is continued. The default is 1. You can set <code>enable_value</code> as 1 0, yes/no or true/false.
<code>-port 0 1</code>	When set to 1, enables wildcard matching of a subcircuit port in a signal name. The default is 0.
<code>-report report_type</code>	Chooses to report the following additional information for each device: • <code>conn</code> to report connectivity information. • <code>subinfo</code> to report the subcircuit hierarchy information.

Argument	Description
<code>-numvd num_value</code>	Limits the number of violations per device or node to report. The default is 1. To print all violations, specify the <code>all</code> keyword. To disable violation reporting, set the value to 0. To print the three most significant violations, such as minimum value, maximum value and longest duration of each individual device, specify the <code>boundary</code> keyword. Note: If the <code>-check</code> expression is specified, only the longest violation is reported since the minimum and maximum violations have no relevance. If a device has only one violation, the <code>cck_signal</code> command still reports three violations since this violation meets the criteria of the minimum value, maximum value and longest duration. If the check expression is device-based (that is, <code>-check {v(c1) - v(c2) <= 0.2}</code>), the <code>-numvd</code> option specifies the number of violations per device. If the check expression is node-based (that is, <code>-check {v(node1) <= 0}</code>), the <code>-numvd</code> option specifies the number of violations per node.
<code>-sort sort_order</code>	Specifies the sorting order to report the violations of each individual device. The supported values are: • minimum • maximum • longest Note: You cannot use the <code>-sort</code> option together with the <code>-numvd boundary</code> option.
<code>-intr interactive_mode</code>	When set to <code>true yes 1</code>, the tool interrupts the simulation when a violation is found. Note: The interactive mode is triggered only at the beginning of a violation. Not every violated data point triggers the interactive mode. The default is <code>false no 0</code>.

Argument	Description
-avg 0 1	When set to 1, the tool calculates violations by dividing "total violation time" by "total transient time" in percentage. The default is 0. This -avg option is mutually exclusive with the -integ option. Example: <pre>cck_signal -label testavg \ -check "vbd(x1.mn1)>-1.00 && \ vbd(x1.mn1)<-0.4" -avg 1</pre> The report shows the following information: <pre>Constraint=(vbd(x1.mn1)>-1.00 && \ vbd(x1.mn1)<-0.4, *, *) avg=1 \ *</pre> Dev #1: x1.mn1 Attributes: average=0.584%
-integ 0 1	When set to 1, the tool calculates violations as the total violation time. The default is 0. This -avg option is mutually exclusive with the -integ option. Example: <pre>cck_signal -label testinteg \ -check "vbd(x1.mn1)>-1.00 && \ vbd(x1.mn1)<-0.4" -integ 1</pre> The report shows the following information: <pre>Constraint=(vbd(x1.mn1)>-1.00 && \ vbd(x1.mn1)<-0.4, *, *) integ=1 *</pre> Dev #1: x1.mn1 Attributes: integral=1.867ns

Argument	Description
<code>-time_factor</code> <code>signal_name</code>	Specifies the weighting factor to calculate a duty cycle. This option can only be used together with the <code>-avg</code> or <code>-integ</code> option. Example: <pre>cck_signal -label testifx \ -check "vbd(x1.mn1)>-1.00 && \ vbd(x1.mn1)<-0.4" -integ 1 \ -time_factor test</pre> In the above example, - Transient time is 10us - Logical expression is true between 1.2u and 1.4u, and between 5u and 5.5u - Total duration of (expression=true) is: 0.2u+0.5u=0.7u - Real life time factor is 1 between 0 and 5u, and 2 between 5u and 10u - Duration of violation is: 0.2u*1+0.5u*2=1.2u - Total duration is: 5u*1+5u*2=15u - Duty cycle is: 1.2u/15u=0.08 The report includes the total duration of the violation weighted by the "test" signal. For example, <pre>Constraint=(vbd(x1.mn1)>-1.00 && \ vbd(x1.mn1)<-0.4, *, *) integ=1 \ time_factor=test * Dev #1: x1.mn1 Attributes: weighted integral=0.924ns</pre>

Description

You can use the asterisk ("*") wildcard character in the check expression in a voltage access function if the other terms of the expression have a constant value. The following scenarios are supported:

```
v(*)
"v(*) - 3.0"
"v(top.x1.*.port.+)/2"
```

The following scenarios are not supported because they contain a second signal access function:

```
"v(*) - v(node1) + 1.2"
"v(*) - v(node1) + 1.2"
```

[Table 28](#) lists the conditions for the `-logic` `safe` and `violation` keywords.

Table 28 *-logic conditions*

Expression Value/Logic Setting	-logic safe (default)	-logic violation
TRUE		Report a violation.
FALSE	Report a violation.	

[Table 29](#) lists the accessing common instance conditions.

Table 29 *Constraint Functions*

Constraint Function	Description
<code>abs(x)</code>	Returns the absolute value of x: $ x $.
<code>ceil(x)</code>	Returns the integer that is greater than or equal to value of x.
<code>Vg(inst)</code>	Gate voltage
<code>Vd(inst)</code>	Drain voltage
<code>Vs(inst)</code>	Source voltage
<code>Vb(inst)</code>	Bulk voltage
<code>Vgs(inst)</code>	Gate-source voltage
<code>Vsg(inst)</code>	Source-gate voltage
<code>Vds(inst)</code>	Drain-source voltage
<code>Vsd(inst)</code>	Source-drain voltage
<code>Vgd(inst)</code>	Gate-drain voltage
<code>Vdg(inst)</code>	Drain-gate voltage
<code>Vdb(inst)</code>	Drain-bulk voltage
<code>Vbd(inst)</code>	Bulk-drain voltage
<code>Vsb(inst)</code>	Source-bulk voltage
<code>Vbs(inst)</code>	Bulk-source voltage
<code>iN(inst)</code>	N-terminal current
<code>V(node)</code>	Node voltage
<code>X(inst.port)</code>	Subcircuit port current

Table 29 Constraint Functions (Continued)

Constraint Function	Description
Isub(inst.port)	Subcircuit port current

Table 30 lists the mathematical functions are supported to construct the constraint expressions.

Table 30 Mathematical Functions

Built-In Function	Description
abs(x)	Returns the absolute value of x: x .
ceil(x)	Returns the integer that is greater than or equal to value of x.
exp(x)	Returns e, raised to the power of the value of x: e ^x .
floor(x)	Returns the integer that is less than or equal to value of x.
log(x)	Returns the natural logarithm of the absolute value of x, with the sign of x: (sign of x)log(x).
log10(x)	Returns the base 10 logarithm of the absolute value of x, with the sign of x: (sign of x)log ₁₀ (x).
pow(x,y)	Returns the value of x raised to the power of the value of y.
sqrt(x)	Returns the square root of the absolute value of x: sqrt(-x) = -sqrt(x).

To conform to the Tcl convention, enclose the constraint expression in double quotes or braces if it contains any white space.

You can use `cck_signal` to scope to specific instances using the `-inst`, `-subckt`, `-except_inst`, and `-except_subckt` arguments. You can use wildcards only with the `-inst` and `-except_inst` arguments. Wildcard characters are not supported for the `-subckt` and `-except_subckt` arguments.

The rule between `-except_subckt` and `-except_inst` arguments accumulates in a logical OR. If you use the `-except_subckt` argument with `-except_inst`, the checks exclude all the instances inside the named subcircuit by the `-except_subckt` argument and all named instances by `-except_inst`.

When you specify the `-flush` argument, on-the-fly auto-reporting is enabled. Each `cck_signal` command for which auto-flushing is enabled writes its output to a unique file. These files are named as `prefix.ccksoa.label`, where `prefix` is the input netlist file name

prefix, or the name specified with `-o` on the command line. The label is the command label specified by `-label` argument of that command. Each file is updated as violations are detected.

Examples

```
cck_signal -label simple_check-check vgs(x1.m1) -min 0 -max 0.6
```

The following example checks a MOS terminal voltage and a node voltage.

```
cck_signal -label logic_check -check {VGS(x1.m1) < 0.3 && v(x1.vddhp) > 3.0}
```

Flags a violation if `v(vreg)` is outside the `-min` and `-max` values for more than 10 ns.

```
cck_signal -label simple_check_3n -subckt regulator -check v(vreg) -min 1.1 -max 1.2 -tth 10n
```

Checks two windows: the first is from 1–2 us, and the second is from 3–4 us.

```
cck_signal -label window_check -subckt regulator -check v(creg) -min 1.1 -max 1.2 -twindow 1u 2u 3u 4u
```

```
cck_signal -label step_check -check v(x1.vreg) -min 0 -max 0.6 -tstep 500n
```

```
cck_signal -label window_step -inst x1 -check v(ctrl) -min 0 -max 0.6 -twindow 500n 5u -tstep 200n
```

In the following example, checks `v(ctrl)` on every instance of the `mysub` subcircuit, except `x1.xsub1` and `x1.xsub2`.

```
cck_signal -label except_inst_check -subckt mysub -except_inst x1.xsub1 x1.xsub2 -check v(ctrl) -min 0.5 -max 0.75
```

In the following example, the `-inst` option scopes to the `xcmod0`, `xcmod1`, `xcmod2`, ..., and `xcmod15` instances. The `-except_subckt` option removes the `xcmod4` instance through the `xcmod15` instance. The resulting checked signals are:

- `xctrl_top.xcmod0.vout`
- `xctrl_top.xcmod1.vout`
- `xctrl_top.xcmod2.vout`
- `xctrl_top.xcmod3.vout`

```
cck_signal -label except_subckt_check -inst xctrl_top.xcmod*
-except_subckt dummy_ctrl -check v(vout) -min 0.5 -max 1.2
xctrl_top a b c d .... ctrl_top .subckt ctrl_top

xcmod0 1 2 3 log_ctrl
xcmod1 4 5 6 log_ctrl
```

```
xcm0d2 7 8 9 log_ctrl
xcm0d3 10 11 12 dummy_ctrl
xcm0d4 ... dummy_ctrl
...
xcm0d15 ... dummy_ctrl
.ends
.subckt log_ctrl a b c
ro vout c r=10
...
.ends
.subckt dummy_ctrl a b c
ro vout c r=10
evout vout 0 vcvs a b 0.5
.ends
```

In the following example, the wildcard in `-inst` scopes to the `xcm0d0` and `xcm0d4` instances, but the `vout` signal does not exist in the `xcm0d4` instance. The tool raises a zero scope error.

```
cck_signal -inst xctrl_top.xcm0d* -check v(vout) -min 0.5 -max 1.2

xctrl_top a b c d ... ctrl_top
.subckt ctrl_top
xcm0d0 1 2 3 log_ctrl
xcm0d4 .... dummy_ctrl
.ends
.subckt log_ctrl a b c
ro vout c r=10
...
.ends
.subckt dummy_ctrl a b c
*does not contain any noted vout
ro cout c r=10
evout cout 0 vcvs a b 0.5
.ends
```

In the following example, the interactive mode is triggered whenever a MOSFET's `vg` crosses the 3.3v threshold.

```
cck_signal -label test_intr -check vg -max 3.3 -intr true
```

The following example prints the checking message in the `.ccksignal` file header:

```
cck_signal -label sig1 -check abs(v(s[3])-v(s[2])) -min 1 -max 2 \
  -twindow 10n 800n -message "verify the s3 and s2 signals"
```

Here is the message printed in the `.ccksignal` file header:

```
Tag: sig1
* Check Windows:
*   start=10n stop=0.8u
* message="verify the s3 and s2 signals"
```

```
* check=abs(v(s[3])-v(s[2])) min=1 max=2s
*
```

cck_soa

Runs a safe operating area check.

Syntax

```
cck_soa -label lname
        [-message "string"]
        [scope]
        [-numvd num_val]
        constraint
        [twindow]
        [-flush auto|auto2]
        [-flush_merge enable_value]
        [report]
        [-numd num_val]
        [-filteralert enable_value]
        [-portalert enable_value]
        [-limit value]
        [-notation scientific]
        [-intr interactive_mode]
        [-avg 0|1]
        [-integ 0|1]
        [-time_factor ]
        [-probe enable_value]
        [-probe_hier n]
        [-time_prec <prec><unit>]
        [-voltage_prec <prec><unit>]
        [-current_prec <prec><unit>]
        [-numeric_prec <prec><unit>]

scope      ::= [model][inst][subckt]
              [instparam][except_i][except_s]
model      ::= -model model_name {model_name}
inst       ::= -inst inst_pattern {inst_pattern}
subckt     ::= -subckt subckt_name {subckt_name}
instparam  ::= -instparam inst_expression
except_i   ::= -except_inst inst_pattern {inst_pattern}
except_s   ::= -except_subckt subckt_name [.inst_pattern]
              {subckt_name [.inst_pattern]}

constraint ::= real_value_check|logic_check|if-then-else-stmt
real_value_check ::= -check real_expr
                  -min min_value
                  -max max_value [-tth value]
logic_check ::= -check logic_expr
               [-logic safe|violation]
```

```

                                [-tth value]
if-then-else-stmt ::= -check if logic_expr then
                                [if-then-else-stmt | check_opts]
                                [else [if-then-else-stmt | check_opts]] endif
check_opts ::= check (real_expr, min_val, max_val [, tth_val])
               check (logic_expr                                [, tth_val])

twindow      ::= [-twindow {tstart tstop} tstart [tstop]]
               [-tstep tstep_val]
report       ::= -report report_type {report_type}
report_type  ::= wl|conn|subinfo|operand|variation
enable_value ::= 1|0|on|off|yes|no|true|false
interactive_mode ::= 0|1|yes|no|true|false

```

Arguments

Argument	Description
-label label_name	Specifies the label name that appears in the report file, which makes it easier to search the report. If on-the-fly reporting is enabled, the label forms part of the report file name.
-message "string"	Add a message in the violation header to explain the purpose of the violation check. It is a sentence written enclosed in double quotes. You can put keywords in the sentence that help them to sort the violations.
-model model_name {model_name}	Scopes to devices of the named models. all_resistors, all_capacitors, and all_diodes are reserved keywords that are used to scope to all the resistors, capacitors, and diodes respectively. Wildcard expression is supported.
-inst inst_pattern {inst_pattern}	Scopes to the named instances. You can use wildcards in instance names as specified by the PrimeSim XA set_wildcard_rule command. Wildcard behavior is determined by the set_wildcard_rule command.
-subckt subckt_name {subckt_name}	Scopes to instances of the named subcircuits. When you use the -subckt argument, the -model and -inst arguments become local to the named subcircuits.

Argument	Description
<code>-instparam inst_expr</code>	As an expression of any instance parameter, this argument further scopes to devices whose instance parameter value causes the expression to evaluate to logical true (1). Note that this argument only provides additional scoping to the <code>cck_soa</code> command. To conform to Tcl conventions, its expression should be enclosed within double quotes or braces if it contains any white space. This expression is defined as a single group for the Tcl parser. This means that it should not contain white space or be grouped with the Tcl grouping operators, double quotes (""), or braces ({}). Note: The <code>-instparam</code> argument can support any instance parameter. It is not limited to <code>w</code> and <code>l</code> parameters.
<code>-except_inst inst_pattern</code> <code>{inst_pattern}</code>	Excludes instances from the <code>cck_soa</code> command scope. Any instance whose hierarchical instance name matches the pattern is excluded from the scope. Wildcards are supported.
<code>-except_subckt</code> <code>sub_name[.inst_pattern]</code> <code>{sub_name}[.inst_pattern]</code>	Excludes all instances of the named subcircuits from the <code>cck_soa</code> command scope. Any subcircuit whose name matches the pattern is excluded from the scope. You can also exclude only the named instances of the subcircuit. Wildcards are not supported in subcircuit names, but are supported in the <code>inst_pattern</code>. Use the following syntax to exclude multiple subcircuits in the same <code>cck_soa</code> command statement: <pre>cck_soa ... -except_subckt sub_a sub_b sub_c</pre> Where <code>sub_a</code>, <code>sub_c</code>, and <code>sub_c</code> are the explicit subcircuit names to be excluded.
<code>-numvd num_val</code>	Limits the number of violations per device to report. The default is 1. To print all violations, set <code>num_val</code> to <code>all</code>. To turn off violation reporting, set it to 0.

Argument	Description
<pre>-check real_expr -min min_value -max max_value [-tth tval] logic_expr [-logic safe violation] [-tth tval] if-then-else-stmt</pre>	You must provide a constraint expression which can be a logic expression, a real-value expression or an if-then-else statement. - A real value expression can also be an arithmetic expression. If the expression contains a logical operator, it is considered a logic expression. - Any logic expression that evaluates to logical false (0) for a period longer than the <i>tval</i> is reported as a violation. You can change it with the <i>-logic</i> option. By default, the <i>-logic</i> option is set to <i>safe</i>. For information about the <i>safe</i> and <i>violation</i> keywords, see Table 28. - An if-then-else statement that allows you to switch the check expressions dynamically to check either waveforms or geometrical data. A constraint that is a real-value expression is reported as a violation if it is outside of the safe operating range specified by <i>-min</i> and <i>-max</i> arguments. To conform to Tcl conventions, enclose the constraint expression in double quotes or braces if it contains any white space. This expression is defined as a single group for the Tcl parser. This means that it should not contain white space or be grouped with the Tcl grouping operators, double quotes (""), or braces ({}). Example 1: <pre>cck_soa -label example_1 ... -check "if vgs > 2 then check (id, 0.1u, 1u) endif" ...</pre> Example 2: <pre>cck_soa -label example_2 ... -check "if vgs > 2 then check (vds > 1.5 && vg < 0.5) endif" ...</pre> Example 3: <pre>cck_soa -label example_3 ... -check "if vgs > 2 then if vd < 0 then check (id, 0.1u, 1u) endif else if vd > 0 then check (id, 0.2u, 2u) endif endif"</pre> Note: The <i>-check</i> expression and the <i>-min</i> and <i>-max</i> arguments all support local instance parameters.
<pre>-twindow {tstart tstop} tstart tstop</pre>	If you specify this argument, the tool performs the <i>cck_soa</i> check within the time window defined by <i>tstart tstop</i> {<i>tstart tstop</i>}. The <i>tstart</i> and <i>tstop</i> must come in pairs, except for the final window. If <i>tstop</i> is not specified, it is assumed to be the end of the simulation. The final <i>tstop</i> can be the <i>end</i> or <i>END</i> keyword. For example: <pre>-twindow 10n 20n 50n.</pre>

Argument	Description
<code>-tstep tstep_val</code>	When specified, the <code>cck_soa</code> check is performed at discrete time points, which are integer multiples of <code>tstep_val</code>. You can use this argument with the <code>-twindow</code> argument. The <code>-tstep</code> argument must be used with the <code>-twindow</code> argument,
<code>-flush auto auto2</code>	When set to <code>auto</code>, the tool reports violations according to all of the following criteria: • Reports violations as they occur. • Reports violations if the violation duration satisfies the time period specified by the <code>cck_soa</code> command constraint expression and the violation state change. When set to <code>auto2</code>, the tool reports violations according to all of the following criteria: • Reports violations as they occur. • Reports violations if the violation duration satisfies the time period specified by the <code>cck_soa</code> command constraint expression.
<code>-flush_merge enable_value</code>	When set to 1, and the <code>-flush</code> option is set to <code>auto</code> or <code>auto2</code>, a complete main <code>.ccksoa</code> file is generated at the end of the simulation. This file is the same as the one when <code>-flush auto auto2</code> is not used. The <code>-flush_merge</code> argument ensures that all violations are listed in the final main file. If <code>-flush auto auto2</code> is used and <code>-flush_merge 1</code> is not used, the violations are not merged in the final main file. The results are in multiple files, which is the default behavior in the tool. In the case of the following commands: <pre>cck_soa ... -flush auto auto2 -flush_merge 1 cck_soa ... -flush auto auto2</pre> The results of the second <code>cck_soa</code> command are not written to the main file because the <code>-flush_merge 1</code> argument is not set.
<code>-report report_type {report_type}</code>	Specifies which information to report for each device. The supported values are: • <code>wl</code>: Reports length and width information. • <code>conn</code>: Reports connectivity information. • <code>subinfo</code>: Reports the subcircuit hierarchy information. • <code>operand</code>: Reports the atomic values of the logical expression. For example, when you specify: <code>Vdb>3&&Vgs>0.5</code>, <code>operand</code> reports the peak values of <code>Vdb</code> and <code>Vgs</code> at the period of the violation. • <code>variation</code>: Reports the following information: ◦ The minimum and maximum arguments in the header section ◦ The variation line in the device or data section of the output report in JSON or CCK format, respectively.
<code>-numd num_value</code>	Limits the total number of violated devices to report. The default is 300. To print all violated devices, set the option to <code>all</code>. To turn off violation reporting, set the option to 0.

Argument	Description
<code>-filteralert 0 1</code>	Issues the <code>zero scoping error</code> message if there are no devices that match with the user-specified scoping rule. This is caused by an unintended typo or the exclusion of the matched device by the <code>-except_inst</code> or the <code>-except_subckt</code> expression in the same <code>cck_soa</code> statement. By default, the <code>-filterAlert</code> option is set to 1, meaning that such a scenario is considered an error. The tool displays the <code>zero scoping error</code> message and terminates the simulation. If <code>-filterAlert</code> to 0, only the <code>zero scoping error</code> message is displayed and the simulation is not terminated.
<code>-portalert enable_value</code>	Enables a warning or error if the Verilog-A port that is specified in the constraint expression cannot be matched. By default, it is set to 1 to issue an error message and terminate the simulation. To issue a warning message for the statement and continue the simulation, set it to 0.
<code>-limit value</code>	Specifies the hierarchy level down to which the scope is applied. When you specify <code>-subckt</code>, the <code>-limit</code> option is relative to where the subcircuit is located in the hierarchy. A value of 0 specifies the top level of the subcircuit. The default is for all levels. This option is only effective when scoping includes wildcard characters.
<code>-notation scientific</code>	Specifies the <code>-notation</code> option as <code>scientific</code> to output the voltage and current values in exponential format. Note: If you specify any other value other than <code>scientific</code>, the simulation results in an error and is terminated.
<code>-intr interactive_mode</code>	Specifies the <code>-intr</code> option as <code>true yes 1</code> to interrupt the simulation when a violation is found. Note: The interactive mode is triggered only at the beginning of a violation. Not every violated data point triggers the interactive mode. The default is <code>false no 0</code>.

Argument	Description
<code>-avg 0 1</code>	When the <code>-avg</code> option is set to 1, the tool calculates violations by dividing "total violation time" by "total transient time" in percentage. The default is 0. Note: When <code>-check</code> option is a numeric number (e.g., Vgs), the calculated result is the average value of the <code>-check</code> option during the whole simulation time. When the <code>-check</code> option is logical (e.g., Vgs>3), the calculated result is the percentage of the violation period over the whole simulation time. Example: <pre>cck_soa -label testavg -model n \ -check "vbd>-1.00 && vbd<-0.4" -avg 1</pre> The .ccksoa report shows the following: <pre>Constraint=(vbd>-1.00 && vbd<-0.4, *, *) avg=1 * Dev #1: x1.mn1 Attributes: model=n, average=0.584%</pre>
<code>-integ 0 1</code>	When the <code>-integ</code> option is set to 1, the tool calculates violations as the total violation time. The default is 0. Example: <pre>cck_soa -label testinteg -model n \ -check "vbd>-1.00 && vbd<-0.4" -integ 1</pre> The .ccksoa report shows the following: <pre>Constraint=(vbd>-1.00 && vbd<-0.4, *, *) integ=1 * Dev #1: x1.mn1 Attributes: model=n, integral=1.867ns</pre>

Argument	Description
<code>-time_factor signal_name</code>	Specifies the weighting factor to calculate a duty cycle. This option can only be used together with the <code>-avg</code> or <code>-integ</code> option. Example: <pre>cck_soa -label testifx -model n -check \ "vbd>-1.00 && vbd<-0.4" -integ 1 \ -time_factor test</pre> In the above example, • Transient time is 10us • Logical expression is true between 1.2u and 1.4u, and between 5u and 5.5u • Total duration of (expression=true) is: 0.2u+0.5u=0.7u • Real life time factor is 1 between 0 and 5u, and 2 between 5u and 10u • Duration of violation is: 0.2u*1+0.5u*2=1.2u • Total duration is: 5u*1+5u*2=15u • Duty cycle is: 1.2u/15u=0.08 The result is the total duration of the violation weighted by the 'test' signal. The .ccksoa report contains the following information: <pre>Constraint=(vbd>-1.00 && \ vbd<-0.4, *, *) integ=1 \ time_factor=test * Dev #1: x1.mn1 Attributes: model=n, weighted integral=0.924ns</pre>
<code>-probe enable_value</code>	When enabled, the tool writes terminal voltage waveforms for each of the violated devices, in the same format as defined by the <code>.probe</code> or <code>.plot</code> commands. The default waveform format is FSDB. To write output waveforms in a different format, use the set_waveform_option command. For example, to write output waveforms in the WDF format, use the <code>set_waveform_option -format wdf</code> command. Waveforms are saved to an output file named <i>prefix.cck_soa.label</i>. For example, if the output format is FSDB, the output file will be named <i>prefix.cck_soa.label.fsdb</i>.

Argument	Description
<code>-probe_hier n</code>	Writes node voltage waveforms for the violating devices, including subcircuits in the higher level of hierarchy in the design. The supported value can be any integer greater than zero. The default is 1. When <code>-probe_hier 1</code> is set, all nodes under the subcircuit that contains the violating device will be probed. Taking a violating device <code>x1.x2.mn</code> as an example, • Setting <code>-probe_hier 1</code> writes voltage waveforms of <code>x1.x2.*</code> (as well as <code>x1.x2.*.*</code>). • Setting <code>-probe_hier 2</code> writes waveforms for every single nodes under the <code>x0.x1</code> up to two levels up from the device <code>mn</code>. That is, the waveform of the following nodes would be written: ◦ <code>x0.x1.net01</code> (two-level-up parent subcircuit) ◦ <code>x0.x1.x2.net01</code> (one-level-up parent subcircuit) ◦ <code>x0.x1.x2.x3.net01</code> (sibling subcircuit) ◦ <code>x0.x1.x2_1.net01</code> (sibling of 1-level-up subcircuit) Waveforms are saved to an output file named <i>prefix.cck_soa.label</i>. For example, if the output format is FSDB, the output file will be named <i>prefix.cck_soa.label.fsdb</i>.
<code>-time_prec <prec><unit></code>	Controls precision and data unit when reporting violation time. • <code>prec</code>: Specifies the number of digits after the decimal. The default is 4. • <code>unit</code>: Specifies the unit for the violation time. When not specified, the tool automatically assigns a unit based on the value scale. The supported units are: ◦ <code>x</code>: Prints violation time data with no unit. ◦ <code>m</code>: Prints violation time data in milli unit. ◦ <code>u</code>: Prints violation time data in micro unit. ◦ <code>n</code>: Prints violation time data in nano unit. ◦ <code>p</code>: Prints violation time data in pico unit. ◦ <code>f</code>: Prints violation time data in fento unit. Example: The following command reports violation time for violated devices, in nanosecond with 6 digits after the decimal. <pre>cck_soa -time_prec 6n ...</pre>

Argument	Description
<code>-voltage_prec <prec>< unit></code>	Controls precision and data unit when reporting voltage data. • <code>prec</code>: Specifies the number of digits after the decimal. The default is 3. • <code>unit</code>: Specifies the unit for the voltage data. When not specified, the tool automatically assigns a unit based on the value scale. The supported units are: ◦ <code>x</code>: Prints voltage values with no unit. ◦ <code>m</code>: Prints voltage values in milli unit. ◦ <code>u</code>: Prints voltage values in micro unit. ◦ <code>n</code>: Prints voltage values in nano unit. ◦ <code>p</code>: Prints voltage values in pico unit. ◦ <code>f</code>: Prints voltage values in fento unit. Example: The following command reports voltage values for violated devices, with 6 digits after the decimal without any unit. <pre>cck_soa -voltage_prec 6x ...</pre>
<code>-current_prec <prec>< unit></code>	Prints current data and specifies the following: • <code>prec</code>: Specifies the number of digits after the decimal. The default is 3. • <code>unit</code>: Specifies the unit for the current data. When not specified, the tool automatically assigns a unit based on the value scale. The supported units are: ◦ <code>x</code>: Prints current values with no unit. ◦ <code>m</code>: Prints current values in milli unit. ◦ <code>u</code>: Prints current values in micro unit. ◦ <code>n</code>: Prints current values in nano unit. ◦ <code>p</code>: Prints current values in pico unit. ◦ <code>f</code>: Prints current values in fento unit.
<code>-numeric_prec <prec>< unit></code>	Controls precision and data unit when reporting numeric data. • <code>prec</code>: Specifies the number of digits after the decimal. The default is 3. • <code>unit</code>: Specifies the unit for the numeric data. When not specified, the tool automatically assigns a unit based on the value scale. The supported units are: ◦ <code>x</code>: Prints numeric data with no unit. ◦ <code>m</code>: Prints numeric data in milli unit. ◦ <code>u</code>: Prints numeric data in micro unit. ◦ <code>n</code>: Prints numeric data in nano unit. ◦ <code>p</code>: Prints numeric data in pico unit. ◦ <code>f</code>: Prints numeric data in fento unit.

Description

This command checks primitive devices for safe operating area conditions. Violations are reported to an output file, named `*.ccksoa`.

The `cck_soa` command checks the following elements:

- MOSFETs
- BJTs
- JFETs
- Diodes
- Resistors and capacitors
- Verilog-A modules

The constraint expression is a required argument that specifies the check to be performed. You can construct the expression with a single or group of constraint functions (listed in [Table 31](#)) with either arithmetic operators (+, -, *, /) or logical operators (>, <, >=, <=, &&, ||, ==, !=). It can be a real-value constraint or a logical constraint.

A real-value constraint expression only involves arithmetic operators. The expression return value is a normal decimal real value. A violation is reported if the normal decimal real value is outside a safe operating area specified by `-min` and `-max` arguments. However, a logical constraint expression involves logical operators. The expression evaluation return is either logical true (1) or false (0). A logical expression that evaluates to logical false (0) is reported as a violation.

[Table 31](#) lists the keywords that are used for accessing common instance conditions.

Table 31 *Constraint Functions*

Constraint Function	Description
I	Current of a MOSFET, JFET, BJT, DIODE, RESISTOR or CAPACITOR
Ib	Bulk current of a MOSFET or base current of a BJT
Ic	Collector current of a BJT
Id	Drain current of MOSFET or BJT
Ie	Emitter current of a BJT
Ig	Gate current of a MOSFET or JFET
Is	Source current of a MOSFET, BJT, or JFET
Vb	Bulk voltage of a MOSFET or base voltage of a BJT
Vbc	Base/collector voltage difference of a BJT ($V_b - V_c$)
Vcb	Collector/base voltage difference of a BJT ($V_c - V_b$)

Table 31 Constraint Functions (Continued)

Constraint Function	Description
Vbd	Bulk/drain voltage difference of a MOSFET or JFET ($V_b - V_d$)
Vdb	Drain/bulk voltage difference of a MOSFET or JFET ($V_d - V_b$)
Vbe	Base/emitter voltage difference of a BJT ($V_b - V_e$)
Veb	Emitter/base voltage difference of a BJT ($V_e - V_b$)
Vbs	Bulk/source voltage difference of a MOSFET or base/source voltage difference of a BJT ($V_b - V_s$)
Vsb	Source/bulk voltage difference of a MOSFET or source/base voltage difference of a BJT ($V_s - V_b$)
Vc	Collector voltage of a BJT
Vce	Collector/emitter voltage difference of a BJT ($V_c - V_e$)
Vec	Emitter/collector voltage difference of a BJT ($V_e - V_c$)
Vcs	Collector/source voltage difference of a BJT ($V_c - V_s$)
Vsc	Source/collector voltage difference of a BJT ($V_s - V_c$)
Vd	Drain voltage of a MOSFET or JFET
Vdip	Bias voltage of DIODE, RESISTOR, or CAPACITOR
Vds	Drain/source voltage difference of a MOSFET or JFET ($V_d - V_s$)
Vsd	Source/drain voltage difference of a MOSFET or JFET ($V_s - V_d$)
Ve	Emitter voltage of a BJT
Ves	Emitter/source voltage difference of a BJT ($V_e - V_s$)
Vse	Source/emitter voltage difference of a BJT ($V_s - V_e$)
Vse	Source/emitter voltage difference of a BJT ($V_s - V_e$)
Vg	Gate voltage of a MOSFET or JFET
Vgb	Gate/bulk voltage difference of a MOSFET or JFET ($V_g - V_b$)
Vbg	Bulk/gate voltage difference of a MOSFET or JFET ($V_b - V_g$)
Vgd	Gate/drain voltage difference of a MOSFET or JFET ($V_g - V_d$)
Vdg	Drain/gate voltage difference of a MOSFET or JFET ($V_d - V_g$)

Table 31 *Constraint Functions (Continued)*

Constraint Function	Description
Vgs	Gate/source voltage difference of a MOSFET or JFET ($V_g - V_s$)
Vsg	Gate/source voltage difference of a MOSFET or JFET ($V_s - V_g$)
Vneg	Cathode voltage of a DIODE or low-voltage terminal voltage of RESISTOR or CAPACITOR
Vpos	Anode voltage of a DIODE or high-voltage terminal voltage of RESISTOR or CAPACITOR
Vs	Source voltage of a MOSFET, JFET, or BJT
W	Width of a MOSFET
L	Length of a MOSFET
V(port)	Port voltage of Verilog-A Module

[Table 32](#) lists the mathematical functions that are supported to construct the constraint expressions.

Table 32 *Mathematical Functions*

Built-In Function	Description
abs(x)	Returns the absolute value of x: $ x $
ceil(x)	Returns the integer that is greater than or equal to the value of x
exp(x)	Returns e, raised to the power of the value of x: e^x
floor(x)	Returns the integer that is less than or equal to the value of x
log(x)	Returns the natural logarithm of the absolute value of x with the sign of x: $(\text{sign of } x)\log(x)$
log10(x)	Returns the base 10 logarithm of the absolute value of x with the sign of x: $(\text{sign of } x)\log_{10}(x)$
pow(x,y)	Returns the value of x raised to the power of the value of y
sqrt(x)	Returns the square root of the absolute value of x: $\text{sqrt}(-x) = -\text{sqrt}(x)$

To conform to the Tcl convention, enclose the constraint expression in double quotes or braces if it contains any white space.

You can use `cck_soa` to scope to specific instances using the `-model`, `-inst`, `-subckt`, `-except_inst`, `-except_subckt`, and `-instparam` arguments. You can use wildcards only with the `-inst` and `except_inst` arguments. Wildcard characters are not supported for the `-model`, `-subckt`, and `-except_subckt` arguments. If you use the `-subckt` argument with the `-inst` argument or the `-model` argument, the named instances or named models are local to the specified subcircuits.

The rule between `-except_subckt` and `-except_inst` arguments accumulates in a logical OR. If you use the `-except_subckt` argument with `-except_inst`, the checks exclude all instances inside the named subcircuit by the `-except_subckt` argument and all the named instances by `-except_inst`.

You can specify resistors, capacitors, and diodes for the `-model` argument, by using the reserved keywords `all_resistors`, `all_capacitors` and `all_diodes`, respectively. The tool checks the specified resistors, capacitors and diodes when:

- All resistors are checked if the `-model all_resistors` argument is specified.
- All capacitors are checked if the `-model all_capacitors` argument is specified.
- All diodes are checked if `-model all_diodes` argument is specified.

The names of the resistors, capacitors, and diodes models can also be specified in the in the `-model` argument if only certain models are to be checked.

The Verilog-A module also is supported by specifying the names of the Verilog-A module in the `-model` argument. It uses `v (portName)` to monitor the port voltage of the Verilog-A module in the `-check` argument.

If you specify the `-instparam` argument, you can construct the instance parameter expression with a single or a group of instance parameter functions with logical operators (`>`, `<`, `>=`, `<=`, `&&`, `||`, `==`, and `!=`). These two instance parameter functions can be used to create an instance parameter expression.

Table 33 Instance Parameters

Instance Parameter Function	Description
L	Length of a MOS device
W	Width of a MOS device

When you specify the `-flush` argument, on-the-fly auto-reporting is enabled. Each `cck_soa` command for which auto-flushing is enabled writes its output to a unique file. These files are named as `prefix.ccksoa.label`, where `prefix` is the input netlist file name `prefix`, or the name specified with `-o` on the command line. The label is the command label specified by `-label` argument of that command. Each file is updated as violations are detected.

When you specify the `-report` argument, it triggers additional information to be reported in the output file. The `cck_soa` command supports reporting the width and length and connectivity information for each device that violates the check. You can enable the `-report wl` argument and connectivity information by using the `-report conn` command. The following example shows a sample report:

```
Dev #1: N1
Attributes: model=NMOS_test w=10u l=0.5u
Connectivity:
  D: net052
  G: net17
  S: 0
  G: 0
```

Examples

Example 1:

The following example checks for the voltage difference between drain and source of MOSFET and JFET. A violation is reported if the voltage difference is out of safe operating range of 0.3 V – 0.5 V.

```
cck_soa -label vdschk -check vds -min 0.3 -max 0.5
```

Example 2:

The following example shows a logical expression. If `vds > 0.3` and `vds < 0.5`, and `vgs > 0` and `vgs < 0.5`, the return value is logical true (1), and no violation is reported. Otherwise a violation is reported.

```
cck_soa -label logcheck -check "vds>0.3 && vds<0.5 && vgs>0 && vgs<0.5"
```

Example 3:

The following example shows a logical violation. If `vgs < 0`, the returned value is logical true (1), and a violation is reported.

```
cck_soa -label vgscheck -check "vgs<0" -logic violation
```

Example 4:

The following example applies the check to all instances and devices of `nchmos_mac` subcircuit.

```
cck_soa -label subckt_only -subckt nchmos_mac -check vgs -min 0 -max 3 ...
```

Example 5:

The following example applies the check only to the x1 instance of the mysub subcircuit. All other instances are excluded from the check.

```
cck_soa -label subinst -subckt mysub -inst x1 -check vgs -min 0 -max 3 ...  
.subckt mysub port1 port2 port3  
  
X1 port1 port2 somesub  
X2 port3 port4 somesub  
...  
.ends
```

Example 6:

The following example checks all MOS devices which are named nch and have length less than 0.1um.

```
cck_soa -label chk_model_length -model nch -instparam { l < 0.1u } -check  
vgs -min -1.0 -max 1.0
```

Example 7:

In the following example, you can exclude instances from scoping using the `-except_inst` and `-except_subckt` arguments. When the `-except_inst` argument and `-except_subckt` argument are used in the same command, the checks exclude the instances specified by the `-except_inst` argument and all the instances inside the subcircuit specified by `-except_subckt`. This example applies the check to all the devices with model name nch, except for those instances of x1.x2 instance and all the instances inside the dummy_cell subcircuit.

```
cck_soa -label except -model nch \  
-except_inst x1.x2.* -except_subckt dummy_cell -check "vgs < 0.6"
```

Example 8:

The following example applies the check to all resistor devices.

```
cck_soa -label res_model -model all_resistors -check { vpos-vneg > 2 }
```

Example 9:

The following example applies the check to Verilog-A module named mos_va.

```
cck_soa -label chk_model -model mos_va -check { v(port1) > 10 }
```

Example 10:

The following example shows a real-value check.

```
cck_soa -label simple_check -model nch -check vgs -min 0 -max 0.6
```

Example 11:

The following example shows a logic check.

```
cck_soa -label logic_check -model nch -check {VGS > 0 && VGS < 0.6}
cck_soa logic_check2 -model nch \
  -check "(VGS>0) && (VGS<0.6) && (ID<200n)"
```

Example 12:

The following example shows a real-value check with time window.

```
cck_soa -label simple_check_3n -model nch -check vgs \
  -min 0 -max 0.6 -tth 3n \
cck_soa windows -model nch -check vgs -min 0 -max 0.6 \
  -twindow 1u 2u 3u 4u
```

Example 13:

The following example shows the discrete points check.

```
cck_soa -label step -model nch -check vgs -min 0 -max 0.6 -tstep 500n
cck_soa window_step -model nch -check vgs -min 0 -max 0.6 \
  -twindow 500n 5u -tstep 200n
```

Example 14:

The following example outputs violations in exponential format.

```
cck_soa -label "normal" -model PMOS_test -inst * -check id -min 0 -max \
  10u -numvd all -notation scientific
```

Example 15:

In the following example, the interactive mode is triggered whenever a MOSFET's `vg` crosses the 3.3v threshold.

```
cck_soa -label test_intr -check vg -max 3.3 -intr true
```

Example 16:

The following command reports this message in the `.ccksoa` file header:

```
cck_soa -label test2 -subckt mp4s -check "Vg < 0.8" -flush auto \
  -message "Check vg"
```

```
* CCK Command: cck_soa
*
* Tag: test2
* Scopes:
*   subckt=mp4s
* Constraint=Vg < 0.8
* message="Check vg"
* flush=auto
```

Example 17:

The following example merges all the results into a main file at the end of the simulation and generates a `.csv` main file.

```
cck_soa -label test3 -subckt s9mp4s -check "(Vg-Vb) < 0.4" -flush auto \
-flush_merge 1
cck_soa -label test4 -subckt s9mp4s -check "(Vg-Vb) < 0.7" -flush auto \
-flush_merge 1
```

The following files are generated:

```
xa.ccksoa_test3
xa.ccksoa_test3.csv
xa.ccksoa_test4
xa.ccksoa_test4.csv
xa.ccksoa
xa.ccksoa.csv
```

The `xa.ccksoa` and `xa.ccksoa.csv` files are the main files that contain all violations.

Example 18:

The following example merges all the results into a main file at the end of the simulation and generates the `.csv` main file.

```
cck_soa -label test3 -subckt s9mp4s -check "(Vg-Vb) < 0.4" -flush auto \
-flush_merge 1
cck_soa -label test4 -subckt s9mp4s -check "(Vg-Vb) < 0.7"
```

The following files are generated:

```
xa.ccksoa_test3
xa.ccksoa_test3.csv
xa.ccksoa
xa.ccksoa.csv
```

The `xa.ccksoa` and `xa.ccksoa.csv` files are the main files that contain all violations (for test3 and test4).

Example 19:

The following example merges the test2 and test3 results into a main file at the end of the simulation and generates the `.csv` main file. However the test4 results are not merged to the main file.

```
cck_soa -label test2 -subckt s9mp4s -check "(Vg-Vb) < 1.8" -flush auto \
-flush_merge 1
cck_soa -label test3 -subckt s9mp4s -check "(Vg-Vb) < 0.4" -flush auto \
-flush_merge 1
cck_soa -label test4 -subckt s9mp4s -check "(Vg-Vb) < 0.7" -flush auto
```

The following files are generated:

```
xa.ccksoa_test2
xa.ccksoa_test2.csv
xa.ccksoa_test3
xa.ccksoa_test3.csv
xa.ccksoa_test4
xa.ccksoa_test4.csv
xa.ccksoa
xa.ccksoa.csv
```

The xa.ccksoa and xa.ccksoa.csv files are the main files that contain all violations for test2 and test3 (not test4).

Example 20:

The following example reports the minimum argument in the header section and the variation line in the device section in the output JSON report.

```
cck_soa -label diode -model ALL_diodes -check vpos -min 5 -numvd 10
-report variation
```

Following is a snippet of the output report in the JSON format:

```
"Header": {
  "CCK Command": "cck_soa",
  "Tag": "diode",
  "Scopes": [
    "model=ALL_DIODES "
  ],
  "Arguments": {
    "Constraint": "(vpos, 5, *)",
    "min": "5",
    "numvd": "10",
    "report": "variation"
  }
},
"Devices": [
  {
    "Seq": 1,
    "Name": "dd",
    "Attributes": {
      "model": "d"
    },
    "Data": {
      "type": "dyn",
      "content": {
        "Violations": [
          {
            "Label": "Constraint: (vpos, 5, *)",
            "Durations": [
              {
                "Seq": 1,
```

```

"Start": "0",
"Stop": "6.25n",
"Span": "6.25n",
"Points": {
  "Start": {
    "expr": "0V",
    "t": "0s"
  },
  "End": {
    "expr": "5V",
    "t": "6.25ns"
  },
  "Min(expr)": {
    "expr": "0V",
    "t": "0s"
  },
  "Max(expr)": {
    "expr": "5V",
    "t": "6.25ns"
  },
  "Variation(expr)": {
    "expr": "5V"
  }
},

```

```

... ..
... ..

```

Example 21:

The following example reports the minimum argument in the header section and the variation line in the data section of the output CCK report.

```

cck_soa -label two_term -except_inst v* -check vpos -min 5 -numvd 10
-report variation

```

Following is a snippet of the output file in the CCK format:

```

* CCK Command: cck_soa
*
* Tag: diode
* Scopes:
*   model=ALL_DIODES
* Constraint=(vpos, 5, *) min=5 numvd=10
* report=variation
*
Dev #1: dd
Attributes: model=d
Data:
  Constraint: (vpos, 5, *)
  Duration: seq=1, t1=0s, t2=6.25ns, span=6.25ns
  Start: expr=0V, t=0s

```



```
End: expr=5V, t=6.25ns
Min(expr): expr=0V, t=0s
Max(expr): expr=5V, t=6.25ns
Variation(expr): expr=5V
...
...
```

cck_substrate

Checks for a forward-bias bulk condition on MOS devices.

Syntax

```
cck_substrate -label lname
               [scope]
               [-numv num]
               [twindow]
               [-tstep tstep_val]
               [-vth value]
               [-ith value]
               [-tth value]
               [-flush auto]
               [report]
               [-intr interactive_mode]

scope          ::= [model] [inst] [subckt] [except_i] [except_s]
model          ::= -model model_name {model_name}
inst           ::= -inst inst_pattern {inst_pattern}
subckt         ::= -subckt subckt_name {subckt_name}
except_i       ::= -except inst inst_pattern {inst_pattern}
except_s       ::= -except subckt subckt_name [.inst_pattern]
               {subckt_name [.inst_pattern]}
twindow        ::= [-twindow {tstart tstop} tstart [tstop]]
report         ::= -report report_type {report_type}
report_type    ::= wl|conn
interactive_mode ::= 0|1|yes|no|true|false
```

Arguments

Argument	Description
-label label_name	Specifies the label name that appears in the report file, which makes it easier to search the report. If on-the-fly reporting is enabled, the label forms part of the report file name.
-model model_name {model_name}	Scopes to devices of the named models.

Argument	Description
<code>-inst inst_pattern</code> <code>{inst_pattern}</code>	Scopes to the named instances. You can use wildcards in instance names as specified by the PrimeSim XA <code>set_wildcard_rule</code> command. Wildcard behavior is determined by the <code>set_wildcard_rule</code> command.
<code>-subckt subckt_name</code> <code>{subckt_name}</code>	Scopes to instances of the named subcircuits. When you use the <code>-subckt</code> argument, the <code>-model</code> and <code>-inst</code> arguments become local to the named subcircuits.
<code>-except_inst inst_name</code> <code>{sub_name}</code>	Excludes all instances of the named subcircuits from the <code>cck_substrate</code> command scope. Any subcircuit whose name matches the pattern is excluded from the command scope. Wildcards are not supported.
<code>-except_subckt</code> <code>sub_name[.inst_pattern]</code> <code>{sub_name}</code> <code>[.inst_pattern]</code>	Excludes all instances of the named subcircuits from the <code>cck_substrate</code> command scope. Any subcircuit whose name matches the pattern is excluded from the scope. You can also exclude only the named instances of the subcircuit. Wildcards are not supported in subcircuit names, but are supported in the <code>inst_pattern</code> .
<code>-numv num_val</code>	Limits the total number of violations reported. To print all violations, specify the <code>all</code> keyword. To turn off violation report checking, specify <code>0</code> . The default is <code>300</code> .
<code>-twindow {tstart tstop}</code> <code>tstart tstop</code>	If you specify this argument, PrimeSim XA performs the <code>cck_substrate</code> check within the time window defined by <code>tstart tstop {tstart tstop}</code> . The <code>tstart</code> and <code>tstop</code> must come in pairs, except for the final window where: if <code>tstop</code> is not specified, it is assumed to be the end of the simulation. The final <code>tstop</code> can also be the <code>end</code> or <code>END</code> keyword. <code>-tstep</code> must be used with <code>-twindow</code> , except when <code>-twindow</code> has no explicit stop time (for example, <code>-twindow 10n 20n 50n</code>).
<code>-tstep tstep_val</code>	If you specify <code>-tstep</code> this check is only performed at integer multiple of <code>tstep_val</code> . You can combine this setting with <code>-twindow</code> .
<code>-vth vth</code>	Specifies that the bulk must be forward biased by the specified amount before a violation is reported. The default is <code>0.5 V</code> .
<code>-ith ith</code>	Specifies that the bulk current must exceed the specified value for a violation to be reported. Note, if you want to apply the bulk current as a substrate violation criteria, it is recommended to set higher accuracy model level (such as model level 6 or 7) in order to obtain an accurate bulk current value. There is no default value for this control option. If this option is not specified, the command will disable the bulk current check.

Argument	Description
<code>-tth value</code>	Specifies the time duration threshold value to be reported. The violation must last longer than the specified time value.
<code>-flush auto</code>	Reports the violations that are reported after the completion of the simulation, if you do not specify this argument. To enable Auto-flushing, specify the <code>auto</code> keyword that specifies to report violations as they occur on-the-fly in the simulation.
<code>-report report_type</code> <code>{report_type}</code>	Enables additional information to be reported for each device. To enable the Width and length information, specify the <code>wl</code> keyword. To enable the Connectivity information, specify the <code>conn</code> keyword.
<code>-intr interactive_mode</code>	Specify the <code>-intr</code> option as <code>true yes 1</code> to interrupt the simulation when a violation is found. Note: The interactive mode is triggered only at the beginning of a violation. Not every violated data point triggers the interactive mode. The default value is <code>false no 0</code> .

Description

The `cck_substrate` command checks for a forward-bias bulk condition on MOS devices. A violation is reported when a MOSFET meets the following conditions:

$$\begin{cases} pmos, v_s - v_b > v_t \text{ or } v_d - v_b > v_t \\ nmos, v_b - v_s > v_t \text{ or } v_b - v_d > v_t \end{cases}$$

Violations are reported to an output file, `*.substrate`.

You can use the `cck_substrate` to scope to specific instances using the `-model`, `-inst`, `-subckt`, `-except_inst`, `-except_subckt`, and `-instparam` arguments. You can use wildcards only with the `-inst` and `-except_inst` arguments. Wildcard characters are not supported for the `-model`, `-subckt`, and `-except_subckt` arguments. If you use the `-subckt` argument with the `-inst` argument or the `-model` argument, the named instances or named models are local to the specified subcircuits.

The rule between `-except_subckt` and `-except_inst` arguments accumulates in a logical OR. If you use the `-except_subckt` argument is with `-except_inst`, the checks exclude all the instances inside the named subcircuit by the `-except_subckt` argument and all the named instances by the `-except_inst` argument.

When you specify the `-flush` argument, on-the-fly auto-reporting is enabled. Each `cck_substrate` command for which auto-flushing is enabled writes its output to a unique file. These files are named as `prefix.ccksubstrate.label`, where *prefix* is the input

netlist filename *prefix*, or the name specified with `-o` on the command line. The label is the command label specified by `-label` argument of that command. Each file is updated as violations are detected.

When you specify the `-report` argument, it triggers additional information to be reported in the output file. For each `cck_substrate` command in which `-report wl` is specified, width and length information is reported for each device that violates the check as shown by:

```
Dev #1: mp
```

```
Attributes: model=p w=1u l=0.5u
```

When you specify the `-report conn` argument, connectivity information is reported for each device as follows:

```
Dev #1: xml.ml
```

```
Attributes: type=PMOS
```

```
Connectivity:
```

```
D: 0
```

```
G: VDD
```

```
S: VDD
```

```
B: BODY
```

Examples

The following example applies the check to all instances and devices of `nchmos_mac` subcircuit.

```
cck_substrate subcktonly -subckt nchmos_mac ...
```

In the following example, when you use `-except_inst` and `-except_subckt` in the same command, the checks exclude the instances specified by `-except_inst` and all the instances inside the subcircuit specified by `-except_subckt`. This example applies the check to all the devices with model name `nch`, except for those instances of `x1.x2` and all the instances inside `dummy_cell` subcircuit.

```
cck_substrate except -model nch \
-except_inst x1.x2.* -except_subckt dummy_cell ...
```

The following example checks only MOS instances of the model `nch` for forward biased bulk conditions. The check is performed from `t = 0` to the end of the simulation.

```
cck_substrate -label nmos_bulk_chk -model nch
```

The following example checks all MOS instances for forward biased bulk conditions only during the time window 550– 600 ns.

```
cck_substrate -label bulk_check_550_600 -twindow 550n 600n
```

In the following example, the interactive mode is triggered whenever a MOSFET's `vg` crosses the 3.3v threshold.

```
cck_substrate -label test_intr -check vg -max 3.3 -intr true
```

cck_toggle_count

Checks toggling activity on the specified nodes.

Syntax

```
cck_toggle_count -label lname
                  [report_type]
                  [scope]
                  [twindow]
                  [-flush value%]
                  [-report_first_toggle value]
                  [-report_most_toggle value]
                  [-report_zero_toggle value|all]
                  [-report_multi_toggle_period num_toggle]
                  [-inst instance_name]
                  [-except_inst instance_name]

report_type ::= basic_report|detail_report
basic_report ::= -enable_detail 0|no|false|off
               [-vth value]
detail_report ::= -enable_detail 1|yes|true|on
               [-loth|-vol value]
               [-hith|-voh value]

scope ::= [node] [subckt] [except_n] [except_s]
node ::= -node node_pattern {node_pattern}
subckt ::= -subckt subckt_name {subckt_name}
except_n ::= -except_node node_pattern {node_pattern}
except_s ::= -except_subckt subckt_name [.node_pattern]
               {subckt_name [.node_pattern]} [-port value]
twindow ::= -twindow {tstart tstop} tstart [tstop]
```

Arguments

Argument	Description
-label label_name	Specifies the label name that appears in the report file, which makes it easier to search the report. If on-the-fly reporting is enabled, the label forms part of the report file name.

Argument	Description
<code>-enable_detail</code> <code>enable_value</code>	Enables the type of report to be generated. A basic report is generated by default, or when the argument is set to 0 (off). A detail report is generated when the argument is set to 1 (on).
<code>-vth vth</code>	Specifies the voltage reference if the basic report is enabled. The default is $(V_{max}+V_{min})*0.5$. V_{max} and V_{min} are different among various voltage domains.
<code>-loth -vol vlth</code>	Specifies the low voltage threshold if the detail report is enabled. The default follows the value specified in the <code>set_logic_threshold</code> command.
<code>-hith -voh vhth</code>	Specifies the high voltage threshold if the detail report is enabled. The default follows the value specified in <code>set_logic_threshold</code> command.
<code>-node node_pattern</code> <code>{node_pattern}</code>	Scopes to the named nodes. You can use wildcard characters in the node pattern. Wildcard behavior is determined by the <code>set_wildcard_rule</code> command.
<code>-subckt subckt_name</code> <code>{subckt_name}</code>	Scopes to nodes of the named subcircuits. When you specify <code>-subckt</code> , node arguments become local to the named subcircuits. Wildcard characters are not supported in subcircuit names.
<code>-except_node node_pattern</code> <code>{node_pattern}</code>	Excludes nodes from the command scope. Any node whose name matches the pattern is excluded from the command scope. You can use wildcard characters in the node patterns.
<code>-except_subckt</code> <code>sub_name[.node_pattern]</code> <code>{sub_name}[.node_pattern]</code>	Excludes all instances of the named subcircuits from the <code>cck_toggle_count</code> command scope. Any subcircuit whose name matches the pattern is excluded from the scope. You can also exclude only the named instances of the subcircuit. Wildcards are not supported in subcircuit names but are supported in the <code>node_pattern</code> .
<code>-twindow {tstart tstop}</code> <code>tstart [tstop]</code>	Limits checking to the specified time windows. The default for <code>tstart</code> is 0 and <code>tstop</code> is the end of transient simulation time.
<code>-flush value%</code>	Reports violations after the completion of the simulation if you do not specify this argument. Flushing is enabled by providing the frequency in a percentage value to report violations. This option is not supported in the mixed-signal simulation mode.
<code>-report first_toggle_time</code>	Reports the first switching/toggling time for a node.
<code>-report_most_toggle value</code>	Specifies the number of the most toggling nodes to be reported.

Argument	Description
<code>-report_zero_toggle value all</code>	Specifies the number of zero toggling node to be reported. To report all zero toggling nodes, specify the <code>all</code> keyword.
<code>-report_multi_toggle period [num_toggle]</code>	Reports nodes that toggle more than <code>num_toggle</code> times within a specified period.
<code>-inst instance_name</code>	Specifies to apply the check to an instanced-based scope. You can specify wildcard characters in instance names and multiple instance names,
<code>-except_inst instance_name</code>	Limits the check to the specified instance names. You can specify wildcard characters in instance names and multiple instance names,
<code>-port value</code>	Specify one of the following values. • 0 (default) specifies to check if the nodes are internal nodes and only report if the internal nodes. • 1 specifies to report all nodes (internal nodes and subcircuit ports). • 2 checks if the nodes are subcircuit ports and only reports that the nodes are subcircuit ports.

Description

You can use `cck_toggle_count` to scope to specific instances using the `-node`, `-subckt`, `-except_node`, and `-except_subckt` arguments. You can use wildcards only with the `-node` and `-except_node` arguments. Wildcard characters are not supported for the `-subckt` and `-except_subckt` arguments. If you use the `-subckt` argument with the `-node` argument, the named nodes are local to the specified subcircuits.

The rule between `-except_subckt` and `-except_node` arguments accumulates in a logical OR. If you use the `-except_subckt` argument with `-except_node`, the checks exclude all the nodes inside the named subcircuit by the `-except_subckt` argument and all the named nodes by the `-except_node` argument.

You can use the `cck_toggle_count` command to create a basic report or a detail report. A basic report uses the single voltage threshold to determine toggling. It contains only the number of toggles per node. A detail report uses two voltage thresholds to determine toggling and contains the following information:

- Rising toggle
- Falling toggle
- Average overshoot
- Average undershoot

- Rise time (max/min/avg)
- Fall time (max/min/avg)
- Probability high/low

The following three arguments (`-vth`, `-loth`, and `-hith`) are used to collect statistic data about toggling activities as follows:

- Most active toggling nodes
- Zero toggling nodes
- Nodes of specified number of toggles within a specified period

When you specify the `-flush` argument, on-the-fly auto reporting is enabled. Each `cck_toggle_count` command for which auto flushing is enabled writes its output to a unique file. These files are named as `prefix.ccktogle.label`, where *prefix* is the input netlist file name *prefix*, or the name specified with `-o` on the command line. The label is the command label specified by `-label` argument of that command. Each file is updated as violations are detected.

As shown in [Figure 13 on page 408](#), [Figure 14 on page 409](#), and [Figure 15 on page 409](#), the calculation of the toggle count depends on the number of thresholds. [Figure 13](#) illustrates that the basic report uses a single threshold:

Figure 13 Single Threshold

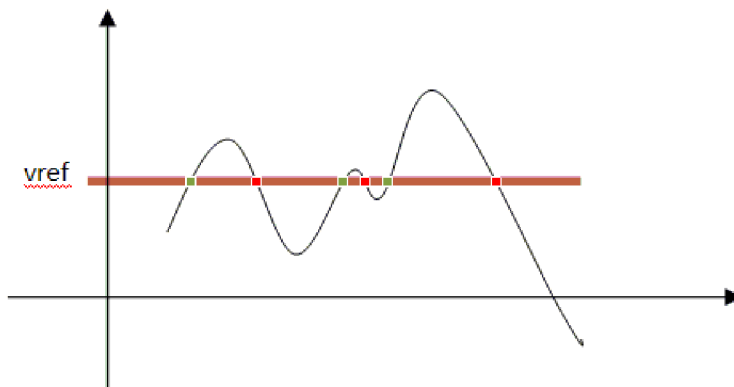
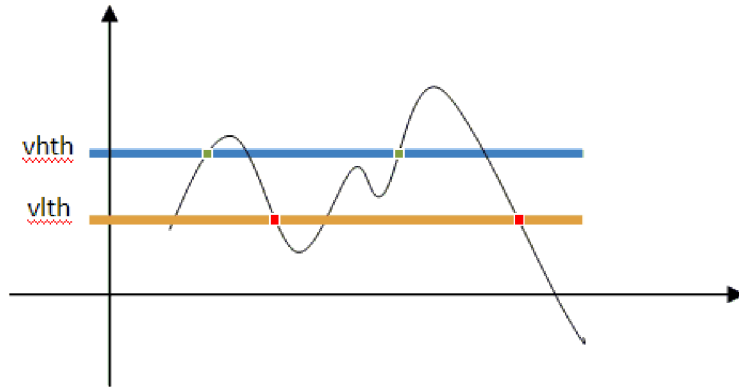


Figure 14 illustrates that the detail report uses two thresholds:

Figure 14 Two Thresholds



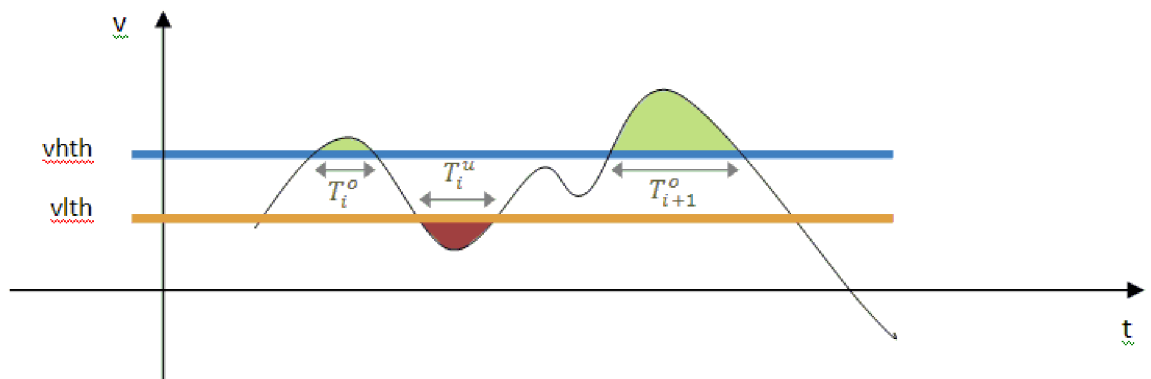
In Figure 15, the colored areas illustrate undershoots (red) and overshoots (green). The time average values are obtained from dividing the colored areas by time t .

Figure 15 Overshoot and Undershoot

Formula:

$$\text{Overshoot} = \frac{\sum_i \int_{\Delta T_i^o} v_o(t) dt}{T}, \text{ where } v_o(t) = v(t) - v_{hth}, t \in \Delta T_i^o \text{ s.t. } v(t) > v_{hth}$$

$$\text{Undershoot} = \frac{\sum_i \int_{\Delta T_i^u} v_u(t) dt}{T}, \text{ where } v_u(t) = v_{lth} - v(t), t \in \Delta T_i^u \text{ s.t. } v(t) < v_{lth}$$



The rise time is the time span of the green area; while the falling time is of the red area. The green areas can be defined by two time points when the voltage is rising, where:

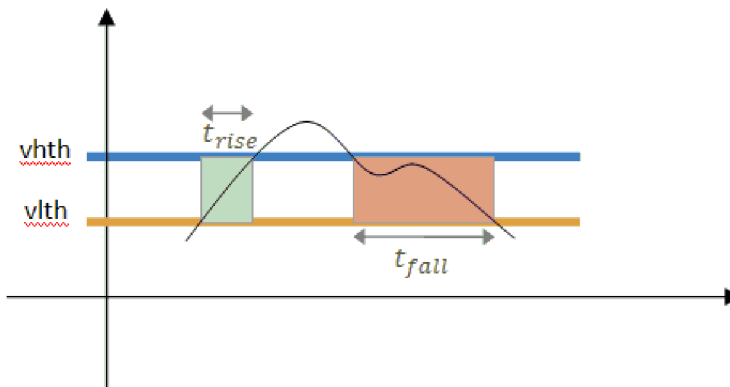
- t_1 is the first moment when $v(t) > vlth$
- t_2 is the first moment when $v(t) > vhth$ after t_1

The red areas are also defined by two time points when the voltage is falling, where:

- t_1 is the first moment when $v(t) < vhth$
- t_2 is the first moment when $v(t) < vlth$ after t_1

See [Figure 16](#).

Figure 16 Rising and Falling Times



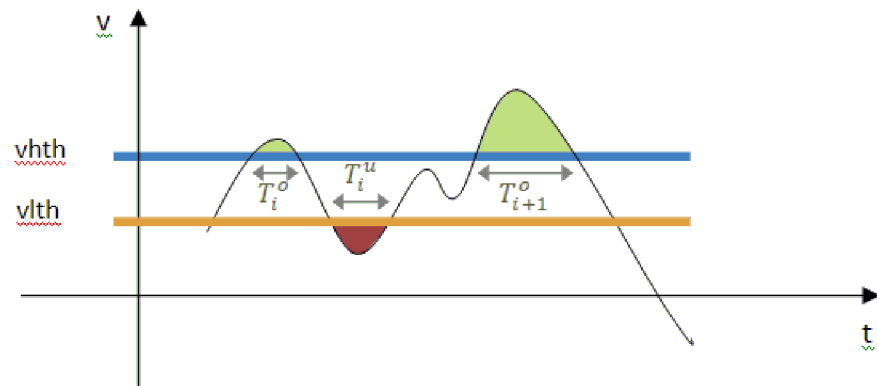
The probability of high is obtained by dividing the whole simulation time by the total time of the green areas. If the denominator is replaced by the total time of the red areas, then the probability of low is obtained. See [Figure 17](#).

Figure 17 Probability of High and Low

Formula:

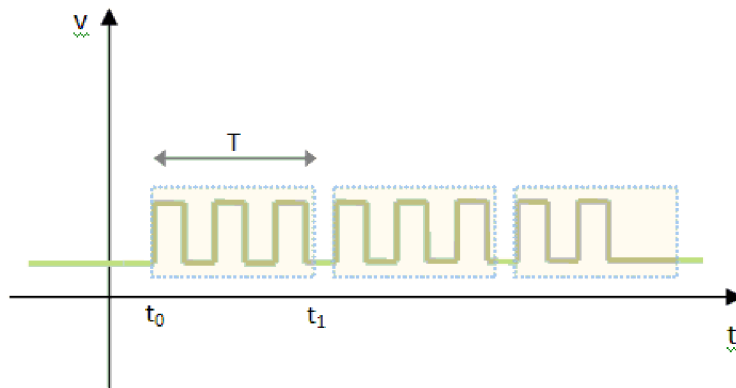
$$\text{Prob. of High} = \frac{\sum_i \Delta T_i^o}{T}, \text{ where } t \in \Delta T_i^o \text{ s.t. } v(t) > v_{hth}$$

$$\text{Prob. of Low} = \frac{\sum_i \Delta T_i^u}{T}, \text{ where } t \in \Delta T_i^u \text{ s.t. } v(t) < v_{lth}$$



Multiple toggling means a node toggles multiple times within a given time period. A time period starts at the point when the first toggle occurs and lasts for a user-specified time. The next time period starts when the next toggle occurs. See [Figure 18](#).

Figure 18 Multiple Toggling



Examples

The following example checks the toggling activity on the `x1.sout` node. On-the-fly reporting is enabled and updated with the basic node activity every 10% of the simulation.

```
cck_toggle_count -label test1 -node x1.sout -flush 10%
```

Checks the toggling activity on all nodes, except for nodes with names that start with `n`. A basic report is printed at the end of the simulation.

```
cck_toggle_count -label test2 -except_node n*
```

Checks the toggling activity on all nodes with names that start with `ain` inside the `dummy` subcircuit. A detail report is generated at the end of the simulation.

```
cck_toggle_count test4 -enable_detail 1 -subckt dummy -node ain*
```

Checks the toggling activity on all nodes. A basic report is printed to an output file at the end of the simulation with additional information on the two most toggling nodes and three zero toggling nodes.

```
cck_toggle_count test5 -report_most_toggle 2 -report_zero_toggle 3
```

check_measure

Checks for failed measurements.

Syntax

```
check_measure \  
  -label meas_label_1 [ meas_label_2 ... ] \  
  -except meas_label_1 [ meas_label_2 ... ] \  
  [-action cont|warn|exit]
```

Arguments

Argument	Description
<code>-label meas_label_1 [meas_label_2 ...]</code>	Specifies a space-delimited list of measurements to check for the “failed” state. Wildcard is supported. When used with the <code>-except</code> option, the effective measurements would be ones in the intersection of these two options.
<code>-except meas_label_1 [meas_label_2 ...]</code>	Specifies a space-delimited list of measurements to exclude from check. Wildcard is supported. When used with the <code>-label</code> option, the effective measurements would be ones in the intersection of these two options.

Argument	Description
<code>-action cont warn exit</code>	Specifies the action when a measurement failure is detected. This option has no default. When this option is specified, you must specify which action to take place. <code>cont</code> : Issues a status message and continues the simulation. <code>warn</code> (Default): Issues a warning message and continues the simulation. <code>exit</code> : Issues an error message and exits the simulation.

Example

In the following example, when running Monte Carlo simulation in DP mode, a DP task that runs through completion is additionally checked for failed measurements. If there is any failure, PrimeSim XA considers the task a failed task and proceeds with auto-replacing, depending on the settings of the `-auto_replace` option of the `set_monte_carlo_option` command.

```
check_measure -label *
```

For more usage examples, see the Auto-Replacing Monte Carlo Samples When Measurements Fail section in *PrimeSim XA User Guide*.

check_node_excess_rf

Performs a timing check for excessive rise and fall time and reports the results in an output file with an *.errt extension.

Syntax

```
check_node_excess_rf -node node_name {node_name} \  
  -title title_name \  
  [-fanout fanout_value] \  
  [-rtime rise_time] \  
  [-ftime fall_time] \  
  [-utime ustate_time] \  
  [-loth low_voltage] \  
  [-hith high_voltage] \  
  [-twindow tstart [{tstop}] {tstart [tstop]}] \  
  [-subckt subckt_name {subckt_name}] \  
  [-error_file output_file_name] \  
  [-limit limit_value] \  
  [-numv value] \  
  [-report totalv]
```

Arguments

Argument	Description
<code>-node node_name</code> <code>{node_name}</code>	Specifies the node names to check for excessive rise and fall time. You must specify this option first. See the Command Scoping section for more information.
<code>-title title_name</code>	Specifies the title name of this excessive rise and fall time check. The name is useful when there are multiple <code>check_node_excess_rf</code> commands in one simulation.
<code>-fanout fanout_value</code>	Defines the fanout of the driver nodes. It can be set to: • 0 (default) to check for all nodes. • 1 to check for nodes that have a direct connection to a transistor gate. • 2 to check for all nodes that have direct connection to a transistor bulk.
<code>-rtime rise_time</code>	Defines the rise time of the signal when it crosses the <i>low_voltage</i> to when it crosses the <i>high_voltage</i> . The default is 1ns.
<code>-ftime fall_time</code>	Defines the fall time of the signal when it crosses the <i>high_voltage</i> to when it crosses the <i>low_voltage</i> . The default is 1ns.
<code>-utime ustate_time</code>	Defines the time period when the signal voltage is between <i>low_voltage</i> and <i>high_voltage</i> . The default is 1ns. U-state checking is only performed when a transition is incomplete, which means it fails to go from <i>high_voltage</i> to <i>low_voltage</i> or vice versa.
<code>-loth low_voltage</code>	Defines the logic low voltage for the signal. The default is the same as the set_logic_threshold value.
<code>-hith high_voltage</code>	Defines the logic high voltage for the signal. The default is the same as the set_logic_threshold value.
<code>-twindow tstart tstop</code> <code>{tstart tstop}</code>	Specifies the time windows for the check. The default for <i>tstart</i> is 0s and <i>tstop</i> is the end of transient time.
<code>-subckt subcircuit_name</code> <code>{subcircuit_name}</code>	Specifies the subcircuit name of the specified <i>node_name</i> where the check is performed. See the Command Scoping section for more information.
<code>-error_file</code> <code>output_file_name</code>	Specifies the file name of the output file for reporting any rise and fall time violations. The output file has a *.errt extension.
<code>-limit limit_value</code>	Specifies the hierarchical level down to which the nodes are checked when you use wildcard characters. When you specify <code>-subckt subckt_name</code> and <code>-limit limit_value</code> <i>limit_value</i> is relative to the scope of <i>subckt_name</i> . The default is 3.

Argument	Description
<code>-numv value</code>	Limits the number of reported violations. The default is to report all violations.
<code>-report totalv</code>	When you specify the <code>-report totalv</code> option, the report file contains a summary count of all violations detected from the analysis of the <code>check_node_excess_rf</code> check. You can limit reporting with the <code>-numv</code> option, but the <code>-report totalv</code> option provides the complete count of all violations detected. For example, if you specify: <pre>check_node_excess_rf ... -numv 10 -report totalv</pre> The .errt file shows: <pre>* edge: Total number of violations reported = 10 * edge: Total number of violations detected = 6105</pre>

Description

When performing a timing check for excessive rise and fall time, the PrimeSim XA tool reports the results if violations occur. You can modify the check with the [set_logic_threshold](#) command, or the `-loth` and `-hith` options of the `check_node_excess_rf` command. The `-loth` and `-hith` options take precedence over the [set_logic_threshold](#) command.

Examples

The following example sets up the timing check for the rise and fall time of node output with the `exrf_check1` title. If the rise time exceeds 2 ns or the fall time exceeds 2 ns, the PrimeSim XA tool reports the violations.

```
check_node_excess_rf -node output -title exrf_check1 \
  -rtise 2ns -ftime 2ns
```

The following example sets up the timing check for the rise and fall time of the `dout*` node inside the `cpump` subcircuit with the `exrf_check2` title. If the rise time exceeds 2 ns or the fall time exceeds 2 ns, the PrimeSim XA tool reports the violations to a file, named `rf_check.errt`.

```
check_node_excess_rf -node dout* -title exrf_check2 \
  -rtise 2ns -ftime 2ns -subckt cpump -error_file rf_check
```

check_node_hotspot

Performs a hotspot check in a design and reports the results to an output file with a *.hotspot extension.

Syntax

```
check_node_hotspot -node node_name {node_name}
```

Arguments

Argument	Description
-node node_name {node_name}	Specifies the node names to apply the hot spot check. See the Command Scoping section for more information.

Description

When performing a hot spot check, the PrimeSim XA tool reports the result if violations occur. The following information is reported when there is a hotspot violation:

- Cap (fF) - Average node capacitance lumped from netlist, wiring, transistor gate, and diffusion capacitances
- Toggle - Number of logic toggles of the node
- Icin (uA) - Average charging current flowing into the capacitances connected to the node
- Icout(uA) - Average discharging current flowing out of the capacitances connected to the node
- Total non-input nodes
- Total node toggles
- Total charging current
- Total discharging current

By default, nodes are reported when Icin(uA) + Icout(uA) of the node is greater than the hotspot factor of 0.5 multiplied by Icin(uA) + Icout(uA) of the node with the largest capacitive current. You can modify the hotspot factor with the [set_hotspot_option](#) command. The hotspot check does not apply to input nodes.

Examples

The following example sets up the hotspot check for all nodes in the design.

```
check_node_hotspot -node *
```

The PrimeSim XA tool reports nodes with the sum of capacitive charging and discharging currents greater than the hotspot factor of 0.5 multiplied by the sum of the node with the largest capacitive current.


```
*-----*
Hotspot Check
* node      = *
* factor    = 0.5
*-----*

Node Name          Cap(fF)      Toggle      Icin(uA)      Icout(uA)
*-----*
x4.bn              745.6        19          20.08         22.31
b[0]               604.1        19          15.68         14.11
x4.b               519.4        19          14.38         12.94
x4.p               405.9        18          11.49         11.51
Total non-input nodes      : 81
Total node toggles         : 406
Total charging current      : 185.9uA
Total discharging current   : 186.1uA
```

check_node_quick_rf

Performs a timing check for when rise or fall time is less than the specified value. The results are reported to an output file with a *.errt extension.

Syntax

```
check_node_quick_rf -node node_name {node_name} \
  -title title_name \
  [-except_node x_node_name {x_node_name}] \
  [-fanout fanout_value] \
  [-rtime rise_time] \
  [-ftime fall_time] \
  [-lovh low_voltage] \
  [-hivh high_voltage] \
  [-twindow tstart [{tstop}] {tstart [tstop]}] \
  [-subckt subckt_name {subckt_name}] \
  [-error_file output_file_name] \
  [-limit limit_value] \
  [-numv value] \
  [-report totalv]
```

Arguments

Argument	Description
-node node_x_name {x_ode_name}	Specifies the node names to check for rise and fall time. You must specify this option first. See the Command Scoping section for more information.
-title title_name	Specifies the title name of this timing check for rise and fall times. The name is useful when there are multiple check_node_quick_rf commands in one simulation.

Argument	Description
<code>-except_node</code> <code>x_node_name</code> { <code>x_node_name</code> }	Specifies the node names to be excluded from the timing check of rise and fall times. See the Command Scoping section for more information.
<code>-fanout fanout_value</code>	Defines the fanout of the driver nodes. It can be set to: • 0 (default) to check for all nodes. • 1 to check for nodes that have a direct connection to a transistor gate. • 2 to check for all nodes that have direct connection to a transistor bulk.
<code>-rtime rise_time</code>	Defines the rise time of the signal when it crosses the <i>low_voltage</i> to when it crosses the <i>high_voltage</i> . The default is 1ns.
<code>-ftime fall_time</code>	Defines the fall time of the signal when it crosses the <i>low_voltage</i> to when it crosses the <i>high_voltage</i> . The default is 1 ns.
<code>-loth low_voltage</code>	Defines the logic low voltage for the signal. The default is the same as the set_logic_threshold value.
<code>-hith high_voltage</code>	Defines the logic high voltage for the signal. The default is the same as the set_logic_threshold value.
<code>-twindow tstart tstop</code> { <code>tstart tstop</code> }	Specifies the time windows for the check. The default for <i>tstart</i> is 0 s and <i>tstop</i> is the end of transient time.
<code>-subckt subckt_name</code> { <code>subckt_name</code> }	Specifies the subcircuit name of the specified <i>node_name</i> where the timing check is performed. See the Command Scoping section for more information.
<code>-error_file</code> <code>output_file_name</code>	Specifies the file name of the output file for reporting any rise and fall time violations. The output file has a *.errt extension.
<code>-limit limit_value</code>	Specifies the hierarchical level down to which the nodes are checked when you use wildcard characters. When you specify <code>-subckt subckt_name</code> and <code>-limit limit_value</code> (<i>limit_value</i> is relative to the scope of <i>subckt_name</i>). The default is 3.
<code>-numv value</code>	Limits the number of reported violations. The default is to report all violations.

Argument	Description
<code>-report totalv</code>	When you specify the <code>-report totalv</code> option the report file contains a summary count of all violations detected from the analysis of the <code>check_node_quick_rf</code> check. You can limit reporting with the <code>-numv</code> option, but the <code>-report totalv</code> option provides the complete count of all violations detected. For example, if you specify: <pre>check_node_quick_rf ... -numv 10 -report totalv</pre> The <code>.errt</code> file shows: <pre>* edge: Total number of violations reported = 10 * edge: Total number of violations detected = 6105</pre>

Description

When performing a timing check for rise and fall time, the PrimeSim XA tool reports the results if the rise time is less than *rise_time* or the fall time is less than *fall_time*. By default, the PrimeSim XA tool uses 30% and 70% of the rail-to-rail voltage of the signal as the logic threshold value. You can modify it with the [set_logic_threshold](#) command or the `-loth` and `-hith` options of the `check_node_quick_rf` command. The `-loth` and `-hith` options take precedence over the [set_logic_threshold](#) command.

Examples

The following example sets up the timing check for rise and fall time of node output with the `quickrf_check1` title. If the rise time is less than 2 ns or the fall time is less than 2 ns, the PrimeSim XA tool reports the results.

```
check_node_quick_rf -node output -title quickrf_check1 \
  -rtise 2ns -ftime 2ns
```

The following example sets up the timing check for rise and fall time of all nodes (except nodes with name beginning with *dout*) inside the `cpump` subcircuit with the `quickrf_check2` title. If the rise time is less than 2 ns or the fall time is less than 2 ns, the PrimeSim XA tool reports the results to an output file, named `rf_check.errt`.

```
check_node_quick_rf -node * -title quickrf_check2 \
  -except_node dout* -rtise 2ns -ftime 2ns -subckt cpump \
  -error_file rf_check
```

check_node_zstate

Performs a high-impedance check of the specified nodes. The results are reported to an output file with a `*.errz` extension.

Syntax

```
check_node_zstate -node node_name {node_name} \  
-title title_name \  
[-except_node x_node_name {x_node_name}] \  
[-ztime z_time] \  
[-fanout 0|1|2] \  
[-tstep tstep_value] \  
[-twindow tstart [tstop] {tstart [tstop]}] \  
[-subckt subckt_name {subckt_name}] \  
[-except_subckt subckt_name {subckt_name}] \  
[-limit limit_val] \  
[-skippcap 0|1] \  
[-report report_switch] \  
[-error_file output_file_name] \  
[-numv value]
```

Arguments

Argument	Description
-node node_name {node_name}	Specifies the node names to check for a high-impedance state. You must specify this option first. See the Command Scoping section for more information.
-title title_name	Specifies the title name of this high-impedance check. The name is useful when there are multiple check_node_zstate commands in one simulation.
-except_node x_node_name {x_node_name}	Specifies the node names to be excluded from the high-impedance check. See the Command Scoping section for more information.
-ztime z_time	Specifies the time period for which a node must remain in a high-impedance state to be reported. The default is 5 ns.
-fanout 0 1 2	Defines the fanout of the driver nodes. It can be set to: • 0 (default) to check for all nodes. • 1 to check for nodes that have a direct connection to a transistor gate. • 2 to check for all nodes that have direct connection to a transistor bulk.
-tstep tstep_value	Performs a high-impedance check at discrete time points that are integer multiples of the specified <i>tstep_value</i> . The first time point default is the value of <i>tstart</i> . If you do not specify the <i>-ztime</i> argument, the high-impedance node of each discrete time is reported. If you use the <i>-ztime</i> argument, it still performs discrete time checking, but only for nodes remaining at a high-impedance state at least to the <i>ztime_value</i> or longer are reported.

Argument	Description
<code>-twindow tstart tstop</code> <code>{tstart tstop}</code>	Specifies the time windows for timing check. The default for <i>tstart</i> is 0s and <i>tstop</i> is the end of transient time.
<code>-subckt subckt_name</code> <code>{subckt_name}</code>	Specifies the subcircuit name of the specified <i>node_name</i> where the high-impedance check is performed. See the Command Scoping section for more information.
<code>-except_subckt</code> <code>subckt_name</code> <code>{subckt_name}</code>	Specifies the subcircuit name of the specified <i>node_name</i> to be excluded from high-impedance check. See the Command Scoping section for more information.
<code>-limit limit_val</code>	Specifies the hierarchy level down to which currents are probed when you use wildcard characters. A value of 0 specifies the top level. The default setting is unlimited, which means a wildcard can match to any hierarchical level. When you specify <code>-subckt</code> , <i>limit_val</i> is relative to the scope of <i>subckt_name</i> .
<code>-skippcap 0 1</code>	You can specify one of the following values. • 0 (default) specifies that pseudo capacitors have no effect on how HiZ nodes are reported. A pseudo capacitor is defined as a MOSFET with drain and source shorted. • 1 specifies that if a node is considered HiZ, and only pseudo capacitors are connected through its MOSFET gates, the node is not reported. Note that the <code>-skippcap</code> option has higher priority than the <code>-fanout</code> option.
<code>-report report_switch</code>	Reports all nodes or only subcircuit ports. The supported values are: • all (default): Reports all nodes. • port: Reports only subcircuit ports. • totalv: Specifies that the report file contains a summary count of all violations detected from the analysis of the <code>check_node_zstate</code> check. You can limit reporting with the <code>-numv</code> argument, but the <code>-report totalv</code> option provides the complete count of all violations detected. For example, if you specify: <pre>check_node_zstate ... -numv 10 -report totalv</pre> The .errt file shows: <pre>* edge: Total number of violations reported = 10 * edge: Total number of violations detected = 6105</pre>
<code>-error_file</code> <code>output_file_name</code>	Specifies the file name of the output file for reporting any rise time and fall time violations. The output file has a *.errz extension.
<code>-numv value</code>	Limits the number of reported violations. The default is to report all violations.

Description

A node stays in a high-impedance state if there is no conducting path from any voltage source to the node. A conducting path is formed by conducting elements. An element is conducting if that specific element meets the following conditions listed in the following table. You can adjust the conditions with the [set_zstate_option](#) command.

Table 34 Conducting Conditions for Elements

Device	Conducting Condition
NMOS	Conducting when one of the following rules is met: • Rule 1: $V_{gs} > V_{th}$ • Rule 2: $I_{ds} > I_{dsth}$ (The default of I_{dsth} is 1e-8 ampere) • Rule 3: $V_g > V_{DD}-0.1$ By default, Rule 1 or Rule 2 must be met for the NMOS to be conducting. You can change the condition with the set_zstate_option command.
PMOS	Conducting when one of the following rules is met: • Rule 1: $V_{gs} < V_{th}$ • Rule 2: $I_{ds} > I_{dsth}$ (The default of I_{dsth} is 1e-8 ampere) • Rule 3: $V_g < 0.1$ By default, Rule 1 or Rule 2 must be met for the element to be conducting. You can change the condition with the set_zstate_option command.
NMOS diode (MOS with drain connected to gate or source connected to gate)	Conducting when one of the following rules is met: • $V_g \text{ (or } V_d) - V_s > \text{mos_diode_vth}$ • $V_g \text{ (or } V_s) - V_d > \text{mos_diode_vth}$ The default for mos_diode_vth is 0.2. You can change it with the set_zstate_option command.
PMOS diode (MOS with drain connected to gate or source connected to gate)	Conducting when one of the following rules is met: • $V_d - V_g \text{ (or } V_s) > \text{mos_diode_vth}$ • $V_s - V_g \text{ (or } V_d) > \text{mos_diode_vth}$ The default for mos_diode_vth is 0.2 volt. You can change it with the set_zstate_option command.
NPN	Conducting when $V_{be} > V_{beth}$. The default of V_{beth} is 0.64 volt. You can change it with the set_zstate_option command.
PNP	Conducting when $V_{be} < -V_{beth}$. The default of V_{beth} is 0.64 volt. You can change it with the set_zstate_option command.
Diode	Conducting when $V(a,c) > \text{diode_vth}$. The default of diode_vth is 0.2 volt. You can change it with the set_zstate_option command.

Table 34 Conducting Conditions for Elements (Continued)

Device	Conducting Condition
Resistor	Conducting by default. You can change it with the set_zstate_option command.
Inductor	Always conducting
Capacitor	Always non-conducting
Verilog-A	Always non-conducting. The reporting of output node from Verilog-A can be controlled by the set_zstate_option command.
Controlled sources	Always conducting

The following table shows the conducting rules for Spectre bsource elements.

Table 35 Conducting Rules for Spectre Bsource Elements

B-source Type	Conducting Rule
resistor	Always conducting by default. You can change it with the set_zstate_option command.
conductance	Always conducting by default. You can change it with the set_zstate_option command.
capacitor	Always non-conducting
inductor	Always conducting
vsource	Always conducting
isource (independent value)	Non-conducting by default. You can change it with the set_zstate_option command.
isource (dependent value)	Always conducting
phi	Always non-conducting
q	Always non-conducting

Examples

The following example sets up the high-impedance check for all nodes with the hiz_check1 title, and reports the high-impedance nodes to an output file, named hiz_out.errz.

```
check_node_zstate -node * -title hiz_check1 -error_file hiz_out
```

check_timing_edge

Performs a timing check for rising and falling edges of specified data (or target) nodes relative to the reference node. The timing difference ($T_{\text{target}} - T_{\text{reference}}$) is calculated from the permissible transition edge of the reference node at $T_{\text{reference}}$ to the permissible transition edge of the target node at T_{target} . The result is reported to an output file with a *.errt extension.

Syntax

```
check_timing_edge \
  -title title_name node_terminal_spec \
  -min_time min_time_value \
  -max_time max_time_value \
  [-window window_value] \
  [-trigger trigger_switch] \
  [-lovh low_voltage] \
  [-hith high_voltage] \
  [-twindow tstart [tstop] {tstart [tstop]}] \
  [-subckt subckt_name] \
  [-error_file output_file_name] \
  [-ref_edge_type edge_type] \
  [-data_edge_type edge_type] \
  [-rule rule_value] \
  [-numv value] \
  [-report totalv] \
```

where

```
node_terminal_spec ::= node_spec | terminal_spec
node_spec ::= -node node_name {node_name} -ref_node node_name
terminal_spec ::= -termn device_name {device_name} -ref_termn device_name
```

Arguments

Argument	Description
-title title_name	Specifies the title name of this timing check. The name is useful when there are multiple check_timing_edge commands in one simulation.
-node node_name {node_name}	Specifies the node names of the target signal to check for rising and falling edges relative to the reference signal. See the Command Scoping section for more information.
-ref_node node_name	Specifies the node name of the reference signal. It can only be one node name, and a wildcard is not allowed.

Argument	Description
<code>-termn device_name</code> <code>{device_name}</code>	Specifies the terminal number <i>n</i> of the named devices as target signals to check rising and falling edges relative to the reference signal. You can use a wildcard limited to one level of hierarchy.
<code>-ref_termn device_name</code>	Specifies device terminal number <i>n</i> as the reference signal to check the rising and falling edges. You cannot specify a wildcard character in the <i>device_name</i> .
<code>-min_time</code> <code>min_time_value</code>	Specifies the lower boundary of the timing check. A violation is reported when the timing difference ($T_{\text{target}} - T_{\text{reference}}$) is less than <i>min_time_value</i> .
<code>-max_time</code> <code>max_time_value</code>	Specifies the upper boundary of the timing check. A violation is reported when the timing difference ($T_{\text{target}} - T_{\text{reference}}$) is greater than <i>max_time_value</i> .
<code>-window window_value</code>	Specifies the window boundary of the timing check. A violation is reported when the timing difference ($T_{\text{target}} - T_{\text{reference}}$) is less than <i>window_value</i> .
<code>-trigger trigger_switch</code>	Specifies the trigger for execution of the timing check. It can be set to: • <i>ref</i> - Checking is executed only when the reference edge occurs. Use this setting if the target node changes edges before the reference node. • <i>input</i> - Checking is executed when the target edge occurs. Use this setting if the target node changes edges after the reference node. • <i>both</i> (default) - Checking is executed when either the reference edge or the target edge occurs. Use this setting with caution as it is pessimistic and might report false errors. You can use the <code>-window</code> argument to remove the false errors.
<code>-loth low_voltage</code>	Defines the logic low voltage for the signal. The default is the same as the set_logic_threshold command.
<code>-hith high_voltage</code>	Defines the logic high voltage for the signal. The default is the same as the set_logic_threshold command.
<code>-twindow tstart tstop</code> <code>{tstart tstop}</code>	Specifies the time windows for timing check. The default for <i>tstart</i> is 0s and <i>tstop</i> is the end of transient time.
<code>-subckt subckt_name</code>	Specifies the subcircuit name of the specified <i>node_name</i> where timing check is performed. See the Command Scoping section for more information.
<code>-error_file</code> <code>output_file_name</code>	Specifies the file name of the output file for reporting any violation. The output file has a *.errt extension.

Argument	Description
<code>-ref_edge_type edge_type</code>	Specifies the edge type of the reference signal. It can be set to: • <code>rise</code> (default) - Rising edge • <code>fall</code> - Falling edge • <code>rf</code> - Both the rising and falling edge • <code>x</code> - Either the rising or the falling edge. This only works with the <code>-rule 2</code> setting. See Figure 19 for a graphical representation of <code>rf</code> and <code>x</code>.
<code>-data_edge_type edge_type</code>	Specifies the edge type of the target signal. It can be set to: • <code>rise</code> (default) - Rising edge • <code>fall</code> - Falling edge • <code>rf</code> - Both the rising and falling edge • <code>x</code> - Either the rising or the falling edge. This only works with the <code>-rule 2</code> setting. See Figure 19 for a graphical representation of <code>rf</code> and <code>x</code>.
<code>-rule rule_value</code>	Selects the rule of timing check. It can be set to: • <code>1</code> (default) - Checks at every transition • <code>2</code> - Checks at the most recent transition between reference signal See Figure 20 for a graphical representation of <code>-rule 1</code> and <code>-rule 2</code>.
<code>-numv value</code>	Limits the number of reported violations. The default is to report all violations.
<code>-report totalv</code>	When you specify the <code>-report totalv</code> option, the report file contains a summary count of all violations detected from the <code>check_timing_edge</code> check. You can limit reporting with the <code>-numv</code> argument, but the <code>-report totalv</code> option provides the complete count of all violations detected. For example, if you specify: <pre>check_timing_edge ... -numv 10 -report totalv</pre> The <code>.errt</code> file shows: <pre>* edge: Total number of violations reported = 10 * edge: Total number of violations detected = 6105</pre>

Description

When performing the timing check for the rising edge and falling edge between the reference and target signals, the PrimeSim XA tool reports the results if the timing is less than *min_time_value* or greater than *max_time_value*. By default, the PrimeSim XA tool uses 30% and 70% as the logic threshold values. You can modify the threshold value with the [set_logic_threshold](#) command, or the `-loth` and `-hith` arguments of the `check_timing_edge` command. The `-loth` and `-hith` options take precedence over the [set_logic_threshold](#) command.

Figure 19 Graphical Representation of the Difference Between rf and x

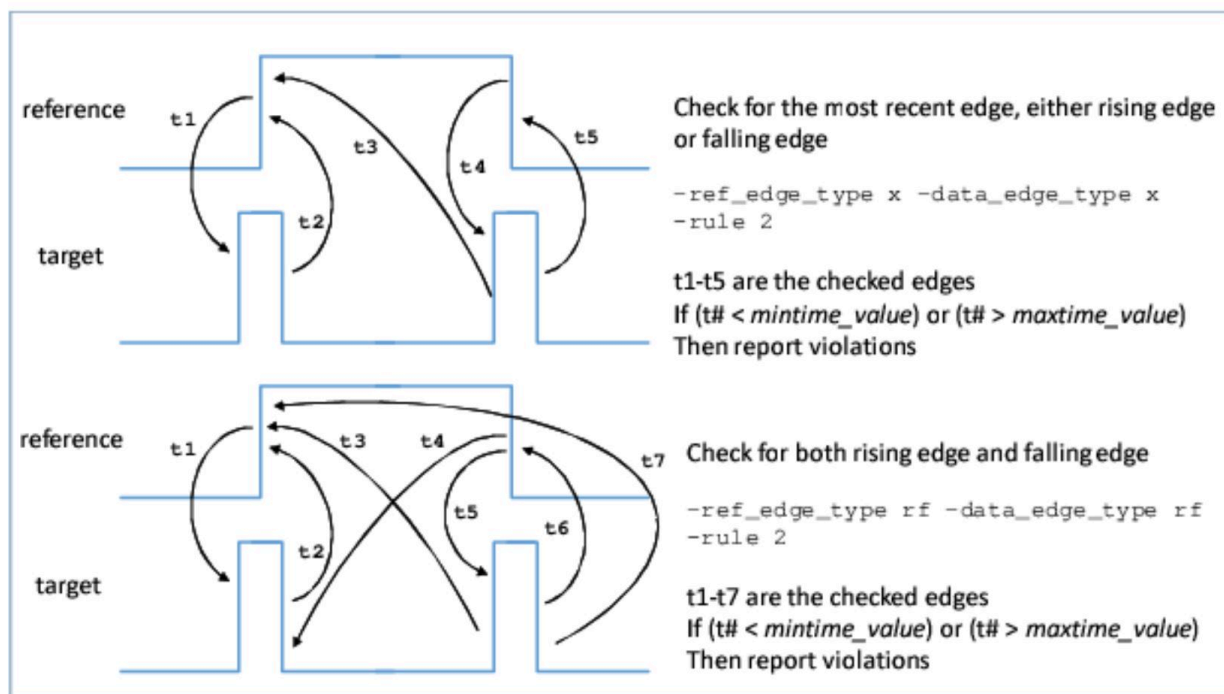
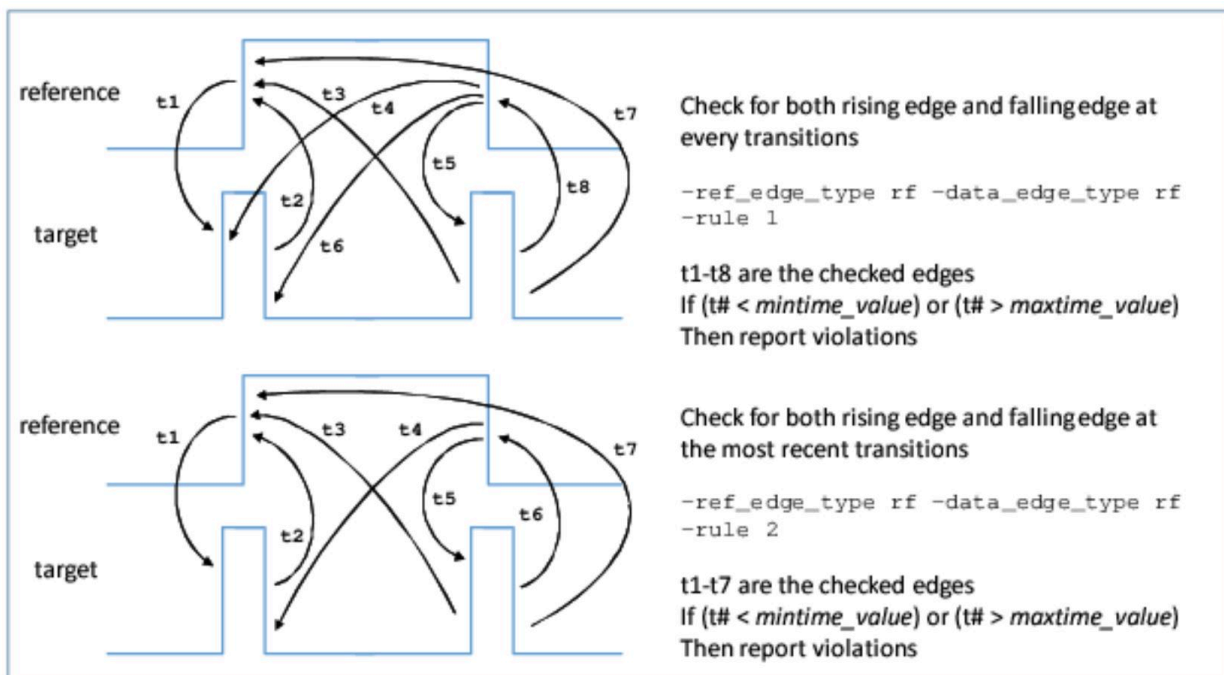


Figure 20 Graphical Representation of -rule 1 and -rule 2



Examples

The following example sets up the timing check for rise edge of the signal ctrl to the falling edge of signal data with the edge_check1 title. If the timing difference is less than 2 ns or greater than 5 ns, the PrimeSim XA tool reports the results to an output file, named xa_edge.errt.

```
check_timing_edge -node data -title edge_check1 -ref ctrl \  
  -ref_edge_type rise -data_edge_type fall \  
  -min_time 2ns -max_time 5ns -error_file xa_edge
```

check_timing_hold

Performs a timing check for the hold time of specified data (or target) nodes. The hold time is defined as the minimum amount of time a target signal must remain stable (not transition to the specified edge) after the reference signal active edge. Hold time violations are reported to an output file with *.errt extension. Also, if you specify `-report min`, `-report max`, or `-report min max`, the command calculates all node hold times per reference event and measures the minimum or maximum values of these hold times. The minimum and maximum hold times for all nodes are reported in a separate file with a *.tckrpt extension.

Syntax

```
check_timing_hold -title title_name node_terminal_spec \  
  -hold_time time [-subckt subckt_name] \  
  [-ref_subckt 0|1] [-data_edge_type edge_type] \  
  [-ref_edge_type edge_type] [-loth logic_low_voltage] \  
  [-hith logic_high_voltage] [-window window_limit] \  
  [-twindow tstart [tstop] {tstart [tstop]}] \  
  [-error_file output_file_name] [-numv value] [-report totalv]
```

or

```
check_timing_hold -title title_name node_terminal_spec \  
  -report min|max|min max [-subckt subckt_name] [-ref_subckt 0|1] \  
  [-data_edge_type edge_type] [-ref_edge_type edge_type] \  
  [-loth logic_low_voltage] [-hith logic_high_voltage] \  
  [-twindow tstart [tstop] {tstart [tstop]}] [-report_file file_name]
```

where

```
node_terminal_spec ::= node_spec | terminal_spec  
node_spec ::= -node node_name {node_name} -ref_node node_name  
terminal_spec ::= -termn device_name {device_name} -ref_termn device_name
```

Arguments

Argument	Description
<code>-title title_name</code>	Specifies the title name of this timing check. The name is useful when there are multiple <code>check_timing_setup</code> commands in one simulation.
<code>-node node_name</code> <code>{node_name}</code>	Specifies the node names of the target signal to check for hold time relative to the reference signal. See the Command Scoping section for more information.
<code>-ref_node node_name</code>	Specifies the node name of the reference signal. It can only be one node name, and a wildcard is not allowed.
<code>-termn device_name</code> <code>{device_name}</code>	Specifies the terminal number <i>n</i> of the named devices as target signals to check hold time relative to the reference signal. You can use a wildcard supported limited to one level of hierarchy.
<code>-ref_termn device_name</code>	Specifies device terminal number <i>n</i> as the reference signal to check hold time. You cannot specify a wildcard character in the <i>device_name</i> .
<code>-hold_time hold_time</code>	Specifies the hold time and can be a positive or negative value. When it is set to a negative value, you must also specify the <code>-window</code> argument. It follows these rules: - When <i>hold_time</i> is set to a positive value, the target signal must not transition to the specified edge between $T_{\text{reference}}$ and $(T_{\text{reference}} + \text{hold_time})$. Otherwise, a hold time violation is reported. See Figure 21 or a graphical representation. - When <i>hold_time</i> is set to a negative value, the target signal must not transition to the specified edge between $(T_{\text{reference}} + \text{hold_time})$ and $(T_{\text{reference}} + \text{window_value})$ and if $T_{\text{reference}}$ is an active edge time of a reference signal. Otherwise, a hold time violation is reported. See Figure 22 for a graphical representation.
<code>-report min max min max</code>	Measures the minimum and/or maximum hold time of the target signals for each event of the reference signal.
<code>-subckt subckt_name</code>	Specifies the subcircuit name of the specified <i>node_name</i> where timing check is performed. See the Command Scoping section for more information.
<code>-ref_subckt 0 1</code>	When <code>-ref_subckt 0</code> is set, the <code>-subckt</code> scoping is not applied to <code>-ref ref_node_name</code> . The <code>-ref ref_node_name</code> is effectively the full path name. When the <code>-ref_subckt</code> option is set to 1, the <code>-subckt</code> scoping is applied to the <code>-ref ref_node_name</code> option (default).

Argument	Description
<code>-data_edge_type edge_type</code>	Specifies the transition edge type of the target signal. It can be set to: • rise (default) - Rising edge • fall - Falling edge. • rf - Both the rising and falling edge.
<code>-ref_edge_type edge_type</code>	Specifies the transition edge type of the reference signal. It can be set to: • rise (default) - Rising edge. • fall - Falling edge • rf - Both the rising and falling edge
<code>-loth logic_low_voltage</code>	Defines the logic low voltage for the signal. The default is the same as the set_logic_threshold value.
<code>-hith logic_high_voltage</code>	Defines the logic high voltage for the signal. The default is the same as the set_logic_threshold value.
<code>-window window_limit</code>	Specifies the width of a window to check for setup time if <i>setup_time</i> is a negative value. This option is ignored when <i>setup_time</i> is set to a positive value.
<code>-twindow tstart tstop {tstart tstop}</code>	Specifies the time windows for timing check. The default for <i>tstart</i> is 0 s and <i>tstop</i> is the end of transient time.
<code>-error_file output_file_name</code>	Specifies the file name of the output file for reporting any violation. The output file has a *.ert extension.
<code>-numv value</code>	Limits the number of reported violations. The default is to report all violations.
<code>-report totalv</code>	When you specify the <code>-report totalv</code> option, the report file contains a summary count of all violations detected from the <code>check_timing_setup</code> check. You can limit reporting with the <code>-numv</code> option, but the <code>-report totalv</code> option provides the complete count of all violations detected. For example, if you specify: <pre>check_timing_setup ... -numv 10 -report totalv</pre> The .ert file shows: <pre>* edge: Total number of violations reported = 10 * edge: Total number of violations detected = 6105</pre>
<code>-report_file file_name</code>	Specifies the output text file name in a format of <i>file_name.tckrpt</i> .

Description

When performing the timing check for hold time, the PrimeSim XA tool reports the results of the hold time violations by the specified conditions. By default, the PrimeSim XA tool

uses 30% and 70% of the rail-to-rail voltage of the signal as the logic threshold values. You can modify the value with the [set_logic_threshold](#) command or the `-loth` and `-hith` options of the `check_timing_hold` command. The `-loth` and `-hith` options take precedence over the [set_logic_threshold](#) command.

Figure 21 Graphical Representation of Positive `hold_time`

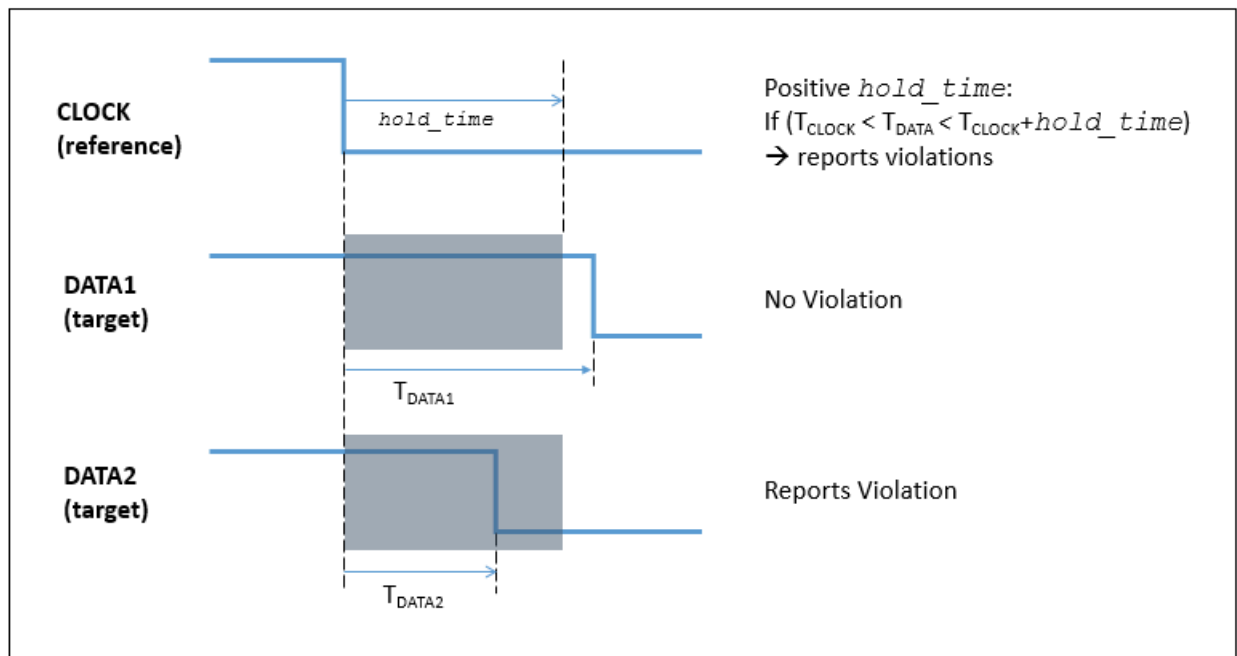
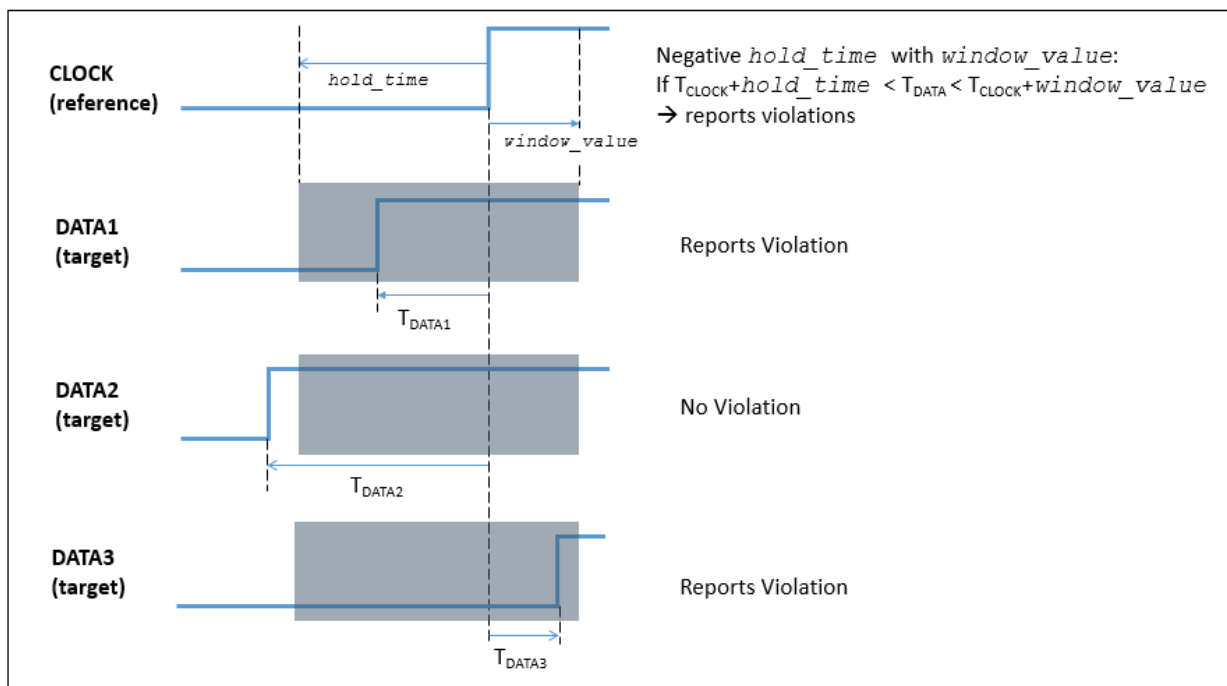


Figure 22 Graphical Representation of Negative hold_time with window_value



Examples

The following example sets up the timing check for hold time from the clk node to the output node with the hold_check1 title. If the hold time is less than 2 ns, the PrimeSim XA tool reports the results.

```
check_timing_hold -node output -title hold_check1 -ref clk \
  -hold_time 2ns
```

The following example sets up the timing check for hold time from the clk node to all nodes with names beginning with "dout" inside the ram subcircuit with the hold_check2 title. If the hold time is less than 2 ns, the PrimeSim XA tool reports the results to a file, named ram_hold_check.errt.

```
check_timing_hold -node dout* -title hold_check2 -ref clk \
  -hold_time 2ns -subckt ram -error_file ram_hold_check
```

The following example enables the timing check for hold time from the bus<*> nodes to node outputs with the hold_bus title. The -report min max option enables a check of the hold time for all nodes per reference signal ref_clk. The PrimeSim XA tool reports the results in the bus_tchk0.tckrpt file.

```
check_timing_hold -node bus<*> -ref ref_clk -ref_edge_type rise \
  -title hold_bus -report min max -report_file bus_tchk0
```


check_timing_post

Description

Enables postprocessing for the timing check commands. It supports the FSDB, WDF, and GRP waveform file formats.

Syntax

```
check_timing_post -waveform file_name
```

check_timing_pulse_width

Performs a timing check for the pulse width of specified nodes. The violations are reported in an output file with the *.errt extension.

where

Syntax

```
check_timing_pulse_width -title title_name node_terminal_spec
  [-fanout fanout_value] -low_min_time low_min_time
  -low_max_time low_max_time -high_min_time high_min_time
  -high_max_time high_max_time [-loth logic_low_voltage]
  [-hith logic_high_voltage] [-twindow tstart [tstop] {tstart [tstop]}]
  [-subckt subckt_name] [-error_file output_file_name]
  [-numv value] [-report totalv]
node_terminal_spec ::= node_spec | terminal_spec
node_spec ::= -node node_name {node_name}
terminal_spec ::= -termn device_name {device_name}
```

Arguments

Argument	Description
-title title_name	Specifies the title name of this pulse width check. The name is useful when there are multiple check_timing_pulse_width commands in one simulation.
-node node_name {node_name}	Specifies the node names to check for pulse width. See the Command Scoping section for more information.
-termn device_name {device_name}	Specifies the terminal number <i>n</i> of the devices to check for pulse width. You can specify a wildcard character in the device_name1 limited to one hierarchical level.

Argument	Description
<code>-fanout fanout_value</code>	Defines the fanout of the driver nodes. It can be set to: • 0 (default) to check for all nodes. • 1 to check for nodes that have a direct connection to a transistor gate. • 2 to check for all nodes that have direct connection to a transistor bulk.
<code>-low_min_time</code> <code>low_min_time</code> <code>-low_max_time</code> <code>low_max_time</code>	Specifies the minimum and maximum pulse width allowed for a logic-low state. The option is ignored if it is set to a negative value. See Figure 23 .
<code>-high_min_time</code> <code>high_min_time</code> <code>-high_max_time</code> <code>high_max_time</code>	Specifies the minimum and maximum pulse width allowed for a logic-high state. The option is ignored if it is set to a negative value. See Figure 23 .
<code>-loth logic_low_voltage</code>	Defines the logic low voltage for the signal. The default is the same as the set_logic_threshold value.
<code>-hith</code> <code>logic_high_voltage</code>	Defines the logic high voltage for the signal. The default is the same as the set_logic_threshold value.
<code>-twindow tstart tstop</code> <code>{tstart tstop}</code>	Specifies the time windows for the pulse width check. The default for <code>tstart</code> is 0s and <code>tstop</code> is the end of transient time.
<code>-subckt subcircuit_name</code>	Specifies the subcircuit name of the specified <code>node_name</code> where the pulse width check is performed. See the Command Scoping section for more information.
<code>-error_file</code> <code>output_file_name</code>	Specifies the file name of the output file for reporting any violations. The output file has a *.errt extension.
<code>-numv value</code>	Limits the number of reported violations. The default is to report all violations.
<code>-report totalv</code>	When you specify the <code>-report totalv</code> command, the report file contains a summary count of all violations detected from the <code>check_timing_pulse_width</code> check. You can limit reporting with the <code>-numv</code> option, but the <code>-report totalv</code> command provides the complete count of all violations detected. For example, if you specify: <pre>check_timing_pulse_width ... -numv 10 -report totalv</pre> The .errt file shows: <pre>* edge: Total number of violations reported = 10 * edge: Total number of violations detected = 6105</pre>

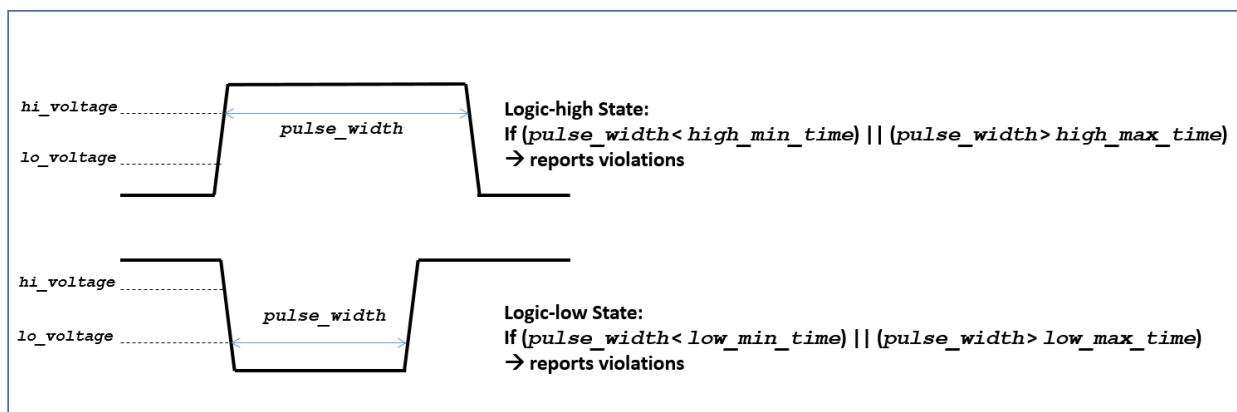
Description

When performing a timing check for pulse width, the PrimeSim XA tool reports the results if:

- The pulse width of a logic-low state is less than *low_min_time* or greater than *low_max_time*.
- The pulse width of a logic-high state is less than *high_min_time* or greater than *high_max_time*.

By default, the PrimeSim XA tool uses 30% and 70% of the rail-to-rail voltage of the signal as the logic threshold values. You can modify the values with the [set_logic_threshold](#) command or the `-loth` and `-hith` options of [check_timing_hold](#) command. The `-loth` and `-hith` options take precedence over the [set_logic_threshold](#) command.

Figure 23 Graphical Representation of `check_timing_pulse_width`



Examples

The following example sets up the timing check for pulse width for the output node with the `pw_check1` title. If the pulse width of logic-low state is less than 2 ns or greater than 10 ns, or if the pulse width of the logic-high state is less than 5 ns or greater than 50 ns, the PrimeSim XA tool reports the results.

```
check_timing_pulse_width -node output -title pw_check1 \  
  -low_min_time 2ns -low_max_time 10ns \  
  -high_min_time 5ns -high_max_time 50ns
```

check_timing_setup

Performs a timing check for the setup time of specified data (or target) nodes. The setup time is defined as the minimum amount of time a target signal must remain stable (that

is, not transition to the specified edge) before the reference signal active edge. Setup time violations are reported to an output file with a *.errt extension. Also, if you specify the -report min, -report max, or -report min max option, the command calculates all node setup times per reference event and measures the minimum or maximum values of these setup times. The minimum and maximum setup times for all nodes are reported in a separate file that has a *.tckrpt extension.

Syntax

```
check_timing_setup -title title_name node_terminal_spec
  -setup_time time [-subckt subckt_name] [-ref_subckt 0|1]
  [-data_edge_type edge_type] [-ref_edge_type edge_type]
  [-loth logic_low_voltage] [-hith logic_high_voltage]
  [-window window_limit] [-twindow tstart [tstop] {tstart [tstop]}]
  [-error_file output_file_name] [-numv value] [-report totalv]
```

or

```
check_timing_setup -title title_name node_terminal_spec
  -report min|max|min max [-subckt subckt_name] [-ref_subckt 0|1]
  [-data_edge_type edge_type] [-ref_edge_type edge_type]
  [-loth logic_low_voltage] [-hith logic_high_voltage]
  [-twindow tstart [tstop] {tstart [tstop]}] [-report_file file_name]
```

where

```
node_terminal_spec ::= node_spec | terminal_spec
node_spec ::= -node node_name {node_name} -ref_node node_name
terminal_spec :: -termn device_name {device_name} -ref_termn device_name
```

Arguments

Argument	Description
-title title_name	Specifies the title name of this timing check. The name is useful when there are multiple check_timing_setup commands in one simulation.
-node node_name {node_name}	Specifies the node names of the target signal to check for setup time relative to the reference signal, named ref_node_name. See the Command Scoping section for more information.
-ref_node node_name	Specifies the node name of the reference signal. It can only be one node name, and using a wildcard character is not allowed.
-termn device_name {device_name}	Specifies the terminal number <i>n</i> of the named devices as target signals to setup time relative to the reference signal. Using a wildcard character is supported only to one level of hierarchy.

Argument	Description
<code>-ref_termn device_name</code>	Specifies device terminal number <i>n</i> as the reference signal to check setup time. You cannot specify a wildcard character in the <i>device_name</i> .
<code>-setup_time time</code>	
<code>-report min max min max</code>	
<code>-subckt subckt_name</code>	Specifies the setup time and can be a positive or negative value. When it is set to a negative value, you must also specify <code>-window</code> argument. It follows these rules: - When <i>time</i> is set to a positive value, the target signal must not transition to the specified edge between ($T_{\text{reference}} - \text{setup_time}$) and $T_{\text{reference}}$. Otherwise, a setup time violation is reported. See Figure 24 or a graphical representation. - When <i>setup_time</i> is set to a negative value, the target signal must not transition to the specified edge between ($T_{\text{reference}} - \text{setup_time}$) and ($T_{\text{reference}} + \text{window value}$). Otherwise, a setup time violation is reported. See Figure 25 for a graphical representation. In addition, $T_{\text{reference}}$ is an active edge time of a reference signal.
<code>-ref_subckt 0 1</code>	Measures the minimum or maximum setup time of the target signals for each event of the reference signal.
<code>-data_edge_type edge_type</code>	Specifies the transition edge type of the target signal. It can be set to: <code>rise</code> (default) - Rising edge. <code>fall</code> - Falling edge. <code>rf</code> - Both the rising and falling edge.
<code>-ref_edge_type edge_type</code>	Specifies the transition edge type of the reference signal. It can be set to: <code>rise</code> (default) - Rising edge. <code>fall</code> - Falling edge. <code>rf</code> - Both the rising and falling edge.
<code>-loth logic_low_voltage</code>	Defines the logic low voltage for the signal. The default is the same as the set_logic_threshold value.
<code>-hith logic_high_voltage</code>	Defines the logic high voltage for the signal. The default is the same as the set_logic_threshold value.
<code>-window window_limit</code>	Specifies the width of a window to check for setup time if <i>setup_time</i> is a negative value. This option is ignored when <i>setup_time</i> is set to a positive value.
<code>-twindow tstart tstop {tstart tstop}</code>	Specifies the time windows for timing check. The default value for <i>tstart</i> is 0 and <i>tstop</i> is the end of transient time.

Argument	Description
<code>-error_file</code> <code>output_file_name</code>	Specifies the file name of the output file for reporting any violation. The output file has a *.errt extension.
<code>-numv value</code>	Limits the number of reported violations. The default is to report all violations.
<code>-report totalv</code>	When you specify the <code>-report totalv</code> option, the report file contains a summary count of all violations detected from the <code>check_timing_setup</code> check. You can limit reporting with the <code>-numv</code> option, but the <code>-report totalv</code> option provides the complete count of all violations detected. For example, if you specify: <pre>check_timing_setup ... -numv 10 -report totalv</pre> The .errt file shows: <pre>* edge: Total number of violations reported = 10 * edge: Total number of violations detected = 6105</pre>
<code>-report_file file_name</code>	Specifies the output text file name in a format of <code>file_name.tckrpt</code> .

Description

When performing the timing check for setup time, the PrimeSim XA tool reports the results of the setup time violations by the specified conditions. By default, the PrimeSim XA tool uses 30% and 70% of the rail-to-rail voltage of the signal as the logic threshold values. You can modify the value with the [set_logic_threshold](#) command or the `-loth` and `-hith` option of the `check_timing_setup` command. The `-loth` and `-hith` options take precedence over the [set_logic_threshold](#) command.

Figure 24 Graphical Representation of Positive setup_time

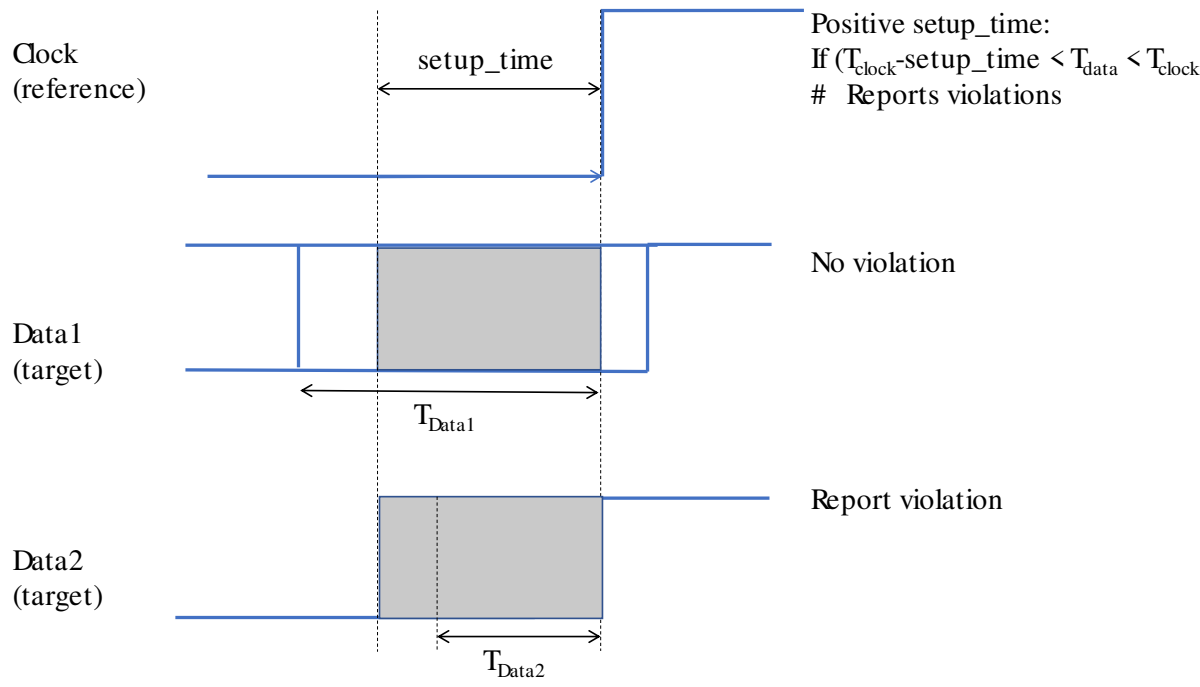
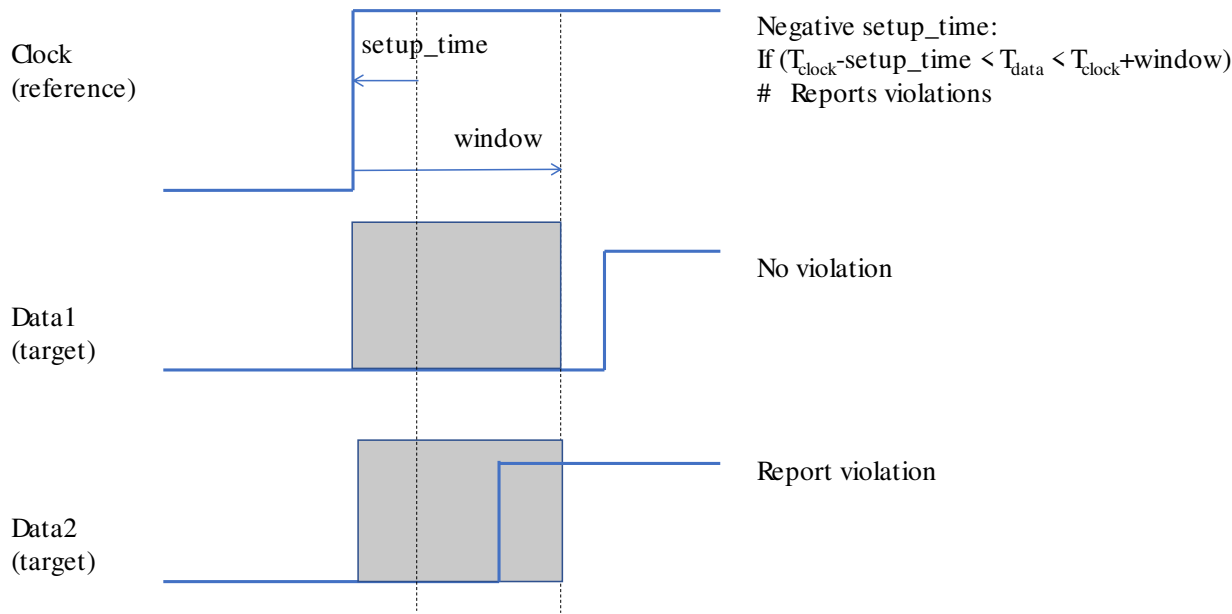


Figure 25 Graphical Representation of Negative setup_time With window_value



Examples

The following example sets up the timing check for setup time from the clk node to the output node with the `setup_check1` title. If the setup time is less than 2 ns, the PrimeSim XA tool reports the results.

```
check_timing_setup -node output -title setup_check1 -ref clk \  
-setup_time 2ns
```

The following example sets up the timing check for setup time from the clk node to all nodes with names beginning with “dout” inside the ram subcircuit with the `setup_check2` title. If the setup time is less than 2 ns, the PrimeSim XA tool reports the results to a file, named `ram_setup_check.errt`.

```
check_timing_setup -node dout* -title setup_check2 -ref clk \  
-setup_time 2ns -subckt ram -error_file ram_setup_check
```

The following example enables the timing check for setup time from the bus<*> nodes to the outputs node with the `setup_bus` title. The `-report min max` option enables a check of the setup time for all nodes per reference signal `ref_clk`. The PrimeSim XA tool reports the results in the `bus_tchk0.tckrpt` file.

```
check_timing_setup -node bus<*> -ref ref_clk -ref_edge_type rise \  
-title setup_bus -report min max -report_file bus_tchk0
```

set_report_option

Specifies the format of the output dynamic CCK analysis report to be CCK (default), JSON or both.

Syntax

```
set_report_option -format cck json
```

Arguments

cck

Writes the dynamic CCK analysis results to an output file in the CCK format.
This is default.

json

Writes the dynamic CCK analysis results to an output file in the JSON format.

Examples

The following example reports dynamic analysis results in both usual CCK and JSON formats.

```
set_report_option -format cck json
```